

Programming in C Manual

Prepared by

Er. Rupesh Shrestha

Lecturer

United Technical College

Affiliated to Pokhara University

1. INTRODUCTION

1.1 Computer

A digital computer is an electronic programmable machine that can process almost all kinds of data. The programmable feature has made this machine unique as compared to other machines. The same machine can be used in a small shop, in medical science, for space-research and various engineering activities. All this is possible by using different programs for different environments/applications.

1.2 Program

A program is a set of instructions, written in a sequence in a computer related language. All digital computers, regardless of their size, are basically electronic devices that can transmit, store, and manipulate **information** (i.e., **data**). Several different types of data can be processed by a computer. These include **numeric data**, **character data** (names, addresses, etc.), **graphic data** (charts, drawings, photographs, etc.), and **sound** (music, speech patterns, etc.). The two most common types, from the standpoint of a beginning programmer, are numeric data and character data. Scientific and technical applications are concerned primarily with numeric data, whereas business applications usually require processing of both numeric and character data.

To process a set of data, the computer must be given an appropriate set of instructions called a **program**. These instructions are entered into the computer and then stored in a portion of the computer's **memory**.

A stored program can be **executed at** any time. This causes the following things to happen.

- A set of information, called the **input data**, will be entered into the computer (from the keyboard, a floppy disk, etc.) and stored in a portion of the computer's memory.
- The input data will be processed to produce certain desired results, known as the **output data**.
- The output data, and perhaps some of the input data, will be printed onto a sheet of paper or displayed on a **monitor** (a television receiver specially designed to display computer output).

1.3 Basic Structure of a Digital Computer

The von-Neumann architecture, which is also known as the von Neumann model and Princeton architecture, is a computer architecture based on that described in 1945 by the mathematician and physicist John von Neumann. This describes a design architecture for an electronic digital computer with parts consisting of a processing unit containing an arithmetic logic unit and processor registers; a control unit containing an instruction register and program counter; a memory to store both data and instructions; external mass storage; and input and output mechanisms. The key idea of the Von-Neumann architecture is the stored program concept. A stored-program digital computer is one that keeps its program instructions, as well as its data, in read-write, random-access memory (RAM). Stored-program computers were an advancement over the program-controlled computers of the 1940s.

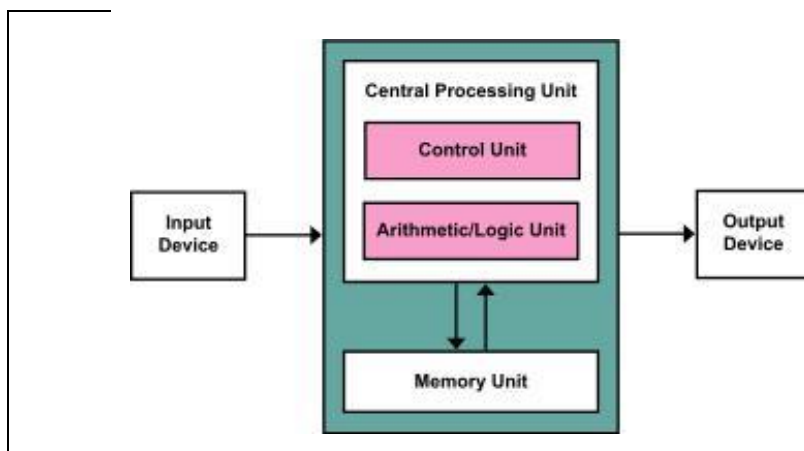


Figure Block Diagram of Computer

The main components of a computer are **Input unit (IU)**, **Central Processing unit (CPU)** and **Output unit (OU)**. The information like data, programs etc. are passed to the computer through input devices. The keyboard, mouse, oppy disk, CD, DVD, joystick etc. are certain input devices. The output device is to get information from a computer after processing. VDU (Visual Display Unit), Printer, Floppy disk, CD etc. are output devices.

The brain of a computer is CPU. It has three components- **Memory unit**, **Control unit** and **Arithmetic and Logical unit (ALU)**. Memory unit also called storage device is to store information. Two types memory are there in a computer. They are RAM (random access memory) and ROM (read only memory). When a program is called, it is loaded and processed in RAM. When the computer is switched-off, whatever stored in RAM will be deleted. So, it is a temporary memory. Whereas, ROM is a permanent memory, where data, program etc. are stored for future use. Inside a computer there is storage device called Hard disk, where data are stored and can be accessed at any time.

The control unit is for controlling the execution and interpreting of instructions stored in the memory. ALU is the unit where the arithmetic and logical operations are performed. The information to a computer is transformed to groups of binary digits, called

bit. The length of bit varies from computer to computer, from 8 to 64. A group of 8 bits is called a *Byte* and a byte generally represents one alphanumeric (Alphabets and Numerals) character. The physical components of a computer are called hardware. But for the machine to work it requires certain programs. They are called software.

1.3 Software

Software is the collection of computer programs, procedures and documentation that performs different tasks on a computer system. The types of computer software are system software and application software.

1.3.1 System Software

System software is the computer software designed to operate the computer hardware and to provide a platform for running application software. System software is the collection of operating system, device drivers, servers, windowing systems and utilities. Some Common types of system software are given below:

i. Operating system

The OS is the most important program that runs on a computer. Every general-purpose computer must have an operating system. Some functions of OS are:

- Recognize input from keyboard, send output to display screen.
- Keep track of files and directories on the disk.
- Controlling peripheral devices such as disk drives and printers.
- Provides interface between hardware and software.
- Provides security and backup, memory management, scheduling processes etc.

Some examples of OS are: Windows 2000, Mac-OS, UNIX, DOS etc.

ii. Utility Software

Utility software helps to manage, maintain and control computer resources. These programs are available to help you with the day-to-day chores associated with personal computing and to keep your system running at peak performance. Some examples of utility programs include:

Antivirus: designed to protect your computer from viruses.

Backup Software: designed to assist you in backing up your files and even the entire computer hard drives.

Disk Defragmentation Software: designed to assist you in keeping reorganized your disk drives. After files are saved, deleted and resaved again, the disk becomes fragmented. Disk defragmenters gather those free spaces and put them together to enable you to continue to save your data in the most efficient manner.

Language Processor: used to translate program to machine level from other languages. The types of language translators are *compiler*, *interpreter* and *assembler*.

1.3.2 Application Software

Application software enable the users to accomplish specific tasks. Business software, databases and educational software are some application software. Other application software is word processing, spreadsheet, presentation packages etc. Mainly, there are two types of application software: *package software* and *tailored software*.

i. Package Software

They are mainly designed by software companies to generalize the tasks. Some common types of package software are:

Word Processing Software: enables the users to create and edit documents. The most popular word processors are MS Word, Word-Pad, Notepad and some other text editors.

Database Software: allows the users to store and retrieve data from database. The Oracle, MS Access, MySQL, SQL etc. are database software.

Spreadsheet Software: allows users to perform different kinds of numeric calculations. E.g. MS Excel, Lotus, Apple numbers etc.

Multimedia Software: allows the users to create and play with picture, audio, video media. E.g. Media Player, Real Player etc.

Presentation Software: allows users to present their information in the form of slides. E.g. MS PowerPoint etc.

ii. Tailored Software

It is specific purpose software which is developed for specific task. They are mainly written in high level language such as Java, C, C++, COBOL. E.g. banking software, hospital software, hotel reservation software, billing software etc.

1.4 Programming Languages

Programming language is defined as a set of rules that provides a platform for instructing computer to perform some specific tasks. Programming languages are classified into various ways. Based on nature of language, they are classified into *general purpose* and *specific purpose* language. On the other hand, programming language are classified into *High-level language* and *Low-level language*.

- i. Low Level Language (LLL)
 - a. Machine Language
 - b. Assembly Language
- ii. High Level Language (HLL)
 - a. Third Generation Language (3GL)
 - b. Fourth Generation Language (4GL)
 - c. Fifth Generation Language (5GL)

i) a. Machine Language

Machine language is the language of CPU which consists of set of instructions composed with series of 0s and 1s.

Advantages:

- Machine language does not require translators.
- Execution time of machine language program is extremely fast.

Disadvantages:

- Programming is very complex and tedious job for the programmers.
- They are machine dependent; so, they support only specific hardware.
- It is very difficult to find bugs and hard to debug.

i) b. Assembly Language

Assembly language is special type of low-level language which consists set of alphanumeric instructions, called *mnemonics*. E.g. ADD A, B

Advantages:

- It is easier to understand than machine language. Also, it consumes less amount of memory than high-level language. Thus, ALL executes much faster than HLL.
- It is efficient for solving hardware interfacing problems. E.g. operating system kernel, programming language, utility software, virus, etc.
- Error finding and debugging process is easier than HLL.

Disadvantages

- ALL is hardware dependent language.
- Programmer should have depth knowledge about the computer hardware during programming.
- Program execution is slower than MLL and the program is lengthy and complex than HLL.

ii) a. Third Generation Language (3GL)

3GL is general-purpose high-level language in which the instructions are expressed in more like English structure and mathematical expressions. E.g. FORTRAN, BASIC, Pascal, C etc. are examples of 3GL.

Advantages

- Machine independent language.
- Easy for program development because the program instructions are English like structure and Mathematical expressions.
- Easy for catching bugs and easy to debug and modify.
- Programmer does not require depth knowledge about computer architecture while programming.

Disadvantages

- 3GL requires translator. So, program execution is slower than LLL.
- They are general-purpose language. They are not suitable for solving complex and specific type of real-world problems.
- They are mostly CUI based language. So, they are less user-friendly.
- Less efficient than 4GL and 5GL.

b. Fourth Generation Language (4GL)

4GL is a specific-purpose high-level language in which the instructions are expressed by using graphical components and English like structures and Mathematical expressions. E.g.

- *Database and Query Processing Language* (DBase, SyBase, DB2, MySQL, SQL, MS Access, Oracle etc.)
- *Application Generator Language* (Java, C#, Visual C++, Visual.NET etc.)
- *Web Based Scripting Language*: (HTML, XML, JavaScript, VBScript, ASP, PHP, JSP etc.)

Advantages:

- It is efficient and more user friendly.
- It supports rapid application development. Programmers can design, create and debug an application in very short period.

- IDE (Integrated Development Environment) helps developer to include powerful features of different programming languages into one application.
- They are platform independent.

Disadvantages:

- They require more memory and disk storage, hence, take more execution time.
- It is not suitable for solving hardware interfacing problems.

c. Fifth Generation Language (5GL)

5GL is a special type of future generation language in which the statements will be expressed in more like human language. E.g. PROLOG, LISP etc. are considered as 5GL. Some characteristics of 5GL are:

- Natural languages make human and machine closer than before.
- They are designed to allow the computer more natural and intelligent than before.
- They are used in the areas of research, robotics, defense system, and different types of expert systems.

1.5 Translators

A translator is a special type of system software that converts program written in any programming language into MLL. It is also known as language processor. A program code written using any programming language is called source code. A program code obtained after converting source code by translator is called object code or machine code. Mainly, there are three types of language translators: *Assembler, Compiler and Interpreter*.

i. Assembler

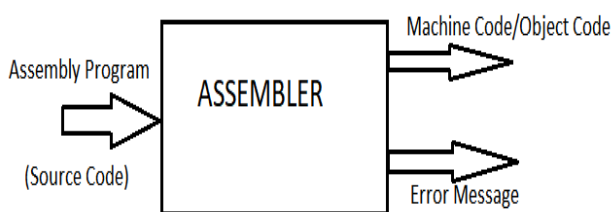


Fig: Assembler

Assembler translates ALL program into MLL program. As computer understands only machine language, assembly language program must be translated into machine language program before execution. Otherwise, computer does not understand the codes. The job of assembler is shown in the figure.

ii. Interpreter

An interpreter translates high level language program into machine language program one instruction at a time. When an error is encountered in the program, the process of translation is halted, and error message is displayed. Programming language such as BASIC, LISP etc. use interpreter. The job of interpreter is shown in figure below.

iii. Compiler

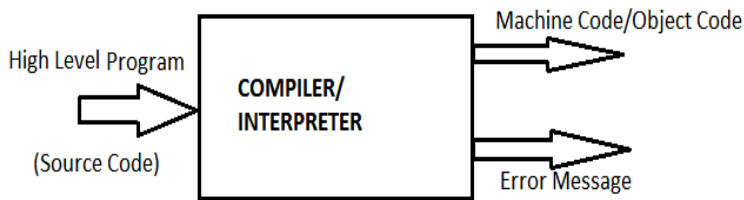


Fig: Compiler/Interpreter

A compiler translates HLL program into MLL program. While translating the program, it checks syntax of whole program, at first, and then translates the code into MLL if no error is occurred. If there are syntax errors, then compiler displays all syntax errors at the same time.

Compiler is more efficient and faster than interpreter. Programming language such as C, C++, C#, Visual Basic etc. use compiler.

1.6 Traditional and Structured Programming

Structured programming is a programming paradigm aimed at improving the clarity, quality, and development time of a computer program by making extensive use of subroutines, block structures, for and while loops—in contrast to using simple tests and jumps such as the go to statement which could lead to “spaghetti code” causing difficulty to both follow and maintain.

It is possible to do structured programming in any programming language, though it is preferable to use something like a procedural programming language. Some of the languages initially used for structured programming include: ALGOL, Pascal, PL/I and Ada – but most new procedural programming languages since that time have included features to encourage structured programming, and sometimes deliberately left out features – notably GOTO – in an effort to make unstructured programming more difficult. Structured programming (sometimes known as modular programming) is a subset of imperative programming that enforces a logical structure on the program being written to make it more efficient and easier to understand and modify.

2. Programming Logic

Problems solving

First and foremost, step of solving a problem is knowing about the nature of problem in order to solve it. Almost all types of problems can be solved with computer. However, correct formulation of the problem in computer understandable terms is essential to formulate the problem and solve it using computer.

2.1 Understanding of problems, Feasibility and Requirement Analysis

In order to start to develop a solution to a problem using computer, we need to first understand the problem about its nature, complexities and others. Problems can be of various nature and we need to understand about the problems to be able to solve it using computer. Once the problem has been understood and we can solve it using computer, then the feasibility of the problem solution needs to be determined. We can solve a problem using various logic but what needs to be worked out is the most feasible solution that can be derived with the least amount of complexities. Another major portion to solving problems using computer is the analysis of the

requirements. The solution to the problem can be solved however the requirements required is beyond the possible technologies of today. Then the solution may not be feasible to the problem.

2.2 Design Flowchart and Algorithm

An algorithm is a solution to a computer programming problem. In other words, a step by step procedure for developing a problem is called an algorithm. Algorithms can be written in two different ways. Pseudocode English like steps that describe the solution. Pseudo code is an artificial and informal language that helps programmers develop algorithms. Pseudo code is a "text-based" detail (algorithmic) design tool. The rules of Pseudocode are reasonably straightforward. All statements showing "dependency" are to be indented. These include while, do, for, if, switch statements.

```
Set total to zero
Set grade counter to one
While grade counter is less than or equal to ten
Input the next grade
Add the grade into the total
Set the class average to the total divided by ten
Print the class average.
```

Flowchart Pictures Detailing with specific blocks detailing out the logical flow of the solution. For a better understanding of an algorithm, it is represented pictorially. The pictorial representation of an algorithm is called a Flowchart. The algorithm and flowchart to add two numbers can be stated as:

Step1: Input numbers as a and b

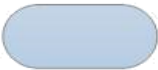
Step2: Sum = a +

b

Step3: Print the sum

used

The
of two
stated

Symbol	Name	Function
	Start/end	An oval represents a start or end point
	Arrows	A line is a connector that shows relationships between the representative shapes
	Input/Output	A parallelogram represents input or output
	Process	A rectangle represents a process
	Decision	A diamond indicates a decision

Various symbols can be to represent different actions like taking input, making decisions and connecting flowcharts. flowchart for the addition numbers whose algorithm has been above can be drawn as:

Advantages of flowchart

- **Communication:** Flowcharts are a good way of communication. descriptions of

They quickly and clearly provide logic, ideas and algorithms to other programmers, students, teacher, computer operators and users.

- **Effective Analysis:** Flowchart provides a clear overview of the entire problem and its algorithm. They show all the major components and their relationships.
- **Proper Documentation:** The flowchart provides a permanent recording of program logic. It documents the steps followed in an algorithm.

- *Efficient Coding*: The flowchart specifies all the steps to be coded and helps to prevent omission or error. Thus, a flowchart acts as a guide or blueprint during the system analysis and program development phase.

Limitations

- *Complex Logic*: The flowchart becomes complex for complicated and large problems.
- *Difficult Modification*: If alternations are required, the flowchart may need to be redrawn completely.

Assignments

- *Write differences between algorithm and flowchart.*
- *What are the significances of algorithm and flowchart?*

2.3 Program Coding

Once algorithm and flowchart has been developed the task now remains is to write programs using some high-level programming language. C, C++, Java, Python are the most popular programming language to develop programs. PHP and ASP remain a popular choice for developing web-based applications.

2.4 Testing and Debugging

Once the program code has been written in a selected programming language of choice the next task to complete is the testing and debugging. Testing and debugging help to find the problems associated with the program behavior under normal and abnormal circumstances. Extensive testing like white box testing and black box testing, integration testing needs to do before the program is deployed into the real-world scenario.

Programming theorists often refer debugging to verification and testing to validation. Verification ensures that the program does what the programmer intends to do. Validation ensures that the program produces the correct results for a set of test data. The different tools like simulators, logic analyzers, breakpoints, trace routines, software interrupts can be used for effective debugging process.

For testing process, test data are supplied to the program and output is observed. If the output is as expected, the program can be considered error free. The error means failure of compiling and executing the program successfully and receiving incorrect results. The types of errors are *syntax error*, *semantic/logical error*, and *runtime error*.

2.5 Implementation

After a program has been written and tested it needs to be implemented to the target environment to solve the problem. Various needs of physical hardware and accessories required by the program to solve the intended problem needs to be present upon implementation. Once the program is implemented it starts to work.

2.6 Evaluation and Maintenance

The evaluation of the performance of the program needs to be done at frequent interval once the program or software is implemented. The advent of new technology, upscaling and downscaling of the business, request for the change from customers, finding of a new bug are the major reasons for the maintenance and change of the software. Once changes have been continuous monitoring of the software performance needs to do to discover the flaws in the software.

2.7 Documentation

The same workforce that developed the program may have left the project and gone in search of opportunities. So, a program must be well documented for the new people to understand how the software was developed and how can it be modified. The documentation starts from the very beginning of the problem formulation to the very end of the Evaluation and Maintenance. Program documentation is the systematic writing of activities from problem analysis to debugging/testing phase including operational information and presenting it as a report. There are two types of documentations: Technical documentation for programmer and user manual for user.

Technical documentation is prepared for future reference to the programmers who maintain, redesign and upgrade the system. It may contain detailed requirements of program, format and reports and forms, logic of the program, data flow diagram (DFD), Entity-Relationship diagram (ERD), algorithm and flowchart, input and output data used for testing and other references.

The user manual supports to the user of the program. This provides instructions for installation of the program and use of it effectively.

Assignment:

Although the documentation is the last phase of problem solving, why documentation is considered as very important phase?

Algorithm and Flowchart (More example)

1. Write an algorithm and draw flowchart to find the number given by user is divisible by 2,3 and 6 or not.

Algorithm

1. Start
2. Declare variables n, r2, r3 and r6.
3. Read a number n.
4. Calculate $r2 = n \text{ mod } 2$.
5. Calculate $r3 = n \text{ mod } 3$.
6. Calculate $r6 = n \text{ mod } 6$.
7. Check if $r2=0$. If true, go to step 8 otherwise go to step 10.
8. Check if $r3=0$. If true, go to step 9 otherwise, go to step 10.
9. Check if $r6=0$. If true, print "n is divisible by 2, 3 and 6". Otherwise go to step 10.
10. Print "n is not divisible by 2 ,3 and 6."
11. Stop.

Flowchart is shown in figure.

2. Write an algorithm and draw flowchart to check if the given number is prime or not.

Algorithm

1. Start.
2. Declare variables n, j, rem.
3. Read a number n.
4. Initialize $j=2$.
5. Calculate $\text{rem}=n \text{ mod } j$.
6. If $\text{rem}=0$ then go to step 8.

7. Increment j by 1.
8. If $j < n$, then go to step 5.
9. Check if $n = j$. If true, then print "n is prime". Otherwise, print "n is no prime".
10. Stop.

Flowchart

3. Write algorithm and draw flowchart to find the Armstrong numbers from 150 to 500.

Algorithm

1. Start.
2. Declare variables rem, j, Sum, Temp.
3. Initialize $j = 150$.
4. Initialize $\text{Sum} = 0$.
5. Assign $\text{Temp} = j$.
6. While $\text{Temp} > 0$, repeat steps 7, 8 and 9. Otherwise, go to step 10.
7. Calculate $\text{rem} = \text{Temp} \bmod 10$.
8. $\text{Sum} = \text{Sum} + \text{rem} * \text{rem} * \text{rem}$.
9. Update $\text{Temp} = \text{Temp} / 10$ and go to step 6.
10. Check if $j = \text{Sum}$. If true, then print j.
11. Increment j by 1. If $j \leq 500$, then go to step 4.
12. Stop.

Flowchart

4. Write an algorithm and draw flowchart to find the smallest number among three.

Algorithm

1. Start.
2. Declare variables a, b, c, and s.
3. Read three numbers a, b, and c.
4. If $a < b$ and $a < c$ then assign $s = a$ and go to step x.
5. Else if $b < c$ then assign $s = b$ and go to step x.
6. Else assign $s = c$.
7. Print "s is the smallest number".
8. Stop.

Flowchart

5. Write an algorithm and develop flowchart to print the even numbers from 150 to 500.

Algorithm

1. Start
2. Declare variable j.
3. Initialize $j = 150$.
4. If $j \bmod 2 = 0$, then print j.
5. Increment j by 1.
6. If $j \leq 500$, go to step 4.
7. Stop.

6. Write an algorithm and draw flowchart to test whether the given number is palindrome or not.

Algorithm

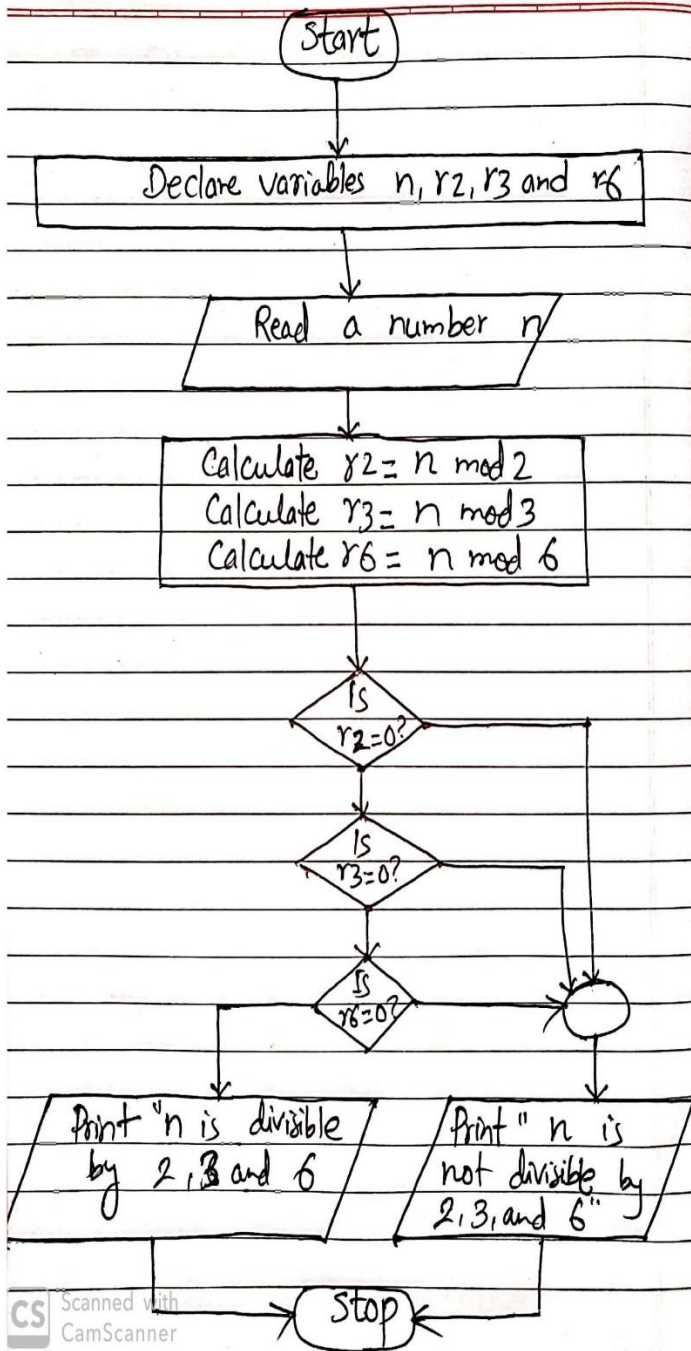
1. Start
2. Declare variables n, rem, rev, temp.
3. Initialize rev=0.
4. Read a number n.
5. Assign temp=n.
6. While temp>0, repeat steps 7,8,9. Otherwise, go to step 10.
7. Calculate rem=temp mod 10.
8. Calculate rev=rev*10+rem.
9. Update temp=temp/10 and go to step 6.
10. Check if n=rev. If true, print "n is palindrome".
Otherwise, print "n is not palindrome."
11. Stop.

Flowcharts

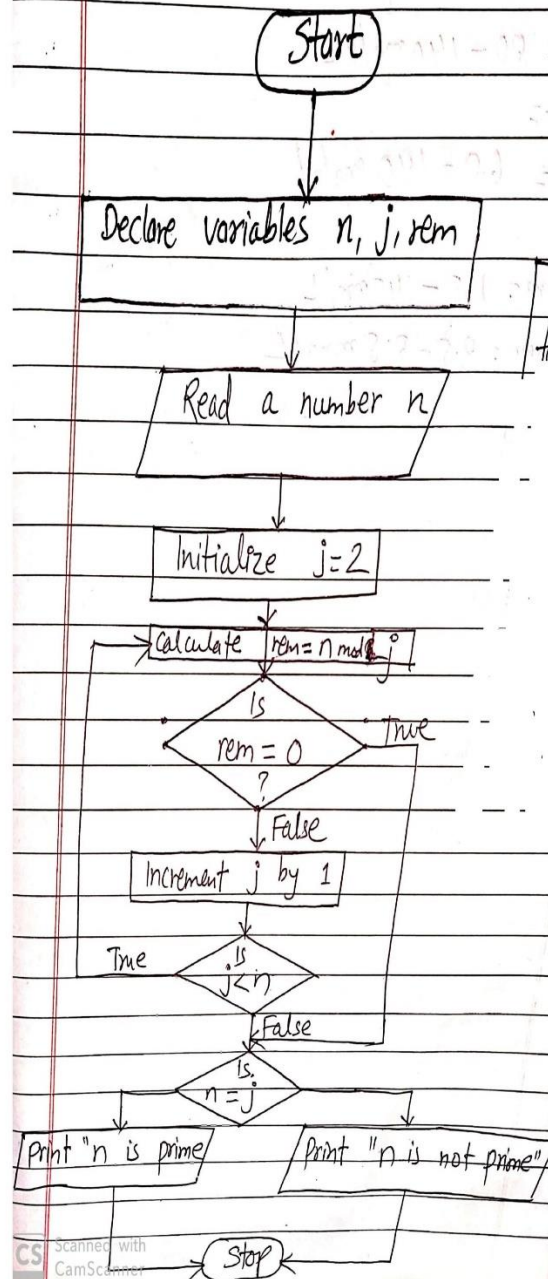
Flowchart 1

Page No.

Date: / /

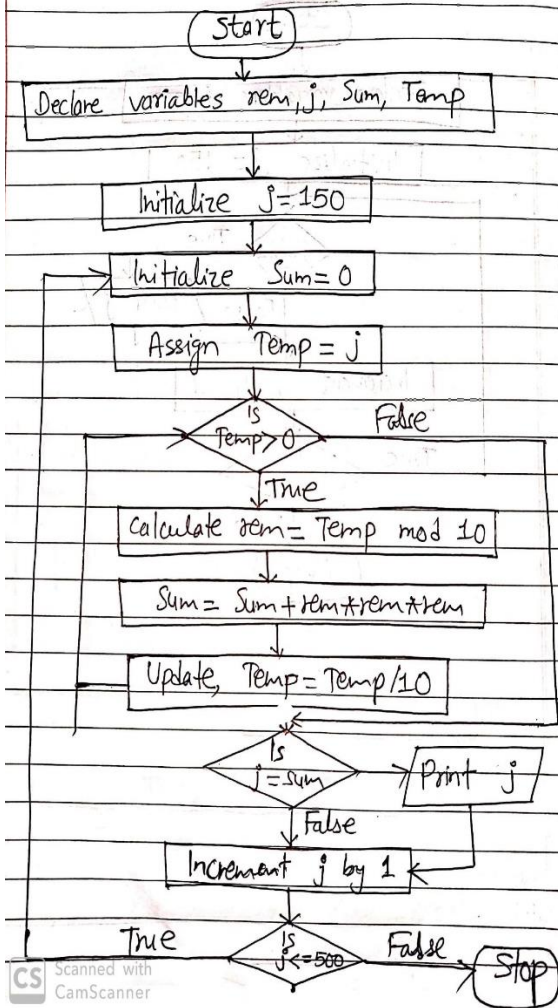


Flowchart 2

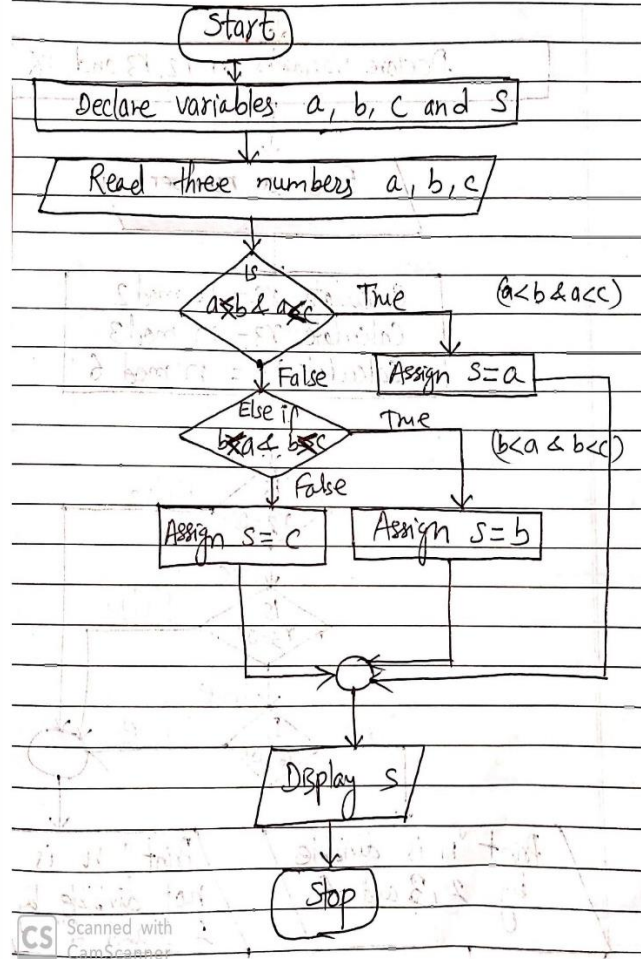


Flowchart-3

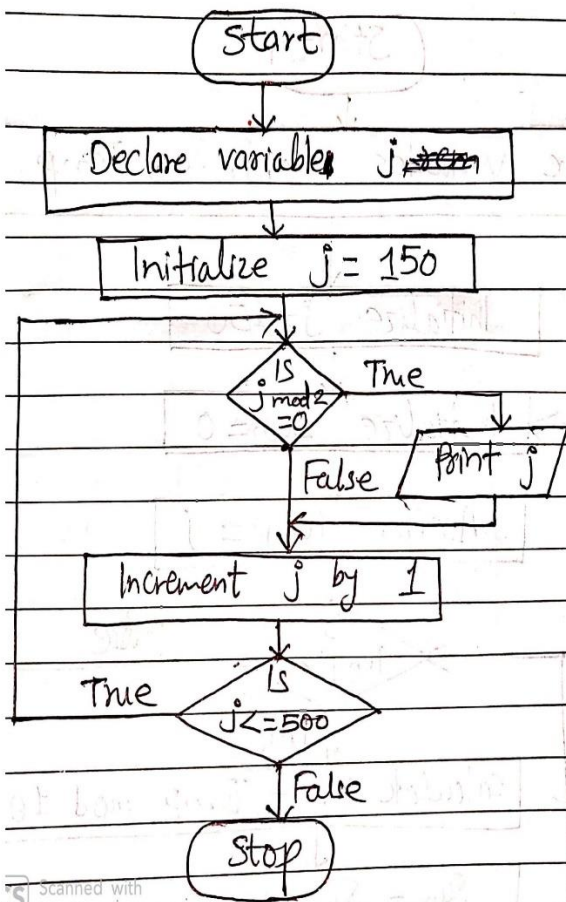
Date: / /



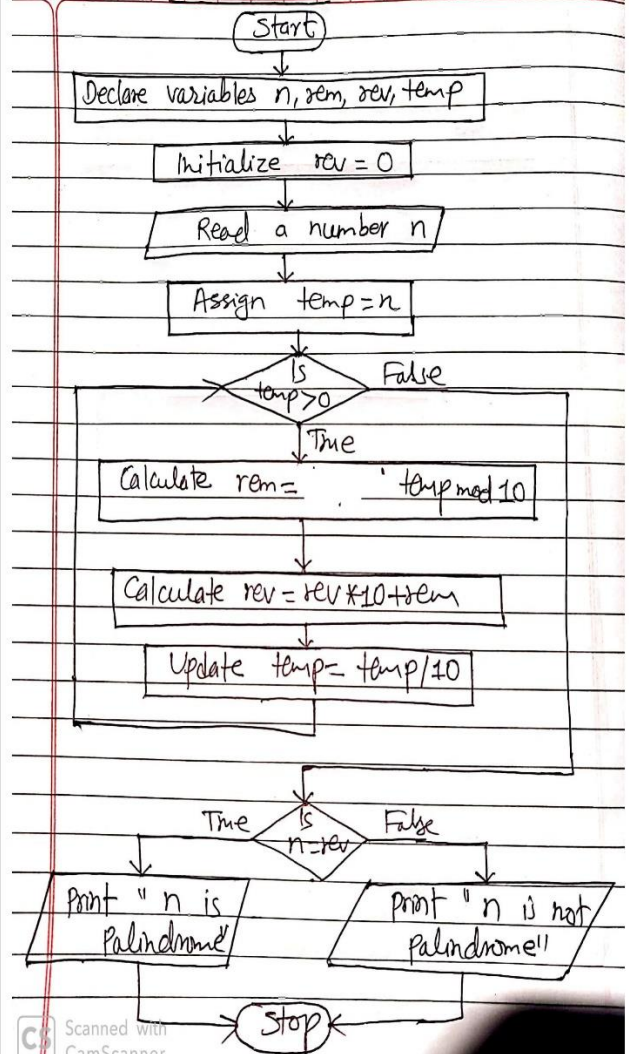
Flowchart 4



Flowchart 5



Flowchart 6



3. VARIABLES AND DATA TYPES

Tokens in C

The basic elements recognized by the C compiler are called as tokens. A token is the source program text that the compiler does not break down into component element. The following elements are types of the token.

- Keywords
- Constants
- Identifiers
- String
- Operators
- Special symbols

The C Character Set

C uses the uppercase letters A to Z, the lowercase letters a to z, the digits 0 to 9, and certain special characters as building blocks to form basic program elements (e.g., constants, variables, operators, expressions, etc.). The special characters are listed below:

+	-	*	/	=	%	&	#	!	?	^	“	‘
	~	\		<	>	(	)	[	]	{	}	:
	;	.	,	-	(blank space)							

Most versions of the language also allow certain other characters, such as @ and \$, to be included within strings and comments.

C uses certain combinations of these characters, such as \b, \n, and \t, to represent special conditions such as backspace, newline and horizontal tab, respectively. These character combinations are called as *escape sequences*, which are also considered as a character.

Identifiers and Keywords

Identifiers are names that are given to various program elements, such as variables, functions, arrays, structure, symbolic constants etc. Identifiers consists of letters and digits, in any order, except that the first character must be a letter. Both upper- and lowercase letters are permitted, though common usage favors the use of lowercase letters for most types of identifiers. Upper- and lowercase letters are not interchangeable (i.e., case sensitive). The underscore character (_) can also be included and is considered to a letter. An underscore is often used in the middle of an identifier. An identifier may also begin with an underscore, though it is rarely done in practice.

The following names are valid identifiers.

x	y12	sum_1	_temperature
names	area	tax_rate	TABLE

The following names are not valid identifiers for the reasons stated.

4th The first character must be a letter.
 X" Illegal character (").
 order-no Illegal character (-).
 error flag Illegal character (blank space).

An identifier can be arbitrarily long. Some implementations of C recognize only the first eight characters, though most implementations recognize more (typically, **31 characters**). Additional characters are carried along for the programmer's convenience. The identifiers *file_manager* and *file_management* are both grammatically valid. Some compilers may be unable to distinguish between them, however, because the first eight letters are the same for each identifier. Therefore, only one of these identifiers should be used in a single C program.

As a rule, an identifier should contain enough characters so that its meaning is readily apparent. On the other hand, an excessive number of characters should be avoided. A C program is being written to calculate the future value of an investment. The identifiers *value* or *future_value* are appropriate symbolic names. However, *v* or *fv* would probably be too brief, since the intended representation of these identifiers is not clear. On the other hand, the identifier *future_value_of_an_investment* would be unsatisfactory because it is too long and cumbersome.

There are certain reserved words, called **keywords**, that have standard, predefined meanings in C. These keywords can be used only for their intended purpose; they cannot be used as programmer-defined identifiers. The standard keywords are:

auto	double	int	struct
break	else	long	switch
case	enum	register	typedef
char	extern	return	union
const	float	short	unsigned
continue	for	signed	void
default	goto	sizeof	volatile
do	if	static	while

Some compiler may also include some or all the following keywords.

ada	far	near	asm
fortran	pascal	entry	huge

Note that the keywords are all lowercase. Since uppercase and lowercase characters are not equivalent, *it is possible to utilize an uppercase keyword as an identifier*. Normally, however, this is not done, as it is considered a poor programming practice.

Assignment:

- Determine which of the following are valid identifiers. If invalid, explain why.

i.record1 ii.1record iii.file_3 iv.return v.\$tax vi.name
vii.name and address viii.name_and_address ix.name-and-address
x.123-45-6789

- Assume that your version of C can recognize only the first 8 characters of an identifier name. Which of the following pairs of identifier names are identical and which are distinct?

i.name, names ii.address, Address iii.identifier_1, identifier_2
iv.list1, list2 v.answer, ANSWER vi.char1, char_1

Datatypes

C supports several different types of data, each of which may be represented differently within the computer's memory. The basic data types are listed below. Typical memory requirements are also given. (The memory requirements for each data type will determine the permissible range of values for that data type. Note that the memory requirements for each data type may vary from one C compiler to another.)

C compilers written for personal computers or small minicomputers (i.e., computers whose natural word size is less than 32 bits) generally represent a word as 4 bytes (32 bits).

The basic data types can be augmented (prefixed) using the data type *qualifiers* **short**, **long**, **signed** and **unsigned**. For example, integer quantities can be defined as *short int*, *long int* or *unsigned int* (these data types are usually written simply as *short*, *long* or *unsigned*, and are understood to be integers).

The interpretation of a qualified integer data type will vary from one C compiler to another, though there are some commonsense relationships. Thus, a *short int* may require less memory than an ordinary *int* or it may require the same amount of memory as an ordinary *int*, but it will never exceed an ordinary *int* in word length. Similarly, a *long int* may require the same amount of memory as an ordinary *int* or it may require more memory, but it will never be less than an ordinary *int*.

If *short int* and *int* both have the same memory requirements (e.g., 2 bytes), then *long int* will generally have double the requirements (e.g., 4 bytes). Or if *int* and *long int* both have the same memory requirements (e.g., 4 bytes) then *short int* will generally have half the memory requirements (e.g., 2 bytes). Remember that the specifics will vary from one C compiler to another.

An *unsigned int* has the same memory requirements as an ordinary *int*. However, in the case of an ordinary *int* (or a *short int* or a *long int*), the leftmost bit **is** reserved for the sign. With an *unsigned int*, all the bits are used to represent the numerical value. Thus, an *unsigned int* can be approximately twice as large as an ordinary *int* (though, of course, *negative values are not permitted*). For example, if an ordinary *int* can vary from -32,768 to +32,767 (which is typical for a 2-byte *int*), then an *unsigned int* will be allowed to vary from 0 to 65,535. The *unsigned* qualifier can also be applied to other qualified integers, e.g., *unsigned short int* or *unsigned long int*.

The `char` type is used to represent individual characters. Hence, the `char` type will generally require only one byte of memory. Each `char` type has an equivalent integer interpretation, however, so that a `char` is a really a special kind of `short integer`. With most compilers, a `char` data type will permit a range of values extending from **0 to 255**. Some compilers represent the `char` data type as having a range of values extending from **-128 to +127**. There may also be `unsigned char` data (with typical values ranging from **0 to 255**), or `signed char` data (with values ranging from **-128 to +127**). Some compilers permit the qualifier `long` to be applied to `float` or to `double`, e.g., `long float`, or `long double`. However, the meaning of these data types will vary from one C compiler to another. Thus, `long float` may be equivalent to `double`. Moreover, `long double` may be equivalent to `double`, or it may refer to a separate, “extra-large” double-precision data type requiring more than two words of memory. Two additional data types, `void` and `enum` are also used frequently. Every identifier that represents a number or a character within a C program must be associated with one of the basic data types before the identifier appears in an executable statement. This is accomplished via a ***type declaration***.

Data Type Summary

<u>Data Type</u>	<u>Description</u>	<u>Typical Memory Requirements</u>
int	<i>Integer quantity</i>	2 bytes or 1 word (varies from one computer to another)
short	<i>Short integer quantity (may contain fewer digits than int)</i>	2 bytes or 1 word (varies from one computer to another)
long	<i>Long integer quantity (may contain more digits than int)</i>	1 or 2 words (varies from one computer to another)
unsigned	<i>Unsigned (positive) integer quantity (maximum permissible quantity is approximately twice as large as int)</i>	2 bytes or 1 word (varies from one computer to another)
char	<i>Single character</i>	1 byte
signed char	<i>Single character, with numerical values ranging from -128 to +127</i>	1 byte
unsigned char	<i>Single character, with numerical values ranging from 0 to 255</i>	1 byte
float	<i>Floating-point number (i.e., a number containing a decimal point and or an exponent)</i>	1 word
double	<i>Double-precision floating-point number (i.e., more significant figures, and an exponent that may be larger in magnitude)</i>	2 words
long	<i>Double-precision floating-point number</i>	2 or more words

double	(may have higher precision than double)	(varies from one computer to another computer)
void	Special data type for functions that do not return any value	(not applicable)
enum	Enumeration constant (special type of int)	2 bytes or 1 word (varies from one computer to another)

CONSTANTS

There are four basic types of constants in C. They are *integer constants*, *floating-point constants*, *character constants* and *string constants* (there are also *enumeration constants*). Moreover, there are several different kinds of integer and floating-point constants, as discussed below.

Integer and floating-point constants represent numbers. They are often referred to collectively as **numeric-type** constants. The following rules apply to all numeric-type constants.

- Commas and blank spaces cannot be included within the constant.
- The constant can be preceded by a minus (-) sign if desired. (Actually, the minus sign is an **operator** that changes the sign of a positive constant, though it can be thought of as a part of the constant itself.)
- The value of a constant cannot exceed specified minimum and maximum bounds. For each type of constant, these bounds will vary from one C compiler to another.

Let us consider each type of constant individually.

Integer Constants

An **integer constant** is an integer-valued number. Thus, it consists of a sequence of digits. Integer constants can be written in three different number systems: decimal (base 10), octal (base 8) and hexadecimal (base 16). Beginning programmers rarely, however, use anything other than decimal integer constants.

A **decimal** integer constant can consist of any combination of digits taken from the set 0 through 9. If the constant contains two or more digits, the first digit must be something other than 0.

Several valid decimal integer constants are shown below.

0 1 743 5280 32767 9999

The following decimal integer constants are written incorrectly for the reasons stated.

12,245	illegal character (,).
36.0	illegal character (.).
10 20 30	illegal character (blank space).
123-45-6789	illegal character (-).
0900	the first digit cannot be a zero.

An **octal** integer constant can consist of any combination of digits taken from the set 0 through 7. However, the first digit must be 0, in order to identify the constant as an octal number.

Several valid octal integer constants are shown below.

0 01 0743 077777

The following octal integer constants are written incorrectly for the reasons stated.

743 Does not begin with 0.

05280 Illegal digit (8).

0777.777 Illegal character (.).

A *hexadecimal* integer constant must begin with either *0x* or *0X*. It can then be followed by any combination of digits taken from the sets *0 through 9* and *A through F* (either upper- or lowercase). Note that the letters *a* through *f* (or *A* through *F*) represent the (decimal) quantities 10 through 15, respectively.

Several valid hexadecimal integer constants are shown below.

0x 0x1 0X7FFF 0xabcd

The following hexadecimal integer constants are written incorrectly for the reasons stated

0x12.34 Illegal Character (.)

0BE38 Does not begin with 0x or 0X

0x. 4bff Illegal Character (.)

0XDEFG Illegal character (G)

The magnitude of an integer constant can range from zero to some maximum value that varies from one computer to another (and from one compiler to another, on the same computer). A typical maximum value for most personal computers and many minicomputers is *32767 decimals* (equivalent to *77777 octal* or *7fff hexadecimal*), which is $2^{15} - 1$.

Mainframe computers generally permit larger values, such as *2,147,483,647* (which is $2^{31} - 1$).

Unsigned and Long Integer Constants

Unsigned integer constants may exceed the magnitude of ordinary integer constants by approximately a factor of 2, though they may not be negative.* An unsigned integer constant can be identified by appending the letter **U** (either upper- or lowercase) to the end of the constant. **Long** integer constants may exceed the magnitude of ordinary integer constants but require more memory within the computer. With some computers (and or some compilers), a long integer constant will automatically be generated simply by specifying a quantity that exceeds the normal maximum value. It is always possible, however, to create a long integer constant by appending the letter **L** (either upper- or lowercase) to the end of the constant. An unsigned long integer may be specified by appending the letters **UL** to the end of the constant. The letters may be written in either upper- or lowercase. However, the **U** must precede the **L**.

Several unsigned and long integer constants are shown below.

<u>Constant</u>	<u>Number System</u>
50000U	decimal (unsigned)
123456789L	decimal (long)
123456789UL	decimal (unsigned long)
0123456L	octal (long)
077777711	octal (unsigned)
0X50000U	hexadecimal (unsigned)
0XFFFFFFUL	hexadecimal (unsigned long)

Floating-Point Constants

A *floating-point constant* is a base-10 number that contains either a decimal point or an exponent (or both).

Several valid floating-point constants are shown below.

0.	1.	0.2	827.602
50000.	0.000743	12.3	315.0066
2E-8	0.006e-3	1.6667E+8	.12121212e12

The following are *not* valid floating-point constants for the reasons stated.

Either a decimal point or an exponent must be present.

1,000.0 Illegal character (,).

The exponent must be an integer quantity (it cannot contain a decimal point).

2E+10.2

3E 10 Illegal character (blank space) in the exponent.

The interpretation of a floating-point constant with an exponent is essentially the same as scientific notation, except that the base 10 is replaced by the letter E (or e). Thus, the number 1.2×10^{-3} would be written as *1.2E-3* or *1.2e-3*. This is equivalent to *0.12e-2*, or *12e-4*, etc. The quantity 3×10^5 can be represented in C by any of the following floating-point constants.

300000.	3e5	3e+5	3E5	3.0e+5
.3e6	0.3E6	30E4	30.E+4	300e3

Similarly, the quantity 5.026×10^{-17} can be represented by any of the following floating-point constants.

5.026E-17	.5026e-16	50.26e-18	.0005026E-13
------------------	------------------	------------------	---------------------

Character Constants

A *character constant* is a single character, enclosed in apostrophes (i.e., single quotation marks). Several character constants are shown below.

'a' 'X' '3' ' '

Notice that the last constant consists of a blank space, enclosed in apostrophes.

Character constants have integer values that are determined by the computer's particular character set. Thus, the value of a character constant may vary from one computer to another. The constants themselves, however, are independent of the character set. This feature eliminates the dependence of a C program on any particular character set (more about this later).

Most computers, and virtually all personal computers, make use of the ASCII (i.e., American Standard Code for Information Interchange) character set, in which each individual character is numerically encoded with its own unique 7-bit combination (hence a total of $2^7 = 128$ different characters). The following Table contains the ASCII character set, showing the decimal equivalent of the 7 bits that represent each character. Notice that the characters are ordered as well as encoded. In particular, the digits are ordered consecutively in their proper numerical sequence (**0** to **9**), and the letters are arranged consecutively in their proper alphabetical order, with uppercase characters preceding lowercase characters. This allows character-type data items to be compared with one another, based upon their relative order within the character set.

Several character constants and their corresponding values, as defined by the ASCII character set, are shown below.

<u>Constant</u>	<u>Value</u>
'A'	65
'x'	120
'3'	51

' ? ' 63
' ' 32 (blank)

These values will be the same for all computers that utilize the ASCII character set. The values will be different, however, for computers that utilize an alternate character set. IBM mainframe computers, for example, utilize the EBCDIC (i.e., Extended Binary Coded Decimal Information Code) character set, in which each individual character is numerically encoded with its own unique 8-bit combination. The EBCDIC character set is distinctly different from the ASCII character set.

ASCII Value	Character	ASCII Value	Character	ASCII Value	Character	ASCII Value	Character
0	NUL	32	(blank)	64	@	96	
1	SOH	33	!	65	A	97	a
2	STX	34	"	66	B	98	B
3	ETX	35	#	67	C	99	c
4	EOT	36	\$	68	D	100	d
5	ENQ	37	%	69	E	101	e
6	ACK	38	&	70	F	102	f
7	BEL	39	'	71	G	103	g
8	BS	40	(	72	H	104	h
9	HT	41	)	73	I	105	i
10	LF	42	*	74	J	106	j
11	VT	43	+	75	K	107	k
12	FF	44	,	76	L	108	l
13	CR	45	-	77	M	109	m
14	SO	46	.	78	N	110	n
15	SI	47	/	79	O	111	o
16	DLE	48	0	80	P	112	p
17	DC1	49	1	81	Q	113	q
18	DC2	50	2	82	R	114	r
19	DC3	51	3	83	S	115	s
20	DC4	52	4	84	T	116	t
21	NAK	53	5	85	U	117	u
22	SYN	54	6	86	V	118	v
23	ETB	55	7	87	W	119	w
24	CAN	56	8	88	X	120	x
25	EM	57	9	89	Y	121	y
26	SUB	58	:	90	Z	122	z
27	ESC	59	;	91	[	123	{
28	FS	60	<	92	\	124	
29	GS	61	=	93	]	125	}
30	RS	62	>	94	^	126	~
31	US	63	?	95	_	127	DEL

Escape Sequences

Certain nonprinting characters, as well as the backslash (\) and the apostrophe ('), can be expressed in terms of *escape sequences*. An escape sequence always begins with a backward slash and is followed by one or more special characters. For example, a line feed (**LF**), which is referred to as a *newline* in C, can be represented as \n. Such escape sequences

always represent single characters, even though they are written in terms of two or more characters. The commonly used escape sequences are listed below.

<u>Character</u>	<u>Escape Sequence</u>	<u>ASCII Value</u>
bell (alert)	\a	007
Backspace	\b	008
horizontal tab	\t	009
vertical tab	\v	011
newline (line feed)	\n	010
form feed	\f	012
carriage return	\r	013
quotation mark (")	\"	034
apostrophe (')	\'	039
question mark (?)	\?	063
backslash ()	\\	092
null	\0	000

Shown below are several character constants, expressed in terms of escape sequences.

'\n' '\t' '\b' '\'' 'x' '\\' 'x' '\"'

Note that the last three escape sequences represent an apostrophe, a backslash and a quotation mark, respectively.

Of interest is the escape sequence \0. This represents the null character (ASCII 000), which is used to indicate the end of a string. Note that, the null character constant '\0' is not equivalent to the character constant '0'.

String Constants

A **string constant** consists of any number of consecutive characters (including none), enclosed in (double) quotation marks. Several string constants are shown below.

"green" "Washington, D.C. 20005" "270-32-3456" "\$19.95"
 "THE CORRECT ANSWER IS:" "2*(I+3)/J" " " " "
 "Line 1\nLine 2\nLine 3"

Note that the string constant "Line 1\nLine 2\nLine 3" extends over three lines, because of the newline characters that are embedded within the string. Thus, this string would be displayed as

Line 1
 Line 2
 Line 3

Also, notice that the string "" is a **null** (empty) string.

Sometimes certain special characters (e.g., a backslash or a quotation mark) must be included as a part of a string constant. These characters must be represented in terms of their escape sequences. Similarly, certain nonprinting characters (e.g., tab, newline) can be included in a string constant if they are represented in terms of their corresponding escape sequences.

The following string constant includes three special characters that are represented by their corresponding escape sequences.

" \ t To continue, press the \"RETURN\" key\n"

The special characters are \ t (horizontal tab), \ " (double quotation marks, which appears twice), and \ n (newline).

The compiler automatically places a null character (\0) at the end of every string constant, as the last character within the string (before the closing double quotation mark). This character is not visible when the string is displayed. However, we can easily examine the individual characters within a string, and test to see whether each character is a null character.

A character constant (e.g., 'A') and the corresponding single-character string constant ("A") are not equivalent. Also remember that a character constant has an equivalent integer value, whereas a single-character string constant does not have an equivalent integer value and, in fact, consists of two characters -the specified character followed by the null character (\0). The character constant 'w' has an integer value of 119 in the ASCII character set. It does not have a null character at the end. In contrast, the string constant "w " consists of two characters -the lowercase letter w and the null character \0. This constant does not have a corresponding integer value.

VARIABLES AND ARRAYS

A **variable** is an identifier that is used to represent some specified type of information within a designated portion of the program. In its simplest form, a variable is an identifier that is used to represent a single data item; i.e., a numerical quantity or a character constant. The data item must be assigned to the variable at some point in the program. The data item can then be accessed later in the program simply by referring to the variable name.

A given variable can be assigned different data items at various places within the program. Thus, the information represented by the variable can change during the execution of the program. However, the data type associated with the variable cannot change.

```
int a, b, c;
char d;

...
a = 3;
b = 5;
c = a + b;
d = 'a';

...
a = 4;
b = 2;
c = a - b;
d = "w";
```

The first two lines are *type declarations*, which state that a , b and c are integer variables, and that d is a char-type variable. Thus a , b and c will each represent an integer-valued quantity, and d will represent a single character. These type declarations will apply throughout the program.

The next four lines cause the following things to happen: the integer quantity 3 is assigned to a, 5 is assigned to b, and the quantity represented by the sum $a + b$ (i.e., 8) is assigned to c. The character 'a' is then assigned to d. In the third line within this group, notice that the values of the variables a and b are accessed simply by writing the variables on the right-hand side of the equal sign. The last four lines redefine the values assigned to the variables as follows: the integer quantity 4 is assigned to a, replacing the earlier value, 3; then 2 is

assigned to *b*, replacing the earlier value, 5 ; then the difference between *a* and *b* (i.e., 2) is assigned to *c*, replacing the earlier value, 8. Finally, the character ' W ' is assigned to *d*, replacing the earlier character, ' a ' .

The **array** is another kind of variable that is used extensively in C. An array is an identifier that refers to a **collection** of data items that all have the same name. The data items must all be of the same type (e.g., all integers, all characters, etc.). The individual data items are represented by their corresponding **array-elements** (i.e., the first data item is represented by the first array element, etc.). The individual array elements are distinguished from one another by the value that is assigned to a **subscript**.

Suppose that *x* is a 10-element array. The first element is referred to as *x*[0], the second as *x*[1], and so on. The last element will be *x*[9]. The subscript associated with each element is shown in square braces. Thus, the value of the subscript for the first element is 0, the value of the subscript for the second element is 1, and so on. For an *n*-element array, the subscripts always range from 0 to *n*-1.

There are several different ways to categorize arrays (e.g., integer arrays, character arrays, one-dimensional arrays, multi-dimensional arrays). For now, we will confine our attention to only one type of array: the one-dimensional, char-type array (often called a one-dimensional *character* array). This type of array is generally used to represent a *string*. Each array element will represent one character within the string. Thus, the entire array can be thought of as an ordered list of characters.

Since the array is one-dimensional, there will be a single *subscript* (sometimes called an *index*) whose value refers to individual array elements. If the array contains *n* elements, the subscript will be an integer quantity whose values range from 0 to *n*-1. Note that an *n-character* string will require an **(*n*+1)-element** array, because of the *null* character (\0) that is automatically placed at the end of the string.

Suppose that the string “Kathmandu” is to be stored in a one-dimensional character array called *letter*. Since “Kathmandu” contains 9 characters, *letter* will be a 10-element array. Thus, *letter*[0] will represent *K*, *letter*[1] represents *a*, and so on. Note that the last (10th) array element, *letter*[9], represents the *null character* (\0) which signifies the end of the string (*letter*).

K	a	T	H	m	a	n	d	u	\0
0	1	2	3	4	5	6	7	8	9

DECLARATIONS

A *declaration* associates a group of variables with a specific data type. All variables must be declared before they can appear in executable statements. A declaration consists of a data type, followed by one or more variable names, ending with a semicolon. Each array variable must be followed by a pair of square brackets, containing a positive integer which specifies the size (i.e., the number of elements) of the array. A C program contains the following type of declarations.

```
int a, b, c;
float root1, root2;
char flag, text[80];
```

Thus, *a*, *b* and *c* are declared to be integer variables, *root1* and *root2* are floating-point variables, *flag* is a char-type variable and *text* is an 80-element, char-type array. Note the square brackets enclosing the size specification for *text*. These declarations could also have been written as follows.

```
int a;
int b;
int c;
```

```
float root1;
float root2;
char flag;
char text[80];
```

Integer-type variables can be declared to be *short integer* for smaller integer quantities, or *long integer* for larger integer quantities. Such variables are declared by writing *short int* and *long int*, or simply *short* and *long*, respectively. A C program contains the following type declarations.

```
short int a, b, c;
long int r, s, t;
int p, q;
```

The above declaration could have been declared as

```
short a, b, c;
long r, s, t;
int p, q;
```

An integer variable can also be declared to be unsigned, by writing unsigned int, or simply unsigned, as the type indicator. Unsigned integer quantities can be larger than ordinary integer quantities (approximately twice as large), but they cannot be negative. A C program contains the following type declarations.

```
int a, b;
unsigned x, y;
```

The unsigned variables x and y can represent values that are twice as large as the values represented by a and b. However, x and y cannot represent negative quantities. For example, if the computer uses 2 bytes for each integer quantity, then a and b may take on values that range from -32768 to +32767, whereas the values of x and y may vary from 0 to +65535.

Floating-point variables can be declared to be *double precision* by using the type indicator *double* or *long float* rather than *float*. In most versions of C, the exponent within a *double-precision* quantity is larger in magnitude than the exponent within an ordinary floating-point quantity. Hence, the quantity represented by a double-precision variable can fall within a greater range. Moreover, a double-precision quantity will usually be expressed in terms of more significant figures. A C program contains the following type declarations.

```
float c1, c2, c3;
double root1, root2; // long float root1, root2;
```

With a C compiler, the double-precision variables *root1* and *root2* represent values that can vary (in magnitude) from approximately 1.7×10^{-308} to $1.7 \times 10^{+308}$. However, the floating-point variables *c1*, *c2* and *c3* are restricted (in magnitude) to the range 3.4×10^{-38} to 3.4×10^{38} . Furthermore, the values represented by *root1* and *root2* will each be expressed in terms of **18** significant figures, whereas the values represented by *c1*, *c2* and *c3* will each be expressed in terms of only **6** significant figures.

Initial values can be assigned to variables within a type declaration. To do so, the declaration must consist of a data type, followed by a variable name, an equal sign (=) and a constant of the appropriate type. A semicolon must appear at the end, as usual.

```
int c = 12;
char star = '*';
float sum = 0.;
double factor = 0.21023e-6;
```

Thus, *c* is an integer variable whose initial value is 12, *star* is a char-type variable initially assigned the character "*", *sum* is a floating-point variable whose initial value is 0., and *factor* is a double-precision variable whose initial value is 0.21023×10^{-6} .

A character-type array can also be initialized within a declaration. To do so, the array is usually written without an explicit size specification (the square brackets are empty). The array name is then followed by an equal sign, the string (enclosed in quotes), and a semicolon. This is a convenient way to assign a string to a character-type array. A C program contains the following declarations.

```
char text[] = "California";
```

This declaration will cause *text* to be a 10-element character array. The first 10 elements will represent the 10 characters within the word California, and the 11th element will represent the null character (`\0`) which is automatically added at the end of the string. The declaration could also have been written

```
char text[11] = "California";
```

where the size of the array is explicitly specified. In such situations it is important, however, that the size be specified correctly. If the size is too small, e.g.,

```
char text[10] = "California";
```

the characters at the end of the string (in this case, the null character) will be lost. If the size is too large, e.g.,

```
char text[20] = "California";
```

the extra array elements may be assigned zeros, or they may be filled with meaningless characters.

EXPRESSIONS

An *expression* represents a single data item, such as a number or a character. The expression may consist of a single entity, such as a constant, a variable, an array element or a reference to a function. It may also consist of some combination of such entities, interconnected by one or more *operators*. The use of expressions involving operators is particularly common in C, as in most other programming languages.

Expressions can also represent logical conditions that are either true or false. However, in C the conditions *true* and *false* are represented by the integer values *1* and *0*, respectively.

Hence, logical-type expressions really represent numerical quantities.

Several simple expressions are shown below.

```
a + b
```

```
x = y
```

```
c = a + b
```

```
x <= y
```

```
x == y
```

```
++i
```

STATEMENTS

A *statement* causes the computer to carry out some action. There are three different classes of statements in C. They are *expression statements*, *compound statements* and *control statements*. An expression statement consists of an expression followed by a semicolon. The execution of an expression statement causes the expression to be evaluated.

Several expression statements are shown below.

```
a = 3;
```

```
c = a + b;
```

```
++i;
```

```
printf("Area = %f", area);
```

```
;
```

The first two expression statements are assignment-type statements. Each causes the value of the expression on the right of the equal sign to be assigned to the variable on the left. The third expression statement is an incrementing-type statement, which causes the value of *i* to increase by 1.

The fourth expression statement causes the *printf* function to be evaluated. This is a standard C library function that writes information out of the computer. In this case, the message *Area =* will be displayed, followed by the current value of the variable *area*. Thus, if *area* represents the value 100., the statement will generate the message

Area = 100.

The last expression statement does nothing, since it consists of only a semicolon. It is simply a mechanism for providing an empty expression statement in places where this type of statement is required. Consequently, it is called a *null statement*.

A compound statement consists of several individual statements enclosed within a pair of braces {}. The individual statements may themselves be expression statements, compound statements or control statements. Thus, the compound statement provides a capability for embedding statements within other statements. Unlike an expression statement, a compound statement does not end with a semicolon. A typical compound statement is shown below.

```
{
    pi = 3.141593;
    circumference = 2. * pi * radius;
    area = pi * radius * radius;
}
```

This compound statement consists of three assignment-type expression statements, though it is considered a single entity within the program in which it appears. Note that the compound statement does not end with a semicolon after the brace.

Control statements are used to create special program features, such as logical tests, loops and branches. Many control statements require that other statements be embedded within them, as illustrated in the following example.

The following control statement creates a conditional loop in which several actions are executed repeatedly, until some condition is satisfied.

```
while (count <= n)
{
    printf ("x = ");
    scanf ("%f" , &x);
    sum + = x;
    ++count;
}
```

This statement contains a compound statement, which in turn contains four expression statements. The compound statement will continue to be executed if the value of *count* does not exceed the value of *n*. Note that *count* increases in value during each pass through the loop.

SYMBOLIC CONSTANTS

A *symbolic constant* is a name that substitutes for a sequence of characters. The characters may represent a numeric constant, a character constant or a string constant. Thus, a symbolic constant allows a name to appear in place of a numeric constant, a character constant or a string. When a program is compiled, each occurrence of a symbolic constant is replaced by its corresponding character sequence. Symbolic constants are usually defined at the beginning of a program. The symbolic constants may then appear later in the program in

place of the numeric constants, character constants, etc. that the symbolic constants represent.

A symbolic constant is defined by writing

```
#define name text
```

where *name* represents a symbolic name, typically written in uppercase letters, and *text* represents the sequence of characters that is associated with the symbolic name. Note that *text* does not end with a semicolon, since a symbolic constant definition is not a true C statement. Moreover, if *text* were to end with a semicolon, this semicolon would be treated as though it were a part of the numeric constant, character constant or string constant that is substituted for the symbolic name.

A C program contains the following symbolic constant definitions.

```
#define TAXRATE 0.23
#define PI 3.141593
#define TRUE 1
#define FALSE 0
#define FRIEND "Susan"
```

Notice that the symbolic names are written in uppercase, to distinguish them from ordinary C identifiers. Also, note that the definitions do not end with semicolons.

Now suppose that the program contains the statement

```
area = PI * radius * radius;
```

During the compilation process, each occurrence of a symbolic constant will be replaced by its corresponding text. Thus, the above statement will become

```
area = 3.141593 * radius * radius;
```

Now suppose that a semicolon had been (incorrectly) included in the definition for PI, i.e.,

```
#define PI 3.141593;
```

The assignment statement for *area* would then become

```
area = 3.141593; * radius * radius;
```

Note the semicolon preceding the first asterisk. This is clearly incorrect, and it will cause an error in the compilation.

The substitution of text for a symbolic constant will be carried out anywhere beyond the *#define* statement, except within a string. Thus, any text enclosed by (double) quotation marks will be unaffected by this substitution process.

A C program contains the following statements.

```
#define CONSTANT 6.023E23
int c;
```

```
.....
```

```
printf ("CONSTANT = %f", c);
```

The *printf* statement will be unaffected by the symbolic constant definition, since the term "CONSTANT = %f" is a string constant. If, however, the *printf* statement was written as

```
printf ("CONSTANT = %f ", CONSTANT);
```

then the *printf* statement would become

```
printf ("CONSTANT = %f ", 6.023E23);
```

during the compilation process.

Symbolic constants are not required when writing C programs. Their use is recommended, however, since they contribute to the development of clear, orderly programs. For example, symbolic constants are more readily identified than the information that they represent, and the symbolic names usually suggest the significance of their associated data items.

Furthermore, it is much easier to change the value of a single symbolic constant than to

change every occurrence of some numerical constant that may appear in several places within the program.

Assignment

Determine which of the following numerical values are valid constants. If a constant is valid, specify whether it is integer or real. Also, specify the base for each valid integer constant.

- ⊕
- | | | |
|-------------|---------------|--------------|
| (a) 0.5 | (e) 12345678 | (i) 0515 |
| (b) 27,822 | (f) 12345678L | (j) 018CDF |
| (c) 9.3e12 | (g) 0.8E+0.8 | (k) OXBCFDAL |
| (d) 9.3e-12 | (h) 0.8E 8 | (l) Ox87e3ha |



Determine which of the following are valid character constants.

- | | | |
|-----------|-----------|-------------|
| (a) 'a' | (e) "\\ " | (h) '\\0' |
| (b) '\$ ' | (f) '\\a' | (i) 'xyz' |
| (c) '\\n' | (g) 'T' | (j) "\\052" |
| (d) '/n' | | |

Determine which of the following are valid string constants.

- | | |
|---------------------------|--|
| (a) '8:15 P.M.' | (e) "1.3e-12" |
| (b) "Red, White and Blue" | (f) "NEW YORK, NY 10020" |
| (c) "Name: 3 (Cont'd)" | (g) "The professor said, "Please don't sleep in class" |

Operators

An *operator* is a symbol that operates on single or multiple data items. They are used to perform mathematical or logical manipulations. The data items that operators act upon are called *operands*. Some operators require two operands, while others act upon only one operand.

1. According to number of operands
 - a) Unary Operators (single operand)

e.g., unary minus (-), unary plus (+), ++, --, !, ~, sizeof, type etc.
 - b) Binary Operators (two operands)

e.g., Binary plus (+), Binary minus (-), *, /, >, <, &&, << etc.
 - c) Ternary Operators (three operands)

e.g., conditional operator (?:)
2. According to Operations
 - a. Arithmetic Operators
 - b. Relational Operators
 - c. Logical Operators
 - d. Assignment Operators
 - e. Increment/Decrement
 - f. Conditional Operator
 - g. Bitwise Operators
 - h. Special operators

a. Arithmetic Operators

The C provides all the Arithmetic Operators (*binary*) which perform arithmetic operations.

Operator	Purpose	Example	Output int a=20, b=6
+	Addition	a + b	26
-	Subtraction	a - b	14
*	Multiplication	a * b	120
/	Division	a / b	3
%	Remainder after integer division	a % b	2

The operands acted upon by arithmetic operators must represent numeric values. The division operator requires that the second operand must be non-zero. The remainder operator (%) requires that both operands must be integers and the second operand should be non-zero.

Division Rule

- int / int = int (e.g., 20 / 3 = 6)
- float / float = float (e.g., 20.0 / 3. = 6.666667)
- int / float =float (e.g., 20 / 3. = 6.666667)
- float / int = float (e.g., 20. / 3 = 6.666667)

Rule for Sign (+ or -)

- When both integer operands of division operator are of the same sign, the result is positive, and it is truncated towards zero. E.g., 4 / 7 = 0, -4 / -7 = 0, -20 / -3 =6, 20 / 3 = 6.
- When one of the operands is negative, then the direction of truncation is machine dependent. That is, -4 / 7 may be 0 or -1 depending upon the machine. The result is negative in most of the machines.
- For modulo division (%) operator, the sign of the result is always the sign of the first operand (i.e., dividend). E.g., 20 % 3 = 2, -20 % 3 = -2, 20 % -3 = 2, -20 % -3 = -2.

CONVERSION RULES

These rules apply to arithmetic operations between two operators with dissimilar data types. There may be some variation from one version of C to another.

1. If one of the operands is *long double*, the other will be converted to *long double* and the result will be *long double*.
2. Otherwise, if one of the operands is *double*, the other will be converted to *double* and the result will be *double*.
3. Otherwise, if one of the operands is *float*, the other will be converted to *float* and the result will be *float*.
4. Otherwise, if one of the operands is *unsigned long int*, the other will be converted to *unsigned long int* and the result will be *unsigned long int*.
5. Otherwise, if one of the operands is *long int* and the other is *unsigned int*, then:
 - If *unsigned int* can be converted to *long int*, the *unsigned int* operand will be converted as such and the result will be *long int*.

- Otherwise, both operands will be converted to *unsigned long int* and the result will be *unsigned long int*.
- 6. Otherwise, if one of the operands is *long int*, the other will be converted to *long int* and the result will be *long int*.
- 7. Otherwise, if one of the operands is *unsigned int*, the other will be converted to *unsigned int* and the result will be *unsigned int*.
- 8. If none of the above conditions applies, then both operands will be converted to *int* (if necessary), and the result will be *int*.

Note that some versions of C automatically convert all floating-point operands to double-precision.

b. Unary Operators

C includes a class of operators that act upon a single operand to produce a new value. Such operators are known as **unary operators**. Unary operators usually precede their single operands, though some unary operators are written after their operands.

Perhaps the most common unary operation is *unary minus*, where a numerical constant, variable or expression is preceded by a minus sign. (Some programming languages allow a minus sign to be included as a part of a numeric constant. In C, however, all numeric constants are positive. Thus, *a negative number is an expression*, consisting of the unary minus operator, followed by a positive numeric constant.)

Note that the unary minus operation is distinctly different from the arithmetic operator which denotes subtraction (-). The subtraction operator requires two separate operands.

Here are several examples which illustrate the use of the unary minus operation.

-723	-0X7FFF	-0.2	-5E-8
-root1	-(x + y)	-3 * (x + y)	

In each case the minus sign is followed by a numerical operand which may be an integer constant, a floating-point constant, a numeric variable or an arithmetic expression.

There are two others commonly used unary operators: The *increment operator*, ++, and the *decrement operator*, --. The increment operator causes its operand to be increased by 1, whereas the decrement operator causes its operand to be decreased by 1. The operand used with each of these operators must be a single variable.

Suppose that *i* is an integer variable that has been assigned a value of 5. The expression ++*i*, which is equivalent to writing *i* = *i* + 1, causes the value of *i* to be increased to 6. Similarly, the expression --*i*, which is equivalent to *i* = *i* - 1, causes the (original) value of *i* to be decreased to 4.

The increment and decrement operators can each be utilized two different ways, depending on whether the operator is written *before* or *after* the operand. If the operator precedes the operand (e.g., ++*i*), then the operand will be altered in value *before* it is utilized for its intended purpose within the program. If, however, the operator *follows* the operand (e.g., *i*++), then the value of the operand will be altered *after* it is utilized.

A C program includes an integer variable *i* whose initial value is 1. Suppose the program includes the following three *printf* statements.

```
printf("i = %d\n", i);
printf("i = %d\n", ++i);
printf("i = %d\n", i);
```

These printf statements will generate the following three lines of output.

```
i = 1
i = 2
i = 2
```

The first statement causes the original value of *i* to be displayed. The second statement increments *i* and then displays its value. The final value of *i* is displayed by the last statement.

Now suppose that the program includes the following three printf statements, rather than the three statements given above.

```
printf("i = %d\n" i);
printf("i = %d\n", i++);
printf("i = %d\n", i);
```

The first and third statements are identical to those shown above. In the second statement, however, the unary operator follows the integer variable rather than precedes it. These statements will generate the following three lines of output.

```
i = 1
i = 1
i = 2
```

The first statement causes the original value of *i* to be displayed, as before. The second statement causes the current value of *i* (1) to be displayed and then incremented (to 2). The final value of *i* (2) is displayed by the last statement.

Another unary operator that is worth mentioning currently is the **sizeof** operator. This operator returns the size of its operand, in bytes. The sizeof operator always precedes its operand. The operand may be an expression, or it may be a cast.

Elementary programs rarely make use of the sizeof operator. However, this operator allows a determination of the number of bytes allocated to various types of data items. This information can be very useful when transferring a program to a different computer or to a new version of C. It is also used for dynamic memory allocation.

Suppose that *i* is an integer variable, *x* is a floating-point variable, *d* is a double-precision variable, and *c* is a character-type variable. The statements

```
printf("integer: %d\n", sizeof i);
printf("float: %d\n", sizeof x);
printf("double: %d\n", sizeof d);
printf("character: %d\n", sizeof c);
```

might generate the following output.

```
integer: 4
float: 4
double: 8
character: 1
```

Thus, we see that this version of C allocates 4 bytes to each integer quantity, 4 bytes to each floating-point quantity, 8 bytes to each double-precision quantity, and 1 byte to each character. These values may vary from one version of C to another.

Another way to generate the same information is to use a cast rather than a variable within each printf statement. Thus, the printf statements could have been written as

```
printf("integer: %d\n", sizeof(int));
printf("float: %d\n", sizeof (float));
```

```
printf("double: %d\n", sizeof (double));
printf("character: %d\n", sizeof (char));
```

These printf statements will generate the same output as that shown above. Note that the cast is enclosed in parentheses.

Finally, consider the array declaration

```
char text [ ] = "Kathmandu";
```

The statement

```
printf ("number of characters = %d", sizeof text);
```

will generate the following output.

```
number of characters = 10
```

A *cast* is also considered to be a unary operator. In general terms, a reference to the cast operator is written as (type).

(data type) expression

This type of construction is known as a cast.

Suppose that *i* is an integer variable whose value is 7, and *f* is a floating-point variable whose value is 5. The expression

```
(i + f)%4
```

is invalid, because the first operand (*i + f*) is floating-point rather than integer.

However, the expression

```
((int)(i + f))%4
```

forces the first operand to be an integer and is therefore valid, resulting in the integer remainder 3.

Note that the explicit type specification applies only to the first operand, not the entire expression.

The data type associated with the expression itself is not changed by a cast. Rather, it is the *value* of the expression that undergoes type conversion wherever the cast appears. This is particularly relevant when the expression consists of only a single variable.

Suppose that *f* is a floating-point variable whose value is 5.5. The expression

```
((int)f) % 2
```

contains two integer operands and is therefore valid, resulting in the integer remainder 1.

Note, however, that *f* remains a floating-point variable whose value is 5.5, even though the value of *f* was converted to an integer (5) when carrying out the remainder operation.

The operators within C are grouped hierarchically according to their **precedence** (i.e., order of evaluation). Operations with a higher precedence are carried out before operations having a lower precedence. The natural order of evaluation can be altered, however, using parentheses.

Among the arithmetic operators, *, / and % fall into one precedence group, and + and - fall into another. The first group has a higher precedence than the second. Thus, multiplication, division and remainder operations will be carried out before addition and subtraction.

Another important consideration is the order in which consecutive operations within the same precedence group are carried out. This is known as **associativity**. Within each of the precedence groups described above, the associativity is left to right. In other words, consecutive addition and subtraction operations are carried out from left to right, as are consecutive multiplication, division and remainder operations.

Unary operators have a higher precedence than arithmetic operators. Hence, if a unary minus operator acts upon an arithmetic expression that contains one or more arithmetic

operators, the unary minus operation will be carried out first (unless, of course, the arithmetic expression is enclosed in parentheses). Also, the associativity of the unary operators is right to left, though consecutive unary operators rarely appear in elementary programs.

Suppose that x and y are integer variables whose values are 10 and 20, respectively. The value of the expression $-x + y$ will be $-10 + 20 = 10$. Note that the unary minus operation is carried out before the addition.

Now suppose that parentheses are introduced, so that the expression becomes $-(10 + 20)$. The value of this expression is $-(10 + 20) = -30$. Note that the addition now precedes the unary minus operation.

c. Relational Operators

They are binary operators which are used to compare two similar operands. They compare their left-hand side operand with their right-hand side operand for comparisons according to magnitudes. The output of a relational operation is either *true* (1) or *false* (0). So, they are used in decision making statements.

Operator	Purpose	Example	Output int a=20, b=6	Output int a=20, b=20
<	Less than	$a < b$	0	0
>	Greater than	$a > b$	1	0
<=	Less than or Equal to	$a <= b$	0	1
>=	Greater than or Equal to	$a >= b$	1	1

These operators all fall within the same precedence group, which is lower than the arithmetic and unary operators. The associativity of these operators is left to right.

Closely associated with the relational operators are the following two *equality operators*.

Operator	Purpose	Example	Output int a=20, b=6	Output int a=20, b=20
==	Equal to	$a == b$	0	1
!=	Not Equal to	$a != b$	1	0

The equality operators fall into a separate precedence group, beneath the relational operators. These operators also have a *left-to-right associativity*.

These six operators are used to form **logical expressions**, which represent conditions that are either true or false. The resulting expressions will be of type integer, since *true* is represented by the integer value 1 and *false* is represented by the value 0.

Suppose that i , j and k are integer variables whose values are 1, 2 and 3, respectively.

Several logical expressions involving these variables are shown below.

Expression	Interpretation	Value
$i < j$	True	1
$(i + j) >= k$	True	1
$(j + k) > (i + 5)$	False	0
$k != 3$	False	0
$j == 2$	True	1

Suppose that i is an integer variable whose value is 7, f is a floating-point variable whose value is 5.5, and c is a character variable that represents the character 'w'. Several logical expressions that make use of these variables are shown below. Each expression involves two different type operands. (Assume that the **ASCII** character set applies.)

Expression	Interpretation	Value
$f > 5$	True	1
$(i + f) <= 10$	False	0

$c == 119$	True	1
$c != 'p'$	True	1
$c \geq 10 * (i + f)$	False	0

d. Logical Operators

In addition to the relational and equality operators, C contains two *logical operators* (also called *logical connectives*). They are

Operator	Meaning
&&	Logical AND
	Logical OR

The logical operators act upon operands that are themselves logical expressions. The net effect is to combine the individual logical expressions into more complex conditions that are either true or false. The result of a *logical and* operation will be true only if both operands are true, whereas the result of a *logical or* operation will be true if either operand is true or if both operands are true. In other words, the result of a *logical or* operation will be false only if both operands are false.

Suppose that i is an integer variable whose value is 7, f is a floating-point variable whose value is 5.5, and c is a character variable that represents the character 'w'. Several complex logical expressions that make use of these variables are shown below.

Expression	Interpretation	Value
$(i \geq 6) \&\& (c == 'w')$	True	1
$(i \geq 6) (c == 119)$	True	1
$(f < 11) \&\& (i > 100)$	False	0
$(c != 'p') ((i + f) \leq 10)$	True	1

The first expression is true because both operands are true. In the second expression, both operands are again true; hence the overall expression is true. The third expression is false because the second operand is false. And finally, the fourth expression is true because the first operand is true.

Each of the logical operators falls into its own precedence group. *Logical and* has a higher precedence than *logical or*. Both precedence groups are lower than the group containing the equality operators. The associativity is left to right. The precedence groups are summarized below.

C also includes the unary operator **!** that negates the value of a logical expression; i.e., it causes an expression that is originally true to become false, and vice versa. This operator is referred to as the *logical negation* (or *logical not*) operator.

Suppose that i is an integer variable whose value is 7, and f is a floating-point variable whose value is 5.5. Several logical expressions which make use of these variables and the logical negation operator are shown below.

Expression	Interpretation	Value
$f > 5$	True	1
$!(f > 5)$	False	0
$i \leq 3$	False	0
$!(i \leq 3)$	True	1
$i > (f + 1)$	True	1
$!(i > (f + 1))$	False	0

The hierarchy of operator precedence covering all the operators discussed so far has become extensive. These operator's precedence is summarized below, from highest to lowest.

Operator category	Operators	Associativity
Unary Operators	-, ++, --, !, sizeof, (type)	$R \rightarrow L$
Arithmetic multiply, division and remainder	*, /, %	$L \rightarrow R$
Arithmetic Add and Subtract	+, -	$L \rightarrow R$
Relational operators	<, >, <=, >=	$L \rightarrow R$
Equality	==, !=	$L \rightarrow R$
Logical AND	&&	$L \rightarrow R$
Logical OR		$L \rightarrow R$

Consider once again the variables *i*, *f*, and *c* whose values are 7, 5.5, and 'w' respectively. Some logical expressions that make use of these variables are shown below.

Expression	Interpretation	Values
<i>i</i> + <i>f</i> <= 10	False	0
<i>i</i> >= 5 && <i>c</i> == 'w'	True	1
<i>c</i> != 'p' <i>i</i> + <i>f</i> <= 10	True	1

Each of these expressions has been presented before though pairs of parentheses were included in the previous examples. The parentheses are not necessary because of the natural operator precedence. Thus, the arithmetic operations will automatically be carried out before the relational or equality operations, and the relational and equality operations will automatically be carried out before the logical connectives.

Consider the last expression. The first operation to be carried out will be addition (i.e., *i* + *f*); then the relational comparison (i.e., *i* + *f* <= 10); then the equality comparison (i.e., *c* != 'p'); and finally, the *logical or* condition.

Complex logical expressions that consist of individual logical expressions joined by the logical operators && and || are evaluated left to right, but only until the overall true/false value has been established. Thus, a complex logical expression will not be evaluated in its entirety if its value can be established from its constituent operands.

Consider the logical expression shown below.

```
error > .0001 && count < 100
```

If *error* > .0001 is *false*, then the second operand (i.e., *count* < 100) will not be evaluated, because the entire expression will be considered *false*. On the other hand, suppose the expression had been written

```
error > .0001 || count < 100
```

If *error* > .0001 is *true*, then the entire expression will be true. Hence, the second operand will not be evaluated. If *error* > .0001 is *false*, however, the second expression (i.e., *count* < 100) *must be evaluated* to determine if the entire expression is true or false.

e. Assignment Operator

There are several different assignment operators in C. All of them are used to form *assignment expressions*, which assign the value of an expression to an identifier. The most used assignment operator is =. Assignment expressions that make use of this operator are written in the form

identifier = expression

where *identifier* generally represents a variable, and *expression* represents a constant, a variable or a more complex expression.

Here are some typical assignment expressions that make use of the = operator.

```
a = 3;
```

```
x = y;
delta = 0.001;
sum = a + b;
area = length * width;
```

The first assignment expression causes the integer value **3** to be assigned to the variable *a*, and the second assignment causes the value of *y* to be assigned to *x*. In the third assignment, the floating-point value **0.001** is assigned to *delta*. The last two assignments each result in the value of an arithmetic expression being assigned to a variable (i.e., the value of **a + b** is assigned to *sum*, and the value of **length * width** is assigned to *area*).

Remember that the *assignment operator* = and the *equality operator* == are distinctly different. The assignment operator is used to assign a value to an identifier, whereas the equality operator is used to determine if two expressions have the same value. These operators cannot be used in place of one another. Beginning programmers often incorrectly use the assignment operator when they want to test for equality. This results in a logical error that is usually difficult to detect.

If the two operands in an assignment expression are of different data types, then the value of the expression on the right (i.e., the right-hand operand) will automatically be converted to the type of the identifier on the left. The entire assignment expression will then be of this same data type.

Under some circumstances, this automatic type conversion can result in an alteration of the data being assigned. For example:

- A floating-point value may be rounded if assigned to an integer identifier.
- A double-precision value may be rounded if assigned to a floating-point (single-precision) identifier.
- An integer quantity may be altered if assigned to a shorter integer identifier or to a character identifier (some high-order bits may be lost).

Moreover, the value of a character constant assigned to a numeric-type identifier will be dependent upon the character set in use. This may result in inconsistencies from one version of C to another. The careless use of type conversions is a frequent source of error among beginning programmers.

In the following assignment expressions, suppose that *i* is an integer-type variable.

<u>Expression</u>	<u>Value</u>
<i>i</i> = 3.3	3
<i>i</i> = 3.9	3
<i>i</i> = -3.9	-3

Now suppose that *i* and *j* are both integer-type variables, and that *j* has been assigned a value of 5. Several assignment expressions that make use of these two variables are shown below.

<u>Expressions</u>	<u>Value</u>
<i>i</i> = <i>j</i>	5
<i>i</i> = <i>j</i> / 2	2
<i>i</i> = 2 * <i>j</i> / 2	5 (L → R associativity)
<i>i</i> = 2 * (<i>j</i> / 2)	4 (truncated division, followed by multiplication)

Finally, assume that *i* is an integer-type variable, and that the **ASCII** character set applies.

<u>Expression</u>	<u>Value</u>
<i>i</i> = 'x'	120
<i>i</i> = '0'	48
<i>i</i> = ('x' - '0') / 3	24

$i = ('y' - '0') / 3$ 24

Multiple assignments of the form

`identifier 1 = identifier 2 = ... = expression`

are permissible in C. In such situations, the assignments are carried out from right to left. Thus, the multiple assignment

`identifier 1 = identifier 2 = expression`

is equivalent to

`identifier 1 = (identifier 2 = expression)`

and so on, with *right-to-left* nesting for additional multiple assignments.

Suppose that *i* and *j* are integer variables. The multiple assignment expression

`i = j = 5;`

will cause the integer value 5 to be assigned to both *i* and *j*. (To be more precise, 5 is first assigned to *j*, and the value of *i* is then assigned to *i*.) Similarly, the multiple assignment expression

`i = j = 5.9;`

will cause the integer value 5 to be assigned to both *i* and *j*. Remember that truncation occurs when the floating-point value 5.9 is assigned to the integer variable *j*.

C contains the following five additional assignment operators: `+=`, `-=`, `*=`, `/=` and `%=`. To see how they are used, consider the first operator, `+=`. The assignment expression

`expression 1 += expression 2`

is equivalent to

`expression 1 = expression 1 + expression 2`

Similarly, the assignment expression

`expression 1 -= expression 2`

is equivalent to

`expression 1 = expression 1 - expression 2`

and so on for all five operators.

Usually, *expression 1* is an identifier, such as a variable or an array element.

Suppose that *i* and *j* are integer variables whose values are 5 and 7, and *f* and *g* are floating-point variables whose values are 5.5 and -3.25. Several assignment expressions that make use of these variables are shown below. Each expression utilizes the original values of *i*, *j*, *f* and *g*.

Expression	Equivalent Expression	Final Value
<code>i += 5</code>	<code>i = i + 5</code>	10
<code>f -= g</code>	<code>f = f - g</code>	8.75
<code>j *= (i - 3)</code>	<code>j = j * (i - 3)</code>	14
<code>f /= 3</code>	<code>f = f / 3</code>	1.833333
<code>i %= (j - 2)</code>	<code>i = i % (j - 2)</code>	0

Assignment operators have a lower precedence than any of the other operators that have been discussed so far. Therefore, unary operations, arithmetic operations, relational operations, equality operations and logical operations are all carried out before assignment operations. Moreover, the assignment operations have a right-to-left associativity.

The hierarchy of operator precedence presented in the last section can now be modified as follows to include assignment operators.

Operator category	Operators	Associativity
Unary Operators	<code>~, ++, --, !, sizeof, (type)</code>	$R \rightarrow L$
Arithmetic multiply, division and remainder	<code>*, /, %</code>	$L \rightarrow R$

Arithmetic Add and Subtract	+, -	L → R
Relational operators	<, >, <=, >=	L → R
Equality	==, !=	L → R
Logical AND	&&	L → R
Logical OR		L → R
Assignment operators	=, +=, -=, *=, /=, %=	R → L

Suppose x, y and z are integer variables which have been assigned to 2, 3 and 4, respectively. The expression

```
x *= -2 * (y + z) / 3;
```

is equivalent to the expression

```
x = x * (-2 * ( y + z ) / 3)
```

Either expression will cause the value -8 to be assigned to x.

Consider the order in which the operations are carried out in the first expression. The arithmetic operations precede the assignment operation. Therefore, the expression (y + z) will be evaluated first, resulting in 7. Then the value of this expression will be multiplied by -2, yielding -14. This product will then be divided by 3 and truncated, resulting in -4. Finally, this truncated quotient is multiplied by the original value of x (i.e., 2) to yield the result of -8.

f. Conditional Operator

Simple conditional operations can be carried out with the *conditional operator* (? :). An expression that makes use of the conditional operator is called a *conditional expression*. Such an expression can be written in place of the more traditional *if-else* statement. The conditional expression is written in the form

```
expression 1 ? expression 2 : expression 3
```

When evaluating a conditional expression, *expression 1* is evaluated first. If *expression 1* is true (i.e., if its value is nonzero), then *expression 2* is evaluated and this becomes the value of the conditional expression. However, if *expression 1* is false (i.e., if its value is zero), then *expression 3* is evaluated and this becomes the value of the conditional expression. Note that only one of the embedded expressions (either *expression 2* or *expression 3*) is evaluated when determining the value of a conditional expression.

In the conditional expression shown below, assume that *i* is an integer variable.

```
(i < 0) ? 0 : 100;
```

The expression (*i* < 0) is evaluated first. If it is true (i.e., if the value of *i* is less than 0), the entire conditional expression takes on the value 0. Otherwise (if the value of *i* is not less than 0), the entire conditional expression takes on the value 100.

In the following conditional expression, assume that *f* and *g* are floating-point variables.

```
(f < g) ? f : g;
```

This conditional expression takes on the value of *f* if *f* is less than *g*; otherwise, the conditional expression takes on the value of *g*. In other words, the conditional expression returns the value of the *smaller* of the two variables.

If the operands (i.e., *expression 2* and *expression 3*) differ in type, then the resulting data type of the conditional expression will be determined by the rules given in previous section (Arithmetic Operator).

Conditional expressions frequently appear on the right-hand side of a simple assignment statement. The resulting value of the conditional expression is assigned to the identifier on the left. Here is an assignment statement that contains a conditional expression on the right-hand side.

```
flag = (i < 0)? 0 : 100;
```

If the value of *i* is negative, then 0 will be assigned to *flag*. If *i* is not negative, however, then 100 will be assigned to *flag*. Here is another assignment statement that contains a conditional expression on the right-hand side.

```
min = (f < g) ? f : g;
```

This statement causes the value of the smaller of *f* and *g* to be assigned to *min*.

The conditional operator has its own precedence, just above the assignment operators. The associativity is *right to left*.

g. Bitwise Operators

Bitwise operators are used for manipulating data at bit level. These operators are used for testing bits or shifting them to the left or to the right. Bitwise operators can be applied only to integer-type operands. There are three types of bitwise operators:

- Bitwise logical operators
- Bitwise shift operators
- One's complement operators

I. Bitwise logical operators

Bitwise logical operators perform logical tests between two integer operands bit by bit starting from Least Significant Bit (LSB) or right most bit. There are three bitwise logical operators: bitwise AND (&), bitwise OR (|) and bitwise Exclusive OR (^).

Bitwise AND (&)

It performs logical *ANDing* between two operands. The result of *ANDing* operation is 1 if both bits are 1, otherwise it is 0. Let us consider two variables, *n1* = 60 and *n2* = 15. The binary representation of these values is:

```
n1 → 0000 0000 0011 1100
```

```
n2 → 0000 0000 0000 1111
```

```
n3 = n1 & n2;
```

```
n3 → 0000 0000 0000 1100 (i.e., 12)
```

Bitwise OR (|)

It performs logical *ORing* between two operands. The result of *ORing* operation is 1 if either of bits is 1, otherwise it is 0. Let us consider two variables, *n1* = 60 and *n2* = 15. The binary representation of these values is:

```
n1 → 0000 0000 0011 1100
```

```
n2 → 0000 0000 0000 1111
```

```
n3 = n1 | n2;
```

```
n3 → 0000 0000 0011 1111 (i.e., 63)
```

Bitwise XOR (^)

The result of Exclusive *ORing* operation is 1 only if one of the bits have the value 1, otherwise it is 0. Let us consider two variables, *n1* = 60 and *n2* = 15. The binary representation of these values is:

```
n1 → 0000 0000 0011 1100
```

```
n2 → 0000 0000 0000 1111
```

```
n3 = n1 ^ n2;
```

$n3 \rightarrow 0000\ 0000\ 0011\ 0011$ (i.e., 51)

II. Bitwise Shift Operators

Bitwise shift operators are used to move bit patterns either to the left or to the right. There are two bitwise shift operators: *left shift* (<<) and *right shift* (>>).

Left Shift (<<)

The left-shift operation causes operand to be shifted to the left by some bit positions. The general form of left-shift operation is:

`operand << n;`

The bits in the operand are shifted to the left by n positions. The *leftmost* n bits in the original bit pattern will be *lost* and the *rightmost* n bits empty positions will be filled with 0s. For example, if $n1 = 60$, the statement $n2 = n1 << 3$ will be processed as follows.

$n1$ `0000 0000 0011 1100`

First Shift `0000 0000 0111 1000`

Second Shift `0000 000 1111 0000`

Third Shift `0000 0001 1110 0000` (i.e., 480)

Right Shift (>>)

The right-shift operation causes operand to be shifted to the *right* by some bit positions. The general form of right-shift operation is:

`operand >> n;`

The bits in the operand are shifted to the *right* by n positions. The *rightmost* n bits in the original bit pattern will be *lost* and the leftmost n bits empty positions will be filled with 0s. For example, if $n1 = 60$, the statement $n2 = n1 >> 3$ will be processed as follows.

$n1$ `0000 0000 0011 1100`

First Shift `0000 0000 0001 1110`

Second Shift `0000 000 0000 1111`

Third Shift `0000 0000 0000 0111` (i.e., 7)

III. Bitwise One's Complement

Bitwise One's complement operator is a unary operator which *inverts* (i.e., 1 to 0 and vice versa) all the bits represented by its operand. For example, if $n1 = 60$, the statement $n2 = \sim n1$;

$n1 \rightarrow 0000\ 0000\ 0011\ 1100$

$\sim n2 \rightarrow 1111\ 1111\ 1100\ 0011$

Operator category	Operators	Associativity
Function, array, structure member, pointer to structure member	<code>(), [], ., -></code>	$L \rightarrow R$
Unary Operators	<code>~, ++, --, !, sizeof, (type), *, &, ~</code>	$R \rightarrow L$
Arithmetic multiply, division and remainder	<code>*, /, %</code>	$L \rightarrow R$
Arithmetic Add and Subtract	<code>+, -</code>	$L \rightarrow R$
Bitwise shift operators	<code><<, >></code>	$L \rightarrow R$
Relational operators	<code><, >, <=, >=</code>	$L \rightarrow R$
Equality	<code>==, !=</code>	$L \rightarrow R$
Bitwise AND	<code>&</code>	$L \rightarrow R$
Bitwise XOR	<code>^</code>	$L \rightarrow R$

Bitwise OR		L → R
Logical AND	&&	L → R
Logical OR		L → R
Conditional Operator	? :	R → L
Assignment operators	=, +=, -=, *=, /=, %=, &=, ^=, =, <<=, >>=	R → L
Comma Operator	,	L → R

Data Input and Output

In this chapter we will make use of six of these functions: *getchar*, *putchar*, *scanf*, *printf*, *gets* and *puts*. These six functions permit the transfer of information between the computer and the standard input output devices (e.g., a keyboard and a **TV** monitor). The first two functions, *getchar* and *putchar*, allow single characters to be transferred into and out of the computer; *scanf* and *printf* are the most complicated, but they permit the transfer of single characters, numerical values and strings; *gets* and *puts* facilitate the input and output of strings. Once we have learned how to use these functions, we will be able to write a few complete, though simple, C programs.

Preliminaries

An input output function can be accessed from anywhere within a program simply by writing the function name, followed by a list of arguments enclosed in parentheses. The arguments represent data items that are sent to the function. Some input output functions do not require arguments, though the empty parentheses must still appear.

The names of those functions that return data items may appear within expressions, as though each function reference were an ordinary variable (e.g., *c = getchar ();*), or they may be referenced as separate statements (e.g., *scanf (. . .);*). Some functions do not return any data items. Such functions are referenced as though they were separate statements (e.g., *putchar (. . .);*).

Most versions of C include a collection of header files that provide necessary information (e.g., symbolic constants) in support of the various library functions. Each file generally contains information in support of a group of related library functions. These files are entered into the program via an **#include** statement at the beginning of the program. As a rule, the header file required by the standard input output library functions is called **stdio.h**. Here is an outline of a typical C program that makes use of several input output routines from the standard C library.

```
/* sample setup illustrating the use of input/ output library
functions */
#include <stdio.h>
main()
{
    char c, d;                /* declarations */
    float x, y;
    int i, j, k;

    c = getchar();            /* character input */
    scanf(" %f", &x);         /* floating-point input */
    scanf(" %d %d", &i, &j);   /* integer input */
    ...
    putchar(d);               /*character output */
}
```

```
    printf("%3d %7.4f", k, y);    /* numerical output */
}
```

The program begins with the preprocessor statement `#include <stdio.h>`. This statement causes the contents of the header file *stdio.h* to be included within the program. The header file supplies required information to the library functions *scanf* and *printf*. (The syntax of the `#include` statement may vary from one version of C to another; some versions of the language use quote instead of angle-brackets, e.g., `#include "stdio.h"`.)

Following the preprocessor statement is the program heading *main ()* and some variable declarations. Several input/output statements are shown in the skeletal outline that follows the declarations. In particular, the assignment statement `c = getchar ( ) ;` causes a single character to be entered from the keyboard and assigned to the character variable *c*. The first reference to *scanf* causes a floating-point value to be entered from the keyboard and assigned to the floating-point variable *x*, whereas the second reference to *scanf* causes two decimal integer quantities to be entered from the keyboard and assigned to the integer variables *i* and *j*, respectively.

The output statements behave in a similar manner. Thus, the reference to *putchar* causes the value of the character variable *d* to be displayed. Similarly, the reference to *printf* causes the values of the integer variable *k* and the floating-point variable *y* to be displayed.

Single Character Input- the *getchar()* Function

Single characters can be entered into the computer using the C library function *getchar*. The *getchar* function is a part of the standard C I/O library. It returns a single character from a standard input device (typically a keyboard). The function does not require any arguments, though a pair of empty parentheses must follow the word *getchar*.

In general terms, a function reference would be written as

```
character variable = getchar ( ) ;
```

where *character variable* refers to some previously declared character variable.

A C program contains the following statements.

```
char c;
.....
```

```
c = getchar ( ) ;
```

The first statement declares that *c* is a character-type variable. The second statement causes a single character to be entered from the standard input device (usually a keyboard) and then assigned to *c*.

If an *end-of-file* condition is encountered when reading a character with the *getchar* function, the value of the symbolic constant **EOF** will automatically be returned. (This value will be assigned within the *stdio.h* file. Typically, EOF will be assigned the value *-1*, though this may vary from one compiler to another.)

The *getchar* function can also be used to read multicharacter strings, by reading one character at a time within a multi-pass loop.

Single Character Output -The *putchar()* Function

Single characters can be displayed (i.e., written out of the computer) using the C library function *putchar*. This function is complementary to the character input function *getchar*. The *putchar* function, like *getchar*, is a part of the standard C I/O library. It transmits a single character to a standard output device (typically a TV monitor). The character being transmitted will normally be represented as a character-type variable. It must be expressed as an argument to the function, enclosed in parentheses, following the word *putchar*.

In general, a function reference would be written as

putchar(character variable)

where `character variable` refers to some previously declared character variable.

A C program contains the following statements.

```
char c;
.....
putchar(c);
```

The first statement declares that `c` is a character-type variable. The second statement causes the current value of `c` to be transmitted to the standard output device (e.g., a TV monitor) where it will be displayed.

The `putchar` function can be used to output a string constant by storing the string within a one-dimensional, character-type array. Each character can then be written separately within a *loop*.

Example: Lowercase to Uppercase Conversion

```
#include <stdio.h>
#include <ctype.h>          // toupper()
main()
{
    char ch;
    printf("enter any character value: ");
    ch = getchar();
    printf("The equivalent uppercase character is: ");
    putchar(toupper(ch));
}
```

OUTPUT

```
enter any character value: a
The equivalent uppercase character is: A
```

The gets and puts Functions

The *gets* and *puts* functions, which facilitate the transfer of strings between the computer and the standard input output devices. Each of these functions accepts *a single argument*. The argument must be a data item that represents a string. (e.g., a character array). The string may include whitespace characters. In the case of *gets*, the string will be entered from the keyboard, and will terminate with *a newline character* (i.e., the string will end when the user presses the **Enter key**).

The *gets* and *puts* functions offer simple alternatives to the use of *scanf* and *printf* for reading and displaying strings, as illustrated in the following example.

EXAMPLE: Reading and Writing a Line of Text

```
#include <stdio.h>
main()
{
    char line[80];
    ...
    gets(line);
    puts(line);
}
```

This program utilizes *gets* and *puts*, rather than *scanf* and *printf*, to transfer the line of text into and out of the computer.

Entering Input Data -the scanf Function

Input data can be entered into the computer from a standard input device by means of the C library function ***scanf***. This function can be used to enter any combination of numerical values, single characters and strings. The function returns the number of data items that have been entered successfully.

In general terms, the ***scanf*** function is written as

scanf(control string, arg1, arg2,...,argn)

where ***control string*** refers to a string containing certain required formatting information, and ***arg1, arg2, . . . argn*** are arguments that represent the individual input data items. (Actually, the arguments represent ***pointers*** that indicate the ***addresses*** of the data items within the computer's memory.)

The control string consists of individual groups of characters, with one-character group for each input data item. Each character group must begin with a *percent sign (%)*. In its simplest form, a single character group will consist of the percent sign, followed by a ***conversion character*** which indicates the type of the corresponding data item.

Within the control string, multiple character groups can be contiguous, or they can be separated by *whitespace characters* (i.e., blank spaces, tabs or newline characters). If whitespace characters are used to separate multiple character groups in the control string, then *all consecutive whitespace characters* in the input data will be read but *ignored*. The use of blank spaces as character-group separators is very common. The more frequently used conversion characters are listed in Table below.

Commonly Used Conversion Characters for Data Input

Conversion Character	Meaning
c	data item is a single character
d	data item is a decimal integer
e	data item is a floating-point value
f	data item is a floating-point value
g	data item is a floating-point value
h	data item is a short integer
i	data item is a decimal, hexadecimal or octal integer
o	data item is an octal integer
s	data item is a string followed by a whitespace character (<i>the null character \0 will automatically be added at the end</i>)
u	data item is an unsigned decimal integer
x	data item is a hexadecimal integer
[...]	data item is a string which may include whitespace characters (see explanation below)

The arguments are written as variables or arrays, whose types match the corresponding character groups in the control string. *Each variable name must be preceded by an ampersand (&).* (The arguments are pointers that indicate where the data items are stored in the computer's memory.) However, array names should *not begin* with an *ampersand*. Here is a typical application of a **scanf** function.

```
#include <stdio.h>
main()

{

    char item[20];
    int partno;
    float cost;
    ...
    scanf("%s %d %f" ,item, &partno, &cost);
    ...

}
```

Within the **scanf** function, the control string is `"%s %d %f"`. It contains three-character groups. The first character group, **%s**, indicates that the first argument (*item*) represents a string. The second character group, **%d**, indicates that the second argument (**&partno**) represents a decimal integer value, and the third character group, **%f**, indicates that the third argument (**&cost**) represents a floating-point value.

Notice that the numerical variables *partno* and *cost* are preceded by ampersands within the `scanf` function. An ampersand does not precede **item**, however, since **item** is an array name. Also, that the **scanf** function could have been written

```
scanf("%s%d%f",item, &partno, &cost);
```

with no whitespace characters in the control string. This is also valid, though the input data could be interpreted differently when using c-type conversions.

The actual data items are numeric values, single characters or strings, or some combination thereof. They are entered from a standard input device (typically a keyboard). The data items must correspond to the arguments in the **scanf** function in number, in type and in order. Numeric data items are written in the same form as numeric constants, though octal values need not be preceded by a **0**, and hexadecimal values need not be preceded by **0x** or **0X**. Floating-point values must include either a decimal point or an exponent (or both).

If two or more data items are entered, they must be separated by whitespace characters. The data items may continue onto two or more lines, since the newline character is a *whitespace character* and can therefore separate consecutive data items.

Moreover, if the control string begins by reading a character-type data item, it is generally a good idea to precede the first conversion character with a blank space. This causes the **scanf** function to ignore any extraneous characters that may have been entered earlier (for example, by pressing the *Enter* key after entering a previous line of data).

Consider once again the skeletal outline of a C program shown in the above Example; i.e.,

```
#include <stdio.h>
main()

{

    char item[20];
    int partno;
    float cost;
    ...
    scanf(" %s %d %f" ,item, &partno, &cost);
    ...

}
```

Notice the blank space that precedes **%s**. This prevents any previously entered extraneous characters from being assigned to **item**. The following data items could be entered from the standard input device when the program is executed.

```
fastener 12345 0.05
```

Thus, the characters that make up the string *fastener* would be assigned to the first eight elements of the array **item**; the integer value *12345* would be assigned to **partno**, and the floating-point value *0.05* would be assigned to **cost**.

Note that the individual data items are entered on one line, separated by blank spaces. The data items could also be entered on separate lines, however, since newline characters are also whitespace characters. Therefore, the data items could also be entered in any of the following ways:

Fastener	fastener	fastener 12345
12345	12345 0.05	0.05
0.05		

Note that the s-type conversion character applies to a string that is terminated by a whitespace character. Therefore, a string that includes whitespace characters cannot be entered in this manner. There are ways, however, to work with strings that include

whitespace characters. One way is to use the **getchar** function within a loop. It is also possible to use the **scanf** function to enter such strings. To do so, the s-type conversion character within the control string is replaced by a sequence of characters enclosed in square brackets, designated as [. . .]. Whitespace characters may be included within the brackets, thus accommodating strings that contain such characters.

When the program is executed, successive characters will continue to be read from the standard input device if each input character matches one of the characters enclosed within the brackets. The order of the characters within the square brackets need not correspond to the order of the characters being entered. Input characters may be repeated. The string will terminate, however, once an input character is encountered that does not match any of the characters within the brackets. A null character (**\0**) will then automatically be added to the end of the string.

This example illustrates the use of the **scanf** function to enter a string consisting of uppercase letters and blank spaces. The string will be of undetermined length, but it will be limited to 79 characters (actually, 80 characters including the null character that is added at the end). Notice the *blank space* that precedes the % sign.

```
#include <stdio.h >
main()
{
    char line[80];
    ...
    scanf(" %[ ABCDEFGHIJKLMNOPQRSTUVWXYZ] ", line);
    /* scanf(" %[ A-Z] ", line); */
    ...
}
```

If the string

NEW YORK CITY

is entered from the standard input device when the program is executed, the entire string will be assigned to the array *line* since the string is comprised entirely of uppercase letters and blank spaces. If the string were written as

New York City

however, then only the single letter *N* would be assigned to *line*, since the first lowercase letter (in this case, *e*) would be interpreted as the first character beyond the string. It would, of course, be possible to include both uppercase and lowercase characters within the brackets, but this becomes cumbersome.

A variation of this feature which is often more useful is to precede the characters within the square brackets by a **Circumflex** (i.e., ^). This causes the subsequent characters within the brackets to be interpreted in the opposite manner. Thus, when the program is executed, successive characters will continue to be read from the standard input device if each input character *does not match* one of the characters enclosed within the brackets.

If the characters within the brackets are simply the circumflex followed by a newline character, then the string entered from the standard input device can contain any ASCII characters except the newline character (line feed). Thus, the user may enter whatever he or she wishes and then press the **Enter** key. The **Enter** key will issue the newline character, thus signifying the end of the string.

Suppose a C program contains the following statements.

```
#include <stdio.h>
```

```
main()
{
    char line[80];
    ...
    scanf("%[^\\n]", line);
    ...
}
```

Notice the blank space preceding `%[^\\n]`, to ignore any unwanted characters that may have been entered previously. When the `scanf` function is executed, a string of undetermined length (but not more than **79** characters) will be entered from the standard input device and assigned to *line*. There will be no restrictions on the characters that comprise the string, except that they all fit on one line. For example, the string

The Mt. Everest is the Highest land of the Earth.
could be entered from the keyboard and assigned to *line*.

More About the scanf Function

The consecutive non-whitespace characters that define a data item collectively define a *field*. It is possible to limit the number of such characters by specifying a maximum *field width* for that data item. To do so, an unsigned integer indicating the field width is placed within the control string, between the *percent sign* (%) and the *conversion character*.

The data item may contain fewer characters than the specified field width. However, the number of characters in the actual data item cannot exceed the specified field width. Any characters that extend beyond the specified field width will not be read. Such leftover characters may be incorrectly interpreted as the components of the *next data item*. The skeletal structure of a C program is shown below.

```
#include <stdio.h>
main()
{
    int a, b, c;
    ...
    scanf("%3d %3d %3d", &a, &b, &c);
    ...
}
```

When the program is executed, three integer quantities will be entered from the standard input device (the keyboard). Suppose the input data items are entered as

1 2 3

Then the following assignment will result

a = 1, b = 2, c = 3

If the data had been entered as

123 456 789

Then the assignments would be

a = 123, b = 456, c = 789

Now suppose that the data had been entered as

123456789

Then the assignments would be

`a = 123, b = 456, c = 789`

as before, since the first three digits would be assigned to *a*, the next three digits to *b*, and the last three digits to *c*. Finally, suppose that the data had been entered as

`1234 5678 9`

The resulting assignments would now be

`a = 123, b = 4, c = 567`

The remaining two digits (8 and 9) would be ignored, unless they were read by a subsequent `scanf` statement.

Consider a C program that contains the following statements.

```
#include <stdio.h>
main()
{
    int i;
    float x;
    char c;
    ...
    scanf("%3d %5f %c", &i, &x, &c);
    ...
}
```

If the data items are entered as

`10 256.875 T`

when the program is executed, then 10 will be assigned to *i*, 256.8 will be assigned to *x* and the character 7 will be assigned to *c*. The remaining two input characters (5 and T) will be ignored.

Most versions of C allow certain conversion characters within the control string to be preceded by a single-letter *prefix*, which indicates the length of the corresponding argument. For example, an *l* (lowercase is used to indicate either a signed or unsigned long integer argument, or a double-precision argument. Similarly, an *h* is used to indicate a signed or unsigned short integer. Also, some versions of C permit the use of an uppercase *L* to indicate a long double.

Suppose the following statements are included in a C program.

```
#include <stdio.h>
main()
{
    short ix, iy;
    long lx, ly;
    double dx, dy;
    ...
    scanf(" %hd %ld %lf", &ix, &lx, &dx);
    ...
    scanf(" %3ho %7lx %15le", &iy, &ly, &dy);
    ...
}
```

The control string in the first **scanf** function indicates that the first data item will be assigned to a *short decimal integer* variable, the second will be assigned to a *long decimal integer* variable, and the third will be assigned to a *double-precision* variable. The control string in

the second **scanf** function indicates that the first data item will have a maximum *field width of 3 characters* and it will be assigned to a *short octal integer* variable, the second data item will have a maximum *field width of 7 characters* and it will be assigned to a *long hexadecimal integer* variable, and the third data item will have a maximum *field width of 15 characters* and it will be assigned to a *double-precision* variable.

In most versions of C it is possible to skip over a data item, *without assigning* it to the designated variable or array. To do so, the % sign within the appropriate control group is followed by an *asterisk (*)*. This feature is referred to as *assignment suppression*.

```
#include <stdio.h>
main()
{
    char item[20];
    int partno;
    float cost;
    ...
    scanf(" %s %*d %f", item, &partno, &cost);
    ...
}
```

Notice the *asterisk* in the second character group. If the corresponding data items are

fastener 12345 0.05

then *fastener* will be assigned to **item** and 0.05 will be assigned to **cost**. However, 12345 will not be assigned to **partno** because of the asterisk, which is interpreted as an assignment suppression character. Note that the integer quantity 12345 will be read into the computer along with the other data items, even though it is not assigned to its corresponding variable.

If the control string contains multiple character groups without interspersed whitespace characters, then some care must be taken with c-type conversion. In such cases a whitespace character within the input data will be interpreted as a data item. To skip over such whitespace characters and read the next non-whitespace character, the conversion group **%1s** should be used.

Consider the following skeletal outline of a C program.

```
#include <stdio.h>
main()
{
    char c1, c2, c3;
    ...
    scanf(" %c%c%c", &c1, &c2, &c3);
    ...
}
```

If the input data consisted of

a b c

(with blank spaces between the letters), then the following assignments would result:

c1 = a, c2 = <blank space>, c3 = b

If the scanf function were written as

```
scanf(" %c%1s%1s", &c1, &c2, &c3);
```

However, then the same input data would result in the following assignments:

```
c1 = a,    c2 = b,    c3 = c
```

as intended. Note that there are some other ways around this problem. We could have written the `scanf` function as

```
scanf(" %c %c %c", &c1, &c2, &c3);
```

with blank spaces separating the `%c` terms, or we could have used the original **`scanf`** function but written the input data as consecutive characters without blanks; i.e., *abc*.

Unrecognized characters within the control string are expected to be *matched* by the same characters in the input data. Such input characters will be read into the computer, but not assigned to an identifier. Execution of the **`scanf`** function will *terminate* if a match is not found.

Consider the following skeletal outline.

```
#include <stdio.h>
main()
{
    int i;
    float x;
    ...

    scanf("%d a %f", &i, &x);
    ...
}
```

If the input data consist of

```
1 a 2.0
```

then the decimal integer **1** will be read in and assigned to *i*, the character **a** will be read in but subsequently *ignored*, and the floating-point value **2.0** will be read in and assigned to *x*. On the other hand, if the input were entered simply as

```
1 2.0
```

then the `scanf` function would stop executing once the expected character (*a*) is not found. Therefore, *i* would be assigned the value **1** but *x* would automatically represent the value **0**.

Writing Output Data- the `printf` Function

Output data can be written from the computer onto a standard output device using the library function *printf*. This function can be used to output any combination of numerical values, single characters and strings. It is like the input function *scanf*, except that its purpose is to display data rather than to enter it into the computer. That is, the *printf* function moves data from the computer's memory to the standard output device, whereas the *scanf* function enters data from the standard input device and stores it in the computer's memory.

In general terms, the `printf` function is written as

```
printf(control string, arg1, arg2, ..., argn)
```

where *control string* refers to a string that contains formatting information, and *arg1*, *arg2*, ..., *argn* are arguments that represent the individual output data items. The arguments can be written as constants, single variable or array names, or more complex expressions. Function

references may also be included. In contrast to the `scanf` function discussed in the last section, the arguments in a `printf` function do not represent memory addresses and therefore are not preceded by ampersands.

The control string consists of individual groups of characters, with one-character group for each output data item. Each character group must begin with a *percent sign (%)*. In its simplest form, an individual character group will consist of the percent sign, followed by a **conversion character** indicating the type of the corresponding data item.

Multiple character groups can be contiguous, or they can be separated by other characters, including whitespace characters. These “other” characters are simply transferred directly to the output device, where they are displayed. The use of blank spaces as character-group separators is particularly common.

Several of the more frequently used conversion characters are listed in Table below.

Commonly Used Conversion Characters for Data Output

Conversion Character	Meaning
c	data item is displayed as a single character
d	data item is as a signed decimal integer
e	data item is displayed as a floating-point value with an exponent
f	data item is displayed as a floating-point value without an exponent
g	data item is as a floating-point value using either <i>e-type</i> or <i>f-type</i> conversion, depending on value. Trailing zeros and trailing decimal point will not be displayed.
i	data item is displayed as a signed decimal integer.
o	data item is displayed as an octal integer, without leading zero.
s	data item is displayed as a string.
u	data item is displayed as an unsigned decimal integer
x	data item is a hexadecimal integer

Here is a simple program that makes use of the `printf` function.

```
#include <stdio.h>
#include <math.h>
main()
{
    float i = 2.0, j = 3.0;
    printf("%f %f %f %f", i, j, i+j, sqrt(i+j));
}
```

Notice that the first two arguments within the `printf` function are *single variables*, the third argument is an *arithmetic expression*, and the last argument is a *function reference* that has a numeric expression as an argument.

Executing the program produces the following output:

```
2.000000 3.000000 5.000000 2.236068
```


The following skeletal outline indicates how several different types of data can be displayed using the *printf* function.

```
#include <stdio.h>
main()
{
    char item[20];
    int partno;
    float cost;
    ...
    printf("%s %d %f", item, partno, cost);
    ...
}
```

Within the *printf* function, the control string is "*%s %d %f*". It contains three-character groups. The first character group, *%s*, indicates that the first argument (*item*) represents a string. The second character group, *%d*, indicates that the second argument (*partno*) represents a decimal integer value, and the third character group, *%f*, indicates that the third argument (*cost*) represents a floating-point value.

Notice that the arguments are not preceded by ampersands. This differs from the *scanf* function, which requires ampersands for all arguments other than array names.

Now suppose that *name*, *partno* and *cost* have been assigned the values ***fastener***, ***12345*** and ***0.05***, respectively, within the program. When the *printf* statement is executed, the following output will be generated.

```
Fastener 12345 0.050000
```

The single space between data items is generated by the blank spaces that appear within the control string in the *printf* statement.

Suppose the *printf* statement had been written as

```
printf("%s%d%f", item, partno, cost);
```

This *printf* statement is syntactically valid, though it causes the output items to run together; i.e.,

```
fastener123450.050000
```

The *f*-type conversion and the *e*-type conversion are both used to output floating-point values. However, the latter causes an exponent to be included in the output, whereas the former does not.

The following program generates the same floating-point output in two different forms.

```
#include <stdio.h>
main() /* display floating-point output 2 different ways */
{
    double x = 5000.0, y = 0.0025;
    printf("%f %f %f %f\n" , x, y, x*y, x/y);
    printf("%e %e %e %e", x, y, x*y, x/y);
}
```

Both *printf* statements have the same arguments. However, the first *printf* statement makes use of *f*-type conversion, whereas the second *printf* statement uses *e*-type conversion. Also, notice the repeated newline character in the first *printf* statement. This causes the output to be double-spaced, as shown below. When the program is executed, the following output is generated.

5000.000000 0.002500 12.500000 2000000.000000

5.000000e+03 2.500000e-03 1.250000e+01 2.000000e+06

The first line of output shows the quantities represented by x , y , $x*y$ and x/y in standard floating-point format, without exponents. The second line of output shows these same quantities in a form resembling scientific notation, with exponents.

Notice that *six decimal places* are shown for each value. The number of decimal places can be altered, however, by specifying the *precision* as a part of each character group within the control string.

The `printf` function interprets *s-type* conversion differently than the `scanf` function. In the `printf` function, *s-type* conversion is used to output a *string* that is terminated by the null character (**0**). Whitespace characters may be included within the string.

Reading and Writing a Line of Text: Here is a short C program that will read in a line of text and then write it back out, just as it was entered. The program illustrates the syntactic differences in reading and writing a string that contains a variety of characters, including whitespace characters.

```
#include <stdio.h>
main()
{
    char line[80];

    scanf("%[^\n]", line);
    printf("%s", line);
}
```

Notice the difference in the control strings within the `scanf` function and the `printf` function. Now suppose that the following string is entered from the standard input device when the program is executed.

The Mt. Everest is the highest land of the Earth.

This string contains lowercase characters, uppercase characters, punctuation characters and whitespace characters. The entire string can be entered with the single `scanf` function, if it is terminated by a newline character (by pressing the **Enter key**). The `printf` function will then cause the entire string to be displayed on the standard output device, just **as** it had been entered. Thus, the message

The Mt. Everest is the highest land of the Earth.
would be generated by the computer.

A *minimum* field width can be specified by preceding the conversion character by an unsigned integer. If the number of characters in the corresponding data item is less than the specified field width, then the data item will be preceded by enough leading *blanks* to fill the specified field. If the number of characters in the data item exceeds the specified field width, however, then *additional space* will be allocated to the data item, so that the entire data item will be displayed. This is just the opposite of the field width indicator in the `scanf` function, which specifies a *maximum* field width.

The following C program illustrates the use of the minimum field width feature.

```
#include <stdio.h>
main()
{
```

```

int i = 12345;
float x = 345.678;

printf("%3d %5d %8d\n\n", i, i, i);
printf("%3f %10f %13f\n\n", x, x, x);
printf("%3e %13e %16e", x, x, x);
}

```

Notice the double newline characters in the first two *printf* statements. They will cause the lines of output to be double spaced, as shown below. When the program is executed, the following output is generated.

```

12345 12345      12345

345.678000 345.678000    345.678000

3.456780e+02 3.456780e+02    3.456780e+02

```

The first line of output displays a decimal integer using three different minimum field widths (three characters, five characters and eight characters). The entire integer value is displayed within each field, even if the field width is too small.

The second value in the first line is preceded by one blank space. This is generated by the blank space separating the first two-character groups within the control string.

The third value is preceded by four blank spaces. One blank space comes from the blank space separating the last two-character groups within the control field. The other three blank spaces fill the minimum field width, which exceeds the number of characters in the output value (the minimum field width is eight, but only five characters are displayed).

A similar situation is seen in the next two lines, where the floating-point value is displayed using f-type conversion (in line 2) and e-type conversion (line 3).

Here is a variation of the program presented in the above Example, which makes use of g-type conversion.

```

#include <stdio.h>
main()
{
    int i = 12345;
    float x = 345.678;

    printf("%3d %5d %8d\n\n", i, i, i);
    printf("%3g %10g %13g\n\n", x, x, x);
    printf("%3g %13g %16g", x, x, x);
}

```

Execution of this program causes the following output to be displayed.

```

12345 12345      12345

345.678    345.678    345.678

345.678    345.678    345.678

```

The floating-point values are displayed with an f-type conversion, since this results in a shorter display. The minimum field widths conform to the specifications within the control string.

More About the printf Function

We have already learned how to specify a minimum field width in a printf function. It is also possible to specify the maximum number of decimal places for a floating-point value, or the maximum number of characters for a string. This specification is known as **precision**. The precision is an unsigned integer that is always preceded by a decimal point. If a minimum field width is specified in addition to the precision (as is usually the case), then the precision specification follows the field width specification. Both integer specifications precede the conversion character.

A floating-point number will be **rounded** if it must be shortened to conform to a precision specification.

Here is a program that illustrates the use of the precision feature with floating-point numbers.

```
#include <stdio.h>
main()
{
    float x = 123.456;

    printf("%7f %7.3f %7.1f\n\n", x, x, x);
    printf("%12e %12.5e %12.3e", x, x, x);
}
```

When this program is executed, the following output is generated.

```
123.456000 123.456    123.5
```

```
1.234560e+02 1.23456e+02    1.235e+02
```

The first line is produced by f-type conversion. Notice the rounding that occurs in the third number because of the precision specification (one decimal place). Also, notice the leading blanks that are added to fill the specified minimum field width (seven characters).

The second line, produced by e-type conversion, has similar characteristics. Again, we see that the third number is rounded to conform to the specified precision (three decimal places). Also, note the leading blanks that are added to fill the specified minimum field width (12 characters).

A minimum field width specification need not necessarily accompany the precision specification. It is possible to specify the precision without the minimum field width, though the precision must still be preceded by a decimal point.

Now let us rewrite the program shown in the last example without any minimum field width specifications, but with precision specifications.

```
#include <stdio.h>
main()
{
    float x = 123.456;
    printf("%f %.3f %.1f\n\n", x, x, x);
    printf("%e %.5e %.3e", x, x, x);
}
```

Execution of this program produces the following output.

```
123.456000 123.456 123.4
```

```
1.234560e+02 1.23456e+02 1.235e+02
```

Notice that the third number in each line does not have multiple leading blanks, since there is no minimum field width that must be satisfied. In all other respects, however, this output is the same as the output generated in the last example.

Minimum field width and precision specifications can be applied to character data as well as numerical data. When applied to a string, the minimum field width is interpreted in the same manner as with a numerical quantity; i.e., leading blanks will be added if the string is shorter than the specified field width, and additional space will be allocated if the string is longer than the specified field width. Hence, the field width specification will not prevent the entire string from being displayed.

However, the precision specification will determine the maximum number of characters that can be displayed. If the precision specification is less than the total number of characters in the string, the excess right-most characters will not be displayed. This will occur even if the minimum field width is larger than the entire string, resulting in the addition of leading blanks to the truncated string.

The following program outline illustrates the use of field width and precision specifications in conjunction with string output.

```
#include <stdio.h>
main()
{
    char line[12];
    ...
    printf("%10s %15s %15.5s %.5s", line, line, line, line);
}
```

Now suppose that the string “**hexadecimal**” is assigned to the character array *line*. When the program is executed, the following output will be generated.

```
hexadecimal      hexadecimal      hexad hexad
```

The first string is shown in its entirety, even though this string consists of 11 characters, but the field width specification is only 10 characters. Thus, the first string overrides the minimum field width Specification. The second string is padded with four leading blanks to fill out the 15-character minimum; hence, the second string is *right justified* within its field. The third string consists of only five nonblank characters because of the five-character precision specification; however, 10 leading blanks are added to fill out the minimum field width specification, which is 15 characters. The last string also consists of five nonblank characters. Leading blanks are not added, however, because there is no minimum field width specification.

Most versions of C permit the use of prefixes within the control string to indicate the length of the corresponding argument. The allowable prefixes are the same as the prefixes used with the scanf function. Thus, an *l* (lowercase) indicates a signed or unsigned integer argument, or a double-precision argument; an *h* indicates a signed or unsigned short integer. Some versions of C permit an *L* (uppercase) to indicate a long double.

Suppose the following statements are included in a C program.

```
#include <stdio.h>
main()
{
    short a, b;
    long c, d;
```

```
...  
printf("%5hd %6hx %8lo %lu", a, b, c, d);  
...  
}
```

The control string indicates that the first data item will be a *short decimal integer*, the second will be a *short hexadecimal integer*, the third will be a *long octal integer*, and the fourth will be a *long unsigned (decimal) integer*. Note that the first three fields have minimum field width specifications, but the fourth does not.

Some versions of C allow the conversion characters **X**, **E** and **G** to be written in uppercase. These uppercase conversion characters cause any letters within the output data to be displayed in uppercase. (Note that this use of uppercase conversion characters is distinctly different than with the `scanf` function.)

The following program illustrates the use of uppercase conversion characters in the *printf* function.

```
#include <stdio.h>
main()
{
    int a = 0x80ec;
    float b = 0.3e-12;
    printf("%4x %10.2e\n\n", a, b);
    printf("%4X %10.2E", a, b);
}
```

Notice that the first *printf* statement contains *lowercase conversion characters*, whereas the second *printf* statement contains *uppercase conversion characters*.

When the program is executed, the following output is generated.

```
80ec    3.00e-13
80EC    3.00E-13
```

The first quantity on each line is a hexadecimal number. Note that the letters *ec* (which are a part of the hexadecimal number) are shown in lowercase on the first line, and in uppercase on the second line.

The second quantity on each line is a decimal floating-point number which includes an exponent. Notice that the letter *e*, which indicates the *exponent*, is shown in lowercase on the first line and uppercase on the second.

You are again reminded that the use of uppercase conversion characters is not supported by all compilers.

In addition to the field width, the precision and the conversion character, each character group within the control string can include a **flag**, which affects the appearance of the output. The flag must be placed immediately after the percent sign (%). Some compilers allow two or more flags to appear consecutively, within the same character group. The more commonly used flags are listed in Table below.

Flag	Meaning
-	Data item is left justified within the field (blank spaces required to fill the minimum field width will be added <i>after</i> the data item rather than <i>before</i> the data item.
+	A sign (either + or -) will precede each signed numerical data item. Without this flag, only negative data items are preceded by a sign.
0	Causes leading zeros to appear instead of leading blanks. Applies only to data items that are right justified within a field whose minimum size is larger than the data item.
' ' (blank space)	A blank space will precede each positive signed numerical data item. This flag is overridden by the + flag if both are present.
# (with o- and x-type conversion)	Causes octal and hexadecimal data items to be preceded by 0 and 0x, respectively.
# (with e-, f- and g-type conversion)	Causes a decimal point to be present in all floating-point numbers, even if the data item is a whole number. Also prevents the truncation of trailing zeros in g-type conversion.

Here is a simple C program that illustrates the use of flags with integer and floating-point quantities.

```
#include <stdio.h>
main()
{
    int i = 123;
    float x = 12.0, y = -3.3;

    printf(":%6d %7.0f %10.1e:\n\n", i, x, y);
    printf(":%-6d %-7.0f %-10.1e:\n\n", i, x, y);
    printf(":%+6d %+7.0f %+10.1e:\n\n", i, x, y);
    printf(":%-+6d %-+7.0f %-+10.1e:\n\n", i, x, y);
    printf(":%7.0f %#7.0f %7g %#7g:", x, x, y, y);
}
```

When the program is executed, the following output is produced. (The colons indicate the beginning of the first field and the end of the last field in each line.)

```
:   123       12   -3.3e+00:
:123   12      -3.3e+00   :
:  +123     +12    -3.3e+00:
:+123   +12     -3.3e+00   :
:      12     12.   -3.3 -3.300000:
```

The first line illustrates how integer and floating-point numbers appear without any flags. Each number is right justified within its respective field. The second line shows the same numbers, using the same conversions, with a - flag included within each character group. Note that the numbers are now left justified within their respective fields. The third line shows the effect of using a + flag. The numbers are now right justified, as in the first line, but each number (whether positive or negative) is preceded by an appropriate sign. The fourth line shows the effect of combining a - and a + flag. The numbers are now left justified and preceded by an appropriate sign. Finally, the last line shows two floating-point numbers, each displayed first without and then with the flag. Note that the effect of the flag is to include a decimal point in the number 12. (which is printed with f-type conversion), and to include the trailing zeros in the number -3.300000(printed with g-type conversion). Now consider the following program, which displays decimal, octal and hexadecimal numbers.

```
#include <stdio.h>
main()
{
    int i = 1234, j = 01777, k = 0xa08c;
    printf(":%8u %8o %8x:\n\n", i, j, k);
    printf(":%-8u %-8o %-8x:\n\n", i, j, k);
    printf(":%#8u %#8o %#8X:\n\n", i, j, k);
    printf(":%08u %08o %08X:\n\n", i, j, k);
}
```

Execution of this program results in the following output. (The colons indicate the beginning of the first field and the end of the last field in each line.)

```
:   1234       1777    a08c:
:1234   1777    a08c   :
:   1234   017777   0XA08C:
```



```
:00001234 00001777 0000A08C:
```

The first line illustrates the display of unsigned integer, octal and hexadecimal output without any flags. Note that the numbers are right justified within their respective fields. The second line shows what happens when you include a - flag within each character group. Now the numbers are left justified within their respective fields.

In the third line we see what happens when the # flag is used. This flag causes the octal number **1777** to be preceded by a 0 (appearing as **01 777**), and the hexadecimal number to be preceded by **OX** (i.e., **OXA08C**). Notice that the unsigned decimal integer **1234** is unaffected by this flag. Also, notice that the hexadecimal number now contains uppercase characters, since the conversion character was written in uppercase (**X**).

The last line illustrates the use of the 0 flag. This flag causes the fields to be filled with leading **0s** rather than leading blanks. We again see uppercase hexadecimal characters, in response to the uppercase conversion character (**X**).

The following program outline illustrates the use of flags with string output.

```
#include <stdio.h>
```

```
main()
```

```
{
    char line[12];
    ...
    printf(":%15s %15.5s %.5s:\n\n", line, line, line);
    printf(":%-15s %-15.5s %-.5s:\n\n", line, line, line);
}
```

Now suppose that the string **“lower-case”** is assigned to the character array *line*. The following output will be generated when the program is executed.

```
:      lower-case          lower lower:
:lower-case      lower          lower:
```

The first line illustrates how strings are displayed when flags are not present. The second line shows the same strings, left justified, in response to the - flag in each character group.

Unrecognized characters within the control string will be displayed just as they appear. This feature allows us to include labels and messages with the output data items, if we wish.

The following program illustrates how printed output can be labeled.

```
#include <stdio.h>
```

```
main()
```

```
{
    float a = 2.2, b = -6.2, x1 = .005, x2 = -12.88;

    printf("$%4.2f %7.1f%\n", a, b);
    printf("x1=%7.3f x2=%7.3f", x1, x2);
}
```

This program causes the value of a (2.2) to be preceded by a dollar sign (\$), and the value of b (-6.2) to be followed by a percent sign (%). Note the two consecutive percent signs in the first printf statement. The first percent sign indicates the start of a character group, whereas the second percent sign is interpreted as a label.

The second printf statement causes the value of **x1** to be preceded by the label **x1=**, and the value of **x2** to be preceded by the label **x2=**. Three blank spaces will separate these two labeled data items. The actual output is shown below.

```
$2.20      -6.2%
x1= 0.0005  x2 =-12.880
```

Interactive (Conversational) Programming

Averaging Student Exam Scores

This example presents a simple, interactive C program that reads in a student's name and three exam scores, and then calculates an average score. The data will be entered interactively, with the computer asking the user for information and the user supplying the information in a free format, as requested. Each input data item will be entered on a separate line. Once all the data have been entered, the computer will compute the desired average and write out all the data (both the input data and the calculated average). The actual program is shown below.

```
#include <stdio.h>

main()
{
    char name[20];
    float score1, score2, score3, avg;

    printf("Please enter your name: ");
    scanf(" %[^\\n]", name);

    printf("Please enter the first score: ");
    scanf("%f", &score1);

    printf("Please enter the second score: ");
    scanf("%f", &score2);

    printf("Please enter the third score: ");
    scanf("%f", &score3);

    avg = (score1 + score2 + score3)/3;

    printf("\\n\\nName: %-s\\n\\n", name);
    printf("Score 1: %-5.1f\\n", score1);
    printf("Score 2: %-5.1f\\n", score2);
    printf("Score 3: %-5.1f\\n", score3);
    printf("Average: %-5.1f\\n\\n", avg);
}
```

Notice that two statements are associated with each input data item. The first is a **printf** statement, which generates a request for the item. The second statement, a **scanf** function, causes the data item to be entered from the standard input device (i.e., the keyboard). After the student's name and all three exam scores have been entered, an average exam score is calculated. The input data and the calculated average are then displayed, as a result of the group of **printf** statements at the end of the program.

A typical interactive session is shown below. To illustrate the nature of the dialog, the user's responses have been underlined.

```
Please enter your name: Raju Shrestha
Please enter your first score: 88
```

Please enter your second score: 62.5

Please enter your third score: 90

Name: Raju Shrestha

Score 1: 88.0

Score 2: 62.5

Score 3: 90.0

Average: 80.2

4. CONTROL STATEMENTS

In most of the C programs we have encountered so far, the instructions were executed in the same order in which they appeared within the program. Each instruction was executed once and only once. Programs of this type are unrealistically simple, since they do not include any logical control structures. Thus, these programs did not include tests to determine if certain conditions are true or false, they did not require the repeated execution of groups of statements, and they did not involve the execution of individual groups of statements on a selective basis. Most C programs that are of practical interest make extensive use of features such as these.

For example, a realistic C program may require that a logical test be carried out at some point within the program. One of several possible actions will then be carried out, depending on the outcome of the logical test. This is known as **branching**. There is also a special kind of branching, called **selection**, in which one group of statements is selected from several available groups. In addition, the program may require that a group of instructions be executed repeatedly, until some logical condition has been satisfied. This is known as **looping**. Sometimes the required number of repetitions is known in *advance*; and sometimes the computation continues *indefinitely* until the logical condition becomes true.

All these operations can be carried out using the various control statements included in C. We will see how this is accomplished in this chapter. The use of these statements will open the door to programming problems that are much broader and more interesting than those considered earlier.

Preliminaries

Before considering the detailed control statements available in C, let us review some concepts presented in chapter 3 that must be used in conjunction with these statements.

Understanding these concepts is essential in order to proceed further.

First, we will need to form logical expressions that are either *true* or *false*. To do so, we can use the four *relational operators*, `<`, `<=`, `>`, `>=`, and the two *equality operators*, `==` and `!=`.

Several logical expressions are shown below.

```
count <= 100
sqrt(a + b + c) > 0.005
answer == 0
balance >= cutoff
ch1 < 'T'
letter != 'x'
```

The first four expressions involve numerical operands. Their meaning should be readily apparent.

In the fifth expression, *ch1* is assumed to be a char-type variable. This expression will be true if the character represented by *ch1* comes before **T** in the character set, i.e., if the numerical value used to encode the character is less than the numerical value used to encode the letter **T**.

The last expression makes use of the char-type variable *letter*. This expression will be true if the character represented by *letter* is something other than **x**.

In addition to the relational and equality operators, C contains two *logical connectives* (also called *logical operators*), **&&** (AND) and **||** (OR), and the *unary negation operator* **!**. The logical connectives are used to combine logical expressions, thus forming more complex expressions. The negation operator is used to reverse the meaning of a logical expression (e.g., from true to false).

Here are some logical expressions that illustrate the use of the logical connectives and the negation operator.

```
(count <= 100) && (ch1 != '*' )
(balance < 1000.0)  || (status == 'R')
(answer < 0) || ((answer > 5.0) && (answer <= 10.0))
!((pay >= 1000.0) && (status == 's'))
```

Note that *ch1* and *status* are assumed to be char-type variables in these examples. The remaining variables are assumed to be numeric (either integer or floating-point).

Since the relational and equality operators have a higher precedence than the logical operators, some of the parentheses are not needed in the above expressions. Thus, we could have written these expressions as

```
count <= 100 && ch1 != '*'
balance < 1000.0      || status == 'R'
answer < 0 || answer > 5.0 && answer <= 10.0
!(pay >= 1000.0 && status == 's')
```

It is a good idea, however, to include pairs of parentheses if there is any doubt about the operator precedence.

The *conditional operator* **?:** also makes use of an expression that is either true or false. An appropriate value is selected, depending on the outcome of this logical expression. This operator is equivalent to a simple *if-else* structure.

Suppose *status* is a char-type variable and *balance* is a floating-point variable. We wish to assign the character C (current) to *status* if *balance* has a value of zero, and O (overdue) if *balance* has a value that is greater than zero. This can be accomplished by writing

```
Status = (balance == 0) ? 'C' : 'O'
```

Finally, recall that there are three different kinds of statements in C: *expression statements*, *compound statements* and *control statements*. An expression statement consists of an expression, followed by a semicolon. A compound statement consists of a sequence of two or more consecutive statements enclosed in braces (**{}**). The enclosed statements can be expression statements, other compound statements or control statements. Most control statements contain expression statements or compound statements, including embedded compound statements.

```
{
    int lower, upper;
```

```

lower = getchar();
upper = toupper(lower);
putchar(upper);
}

```

The control statements presented within this chapter make extensive use of *logical expressions* and *compound statements*. *Assignment operators*, such as the one used in the above example (i.e., +=), will also be utilized.

Branching: the if - else Statement

The if - else statement is used to carry out a logical test and then take one of two possible actions, depending on the outcome of the test (i.e., whether the outcome is true or false). The else portion of the if - else statement is optional. Thus, in its simplest general form, the statement can be written as

if (expression) statement

The *expression* must be placed in parentheses, as shown. In this form, the *statement* will be executed only if the *expression* has a *nonzero* value (i.e., if *expression* is true). If the *expression* has a value of *zero* (i.e., if *expression* is false), then the *statement* will be ignored. Let us try to use if instruction to find out bigger out of two numbers.

```

#include <stdio.h>
main()
{
    int first, second;
    scanf(" %d %d", &first, &second);
    if(first > second)
        printf("first number is bigger");
}

```

In the above program, if the input value of first is higher than second number, only then the printf statement will be executed. So, if the inputs are given as

20 10

We will see the output

first number is bigger

On the other hand, if we give input value as

10 20

No output message will appear on the screen. So, in order to get proper output message for all combinations of inputs, the above program can be extended as

```

#include <stdio.h>
main()
{
    int first, second;
    scanf("%d %d", &first, &second);
    if(first > second)
        printf("first number is bigger\n");
    if(second > first)
        printf("second number is bigger\n");
    if(first == second)
        printf("both numbers are equal\n");
}

```

```
}
```

There can be more than one statements as a part of if statement. In that case all statements need to be collectively put as a compound statement e.g.,

```
if (x <= 3.0)
{
    y = 3 * pow(x, 2);
    printf("%f\n", y);
}
```

The expression used as logical test can include *logical operators* to combine multiple conditions e.g.,

```
if ((balance < 1000) || (status == 'R'))
    printf("%f\n", balance);

if ((a >= 0) && (b <= 5))
{
    xmid = (a+b) / 2;
    ymid = sqrt (xmid);
}
```

In the first statement we see a complex logical expression, which causes the value of *balance* to be displayed if its value is less than **1000** or if *status* has been assigned the character '**R**'.

The last statement involves both a complex logical expression and a compound statement.

Thus, the variables *xmid* and *ymid* will both be assigned appropriate values if the current value of *a* is nonnegative and the current value of *b* does not exceed 5.

Many times, the program requires to take a specific action if a condition is true and another action if the condition is not true (i.e. false). Such a situation can be effectively implemented using if else instruction.

The general form of an if statement which includes the else clause is

if (expression) statement1 else statement2

If the *expression* has a nonzero value (i.e., if *expression* is true), then *statement1* will be executed. Otherwise (i.e., if *expression* is false), *statement2* will be executed.

```
#include <stdio.h>
main()
{
    float first, second;
    scanf("%f %f", &first, &second);
    if (first == second)
        printf("both numbers are equal\n");
    else
        printf("numbers are not equal\n");
}
```

Here are several examples illustrating the full if-else statement.

```
if (status == 's')
    tax = 0.20 * pay;
else
    tax = 0.14 * pay;
```

```

if (past_due > 0)
{
    printf("account number %d is overdue", account_no);
    credit = 0;
}
else
    credit = 1000.0;

if (x <= 3)
    y = 3 * pow(x, 2);
else
    y = 2 * pow((x-3), 2);

if (circle)
{
    scanf(" %f", &radius);
    area = 3.14159 * radius * radius;
    printf("area of circle: %f\n", area);
}
else
{
    scanf(" %f %f", &length, &width);
    area = length * width;
    printf("area of rectangle: %f\n", area);
}

```

In the first example the value of *tax* is determined in one of two possible ways, depending on the character that has been assigned to the variable *status*. Notice the semicolon at the end of each statement, particularly the first statement (*tax* = 0 . 2 * *pay*;). A more concise way to accomplish the same thing is to write

```
tax = (status == 's') ? (0.20 * pay) : (0.14 * pay);
```

though this approach is not as clear.

The second example examines the past-due status of an account. If the value of *past_due* exceeds zero, a message is displayed and the credit limit is set at zero; otherwise, the credit limit is set at 1000.0. In the third example, the value of *y* is computed differently, depending on whether the corresponding value of *x* exceeds 3.

The fourth example shows how an area can be calculated for either of two different geometric figures. If *circle* is assigned a nonzero value, the *radius* of a circle is read into the computer, the *area* is calculated and then displayed. If the value of *circle* is zero, however, then the *length* and *width* of a rectangle are read into the computer, the area is calculated and then displayed. In each case, the type of geometric figure is included in the label that accompanies the value of the area.

It is possible to *nest* (i.e., embed) *if - else* statements, one within another. There are several different forms that nested *if - else* statements can take. The most general form of *two-layer* nesting is

```

if expr1 if expr2 s1
        else s2
else if expr3 s3
        else s4

```

where *expr1*, *expr2* and *expr3* represent logical expressions and *s1*, *s2*, *s3* and *s4* represent statements. Now, one complete if - else statement will be executed if *expr1* is nonzero (true), and another complete if - else statement will be executed if *expr1* is zero (false). It is, of course, possible that *s1*, *s2*, *s3* and *s4* will contain other if - else statements. We would then have multilayer nesting.

Some other forms of two-layer nesting are

```
if expr1 s1
else if expr2 s2
```

```
if expr1 s1
else if expr2 s2
    else s3
```

```
if expr1 if expr2 s1
    else s2
else s3
```

```
if expr1 if expr2 s1
    else s2
```

In the first three cases the association between the else clauses and their corresponding expressions is straightforward. In the last case, however, it is not clear which expression (*expr1* or *expr2*) is associated with the else clause. The answer is *expr2*. The rule is that the *else clause* is always associated with the closest preceding **unmatched if**. This is suggested by the indentation, though the indentation itself is not the deciding factor. Thus, the last example is equivalent to

```
if expr1
{
    if expr2 s1 else s2
}
```

If we wanted to associate the else clause with *expr1* rather than *expr2*, we could do so by writing

```
if expr1
{
    if expr2 s1
}
else s2
```

This type of nesting must be carried out carefully in order to avoid possible ambiguities. One good example of two-level nesting is given below. The code is used to find out biggest out of three numbers, assuming all three numbers to be distinct.

```
if (a > b)
{
    if (a > c) printf("%d is biggest\n", a);
```



```

    else printf("%d is biggest\n", c);
}
else
{
    if (b > c) printf("%d is biggest\n", b);
    else printf("%d is biggest\n", c);
}

```

In some situations, it may be desirable to nest multiple if - else statements, in order to create a situation in which one of several different courses of action will be selected. For example, the general form of four nested if - else statements could be written as

```

if e1 s1
    else if e2 s2
        else if e3 s3
            else if e4 s4
                else s5

```

When a logical expression is encountered whose value is nonzero (true), the corresponding statement will be executed and the remainder of the nested if - else statements will be bypassed. Thus, control will be transferred out of the entire nest once a true condition is encountered.

The final else clause will apply if none of the expressions is true. It can be used to provide a default condition or an error message.

Here is an illustration of three nested *if-else* statements.

```

if((time >= 0.) && (time < 12.)) printf("Good Morning");
else if((time >= 12.) && (time < 18.)) printf("Good Afternoon");
    else if((time >= 18.) && (time < 24.)) printf("Good Evening");
        else printf("Time is out of range.");

```

This example causes a different message to be displayed at various times of the day.

Specifically, the message Good Morning will be displayed if time has a value between 0 and 12; Good Afternoon will be displayed if time has a value between 12 and 18; and Good Evening will be displayed if time has a value between 18 and 24. An error message (Time is out of range) will be displayed if the value of time is less than zero, or greater than or equal to 24.

Looping: the while Statement

The **while** statement is used to carry out looping operations, in which a group of statements is executed repeatedly, until some condition has been satisfied. The general form of the **while** statement is

while (expression) statement

The **statement** will be executed repeatedly, as long as the **expression** is true (i.e., as long as **expression** has a nonzero value). This **statement** can be simple or compound, though it is usually a compound statement. It must include some feature that eventually alters the value of the **expression**, thus providing a stopping condition for the loop.

Consecutive Integer Quantities Suppose we want to display the consecutive digits 0, 1, 2, . . . , 9, with one digit on each line. This can be accomplished with the following program.

```

#include <stdio.h>
main()

```

```

{
    int digit = 0; //initialization

    while (digit <= 9)
    {
        printf("%d\n", digit);
        ++digit; //update loop variable
    }
}

```

Initially, **digit** is assigned a value of 0. The **while** loop then displays the current value of **digit**, increases its value by 1 and then repeats the cycle, until the value of **digit** exceeds 9. The net effect is that the body of the loop will be repeated 10 times, resulting in 10 consecutive lines of output. Each line will contain a successive integer value, beginning with 0 and ending with 9. Thus, when the program is executed, the following output will be generated.

```

0
1
2
3
4
5
6
7
8
9

```

This program can be written more concisely as

```

#include <stdio.h>
main()
{
    int digit = 0;

    while (digit<=9)
    {
        printf("%d\n", digit++);
    }
}

```

When executed, this program will generate the same output as the first program.

In some looping situations, the number of *passes* through the loop is known in advance. The previous example illustrates this type of loop. Sometimes, however, the number of passes through the loop is *not known* in advance. Rather, the looping action continues *indefinitely*, until the specified logical condition has been satisfied. The **while** statement is particularly well suited for this second type of loop.

Lowercase to Uppercase Text Conversion In this example we will read a line of lowercase text

character-by-character and store the characters in a char-type array called **letter**. The program will continue reading input characters until an end-of-line (**EOF**) character has been read. The characters will then be converted to uppercase, using the library function **toupper**, and displayed.

Two separate **while** loops will be used. The first will read the text from the keyboard. Note that the number of passes through this loop is not known in advance. The second **while** loop will perform the conversion and write out the converted text. It will make a known number of passes, since the number of characters to be displayed will be determined by counting the number of passes through the first loop. The complete program is shown below.

```
/* convert a line of lowercase text to uppercase */
#include <stdio.h>
#include <ctype.h> //toupper()

#define EOL '\n'
main()
{
    char letter[80];
    int tag, count = 0;
    printf("Enter your text in lowercase");
    /* read in the lowercase text */
    while ((letter[count] = getchar()) != EOL)
        ++count;
    tag = count;
    /* display the uppercase text */
    count = 0;
    while (count < tag)
    {
        putchar(toupper(letter[count]));
        ++count;
    }
}
```

Notice that *count* is initially assigned a value of zero. Its value increases by 1 during each pass through the first loop. The final value of *count*, at the conclusion of the first loop, is then assigned to *tag*. The value of *tag* determines the number of passes through the second loop. The first **while** loop, i.e.,

```
while ((letter[count] = getchar()) != EOL)
    ++count;
```

is written very concisely. This single-statement loop is equivalent to the following.

```
letter[count] = getchar();
while (letter[count] != EOL)
{
    count = count + 1;
    letter[count] = getchar();
}
```

When the program is executed, any line of text entered the computer will be displayed in uppercase. Suppose, for example, that the following line of text had been entered:

Corona virus killed more than 10,000 people in the Italy.
The computer would respond by printing

CORONA VIRUS KILLED MORE THAN 10,000 PEOPLE IN THE ITALY.

Averaging a List of Numbers Let us now use a **while** statement to calculate the average of a list of n numbers. Our strategy will be based on the use of a partial sum that is initially set equal to zero, then updated as each new number is read into the computer. Thus, the problem very naturally lends itself to the use of a **while** loop. The calculations will be carried out in the following manner (Algorithm).

1. Assign a value of **1** to the integer variable **count**. This variable will be used as a loop counter.
2. Assign a value of **0** to the floating-point variable **sum**.
3. Read in the value for the integer variable **n**.
4. Carry out the following steps repeatedly, as long as **count** does not exceed **n**.
 - a. Read in one of the numbers in the list. Each number will be represented by the floating-point variable **x**.
 - b. Add the value of **x** to the current value of **sum**.
 - c. Increase the value of **count** by **1**,
5. Divide the value of **sum** by **n** to obtain the desired average.
6. Write out the calculated value for the average.

Here is the actual C program. Notice that the input operations are all accompanied by prompts that ask the user for the required information.

```
/* calculate the average of n numbers */
#include <stdio.h>

main()
{
    int n, count = 0;
    float x, average, sum = 0;
    /* initialize and read in a value for n */
    printf("How many numbers?");
    scanf(" %d", &n);
    /* read in the numbers */
    while (count < n)
    {
        printf("x= ");
        scanf(" %f", &x);
        sum += x;
        ++count;
    }
    /* calculating the average and display the answer */
    average = sum / n;
    printf("\nThe average is %f", average);
}
```

Notice that the *while* loop contains a compound statement which, among other things, causes the value of *count* to increase. Eventually, this will cause the logical expression

`count < n`

to become false, thus terminating the loop. Also, note that the loop will not be executed at all if n is assigned a value that is less than 0 (which, of course, would make no sense).

Now suppose that the program will be used to process the following six values: 1, 2, 3, 4, 5, 6. Execution of the program will produce the following interactive dialog. (Note that the user's responses have been underlined.)

```
How many numbers? 6
x = 1
x = 2
x = 3
x = 4
x = 5
x = 6
The average is 3.500000
```

The do-while Statement

When a loop is constructed using the **while** statement, the test for continuation of the loop is carried out at the **beginning** of each pass. Sometimes, however, it is desirable to have a loop with the test for continuation at the **end** of each pass. This can be accomplished by means of the **do - while** statement. The general form of the **do - while** statement is

do statement while (expression) ;

The **statement** will be executed repeatedly, as long as the value of **expression** is true (i.e., is nonzero). Notice that **statement** will always be executed at least once, since the test for repetition does not occur until the end of the first pass through the loop. The **statement** can be either simple or compound, though most applications will require it to be a compound statement. It must include some feature that eventually alters the value of **expression so** the looping action can terminate.

For many applications it is more natural to test for continuation of a loop at the beginning rather than at the end of the loop. For this reason, the **do-while** statement is used less frequently than the **while** statement. For illustrative purposes, however, the programming examples shown in last section are repeated below using the **do-while** statement for the conditional loops.

Consecutive Integer Quantities In the above example, we saw two complete C programs that use the **while** statement to display the consecutive digits 0, 1, 2, . . . , 9. Here is another program to do the same thing, using the **do-while** statement in place of the **while** statement.

```
#include <stdio.h>
main()
{
    int digit = 0;

    do
    {
        printf("%d\n", digit++);
    }while(digit <= 9);
}
```

As in the earlier example, *digit* is initially assigned a value of 0. The **do-while** loop displays the current value of *digit*, increases its value by 1, and then tests to see if the current value of *digit* exceeds 9. If so, the loop terminates; otherwise, the loop continues, using the new value of *digit*. Note that the test is carried out at the end of each pass through the loop.

The net effect is that the loop will be repeated 10 times, resulting in 10 successive lines of output. Each line will appear exactly as shown in the above Example.

Lowercase to Uppercase Text Conversion Now let us rewrite the program shown in the above Example, which converts lowercase text to uppercase, so that the two **while** loops are replaced by **do-while** loops. As in the earlier program, our overall strategy will be to read in a line of lowercase text on a character-by-character basis, store the characters in a char-type array called **letter**, and then write them out in uppercase using the library function **toupper**. We will make use of a **do-while** statement to read in the text on a character-by-character basis, and another **do - while** statement to convert the characters to uppercase and then write them out. Here is the complete C program.

```
/* convert a line of lowercase text to uppercase */
#include <stdio.h>
#include <ctype.h> //toupper()

#define EOL '\n'
main()
{
    char letter[80];
    int tag, count = -1;
    printf("Enter your text in lowercase");
    /* read in the lowercase text */

    do
        ++count;
    while ((letter[count] = getchar()) != EOL)
    tag = count;
    /* display the uppercase text */

    count = 0;
    do
    {
        putchar(toupper(letter[count]));
        ++count;
    } while (count < tag);
}
```

Averaging a List of Numbers The program shown in the previous Example can easily be rewritten to illustrate the use of the **do-while** statement. The logic will be the same, except that the test to determine if all n numbers have been entered into the computer will not be made until the end of the loop rather than the beginning. Thus, the program will always make at least one pass through the loop, even if n is assigned a value of 0 (which would make no sense). Here is the modified version of the program.

```
#include <stdio.h>

main()
{
    int n, count = 0;
    float x, average, sum = 0;
    /* initialize and read in a value for n */
```

```

printf("How many numbers?");
scanf(" %d", &n);
/* read in the numbers */

do
{
    printf("x= ");
    scanf(" %f", &x);
    sum += x;
    ++count;
} while (count < n)

/* calculating the average and display the answer */
average = sum / n;
printf("\nThe average is %f\n", average);
}

```

The for Statement

The **for** statement is the third and perhaps the most used looping statement in C. This statement includes an expression that specifies an initial value for an index, another expression that determines whether the loop is continued, and a third expression that allows the index to be modified at the end of each pass. The general form of the **for** statement is

for (expression 1; expression 2; expression 3) statement

where **expression 1** is used to initialize some parameter (called an **index**) that controls the looping action, **expression 2** represents a condition that must be true for the loop to continue execution, and **expression 3** is used to alter the value of the parameter initially assigned by **expression 1**. Typically, **expression 1** is an assignment expression, **expression 2** is a logical expression and **expression 3** is a unary expression or an assignment expression.

When the **for** statement is executed, **expression 2** is evaluated and tested at the **beginning** of each pass through the loop, and **expression 3** is evaluated at the end of each pass. Thus, the **for** statement is equivalent to

```

expression 1;
while (expression 2)
{
    statement
    expression 3;
}

```

The looping action will continue as long as the value of **expression 2** is not zero, that is, as long as the logical condition represented by **expression 2** is true.

The **for** statement, like the **while** and the **do-while** statements, can be used to carry out looping actions where the number of passes through the loop is not known in advance. Because of the features that are built into the **for** statement, however, it is particularly well suited for loops in which the number of passes is known in advance. As a rough rule of thumb, **while** loops are generally used when the number of passes is **not** known in advance, and **for** loops are generally used when the number of passes is known in advance.

Consecutive Integer Quantities We have already seen several different versions of a C program that will display the consecutive digits 0, 1, 2, . . . , 9, with one digit on each line.

Here is another program which does the same thing. Now, however, we will make use of the **for** statement rather than the **while** statement or the **do-while** statement, as in the earlier examples.

```
#include <stdio.h>
main()
{
    int digit;
    for (digit = 0; digit <= 9; ++digit)
        printf("%d\n", digit);
}
```

The first line of the **for** statement contains three expressions, enclosed in parentheses. The first expression assigns an initial value 0 to the integer variable **digit**; the second expression continues the looping action as long as the current value of **digit** does not exceed 9 at the beginning of each pass; and the third expression increases the value of **digit** by 1 at the end of each pass through the loop. The **printf** function, which is included in the **for** loop, produces the desired output, as shown in the earlier Example.

```
#include <stdio.h>
main()
{
    int digit;
    for (digit = 0; digit <= 9; ++digit )
        printf("%d\n", digit);
}
```

From a syntactic standpoint all three expressions need not be included in the **for** statement, though the semicolons must be present. However, the consequences of an omission should be clearly understood. The first and third expressions may be omitted if other means are provided for initializing the index and or altering the index. If the second expression is omitted, however, it will be assumed to have a permanent value of 1 (true); thus, the loop will continue indefinitely unless it is terminated by some other means, such as a **break** or a **return** statements. As a practical matter, most **for** loops include all three expressions.

Consecutive Integer Quantities Revisited Here is still another example of a C program that generates the consecutive integers 0, 1, 2, ..., 9, with one digit on each line. We now use a **for** statement in which two of the three expressions are omitted.

```
#include <stdio.h>
main()
{
    int digit = 0;
    for (; digit <= 9; )
        printf("%d\n", digit++);
}
```

Lowercase to Uppercase Text Conversion Here once again is a C program that converts lowercase text to uppercase. Now, however, we make use of a **for** loop rather than a **while** loop or a **do-while** loop.

As before, our overall strategy will be to read in a line of lowercase text on a character-by-character basis, store the characters in a char-type array called **letter**, and then write them out in uppercase using the library function **toupper**. Two separate loops will be required: one to

read and store the lowercase characters, the other to display the characters in uppercase. Note that we will now use a **for** statement to build a loop in which the number of passes is not known in advance. Here is the complete C program.

```
/* convert a line of lowercase text to uppercase */
#include <stdio.h>
#include <ctype.h> //toupper()

#define EOL '\n'
main()
{
    char letter[80];
    int tag, count;
    /* read in the lowercase text */
    for (count = 0; (letter[count]=getchar()) != EOL; ++count)
    {}
    tag = count;
    /* display the uppercase text */

    for (count = 0; count < tag; ++count)
    {
        putchar(toupper(letter[count]));
    }
}
```

Averaging a List of Numbers Now let us modify the program given in the earlier Example, which calculates the average of a list of n numbers, so that the looping action is accomplished by means of a **for** statement. The logic will be essentially the same, though some of the steps will be carried out in a slightly different order. In particular:

1. Assign a value of **0** to the floating-point variable *sum*.
2. Read in a value for the integer variable *n*.
3. Assign a value of **1** to the integer variable *count*, where *count* is an index that counts the number of passes through the loop.
4. Carry out the following steps repeatedly, as long as the value of *count* does not exceed *n*.
 - a. Read in one of the numbers in the list. Each number will be represented by the floating-point variable *x*.
 - b. Add the value of *x* to the current value of *sum*.
 - c. Increase the value of *count* by **1**.
5. Divide the value of *sum* by *n* to obtain the desired average.
6. Write out the calculated value for the average.

Here is the complete C program. Notice that steps **3** and **4** are combined in the **for** statement, and that steps **3** and **4(c)** are both carried out in the first line (first and third expressions, respectively).

```
/* calculate the average of n numbers */
#include <stdio.h>

main()
{
```

```

int n, count;
float x, average, sum = 0;
/* initialize and read in a value for n */

printf("How many numbers?");
scanf(" %d", &n);
/* read in the numbers */

for(count = 0; count < n; ++count)
{
    printf("x= ");
    scanf(" %f", &x);
    sum += x;
}
/* calculating the average and display the answer */
average = sum / n;
printf("\n The average is %f\n", average);
}

```

Nested Control Structures

Loops, like if - else statements, can be nested, one within another. The inner and outer loops need not be generated by the same type of control structure. It is essential, however, that one loop be completely embedded within the other -there can be no overlap. Each loop must be controlled by a different index.

Moreover, nested control structures can involve both loops and if - else statements. Thus, a loop can be nested within an if - else statement, and an if - else statement can be nested within a loop. The nested structures may be as complex as necessary, as determined by the program logic.

Repeated Averaging of a List of Numbers Suppose we want to calculate the average of several consecutive lists of numbers. If we know in advance how many lists are to be averaged, then we can use a **for** statement to control the number of times that the inner (averaging) loop is executed. The actual averaging can be accomplished using any of the three methods presented earlier Examples (using a **while**, a **do - while**, or a **for** loop). Let us arbitrarily use the **for** statement to carry out the averaging, as in the last Example. Here is the actual C program.

```

#include <stdio.h>

main()
{
    int n, count, loops, loopcount;
    float x, average, sum;

    /*read in the number of lists */
    printf("How many lists? ");
    scanf(" %d", &loops);

    /* outer loop processes each list of numbers */
    for (loopcount = 1; loopcount <= loops; ++loopcount)
    {

```

```

/* initialize and read in a value for n */
    sum = 0;
    printf("\nList number %d\nHow many numbers? ",
loopcount);
    scanf("%d", &n);
    /* read in the numbers */
    for(count = 1; count <= n; ++count)
    {
        printf("x= ");
        scanf("%f", &x);
        sum += x;
    } //end of inner loop
    /* calculate the average and display the answer */
    average = sum / n;
    printf("\nAverage = %f\n", average);
} //end of outer loop
}

```

Output

How many lists? 3

List number 1

How many numbers? 5

x= 5.4

x= 8.0

x= 2.2

x= 1.7

x= -3.9

Average = 2.680000

List number 2

How many numbers? 3

x= 4

x= -2

x= 7

Average = 3.000000

List number 3

How many numbers? 4

x= 1.5

x= 2.5

x= 6.2

x= 3.0

Average = 3.300000

Converting Several Lines of Text to Uppercase This example illustrates the use of two different types of loops, one nested within the other. Let us extend the lowercase to

uppercase conversion programs presented in the earlier Examples so that multiple lines of lowercase text can be converted to uppercase, with the conversion taking place one line at a time. In other words, we will read in a line of lowercase text, display it in uppercase, then process another line, and so on. The procedure will continue until a line is detected in which the first character is an asterisk.

We will use nested loops to carry out the computation. The outer loop will be used to process multiple lines of text. Two separate inner loops will be embedded within the outer loop. The first will read in a line of text, and the second will display the converted uppercase text. Note that these inner loops are not nested. Let us arbitrarily utilize a **while** statement for the outer loop, and a **for** statement for each of the inner loops.

```
#include <stdio.h>
#include <ctype.h>

#define EOL '\n'

main()
{
    char letter[80];
    int tag, count;

    printf("\nEnter your text here.");
    while((letter[0] = getchar()) != '\*')
    {
        /* read in a line of text */
        for (count=1; (letter[count]=getchar()) != EOL;
            ++count)
        {}
        tag = count;
        /* display the line of text */
        for (count =0; count < tag; ++count)
            putchar(toupper(letter[count]));
        printf("\n Enter your text here.");
    }
    printf("\n Goodbye");
}
```

Output

Enter your text here.

Corona Virus

CORONA VIRUS

Enter your text here.

Aids

AIDS

Enter your text here.

Ebola virus

EBOLA VIRUS

Enter your text here.

*

Goodbye

Encoding a String of Characters Let us write a simple C program that will read in a sequence of **ASCII** characters and write out a sequence of encoded characters in its place. If a character is a letter or a digit, we will replace it with the next character in the character set, except that **Z** will be replaced by **A**, **z** by **a**, and **9** by **0**. Thus **1** becomes **2**, **C** becomes **D**, **p** becomes **q**, and so on. Any character other than a letter or a digit will be replaced by a period (.).

The computation will begin by reading in the characters. The `scanf()` will be used for this purpose. All the characters, up to but not including the newline (**\n**) character that is used to terminate the input, will be entered and stored in an 80-element, character-type array called *line*. The characters will then be encoded and displayed individually within a *for* loop. The loop will process each of the characters in *line*, until the escape character `\0`, which designates the end of the character sequence, is encountered. (Recall that the escape sequence `\0` is automatically added at the end of each string.) Several nested if - else statements will be included within the loop, to carry out the appropriate encoding. Each encoded character will then be displayed using the *putchar* function.

The complete C program is shown below.

```
#include <stdio.h>

#define EOL '\0'
main()
{
    char line[80];
    int count;
    /* read in the entire string */
    printf("Enter a line of text below:\n");
    scanf(" %[^\\n]", line);

    /* encode each individual character and display it */
    for (count = 0; line[count] != EOL ; ++count)
    {
        if (((line[count] >= '0') && (line[count] < '9')) ||
            ((line[count] >= 'a') && (line[count] < 'z')) ||
            ((line[count] >= 'A') && (line[count] < 'Z')))
        {
            putchar(line[count] + 1);
        }
        else if (line[count] == '9') putchar('0');
            else if (line[count] == 'z') putchar('a');
                else if (line[count] == 'Z') putchar('A');
                    else putchar('.');
    }
}
```

Output

```
Enter a line of text below:
UnitedTechnicalCollege990@gmail.com
VojufeUfdiojdbmDpmmfhf001.hnbjm.dpn
```

The switch Statement

The switch statement causes a particular group of statements to be chosen from several available groups. The selection is based upon the current value of an expression which is included within the switch statement. The general form of the switch statement is

switch (expression) statement

where *expression* results in an *integer* value. Note that *expression* may also be of type *char*, since individual characters have equivalent integer values.

The embedded *statement* is generally a compound statement that specifies alternate courses of action. Each alternative is expressed as a group of one or more individual statements within the overall embedded *statement*.

For each alternative, the first statement within the group must be preceded by one or more **case labels** (also called **case prefixes**). The case labels identify the different groups of statements (i.e., the different alternatives) and distinguish them from one another. The case labels must therefore be unique within a given switch statement. In general terms, each group of statements is written as

```
case expression:
    statement 1
    statement 2
    .....
    statement n
```

or, when multiple case labels are required,

```
case expression 1:
case expression 2:
.....
case expression m:
    statement 1
    statement 2
...
    statement n
```

where *expression 1*, *expression 2*, . . . , *expression m* represent constant, integer-valued expressions. Usually, each of these expressions will be written as either an integer constant or a character constant. Each individual *statement* following the case labels may be either simple or complex.

When the switch statement is executed, the *expression* is evaluated, and control is transferred directly to the group of statements whose *case-label* value matches the value of the *expression*. If none of the case-label values matches the value of the *expression*, then none of the groups within the switch statement will be selected. In this case control is transferred directly to the statement that follows the switch statement.

A simple switch statement is illustrated below. In this example, choice is assumed to be a char-type variable.

```
switch (choice = getchar())
{
    case 'r':
    case 'R':
```

```

        printf("red");
        break;
    case 'w':
    case 'W':
        printf("white");
        break;
    case 'b':
    case 'B':
        printf("blue");
}

```

Thus, *RED* will be displayed if **choice** represents either *r* or *R*, *WHITE* will be displayed if **choice** represents either *w* or *W*, and *BLUE* will be displayed if **choice** represents either *b* or *B*. Nothing will be displayed if any other character has been assigned to **choice**.

Notice that each group of statements has two case labels, to account for either upper or lowercase. Also, note that each of the first two groups ends with the **break** statement. The **break** statement causes control to be transferred out of the **switch** statement, thus preventing more than one group of statements from being executed.

One of the labeled groups of statements within the **switch** statement may be labeled **default**. This group will be selected if none of the case labels matches the value of the **expression**. (This is a convenient way to generate error messages or error correction routines.) The **default** group may appear anywhere within the **switch** statement-it need not necessarily be placed at the end. If none of the case labels matches the value of the **expression** and the **default** group is not present (as in the above example), then no action will be taken by the **switch** statement.

```

switch (choice = toupper(getchar()))
{
    case 'R':
        printf("red");
        break;
    case 'W':
        printf("white");
        break;
    case 'B':
        printf("blue");
        break;
    default:
        printf("Error");
}

```

The **switch** statement now contains a **default** group (consisting of only one statement), which generates an error message if none of the case labels matches the original **expression**.

Each of the first three groups of statements now has only one case label. Multiple case labels are not necessary in this example, since the library function **toupper** causes all incoming characters to be converted to uppercase. Hence, **choice** will always be assigned an uppercase character.

Here is another typical **switch** statement. In this example **flag** is assumed to be an integer variable, and **x** and **y** are assumed to be floating-point variables.

```

switch (flag)
{

```

```

case -1:
    y = abs(x);
    break;
case 0:
    y = sqrt(x);
    break;
case 1:
    y = x;
    break;
case 2:
case 3:
    y = 2 * (x - 1);
    break;
default:
    y = 0;
}

```

In this example, **y** will be assigned some value that is related to the value of **x** if **flag** equals -1, 0, 1, 2 or 3. The exact relationship between **y** and **x** will depend upon the particular value of **flag**. If **flag** represents some other value, however, then **y** will be assigned a value of 0. Notice that the case labels are numeric in this example. Also, note that the third group of statements has two case labels, whereas each of the other groups have only one case label. And finally, notice that a default group (consisting of only one statement) is included within this **switch** statement.

In a practical sense, the **switch** statement may be thought of as an alternative to the use of nested **if - else** statements, though it can only replace those **if - else** statements that test for *equality*. In such situations, the use of the **switch** statement is generally much more convenient.

The Break Statement

The **break** statement is used to *terminate loops* or to *exit from a switch*. It can be used within a *for*, *while*, *do - while*, or *switch* statement. The break statement is written simply as

break;

without any embedded expressions or statements.

We have already seen several examples of the use of the break statement within a switch statement, in the last section. The **break** statement causes a transfer of control out of the entire switch statement, to the first statement following the switch statement.

Consider once again the **switch** statement originally presented in the last Example.

```

switch (choice = toupper(getchar()))
{
    case 'R':
        printf("red");
        break;
    case 'W':
        printf("white");
        break;
    case 'B':
        printf("blue");
        break;
}

```



```

    default:
        printf("Error");
        break;
}

```

Notice that each group of statements ends with a **break** statement, in order to transfer control out of the **switch** statement. The **break** statement is required within each of the first three groups, in order to prevent the succeeding groups of statements from executing. The last group does not require a **break** statement, since control will automatically be transferred out of the **switch** statement after the last group has been executed. This last **break** statement is included, however, as a matter of good programming practice, so that it will be present if another group of statements is added later.

If a break statement is included in a *while*, *do - while* or *for* loop, then control will immediately be transferred out of the loop when the break statement is encountered. This provides a convenient way to terminate the loop if an error or other irregular condition is detected.

Here are some illustrations of loops that contain **break** statements. In each situation, the loop will continue to execute as long as the current value for the floating-point variable *x* does not exceed **100**. However, the computation will break out of the loop if a negative value for *x* is detected.

First, consider a **while** loop.

```

scanf(" %f", &x); //initialize
while (x <= 100)
{
    if (x < 0)
    {
        printf("ERROR! NEGATIVE VALUE FOR x");
        break;
    }
    /* process the non-negative value of x */
    ...
    scanf(" %f", &x); //update
}

```

Now consider a *do - while* loop that does the same thing.

```

do
{
    scanf(" %f", &x);
    if (x < 0)
    {
        printf("ERROR! NEGATIVE VALUE FOR x");
        break;
    }
    /* process the non-negative value of x */
    ...
}while (x <= 100);

```

Finally, here is a *for* loop that is similar.

```

for (count = 1; x <= 100; ++count)

```

```

{
    scanf(" %f", &x);
    if (x < 0)
    {
        printf("ERROR! NEGATIVE VALUE FOR x");
        break;
    }
    /* process the non-negative value of x */
    ...
}

```

In the event of several *nested while*, *do - while*, *for* or *switch* statements, a **break** statement will cause a transfer of control out of the immediate enclosing statement, but not out of the outer surrounding statements.

Consider the following outline of a **while** loop embedded within a **for** loop.

```

for (count = 0; count <= n; ++count)
{
    ...
    while (c = getchar() != '\n')
    {
        if (c == '*')
            break;
        ...
    }
}

```

If the character variable *c* is assigned an asterisk (*), then the **while** loop will be terminated. However, the **for** loop will continue to execute. Thus, if the value of *count* is less than *n* when the breakout occurs, the computer will increment *count* and make another pass through the **for** loop.

The continue Statement

The *continue* statement is used to *bypass* the remainder of the current pass through a loop. The loop does not terminate when a *continue* statement is encountered. Rather, the remaining loop statements are skipped and the computation proceeds directly to the next pass through the loop. (Note the distinction between *continue* and *break*.)

The *continue* statement can be included within a *while*, a *do - while* or a *for* statement. It is written simply as

continue;

without any embedded statements or expressions.

Here are some illustrations of loops that contain *continue* statements. First, consider a *do - while* loop.

```

do
{
    scanf(" %f", &x);
    if (x < 0)
    {
        printf("ERROR! NEGATIVE VALUE FOR x");
        continue;
    }
}

```

```

    }
    /* process the non-negative value of x */
    ...
}while (x <= 100);

```

Here is a similar *for* loop

```

for (count = 1; x <= 100; ++count)
{
    scanf(" %f", &x);
    if (x < 0)
    {
        printf("ERROR! NEGATIVE VALUE FOR x");
        continue;
    }
    /* process the non-negative value of x */
    ...
}

```

In each case, the processing of the current value of *x* will be bypassed if the value of *x* is negative. Execution of the loop will then continue with the next pass.

Averaging a List of Nonnegative Numbers In the previous Example, we saw a complete C program that uses a *for* loop to calculate the average of a list of *n* numbers. Let us now modify this program so that it processes only nonnegative numbers.

The earlier program requires two minor changes to accommodate this modification. First, the *for* loop must include an *if* statement to determine whether each new value of *x* is *nonnegative*. A ***continue*** statement will be included in the *if* statement to bypass the processing of negative values of *x*. Secondly, we require a special counter (***navg***) to determine how many nonnegative numbers have been processed. This counter will appear in the denominator when the average is calculated (i.e., the average will be determined as *average* = ***sum*** / ***navg***). Here is the actual C program.

```

/* calculate the average of the non-negative numbers in a list
of n numbers */
#include <stdio.h>

main()
{
    int n, count, navg = 0;
    float x, average, sum = 0;

    /* initialize and read in a value for n */
    printf("How many numbers? ");
    scanf(" %d", &n);

    /* read in the numbers */
    for (count = 1; count <= n; ++count)
    {
        printf("x= ");
        scanf(" %f", &x);
        if (x < 0) continue;
    }
}

```

```

        sum += x;
        ++navg;
    }
    /* calculate the average and write out the answer */
    average = sum / navg;
    printf("\nThe average is %f\n", average);
}

```

Output

How many numbers?6

```

x= 1
x= -1
x= 2
x= -2
x= 3
x= -3

```

The average is 2.000000

This is the correct average of the positive numbers. Note that the average would be *zero* if all of the numbers had been averaged.

The goto Statement

The *goto* statement is used to alter the normal sequence of program execution by transferring control to some other part of the program. In its general form, the *goto* statement is written as

goto label;

where *label* is an identifier that is used to label the target statement to which control will be transferred. Control may be transferred to any other statement within the program. (To be more precise, control may be transferred anywhere within the current *function*. The target statement must be labeled, and the label must be followed by a colon. Thus, the target statement will appear as

label: statement

Each labeled statement within the program (more precisely, within the current function) must have a unique label; i.e., no two statements can have the same label.

The following skeletal outline illustrates how the *goto* statement can be used to transfer control out of a loop if an unexpected condition arises.

```

/* main function */
scanf(" %f", &x);
while (x <= 100)
{
    ...
    if (x < 0) goto errorcheck;
    ...
    scanf(" %f", &x);
}
...
/* error detection routine */
errorcheck:
{

```

```
printf("ERROR! NEGATIVE VALUE FOR x");
...
}
```

In this example control is transferred out of the while loop, to the compound statement whose label is **errorcheck**, if a negative value is detected for the input variable **x**.

The same thing could have been accomplished using the **break** statement, as illustrated in the last Example. The use of the **break** statement is actually the preferred approach. The use of the **goto** statement is presented here only to illustrate the syntax.

All of the popular general-purpose programming languages contain a **goto** statement, though modern programming practice discourages its use. The **goto** statement was used extensively, however, in early versions of some older languages, such as Fortran and BASIC. The most common applications were:

1. Branching around statements or groups of statements under certain conditions.
2. Jumping to the end of a loop under certain conditions, thus bypassing the remainder of the loop during the current pass.
3. Jumping completely out of a loop under certain conditions, thus terminating the execution of a loop.

The structured features in C enable all these operations to be carried out without resorting to the **goto** statement. For example, branching around statements can be accomplished with the **if - else** statement; jumping to the end of a loop can be carried out with the **continue** statement; and jumping out of a loop is easily accomplished using the **break** statement. The use of these structured features is preferable to the use of the **goto** statement, because the use of **goto** tends to encourage (or at least, not discourage) logic that skips all over the program whereas the structured features in C require that the entire program be written in an orderly, sequential manner. For this reason, **use of the goto statement should generally be avoided**.

Occasional situations do arise, however, in which the **goto** statement can be useful. Consider, for example, a situation in which it is necessary to jump out of a *doubly nested loop* if a certain condition is detected. This can be accomplished with two **if - break** statements, one within each loop, though this is awkward. A better solution in this situation might make use of the **goto** statement to transfer out of both loops at once. The procedure is illustrated in the following example.

```
#include <stdio.h>
#include <ctype.h>

#define EOL '\n'

main()
{
    char letter[80];
    int tag, count, linecount = 1;

    while ((letter[0] = getchar()) != '*')
    {
        /* read in a line of text */
        for(count=1; (letter[count]=getchar()) != EOL;
++count)
        {}
        tag = count;
```

```

/* display the line of text */
for(count = 0; count < tag; ++count)
    putchar(toupper(letter[count]));
printf("\n");
++linecount;

/* test for a break condition */
for (count = 1; count < tag; ++count)
    if(letter[count-1]== '$' && letter[count] ==
'$')
        {
            printf("\nBREAK CONDITION IS DETECTED-
TERMINATE EXECUTION\n");
            goto end;
        }
    }
end:
    printf("\nGoodBye!");
}

```

Output

```

my country Nepal
MY COUNTRY NEPAL
corona virus
CORONA VIRUS
pullup$$
PULLUP$$

```

```
BREAK CONDITION IS DETECTED- TERMINATE EXECUTION
```

```
GoodBye!
```

It is interesting to compare this program with the corresponding program presented earlier, in the Example. The present program contains an additional **for** loop embedded at the end of the **while** loop. This **for** loop examines consecutive pairs of characters for a break condition (\$\$), after the entire line has already been written out in uppercase. If a break condition is encountered, then control is transferred to the final **printf** statement (“Goodbye”) which is now labeled end. Note that this transfer of control causes a breakout from the **if** statement, the current **for** loop, and the outer while loop.

You should run this program, using both the regular terminating condition (an asterisk at the start of a new line) and the breakout condition. Compare the results obtained with the output shown in the earlier Example.

The Comma Operator

We now introduce the *comma* operator (,) which is used primarily in conjunction with the **for** statement. This operator permits two different expressions to appear in situations where only one expression would ordinarily be used. For example, it is possible to write

```
for(expression 1a,expression 1b;expression 2;expression 3) statement
```

where *expression 1a* and *expression 1b* are the two expressions, separated by the *comma* operator, where only one expression (*expression 1*) would normally appear. These two expressions would typically initialize two separate indices that would be used simultaneously within the **for** loop.

Similarly, a **for** statement might make use of the comma operator in the following manner.

```
for(expression 1;expression 2;expression 3a, expression 3b)
statement
```

Here *expression 3a* and *expression 3b*, separated by the comma operator, appear in place of the usual single expression. In this application the two separate expressions would typically be used to alter (e.g., increment or decrement) two different indices that are used simultaneously within the loop. For example, one index might count forward while the other counts backward.

Searching for Palindromes A *palindrome* is a word, phrase or sentence that reads the same way either forward or backward. For example, words such as *noon*, *peep*, and *madam* are palindromes. If we disregard punctuation and blank spaces, then the sentence *Rise to vote, sir!* is also a palindrome.

Let us write a C program that will enter a line of text containing a word, a phrase or a sentence, and determine whether the text is a palindrome. To do so, we will compare the *first* character with the *last*, the *second* character with the *next to last*, and so on, until we have reached the *middle* of the text. The comparisons will include punctuation and blank spaces.

In order to outline a computational strategy, let us define the following variables.

Letter = a character-type array containing as many as 80 elements.

Tag = an integer variable indicating the number of characters assigned to letter, excluding the escape character \0 at the end.

Count = an integer variable used as an index when moving forward through letter.

Countback = an integer variable used as an index when moving backward through letters.

Flag = an integer variable that will be used to indicate a true/false condition. True will indicate that a palindrome has been found.

Loop = an integer variable whose value will always equal 1, thus appearing always to be true. The intent here is to continue execution of a main loop, until a particular stopping condition causes a breakout.

```
/* search for a palindrome */
#include <stdio.h>
#include <ctype.h>

#define EOL '\n'
#define TRUE 1
#define FALSE 0

main()
{
    char letter[80];
    int tag, count, countback, flag, loop = TRUE;
    /*main loop*/
    while(loop)        //while(1)
    {
        flag = TRUE;
        /* read the text */
```

```

        printf("\nPlease enter a word, phrase or sentence
below:\n");
        for (count = 0; (letter[count] = getchar()) != EOL;
++count)
        {}
        if ((toupper(letter[0]) == 'E') &&
(toupper(letter[1]) == 'N') && (toupper(letter[2]) == 'D'))
                break;
        tag = count-1;
        /* carry out search */
        for((count = 0, countback = tag); count <=tag/2;
(++count, --countback))
        {
                if (letter[count] != letter[countback])
                {
                        flag = FALSE;
                        break;
                }
        }
        /*display message*/
        for (count = 0; count <= tag; ++count)
                putchar(letter[count]);
        if (flag) printf(" is a palindrome.\n");
        else printf(" is not a palindrome.\n");
    }
}

```

Output:

Please enter a word, phrase or sentence below:
pullup
pullup is a palindrome.

Please enter a word, phrase or sentence below:
false
false is not a palindrome.

Please enter a word, phrase or sentence below:
toot
toot is a palindrome.

Please enter a word, phrase or sentence below:
End

Remember that the comma operator accepts two distinct expressions as operands. These expressions will be evaluated from left to right. In situations that require the evaluation of the overall expression (i.e., the expression formed by the two operands and the comma operator), the type and value of the overall expression will be determined by the type and value of the right operand.

Within the collection of C operators, the comma operator has the *lowest precedence*. Thus, the comma operator falls within its own unique precedence group, beneath the precedence group containing the various assignment operators. Its associativity is *left to right*.

5. Functions

We have already seen that C supports the use of library functions, which are used to carry out several commonly used operations or calculations. However, C also allows programmers to define their own functions for carrying out various individual tasks. This chapter concentrates on the creation and utilization of such programmer-defined functions.

The use of programmer-defined functions allows a large program to be broken down into a few smaller, self-contained components, each of which has some unique, identifiable purpose. Thus, a C program can be **modularized** through the intelligent use of such functions. There are several *advantages* to this modular approach to program development. For example, many programs require that a group of instructions be accessed repeatedly, from several different places within the program. The repeated instructions can be placed within a single function, which can then be accessed whenever it is needed. Moreover, a different set of data can be transferred to the function each time it is accessed. *Thus, the use of a function avoids the need for redundant (repeated) programming of the same instructions.*

Equally important is the **logical clarity** resulting from the decomposition of a program into several concise functions, where each function represents some well-defined part of the overall problem. Such programs are easier to write and easier to debug, and their logical structure is more apparent than programs which lack this type of structure. This is especially true of lengthy, complicated programs. Most C programs are therefore modularized in this manner, even though they may not involve repeated execution of the same tasks. In fact, the decomposition of a program into individual program modules is generally considered to be an important part of good programming practice.

The use of functions also enables a programmer to build a **customized library** of frequently used routines or of routines containing system-dependent features. Each routine can be programmed as a separate function and stored within a special library file. If a program requires a particular routine, the corresponding library function can be accessed and attached to the program during the compilation process. Hence a single function can be utilized by many different programs. This avoids repetitive programming between programs. It also promotes **portability** since programs can be written that are independent of system-dependent features.

In this chapter we will see how functions are defined and how they are accessed from various places within a C program. We will then consider the way how information is passed to a function.

A Brief Overview

A **function** is a self-contained program segment that carries out some specific, well-defined task. Every C program consists of one or more functions. One of these functions must be called **main**. Execution of the program will always begin by carrying out the instructions in **main**. Additional functions will be subordinate to **main**, and perhaps to one another. If a program contains multiple functions, their definitions may appear in any order, though they must be independent of one another. That is, one function definition cannot be embedded within another.

A function will carry out its intended action whenever it is *accessed* (i.e., whenever the function is "called") from some other portion of the program. The same function can be accessed from several different places within a program. Once the function has carried out its intended action, control will be returned to the point from which the function was accessed. Generally, *a function will process information that is passed to it from the calling portion of the program and return a single value.* Information is passed to the function via special identifiers called **arguments** (also called **parameters**) and returned via the **return** statement. Some functions, however, accept information but do not return anything (as, for example, the library function **printf**), whereas other functions (e.g., the library function **scanf**) return multiple values.

Defining a Function

A function definition has two principal components: the *first line* (including the *argument declarations*), and the *body* of the function.

The first line of a function definition contains the type specification of the **value returned** by the function, followed by the **function name**, and (optionally) a **set of arguments**, separated by **commas** and enclosed in **parentheses**. Each argument is preceded by its associated type declaration. An empty pair of parentheses must follow the function name if the function definition does not include any arguments.

In general terms, the *first line* can be written as

data-type name(type 1 arg 1, type 2 arg 2, ..., type n arg n)

where **data - type** represents the data type of the item that is returned by the function, **name** represents the function name, and **type 1, type 2, .. , type n** represent the data types of the arguments **arg 1, arg 2, ..., arg n** respectively. The data types are assumed to be of type **int** if they are not shown explicitly. However, the omission of the data types is considered poor programming practice, even if the data items are integers.

The arguments are called **formal arguments**, because they represent the names of data items that are transferred into the function from the calling portion of the program. They are also known as **parameters** or **formal parameters**. (The corresponding arguments in the function *reference* are called **actual arguments**, since they define the data items that are transferred. Some textbooks refer to actual arguments simply as *arguments*, or as *actual parameters*.)

The identifiers used as formal arguments are "**local**" in the sense that they are not recognized outside of the function. Hence, the names of the formal arguments *need not* be the same as the names of the actual arguments in the calling portion of the program. Each formal argument must be of the same *data type*, however, as the data item it receives from the calling portion of the program.

The remainder of the function definition is a compound statement that defines the action to be taken by the function. This compound statement is sometimes referred to as the **body** of the function. Like any other compound statement, this statement can contain expression statements, other compound statements, control statements, and so on. It should include one or more return statements, in order to return a value to the calling portion of the program. A function can access other functions. In fact, it can even access itself (this process is known as *recursion*).

Consider the function **lowerToUpper**

```
char lowerToUpper(char c1) /* programmer-defined conversion
function */
```

```

{
    char c2;

    c2 = (c1 >= 'a' && c1 <= 'z') ? (c1 - 32) : c1;
    return (c2);
}

```

The first line contains the function name, **lowerToUpper**, followed by the formal argument **c1**, enclosed in parentheses. The function name is preceded by the data type **char**, which describes the data item that is returned by the function. In addition, the formal argument **c1** is preceded by the data type **char**. This later data type, which is included within the pair of parentheses, refers to the *formal argument*. The formal argument, **c1**, represents the lowercase character that is transferred to the function from the calling portion of the program. The body of the function begins on the second line, with the declaration of the local char-type variable **c2**. (Note the distinction between the formal argument **c1**, and the local variable **c2**.) Following the declaration of **c2** is a statement that tests whether **c1** represents a lowercase letter and then carries out the conversion. The original character is returned intact if it is not a lowercase letter. Finally, the **return** statement (see below) causes the converted character to be returned to the calling portion of the program.

Information is returned from the function to the calling portion of the program via the **return** statement. The **return** statement also causes the program logic to return to the point from which the function was accessed. In general terms, the **return** statement is written as

return expression;

The value of the **expression** is returned to the calling portion of the program. The **expression** is optional. If the **expression** is omitted, the **return** statement simply causes control to revert to the calling portion of the program, without any transfer of information.

Only one expression can be included in the **return** statement. Thus, a function can return only one value to the calling portion of the program via return.

A function definition can include multiple **return** statements, each containing a different expression. Functions that include multiple branches often require multiple returns.

Here is a variation of the function **lowerToUpper**

```

char lowerToUpper(char c1)
{
    if (c1 >= 'a' && c1 <= 'z')
        return (c1 - 32);
    else
        return c1;
}

```

This function utilizes the **if - else** statement rather than the conditional operator. It is somewhat less compact than the original version, though the logic is clearer. In addition, note that this form of the function does not require the local variable **c2**. This function contains two different **return** statements. The first returns an expression that represents the uppercase equivalent of the lowercase character; the second returns the original lowercase character, unchanged.

The **return** statement can be absent altogether from a function definition, though this is generally regarded as poor programming practice. If a function reaches the end without encountering a **return** statement, control simply reverts to the calling portion of the program without returning any information. The presence of an empty **return** statement (without the

accompanying expression) is recommended in such situations, to clarify the logic and to accommodate future modifications to the function.

The following function accepts two integer quantities and determines the larger value., which is then displayed. The function does not return any information to the calling program.

```
maximum(int x, int y)
{
    int z;

    z = (x >= y) ? x : y;
    printf("\nThe maximum value is %d", z);
    return;
}
```

Notice that an empty **return** statement is included, as a matter of good programming practice. The function would still work properly, however, if the **return** statement were not present.

The function shown below calculates the factorial of a given positive integer **n**. The factorial is returned as a long integer quantity, since factorials grow in magnitude very rapidly as **n** increases. (For example, **8! = 40,320.**)

```
long int factorial(int n) /* calculates the factorial of n */
{
    int i;
    long int prod = 1;

    if (n > 1)
    {
        for (i = 1; i <= n; ++i)
            prod *= i;
    }
    return prod;
}
```

If the data type specified in the first line is *inconsistent* with the expression appearing in the **return** statement, the compiler will attempt to convert the quantity represented by the expression to the data type specified in the first line. This could result in a compilation error, or it may involve a partial loss of data (e.g., due to *truncation*). In any event, inconsistencies of this type should be avoided.

The following function definition is identical to that in the last Example except that the first line does not include a type specification for the value that is returned by the function.

```
factorial(int n) /* calculates the factorial of n */
{
    int i;
    long int prod = 1;

    if ( n > 1)
    {
        for (i = 1; i <= n; ++i)
            prod *= i;
    }
    return prod;
}
```

```
}
```

The function expects to return an ordinary integer quantity, since there is no explicit type declaration in the first line of the function definition. However, the quantity being returned (*prod*) is declared as a long integer within the function. This inconsistency can result in an error. The problem can be avoided, however, by adding a *long int* type declaration to the first line of the function definition.

The keyword **void** can be used as a type *specifier* when defining a function that does not return anything, or when the function definition does not include any arguments. The presence of this keyword is not mandatory, but it is good programming practice to make use of this feature.

Consider once again the function presented in the earlier Example, which accepts two integer quantities and displays the larger of the two. Recall that this function does not return anything to the calling portion of the program. Therefore, the function can be written as

```
void maximum(int x, int y)
{
    int z;

    z = (x >= y) ? x : y;
    printf("\nThe maximum value is %d", z);
    return;
}
```

Accessing a Function

A function can be *accessed* (i.e., **called**) by specifying its name, followed by a list of arguments enclosed in parentheses and separated by commas. If the function call does not require any arguments, an empty pair of parentheses must follow the name of the function. The function call may be a part of a simple expression (such as an assignment statement), or it may be one of the operands within a more complex expression.

The arguments appearing in the function call are referred to as **actual arguments**, in contrast to the *formal arguments* that appear in the first line of the function definition. (They are also known simply as *arguments*, or as *actual parameters*.) In a normal function call, there will be one actual argument for each formal argument. The actual arguments may be expressed as constants, single variables, or more complex expressions. However, each actual argument must be of the same data type as its corresponding formal argument. Remember that it is the *value* of each actual argument that is transferred into the function and assigned to the corresponding formal argument.

If the function returns a value, the function access is often written as an assignment statement; e.g.,

```
y = polynomial(x);
```

This function access causes the value returned by the function to be assigned to the variable *y*.

On the other hand, if the function does not return anything, the function access appears by itself; e.g.,

```
display(a, b, c);
```

This *function access* causes the values of *a*, *b* and *c* to be processed internally (i.e., displayed) within the function.

Consider once again the program originally shown in the Example, which reads in a single lowercase character, converts it to uppercase using a programmer-defined function, and then displays the uppercase equivalent.

```

#include <stdio.h>
char lowerToUpper(char c1)
{
    char c2;

    c2 = (c1 >= 'a' && c1 <= 'z') ? (c1 - 32) : c1;
    return c2;
}
void main(void)
{
    char lower, upper;

    printf("Please enter a lowercase character: ");
    scanf(" %c", &lower);
    upper = lowerToUpper(lower);
    printf("\nThe uppercase equivalent is %c\n", upper);
}

```

The function call contains one actual argument, the char-type variable **lower**. Note that the corresponding formal argument, **c1**, within the function definition is also a char-type variable.

When the function is accessed, the value of **lower** to be transferred to the function. This value is represented by **c1** within the function. The value of the uppercase equivalent, **c2**, is then determined and returned to the calling portion of the program, where it is assigned to the char-type variable **upper**.

The last two statements in **main** can be combined to read

```
printf("\nThe uppercase equivalent is %c\n", lowerToUpper(lower));
```

The call to **lowerToUpper** is now an actual argument for the library function **printf**. Also, note that the variable **upper** is no longer required.

Finally, notice the way the first line of main is written, i.e., **void main (void)**. This is permitted under the ANSI standard, though some compilers do not accept the **void** return type. Hence, many authors (and many programmers) write the first line of main as **main(void)**, or simply **main()**.

There may be several different calls to the same function from various places within a program. The actual arguments may differ from one function call to another. Within each function call, however, *the actual arguments must correspond to the formal arguments in the function definition; i.e., the number of actual arguments must be the same as the number of formal arguments, and each actual argument must be of the same data type as its corresponding formal argument.*

Largest of Three Integer Quantities The following program determines the largest of three integer quantities. This program makes use of a function that determines the larger of two integer quantities. The function is like that defined in the previous Example, except that the present function returns the larger value to the calling program rather than displaying it. The overall strategy is to *determine the larger of the first two quantities, and then compare this value with the third quantity.* The largest quantity is then displayed by the *main* part of the program.

```

/* determine the largest of three integer quantities */
#include <stdio.h>

```

```
int maximum(int x, int y)
```

Compiled by: **Er. Rupesh Shrestha** rupeshshrestha64@gmail.com

www.arstudymaterials.com

```

{
    int z;
    z = (x >= y) ? x : y;
    return (z);
}
main()
{
    int a, b, c, d;
    /* read the integer quantities */
    printf("\na = ");
    scanf(" %d", &a);
    printf("\nb = ");
    scanf(" %d", &b);
    printf("\nc = ");
    scanf(" %d", &c);

    /* calculate and display the maximum value */
    d = maximum(a, b);
    printf("\nMaximum = %d", maximum(c, d));
}

```

The function **maximum** is accessed from two different places in **main**. In the first call to **maximum** the actual arguments are the variables **a** and **b**, whereas the arguments are **c** and **d** in the second call (**d** is a temporary variable representing the maximum value of **a** and **b**). Note the two statements in main that access **maximum**, i.e.,

```

d = maximum(a, b);
printf("\nMaximum = %d", maximum(c, d));

```

These two statements can be replaced by a single statement; e.g.,

```

printf("\nMaximum = %d", maximum(c, maximum(a, b)));

```

In this statement we see that one of the calls to **maximum** is an argument for the other call. Thus, the calls are embedded, one within the other, and the intermediary variable, **d**, is not required. Such embedded function calls are permissible, though their logic may be unclear.

Function Prototypes

In the programs that we have examined the programmer-defined function has always *preceded* main. Thus, when these programs are compiled, the programmer-defined function will have been defined before the first function access. However, many programmers prefer a "**top-down**" approach, in which main appears ahead of the programmer-defined function definition. In such situations the function access (within main) will precede the function definition. This can be confusing to the compiler, unless the compiler is first alerted to the fact that the function being accessed will be defined later in the program. A *function prototype* is used for this purpose.

Function prototypes are usually written at the beginning of a program, ahead of any programmer-defined functions (including main). The general form of a function prototype is

data-type name(type 1 arg 1, type 2 arg 2..., type n arg n);

where *data-type* represents the data type of the item that is returned by the function, *name* represents the function name, and *type 1*, *type 2*, ..., *type n* represent the data types of the

arguments *arg 1*, *arg 2*, ..., *arg n* respectively. Notice that a function prototype resembles the first line of a function definition (though a function prototype ends with a semicolon).

The names of the arguments within the function prototype need not be declared elsewhere in the program, since these are "***dummy***" argument names that are recognized only within the prototype. In fact, the argument names can be omitted (though it is not a good idea to do so); however, the argument *data types* are essential.

In practice, the argument names are usually included and are often the same as the names of the actual arguments appearing in one of the function calls. The data types of the actual arguments must conform to the data types of the arguments within the prototype.

Function prototypes are not mandatory in C. They are desirable, however, because they further facilitate error checking between the calls to a function and the corresponding function definition.

```
/* calculate the factorial of an integer quantity */
#include <stdio.h>

long int factorial(int n);    /* function prototype */
main()
{
    int n;
    /* read in the integer quantity */
    printf("\n n= ");
    scanf(" %d", &n);

    /* calculate and display factorial */
    printf("\n n! = %ld", factorial(n));
}
long int factorial(int n)
/* calculate the factorial of n */
{
    int i;
    long int prod = 1;

    if (n > 1)
    {
        for (i= 1; i <= n; ++i)
            prod *= i;
    }
    return prod;
}
```

Function calls can span several levels within a program. That is, *function A* can call *function B*, which can call *function C*, etc. Also, *function A* can call *function C* directly, and so on.

Passing Arguments to a Function

When a single value is passed to a function via an actual argument, the value of the actual argument is *copied* into the function. Therefore, *the value of the corresponding formal argument can be altered within the Function, but the value of the actual argument within the*

calling routine will not change. This procedure for passing the value of an argument to a function is known as *passing by value*.

Here is a simple C program containing a function that alters the value of its argument.

```
#include <stdio.h>

void modify(int a);

main()
{
    int a = 2;

    printf("\na = %d (from main, before calling the function)", a);
    modify(a) ;
    printf("\na = %d (from main, after calling the function)", a);
}

void modify(int a)
{
    a *= 3;
    printf("\na = %d (from the function, after being modified)", a);
    return;
}
```

The original value of *a* (i.e., **a = 2**) is displayed when main begins execution. This value is then passed to the function *modify*, where it is multiplied by **3** and the new value displayed. Note that it is the altered value of the formal argument that is displayed within the function. Finally, the value of *a* within main (i.e., the actual argument) is again displayed, after control is transferred back to main from *modify*. When the program is executed, the following output is generated.

```
a = 2 (from main, before calling the function)
a = 6 (from the function, after being modified)
a = 2 (from main, after calling the function)
```

These results show that ***a*** is not altered within main, even though the corresponding value of ***a*** is changed within *modify()*.

Passing an argument by value has advantages and disadvantages. On the plus side, it allows a single-valued actual argument to be written as an expression rather than being restricted to a single variable. Moreover, if the actual argument is expressed simply as a single variable, it protects the value of this variable from alterations within the function. On the other hand, it does not allow information to be transferred back to the calling portion of the program via arguments. Thus, *passing by value is restricted to a one-way transfer of information*.

Array arguments are passed differently than single-valued data items. If an array name is specified as an actual argument, the individual array elements are not copied. Instead, the *location* of the array (i.e., the location of the first element) is passed to the function. If an element of the array is then accessed within the function, the access will refer to the location of that array element relative to the location of the first element.

Thus, *any alteration to an array element within the function will carry over to the calling routine*. We will discuss this in greater detail in next chapter, when we formally consider arrays.

Recursion

Recursion is a process by which a function calls itself repeatedly, until some specified condition has been satisfied. The process is used for repetitive computations in which each action is stated in terms of a previous result. Many iterative (i.e., repetitive) problems can be written in this form.

In order to solve a problem recursively, two conditions must be satisfied. First, the problem must be written in a *recursive form*, and second, the problem statement must *include a stopping condition*.

Suppose, for example, we wish to calculate the factorial of a positive integer quantity. We would normally express this problem as $n! = 1 \times 2 \times 3 \times \dots \times n$, where n is the specified positive integer. However, we can also express this problem in another way, by writing $n! = n \times (n - 1)!$. This is a recursive statement of the problem, in which the desired action (the calculation of $n!$) is expressed in terms of a previous result [the value of $(n - 1)!$, which is assumed to be known]. Also, we know that $1! = 1$. This last expression provides a stopping condition for the recursion.

Calculating Factorials in previous Example, we saw two versions of a program that calculates the factorial of a given input quantity, using a non-recursive function to perform the actual calculations. Here is a program that carries out this same calculation using recursion.

```
/* calculate the factorial of an integer quantity using
recursion */
#include <stdio.h>

long int factorial(int n);    /* function prototype */
main()
{
    int n;

    printf("n = ");
    scanf(" %d", &n);

    /* calculate and display the factorial */
    printf("\nn! = %ld\n", factorial(n));
}

long int factorial(int n)
{
    if (n <= 1)
        return (1);
    else
        return(n * factorial(n - 1));
}
```

The main portion of the program simply reads the integer quantity n and then calls the long-integer recursive function **factorial**. (Recall that we use long integers for this calculation because factorials are such large integer quantities, even for modest values of n .) The function **factorial** calls, itself recursively, with an actual argument $(n-1)$ that decreases in magnitude for each successive call. The recursive calls terminate when the value of the actual argument becomes equal to 1.

Notice that the present form of **factorial** is simpler than the function presented in the earlier Example. The close correspondence between this function and the original problem definition, in recursive terms, should be readily apparent. Note that the **if - else** statement includes a termination condition that becomes active when the value of **n** is less than or equal to **1**. (Note that the value of **n** will never be less than **1** unless an improper initial value is entered into the computer.)

When the program is executed, the function **factorial** will be accessed repeatedly, once in **main** and **(n - 1)** times within itself, though the person using the program will not be aware of this. Only the final answer will be displayed; for example,

```
n = 10
n! = 3628800
```

When a recursive program is executed, the recursive function calls are not executed immediately. Rather, they are placed on a **stack** until the condition that terminates the recursion is encountered. The function calls are then executed in reverse order, as they are “*popped*” off the stack. Thus, when evaluating a factorial recursively, the function calls will proceed in the following order.

```
n! = n x (n - 1)!
(n - 1)! = (n - 1) x (n - 2)!
(n - 2)! = (n - 2) x (n - 3)!
...
```

```
2! = 2 x 1!
```

The actual values will then be returned in the following reverse order.

```
1! = 1
2! = 2 x 1! = 2 x 1 = 2
3! = 3 x 2! = 3 x 2 = 6
4! = 4 x 3! = 4 x 6 = 24
...
```

```
n! = n x (n- 1)! = ...
```

This reversal in the order of execution is a characteristic of all functions that are executed recursively.

[A stack is a last-in, first-out data structure in which successive data items are “pushed down” upon preceding data items. The data items are later removed (i.e., they are “popped”) from the stack in reverse order, as indicated by the last-in, first-out designation.]

If a recursive function contains local variables, a *different* set of local variables will be created during each call. The names of the local variables will, of course, always be the same, as declared within the function. However, the variables will represent a different set of values each time the function is executed. Each set of values will be stored on the stack, so that they will be available as the recursive process “*unwinds*,” i.e., as the various function calls are “*popped*” off the stack and executed.

Printing Backwards The following program reads in a line of text on a character-by-character basis, and then displays the characters in reverse order. The program utilizes recursion to carry out the reversal of the characters.

```
/* read a line of text and write it out backwards, using
recursion */
#include <stdio.h>
```

```

#define EOL '\n'
void reverse(void);

main()
{
    printf("Please enter a line of text below\n");
    reverse() ;
}

void reverse(void)
/* read a line of text and write it out backwards */
{
    char c;
    if ((c = getchar()) != EOL)
        reverse();
    putchar(c);
    return;
}

```

The main portion of this program simply displays a prompt and then calls the function **reverse**, thus initiating the recursion. The recursive function **reverse** then proceeds to read single characters until an end-of-line designation (`\n`) is encountered. Each function call causes a new character (a new value for `c`) to be pushed onto the stack. Once the end of line is encountered, the successive characters are popped off the stack and displayed on a *last-in, first-out* basis. Thus, the characters are displayed in reverse order. Suppose the program is executed with the following line of input:

```

Please enter a line of text below
Swami Mukundananda
adnanadnukuM imawS

```

Sometimes a complicated repetitive process can be programmed very concisely using recursion, though the logic may be tricky. The following example provides a well-known illustration.

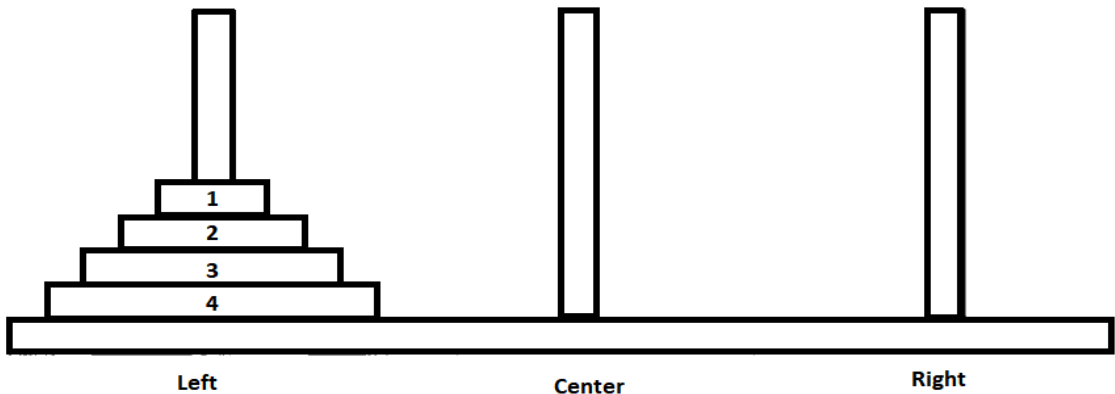
The Towers of Hanoi The Towers of Hanoi is a well-known children's game, played with three poles and a number of different-sized disks. Each disk has a hole in the center, allowing it to be stacked around any of the poles. Initially, the disks are stacked on the leftmost pole in the order of decreasing size, i.e., the largest on the bottom and the smallest on the top, as illustrated in the Figure below.

The object of the game is to transfer the disks from the leftmost pole to the rightmost pole, without ever placing a larger disk on top of a smaller disk. Only one disk may be moved at a time, and each disk must always be placed around one of the poles.

The general strategy is to consider one of the poles to be the origin, and another to be the destination. The third pole will be used for intermediate storage, thus allowing the disks to be moved without placing a larger disk over a smaller one. Assume there are n disks, numbered from smallest to largest, as in the Figure below. If the disks are initially stacked on the left pole, the problem of moving all n disks to the right pole can be stated in the following recursive manner.

1. Move the top $n - 1$ disks from the left pole to the center pole.
2. Move the n th disk (the largest disk) to the right pole.

3. Move the $n - 1$ disks on the center pole to the right pole.



The problem can be solved in this manner for any value of n greater than 0 ($n = 0$ represents a stopping condition).

In order to program this game, we first label the poles so that the left pole is represented as L , the center pole as C and the right pole as R . We then construct a recursive function called **transfer** that will transfer n disks from one pole to another. Let us refer to the individual poles with the char-type variables *from*, *to* and *temp* for the *origin*, *destination*, and *temporary storage*, respectively. Thus, if we assign the character L to **from**, R to **to** and C to **temp**, we will in effect be specifying the movement of n disks from the leftmost pole to the rightmost pole, using the center pole for intermediate storage.

With this notation, the function will have the following skeletal structure.

```
void transfer(int n, char from, char to, char temp)
/* n = number of disks
from = origin
to = destination
temp = temporary storage */
{
    if (n > 0)
    {
        /* move n-1 disks from their origin to the temporary
pole */
        /* move the nth disk from its origin to its
destination */
        /* move the n-1 disks from the temporary pole to
their destination */
    }
}
```

The transfer of the $n-1$ disks can be accomplished by a recursive call to transfer. Thus, we can write

```
transfer(n - 1, from, temp, to);
```

For the first transfer, and

```
transfer(n - 1, temp, to, from);
```

For the second. Note the order of the arguments in each call. The movement of the n th disk from the origin to the destination simply requires writing out the correct values of *from* and *to*. Hence, the complete function can be written as follows.

```

void transfer(int n, char from, char to, char temp)
/* n = number of disks
from = origin
to = destination
temp = temporary storage */
{
    if (n > 0)
    {
        /* move n-1 disks from their origin to the temporary
pole */
        transfer(n - 1, from, temp, to);
        /* move the nth disk from its origin to its
destination */
        printf("move disk %d from %c to %c\n", n, from, to);
        /* move the n-1 disks from the temporary pole to
their destination */
        transfer(n - 1, temp, to, from);
    }
    return;
}

```

It is now a simple matter to add the main portion of the program, which merely reads in a value for n and then initiates the computation by calling **transfer**. In this first function call, the actual parameters will be specified as character constants, i.e.,

```
transfer(n, 'L', 'R', 'C');
```

This function call specifies the transfer of all n disks from the leftmost pole (the origin) to the rightmost pole (the destination), using the center pole for intermediate storage. Here is a complete program.

```

/* the Tower of Hanoi - solved using recursion */
#include <stdio.h>

```

```

void transfer(int n, char from, char to, char temp);
main()
{
    int n;

    printf("How many disks you have? ");
    scanf(" %d", &n);

    transfer(n, 'L', 'R', 'C');
}
void transfer(int n, char from, char to, char temp)
{
    if (n > 0)
    {
        transfer(n-1, from, temp, to);

        printf("\nMove disk %d from %c to %c", n, from, to);
    }
}

```

```

        transfer(n-1, temp, to, from);
    }
    return;
}

```

The function `transfer` receives a different set of values for its arguments each time the function is called. These sets of values will be pushed onto the stack independently of one another, and then popped from the stack at the proper time during the execution of the program. It is this ability to store and retrieve these independent sets of values that allows the recursion to work.

When the program is executed for the case where $n = 3$, the following output is obtained.

How many disks you have? 3

```

Move disk 1 from L to R
Move disk 2 from L to C
Move disk 1 from R to C
Move disk 3 from L to R
Move disk 1 from C to L
Move disk 2 from C to R
Move disk 1 from L to R

```

The use of recursion is not necessarily the best way to approach a problem, even though the problem definition may be recursive in nature. A non-recursive implementation may be more efficient, in terms of memory utilization and execution speed. Thus, the use of recursion may involve a tradeoff between *simplicity and performance*. Each problem should therefore be judged on its own individual merits.

Storage Classes

We have already mentioned that there are two different ways to characterize variables: by **data type**, and by **storage class**. Data type refers to the type of information represented by a variable, e.g., integer number, floating-point number, character, etc. Storage class refers to the **permanence** of a variable, and its **scope** within the program, i.e., the portion of the program over which the variable is recognized.

There are four different storage-class specifications in C: **automatic**, **external**, **static** and **register**. They are identified by the keywords **auto**, **extern**, **static**, and **register**, respectively. The storage class associated with a variable can sometimes be established simply by the location of the variable declaration within the program. In other situations, however, the keyword that specifies a particular storage class must be placed at the beginning of the variable declaration.

Shown below are several typical variable declarations that include the specification of a storage class.

```

auto int a, b, c;
extern float root1, root2;
static int count = 0;
extern char star;

```

The first declaration states that **a**, **b** and **c** are automatic integer variables, and the second declaration establishes **root1** and **root2** as external floating-point variables. The third

declaration states that **count** is a static integer variable whose initial value is 0, and the last declaration establishes **star** as an external character-type variable.

Automatic Variables

Automatic variables are always declared within a function and are local to the function in which they are declared; that is, their scope is confined to that function. Automatic variables defined in different functions will therefore be independent of one another, even though they may have the same name.

Any variable declared within a function is interpreted as an automatic variable unless a different storage-class specification is shown within the declaration. This includes formal argument declarations. All the variables in the programming examples encountered earlier have been automatic variables.

Since the location of the variable declarations within the program determines the *automatic storage class*, the keyword **auto** is not required at the beginning of each variable declaration. There is no harm in including an auto specification within a declaration, though this is normally not done.

Calculating Factorials Consider once again the program for calculating factorials, originally shown in the earlier Example. Within **main**, **n** is an automatic variable. Within **factorial**, **i** and **prod**, as well as the formal argument **n**, are automatic variables.

The storage-class designation **auto** could have been included explicitly in the variable declarations if we had wished. Thus, the program could have been written as follows.

```
/* calculat the factorial of an integer quantity */
#include <stdio.h>

long int factorial(int n);

main()
{
    auto int n;
    /* read in the integer quantity */
    printf("n = ");
    scanf(" %d", &n);
    /* calculate and display the factorial */
    printf("\nn! = %ld", factorial(n));
}

long int factorial(int n)
{
    auto int i;
    auto long int prod = 1;
    if (n > 1)
    {
        for (i = 1; i <= n; ++i)
            prod *= i;
    }
    return prod;
}
```


Either method is acceptable. As a rule, however, the **auto** designation is not included in variable or formal argument declarations, since this is the default storage class. Thus, the program shown in the earlier Example represents a more common programming style.

Automatic variables can be assigned initial values by including appropriate expressions within the variable declarations, as in the above example, or by explicit assignment expressions elsewhere in the function. Such values will be reassigned each time the function is reentered. If an automatic variable is not initialized in some manner, however, its initial value will be *unpredictable*, and probably *unintelligible* (i.e., *garbage* value).

An automatic variable does not retain its value once control is transferred out of its defining function.

Therefore, any value assigned to an automatic variable within a function will be lost once the function is exited. If the program logic requires that an automatic variable be assigned a particular value each time the function is executed, that value will have to be reset whenever the function is reentered (i.e., whenever the function is accessed).

Average Length of Several Lines of Text Let us now write a C program that will read several lines of text and determine the average number of characters (including punctuation and blank spaces) in each line. We will structure the program in such a manner that it continues to read additional lines of text until an *empty line* (i.e., a line whose first character is `\n`) is encountered.

We will utilize a function (**linecount**) that reads a single line of text and counts the number of characters, excluding the newline character (`\n`) that signifies the end of the line. The calling routine (main) will maintain a cumulative sum, as well as a running total of the number of lines that have been read. The function will be called repeatedly (thus reading a new line each time), until an empty line is encountered. The program will then divide the cumulative number of characters by the total number of lines to obtain an average.

Here is the entire program.

```
/* read several lines of text and determine the average number of
character per line */
#include <stdio.h>

int linecount(void);
main()
{
    int n; /* no. of characters in given line */
    int count = 0; /*number of lines */
    int sum = 0; /*total number of characters */
    float avg; /* average no. of characters per line */

    printf("Enter the text below:\n");
    /* read a line of text and update the cumulative counters */

    while ((n = linecount()) > 0)
    {
        sum += n;
        ++count;
    }

    avg = (float) sum / count;
    printf("\nAverage no. of characters per line: %5.2f", avg);
}
```

```

int linecount(void)
/* read a line of text and count the number of characters */
{
    char line[80];
    int count = 0;

    while((line[count] = getchar()) != '\n')
        ++count;
    return count;
}

```

We see that `main` contains four automatic variables: `n`, `count`, `sum` and `avg`, whereas **`linecount`** contains two: `line` and `count`. (Notice that `line` is an 80-element character array, representing the contents of one line of text.) Three of these automatic variables are assigned initial values of zero.

Also, note that `count` has different meanings within each function. Within **`linecount`**, `count` represents the number of characters in a single line, whereas within `main`, `count` represents the total number of lines that have been read. Moreover, `count` is reset to zero within **`linecount`** whenever the function is accessed. This does not affect the value of `count` within `main`, since the variables are independent of one another. It would have been clearer if we had named these variables differently, e.g., `count` and `lines`, or perhaps `chars` and `lines`. We have used the same name for both variables to illustrate the independence of automatic variables within different functions.)

A sample interactive session, resulting from execution of this program, is shown below. As usual, the user's responses are underlined.

```

Enter the text below:
Please help us to keep the
books clean and moving.

```

```

Average no. of characters per line: 24.50

```

The scope of an automatic variable can be smaller than an entire function if we wish. In fact, automatic variables can be declared within a single compound statement. With small, simple programs there is usually no advantage in doing this, but it may be desirable in larger programs.

External (Global) Variables

External variables, in contrast to automatic variables, are not confined to single functions. Their scope extends from the point of definition through the remainder of the program. Hence, they usually span two or more functions, and often an entire program. They are often referred to as **global variables**.

Since external variables are recognized globally, they can be accessed from any function that falls within their scope. They retain their assigned values within this scope. Therefore, an external variable can be assigned a value within one function, and this value can be used (by accessing the external variable) within another function.

The use of external variables provides a convenient mechanism for **transferring information back and forth between functions**. We can transfer information into a function without using

arguments. This is especially convenient when a function requires numerous input data items. Moreover, we now have a way to transfer multiple data items out of a function, since the *return* statement can return only one data item. (We will see another way to transfer information back and forth between functions in later chapter, where we discuss pointers.)

When working with external variables, we must distinguish between external variable **definitions** and external variable **declarations**. An external variable **definition** is written in the same manner as an ordinary variable declaration. It must appear outside of, and usually before, the functions that access the external variables. An external variable definition will automatically allocate the required storage space for the external variables within the computer's memory. The assignment of initial values can be included within an external variable definition if desired (more about this later).

The storage-class specifier **extern** is not required in an external variable definition, since the external variables will be identified by the location of their definition within the program. In fact, many C compilers forbid the use of **extern** within an external variable definition.

If a function requires an external variable that has been defined earlier in the program, then the function may access the external variable freely, without any special declaration within the function. (Remember, however, that *any alteration to the value an external variable within a function will be recognized within the entire scope of the external variable*.) On the other hand, if the function definition **precedes** the external variable definition, then the function must include a **declaration** for that external variable. The function definitions within a large program often include external variable declarations, whether they are needed or not, as a matter of good programming practice.

An external variable **declaration** must begin with the storage-class specifier **extern**. The name of the external variable and its data type must agree with the corresponding external variable definition that appears outside of the function. Storage space for external variables will not be allocated as a result of an external variable declaration. Moreover, an external variable declaration cannot include the assignment of initial values. These distinctions between an external variable **definition** and an external variable **declaration** are very important.

External variables can be assigned initial values as a part of the variable definitions, but the initial values must be expressed as **constants** rather than expressions. These initial values will be assigned only once, at the beginning of the program. The external variables will then retain these initial values unless they are later altered during the execution of the program. If an initial value is not included in the definition of an external variable, the variable will automatically be assigned a value of **zero**. Thus, external variables are never left dangling with undefined, garbled values. Nevertheless, it is good programming practice to assign an explicit initial value of zero when required by the program logic.

Average Length of Several Lines of Text Shown below is a modification of the program previously presented in the last Example, for determining the average number of characters in several lines of text. The present version makes use of external variables to represent the total (cumulative) number of characters read, and the total number of lines.

```
/* read several lines of text and determine the average number of
characters per line */
#include <stdio.h>
```

```

/* global variable */
int sum = 0; /* total number of character */
int lines = 0; /*total number of lines */

int linecount(void);

main()
{
    int n; /* no of chars in given line */
    float avg; /*average number of chars per line */
    printf("Enter the text below:\n");
    /* read a line of text and update the cumulative counters */
    while ((n = linecount()) > 0)
    {
        sum += n;
        ++lines;
    }
    avg = (float) sum / lines;
    printf("\nAverage number of characters per line: %5.2f", avg);
}
/* read a line of text and count the number of characters */
int linecount(void)
{
    char line[80];
    int count = 0;

    while ((line[count] = getchar()) != '\n')
        ++count;
    return count;
}

```

Notice that **sum** and **lines** are external variables that represent the total (cumulative) number of characters read and the total number of lines, respectively. Both variables are assigned initial values of zero. These values are successively modified within main, as additional lines of text are read.

Also, recall that the earlier version of the program used two different automatic variables, each called *count* in different parts of the program. In the present version of the program, however, the variables that represent the same quantities have different names, since one of the variables (*lines*) is now an external variable.

You should understand that **sum** and **lines** need not be assigned zero values explicitly, since external variables are always set equal to zero unless some other initial value is designated. We include the explicit zero initialization in order to clarify the program logic. The output is same as before.

Enter the text below:
Please Help us to Keep the
Book Clean and Moving

Average number of characters per line: **23.50**

Finally, there are inherent dangers in the use of *external* variables, since an alteration in the value of an external variable within a function will be carried over into other parts of the

program. Sometimes this happens inadvertently, as a *side effect* of some other action. Thus, there is the possibility that the value of an external value will be changed unexpectedly, resulting in a subtle programming error. You should decide carefully which storage class is most appropriate for each programming situation.

Static Variables

Static variables are defined within individual functions and therefore have the same scope as automatic variables; i.e., they are local to the functions in which they are defined. Unlike automatic variables, however, static variables retain their values throughout the life of the program. Thus, if a function is exited and then re-entered later, the static variables defined within that function will retain their former values. This feature allows functions to retain information permanently throughout the execution of a program.

Static variables are defined within a function in the same manner as automatic variables, except that the variable declaration must begin with the ***static*** storage-class designation. Static variables can be utilized within the function in the same manner as other variables. They cannot, however, be accessed outside of their defining function.

It is not unusual to define automatic or static variables having the same names as external variables. In such situations the local variables will *take precedence over* the external variables, though the values of the external variables will be unaffected by any manipulation of the local variables. Thus, the external variables maintain their independence from locally defined automatic and static variables. The same is true of local variables within one function that have the same names as local variables within another function.

Shown below is the skeletal structure of a C program that includes variables belonging to several different storage classes.

```
float a, b, c;

void dummy(void);

main()
{
    static float a;
    ...
}

void dummy(void)
{
    static int a;
    int b;
    ...
}
```

Within this program ***a***, ***b*** and ***c*** are external, floating-point variables. However, ***a*** is redefined as a *static* floating-point variable within ***main***. Therefore, ***b*** and ***c*** are the only external variables that will be recognized within ***main***. Note that the static ***local*** variable ***a*** will be independent of the ***external*** variable ***a***.

Similarly, ***a*** and ***b*** are redefined as integer variables within ***dummy***. Note that ***a*** is a static variable, but ***b*** is an automatic variable. Thus, ***a*** will retain its former value whenever ***dummy*** is reentered, whereas ***b*** will lose its value whenever control is transferred out of ***dummy***. Furthermore, ***c*** is the only external variable that will be recognized within ***dummy***.

Since **a** and **b** are local to **dummy**, they will be independent of the external variables **a**, **b** and **c**, and the static variable **a** defined within **main**. The fact that **a** and **b** are declared as integer variables within **dummy** and floating-point variables elsewhere is therefore immaterial.

Initial values can be included in the static variable declarations. The rules associated with the assignment of these values are essentially the same as the rules associated with the initialization of external variables, even though the static variables are defined locally within a function. In particular:

- The initial values must be expressed as constants, not expressions.
- The initial values are assigned to their respective variables at the beginning of program execution. The variables retain these values throughout the life of the program, unless different values are assigned during the computation.
- **Zeros** will be assigned to all static variables whose declarations do not include explicit initial values. Hence, static variables will always have assigned values.

Generating Fibonacci Numbers The Fibonacci numbers form an interesting sequence in which each number is equal to the sum of the previous two numbers. In other words,

$$F_i = F_{i-1} + F_{i-2}$$

where ***Fi*** refers to the ***i*th** Fibonacci number. The first two Fibonacci numbers are defined

$$F_1 = 0$$

$$F_2 = 1$$

Hence,

$$F_3 = F_2 + F_1 = 1 + 0 = 1$$

$$F_4 = F_3 + F_2 = 1 + 1 = 2$$

$$F_5 = F_4 + F_3 = 2 + 1 = 3$$

and so on.

Let us write a C program that generates the first **n** Fibonacci numbers, where **n** is a value specified by the user. The **main** portion of the program will read in a value for **n**, and then enter a loop that generates and writes out each of the Fibonacci numbers. A function called **fibonacci** will be used to calculate each Fibonacci number from its two preceding values. This function will be called once during each pass through the main loop.

When **fibonacci** is entered, the computation of the current Fibonacci number, **f**, is very simple provided the two previous values are known. These values can be retained from one function call to the next if we assign them to the static variables **f1** and **f2**, which represent ***F_{i-1}*** and ***F_{i-2}***, respectively. (We could, of course, have used external variables for this purpose, but it is better to use local variables, since ***F_{i-1}*** and ***F_{i-2}*** are required only within the function.) We then calculate the desired Fibonacci number as

$$f = f_1 + f_2$$

and update the values of **f2** and **f1** using the formulas

$$f_2 = f_1$$

and

$$f_1 = f$$

Here is the complete C program.

```
/* calculate successive Fibonacci series */
#include<stdio.h>
```

```

long int fibonacci(int count);

main()
{
    int count, n;

    printf("How many Fibonacci Numbers? ");
    scanf(" %d", &n);

    for (count = 0; count < n; ++count)
        printf("\ni = %2d\tF = %ld", count+1, fibonacci(count));
}

long int fibonacci(int count)
{
    static int f1 = 0, f2 = 1;
    long int f;

    f = count < 2 ? count : (f1 + f2);
    f1 = f2;
    f2 = f;
    return f;
}

```

Notice that long integers are used to represent the Fibonacci numbers. Also, note that ***f1*** and ***f2*** are static variables that are each assigned an initial value of 0 and 1 respectively. These initial values are assigned only once, at the beginning of the program execution. The subsequent values are retained between successive function calls, as they are assigned. You should understand that ***f1*** and ***f2*** are strictly local variables, even though they retain their values from one function call to another.

The output corresponding to a value of **n = 30** is shown below. As usual, the user's response is underlined.

How many Fibonacci Numbers? 30

```

i = 1   F = 0
i = 2   F = 1
i = 3   F = 1
i = 4   F = 2
i = 5   F = 3
i = 6   F = 5
i = 7   F = 8
i = 8   F = 13
i = 9   F = 21
i = 10  F = 34
i = 11  F = 55
i = 12  F = 89
i = 13  F = 144
i = 14  F = 233
i = 15  F = 377
i = 16  F = 610
i = 17  F = 987

```

```

i = 18   F = 1597
i = 19   F = 2584
i = 20   F = 4181
i = 21   F = 6765
i = 22   F = 10946
i = 23   F = 17711
i = 24   F = 28657
i = 25   F = 46368
i = 26   F = 75025
i = 27   F = 121393
i = 28   F = 196418
i = 29   F = 317811
i = 30   F = 514229

```

Register Variables

Registers are special storage areas within the computer's central processing unit. The actual arithmetic and logical operations that comprise a program are carried out within these registers. Normally, these operations are carried out by transferring information from the computer's memory to these registers, carrying out the indicated operations, and then transferring the results back to the computer's memory. This general procedure is repeated many times during a program's execution.

For some programs, the execution time can be reduced considerably if certain values can be stored within these registers rather than in the computer's memory. Such programs may also be somewhat smaller in size (i.e., they may require fewer instructions), since fewer data transfers will be required. Usually, however, the size reduction will not be dramatic and will be less significant than the savings in execution time.

In C, the values of **register variables** are stored within the registers of the central processing unit. A variable can be assigned this storage class simply by preceding the type declaration with the keyword **register**. There can, however, be only a few register variables (typically, two or three) within any one function. The exact number depends upon the particular computer, and the specific C compiler. Usually, only integer variables are assigned the register storage class.

The register and automatic storage classes are closely related. Their visibility (i.e., their scope) is the same. Thus, **register** variables, like automatic variables, are local to the function in which they are declared. Furthermore, the rules governing the use of register variables are the same as those for automatic variables, except that the address operator (**&**) cannot be applied to register variables.

The similarities between register and automatic variables is not coincidental, because the register storage class can be assigned only to variables that would otherwise have the storage class automatic. Moreover, declaring certain variables to be register variables does not guarantee that they will be treated as register variables. The declaration will be valid only if the requested register space is available. If a register declaration is not honored, the variables will be treated as having the storage class automatic.

A C program contains the variable declaration

```
register int a, b, c;
```

This declaration specifies that the variables **a**, **b** and **c** will be integer variables with storage class register. Hence, the values of **a**, **b** and **c** will be stored within the registers of the computer's central processing unit rather than in memory, provided the register space is available.

If the register space is not available, then the variables will be treated as integer variables with storage class automatic. This is equivalent to the declaration

```
auto int a, b, c;
```

or simply

```
int a, b, c;
```

Unfortunately, there is no way to determine whether a **register** declaration will be honored, other than to run a program with and without the declaration and compare the results. A program that makes use of **register** variables should run faster than the corresponding program without **register** variables. It may also be somewhat smaller in size.

Macros

We have already seen that the **#define** statement can be used to define symbolic constants within a C program. At the beginning of the compilation process, all symbolic constants are replaced by their equivalent text. Thus, symbolic constants provide a form of shorthand notation that can simplify the organization of a program.

The **#define** statement can be used for more, however, than simply defining symbolic constants. It can be used to define *macros*; i.e., single identifiers that are equivalent to expressions, complete statements or groups of statements. Macros resemble functions in this sense. They are defined in an altogether different manner than functions, however, and they are treated differently during the compilation process.

Consider the simple C program shown below.

```
#include <stdio.h>

#define AREA length * width

main()
{
    float length, width;

    printf("Length = ");
    scanf(" %f", &length);
    printf("\nWidth = ");
    scanf(" %f", &width);

    printf("\narea = %.2f", AREA);
}
```

This program contains the macro **AREA**, which represents the expression *length * width*. When the program is compiled, the expression *length * width* will replace the identifier **AREA** within the printf statement, so that the printf statement will become

```
printf("\narea = %.2f", length * width);
```

When the program is executed, the values for length and width are entered interactively from the keyboard, and the corresponding value for area is displayed. A typical interactive session is shown below. The user's responses are underlined.

```
Length = 4
Width = 3
area = 12.00
```

Macro definitions are customarily placed at the beginning of a file, ahead of the first function definition. The scope of a macro definition extends from its point of definition to the end of the file. Multiline macros can be defined by placing a *backward slash* (\) at the end of each line except the last. This feature permits a single macro (i.e., a single identifier) to represent a compound statement. Here is another simple C program that contains a macro.

```
#include <stdio.h>
#define LOOP for (lines = 1; lines <= n; ++lines) \
    { \
        for(count = 1; count <= lines; ++count) \
        { \
            printf("%d ", count); \
        } \
        printf("\n"); \
    }

main()
{
    int count, lines, n;
    printf("number of lines ");
    scanf(" %d", &n);

    LOOP
}
```

This program contains a multiline macro, which represents a compound statement. The compound statement consists of several embedded for loops. Notice the backward slash (\) at the end of each line, except the last.

When this program is compiled, the reference to the macro is replaced by the statements contained within the macro definition. Thus, the above program becomes

```
#include <stdio.h>

main()
{
    int count, lines, n;
    printf("number of lines ");
    scanf(" %d", &n);
    for (lines = 1; lines <=n; ++lines)
    {
        for(count = 1; count <= lines; ++count)
        {
            printf("%d ", count);
        }
        printf("\n");
    }
}
```

Output:

number of lines 5

```

1
1 2
1 2 3
1 2 3 4
1 2 3 4 5

```

A macro definition may include arguments, which are enclosed in parentheses. The left parenthesis must appear immediately after the macro name; i.e., there can be no space separating the macro name from the left parenthesis. When a macro is defined in this manner, its appearance within a program resembles a function call.

Here is another variation of the program shown in the last Example.

```

#include <stdio.h>
#define LOOP(n) for (lines = 1; lines <= n; ++lines) \
    { \
        for(count = 1; count <= lines; ++count) \
        { \
            printf("%d ", count); \
        } \
        printf("\n"); \
    }

main()
{
    int count, lines, n;
    printf("number of lines ");
    scanf(" %d", &n);

    LOOP(n)
}

```

The program now passes the value of ***n*** to the macro, as though it were an actual argument in a function call.

Macros are sometimes used in place of functions within a program. The use of a macro in place of a function eliminates the time delays associated with function calls. If a program contains many repeated function calls, the time savings resulting from the use of macros can become significant.

On the other hand, macro substitution will take place wherever a reference to a macro appears within a program. Thus, a program that contains several references to the same macro may become unreasonably long. We are therefore faced with a tradeoff between execution speed and size of the compiled object program. The use of a macro is most advantageous in applications where there are relatively few function calls, but the function is called repeatedly (e.g., a single function call within a loop).

Many commercial C compilers offer certain library functions both as macros and as true functions. The macros are defined in the various header files. You may then choose which form is most appropriate for each particular application. Keep in mind, however, that there are certain disadvantages associated with the use of macros in place of functions, aside from the potentially significant increase in program length. In particular:

- When passing arguments to a macro, the number of arguments will be checked, but their data types will not. Thus, there is *less error checking* than with a function call.

- A macro identifier is not associated with an address, so that a macro cannot be utilized as a pointer. Thus, a macro identifier cannot be passed to a function as an argument, in the same sense that a function can be passed to another function as an argument. Moreover, a macro cannot call itself recursively.
- There are possible undesirable side effects associated with the use of macros, particularly when calling arguments are involved.

Consider the macro definition

```
#define ROOT(a, b) sqrt(a * a + b * b)
```

Now suppose that this macro is utilized within a program in the following manner.

```
ROOT(a+1, b+2)
```

The intent is to evaluate the formula

```
sqrt((a + 1) * (a + 1) + (b + 2) * (b + 2))
```

However, each appearance of *a* is replaced by the expression *a + 1* (without parentheses), and each appearance of *b* is replaced by *b + 1*. Therefore, the result of the macro substitution will be

```
sqrt(2 * a + 1 + b + 2 * b + 2)
```

this expression is equivalent to

```
sqrt(2 * a + 3 * b + 3)
```

which is clearly incorrect. The source of error can be corrected, however, by placing additional parentheses within the macro definition; i.e.,

```
#define ROOT(a, b) sqrt((a) * (a) + (b) * (b))
```

A more subtle error occurs if we write

```
ROOT(a++, b++)
```

The macro substitution results in the expression

```
sqrt(a * (a + 1) + b * (b + 1))
```

rather than

```
sqrt(a * a + b * b)
```

as intended. This is an example of an undesired side effect. The placement of additional parentheses within the macro definition will not correct this problem.

The C Preprocessor

The C preprocessor is a collection of special statements, called directives, that are executed at the beginning of the compilation process. The **#include** and **#define** statements considered are preprocessor directives. Additional preprocessor directives are **#if**, **#elif**, **#else**, **#endif**, **#if def**, **#ifndef**, **#line** and **#undef**. The preprocessor also includes three special operators: **defined**, **#**, and **##**.

Preprocessor directives usually appear at the beginning of a program, though this is not a firm requirement. Thus, a preprocessor directive may appear anywhere within a program. However, the directive will apply only to the portion of the program following its appearance.

For the beginning programmer, some of the preprocessor directives are relatively unimportant. Hence, we will not describe each preprocessor feature in detail. The more important features are discussed below.

The **#if**, **#elif**, **#else** and **#endif** directives are used most frequently. They permit conditional compilation of the source program, depending on the value of one or more true/false conditions. They are sometimes used in conjunction with the *defined* operator, which is used to determine whether a symbolic constant or a macro identifier has been defined within a program.

The following preprocessor directives illustrate the conditional compilation of a C program. The conditional compilation depends on the status of the symbolic constant **BACKGROUND**.

```
#if defined (BACKGROUND)
    #define BACKGROUND 0
#else
    #define FOREGROUND 0
    #define BACKGROUND 7
#endif
```

Thus, if **BACKGROUND** has already been defined, the symbolic constant **BACKGROUND** will represent the value 0. Otherwise, **FOREGROUND** and **BACKGROUND** will represent the values 0 and 7 respectively.

Here is another way to accomplish the same thing.

```
#ifndef FOREGROUND
    #define BACKGROUND 0
#else
    #define FOREGROUND 0
    #define BACKGROUND 7
#endif
```

The directive **#ifndef** is equivalent to **#if !defined()**. Similarly, the directive **#ifndef** is equivalent to **#if !defined()**, i.e., “if not defined.” The original approach, in which the *defined* operator appears explicitly, is the preferred form.

In each of these examples, the last directive is **#endif**. The preprocessor requires that any set of directives beginning with **#if**, **#ifdef** or **#ifndef** must end with **#endif**.

The directive **#elif** is analogous to an **else - if** clause using ordinary C control statements. An **#if** directive can be followed by any number of **#elif** directives, though there can be only one **#else** directive. The appearance of the **#else** directive is optional, as determined by the required program logic.

Here is another illustration of conditional compilation. In this situation the conditional compilation will depend on the value that is represented by the symbolic constant **BACKGROUND**.

```
#if BACKGROUND == 7
    #define FOREGROUND 0
#elif BACKGROUND == 6
    #define FOREGROUND 1
#else
    #define FOREGROUND 6
```

#endif

In this example we see that **FOREGROUND** will represent **0** if **BACKGROUND** represents **7**, and **FOREGROUND** will represent **1** if **BACKGROUND** represents **6**. Otherwise, **FOREGROUND** will represent **6**.

The **#undef** directive “undefines” a symbolic constant or a macro identifier; i.e., it negates the effect of a **#define** directive that may have appeared earlier in the program.

The following example illustrates the use of the **#undef** directive within a C program.

```
#define FOREGROUND 7
#define BACKGROUND 0

main()
{
    .....
    #undef    FOREGROUND
    .....
    #undef    BACKGROUND
    .....
}
```

The symbolic constants **FOREGROUND** and **BACKGROUND** are defined by the first two directives. These definitions are then negated by the **#undef** directives, when they appear later in the program. Prior to the **#undef** directives, any references to **FOREGROUND** or **BACKGROUND** will be associated with the values **7** and **0**, respectively. After the **#undef** directives, any references to the corresponding identifiers will be ignored.

The “**stringizing**” operator **#** allows a formal argument within a macro definition to be converted to a *string*. If a formal argument in a macro definition is preceded by this operator, the corresponding actual argument will automatically be enclosed in *double quotes*. Consecutive whitespace characters inside the actual argument will be replaced by a single blank space, and any special characters, such as **‘**, **“**, and ****, will be replaced by their corresponding escape sequences; e.g., **\’**, **\”** and ****. In addition, the resulting string will automatically be concatenated (i.e., combined) with any adjacent strings.

Here is an illustration of the use of the “**stringizing**” operator, **#**.

```
#define display(text) printf(#text "\n")

Main()
{
    ...
    display(Please do not sleep in class.);
    ...
    display(Please - don't snore during the professor's
lecture!);
}
```

Within **main**, the macros are equivalent to

```
printf("Please do not sleep in class.\n");
```

and

```
printf("Please - don\'t snore during the professor\'s
lecture!\n");
```

Notice that each actual argument is converted to a string within the printf function. Each argument is concatenated with a newline character (`\n`), which is written as a separate string within the macro definition. Also, notice that the consecutive blank spaces appearing in the second argument are replaced by single blank spaces, and each apostrophe (`'`) is replaced by its corresponding escape sequence (`\'`).

Execution of this program will result in the following output:

```
Please do not sleep in class.
Please - don't snore during the professor's lecture!
```

The "**token-pasting**" operator `##` causes individual items within a macro definition to be concatenated, thus forming a single item. The various rules governing the use of this operator are somewhat complicated. However, the general purpose of the token-pasting operator is illustrated in the following example. A C program contains the following macro definition.

```
#define display(i) printf("x" #i " = %f\n", x##i)
```

Suppose this macro is accessed by writing

```
display(3);
```

The result will be

```
printf("x3 = %f\n", x3);
```

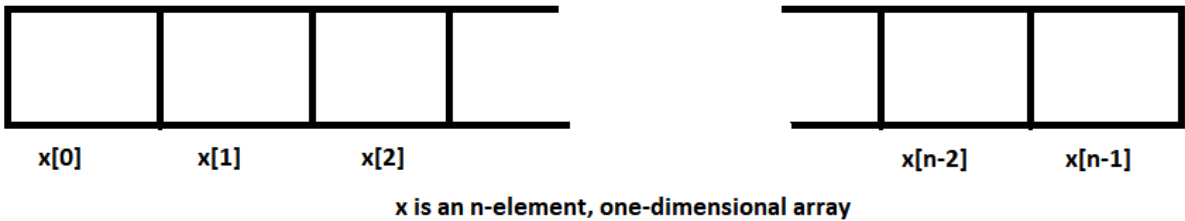
Thus, the expression `x##i` becomes the variable `x3`, since 3 is the current value of the argument `i`.

Notice that this example illustrates the use of both the *stringizing operator* (`#`) and the *token-pasting operator* (`##`).

6. ARRAYS AND STRINGS

Many applications require the processing of multiple data items that have common characteristics (e.g., a set of numerical data, represented by $x_1, x_2, \dots, x_n$). In such situations it is often convenient to place the data items into an **array**, where they will all share the same name (e.g., `x`). The individual data items can be characters, integers, floating-point numbers, etc. However, they must all be of the same type and the same storage class.

Each array element (i.e., each individual data item) is referred to by specifying the array name followed by one or more *subscripts*, with each subscript enclosed in square brackets. Each subscript must be expressed as a *nonnegative integer*. In an n -element array, the array elements are `x[0]`, `x[1]`, `x[2]`, ..., `x[n - 1]` as illustrated in Figure below. The value of each subscript can be expressed as an *integer constant*, an *integer variable* or a more *complex integer expression*.



The number of subscripts determines the dimensionality of the array. For example, **x[i]** refers to an element in the one-dimensional array **x**. Similarly, **y[i][j]** refers to an element in the two-dimensional array **y**. (We can think of a two-dimensional array as a *table*, where **y[i][j]** is the j^{th} element of the i^{th} row.) Higher-dimensional arrays can also be formed, by adding additional subscripts in the same manner (e.g., **z[i][j][k]**).

Recall that we have used one-dimensional character arrays earlier in this note, in conjunction with the processing of strings and lines of text. Thus, arrays are not entirely new, even though our previous references to them were somewhat casual. We will now consider arrays in greater detail. We will discuss the way arrays are defined and processed, the passing of arrays to functions, and the use of multidimensional arrays. Both numerical and character-type arrays will be considered. Initially we will concentrate on one-dimensional arrays, though multidimensional arrays will be then considered later.

Defining an Array

Arrays are defined in much the same manner as ordinary variables, except that each array name must be accompanied by a size specification (i.e., the number of elements). For a one-dimensional array, the size is specified by a positive integer expression, enclosed in square brackets. The expression is usually written as a positive integer constant.

In general terms, a one-dimensional array definition may be expressed as

storage-class data-type array[expression] ;

where *storage-class* refers to the storage class of the array, *data-type* is the data type, *array* is the array name, and *expression* is a positive-valued integer expression which indicates the number of array elements. The *storage-class* is optional; default values are **automatic** for arrays that are defined within a function or a block, and **external** for arrays that are defined outside of a function.

Several typical one-dimensional array definitions are shown below.

```
int x[100];
char text[80];
static char message[25];
static float n[12];
```

The first line states that **x** is a 100-element integer array, and the second defines **text** to be an 80-element character array. In the third line, **message** is defined as a static 25-element character array, whereas the fourth line establishes **n** as a static 12-element floating-point array.

It is sometimes convenient to define an array size in terms of a symbolic constant rather than a fixed integer quantity. This makes it easier to modify a program that utilizes an array, since all references to the maximum array size (e.g., within *for* loops as well as in array definitions) can be altered simply by changing the value of the symbolic constant.

Lowercase to Uppercase Text Conversion

Here is a complete program that reads in a one-dimensional character array, converts all the elements to uppercase, and then displays the converted array. Similar program is shown in example of previous chapter.

```
/* read in a line of lowercase text and convert it into
uppercase */
#include <stdio.h>
#include <ctype.h>

#define SIZE 80

main()
{
    char letter[SIZE];
    int count;
    /*read in a line */
    printf("Enter a line of text ");
    for (count = 0; count < SIZE; ++count)
        letter[count] = getchar();

    /*display the line in uppercase */
    for (count = 0; count < SIZE; ++count)
        putchar(toupper(letter[count]));
}
```

Notice that the symbolic constant **SIZE** is assigned a value of **80**. This symbolic constant, rather than its value, appears in the array definition and in the two *for* statements. (Remember that *the value of the symbolic constant will be substituted for the constant itself during the compilation process.*) Therefore, in order to alter the program to accommodate a different size array, only the **#define** statement must be changed. For example, to alter the above program so that it will process a 60-element array, the original **#define** statement is simply replaced by

```
#define SIZE 60
```

This one change accommodates all the necessary program alterations; there is no possibility that some required program modification will be overlooked.

Automatic arrays, unlike automatic variables, cannot be initialized. However, external and static array definitions can include the assignment of initial values if desired. The initial values must appear in the order in which they will be assigned to the individual array elements, enclosed in braces and separated by commas. The general form is

```
storage-class data-type array[expression] = {value 1, value 2,
..., value n};
```

where **value 1** refers to the value of the first array element, **value 2** refers to the value of the second element, and so on. The appearance of the **expression**, which indicates the number of array elements, is *optional* when initial values are present.

Shown below are several array definitions that include the assignment of initial values.

```
int digits[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
static float x[6] = {0, 0.25, 0, -0.50, 0, 0};
```

```
char color[3] = {'R', 'E', 'D',};
```

Note that **x** is a static array. The other two arrays (**digits** and **color**) are assumed to be external arrays by virtue of their placement within the program. The results of these initial assignments, in terms of the individual array elements, are as follows. (Remember that the subscripts in an **n**-element array range from **0** to **n - 1**.)

digits[0] = 1	x[0] = 0	color[0] = 'R'
digits[1] = 2	x[1] = 0.25	color[1] = 'E'
digits[2] = 3	x[2] = 0	color[2] = 'D'
digits[3] = 4	x[3] = -0.50	
digits[4] = 5	x[4] = 0	
digits[5] = 6	x[5] = 0	
digits[6] = 7		
digits[7] = 8		
digits[8] = 9		
digits[9] = 10		

All individual array elements that are not assigned explicit initial values will automatically be set to **zero**. This includes the remaining elements of an array in which some elements have been assigned nonzero values.

Consider the following array definitions.

```
int digits[10] = {3, 3, 3};
static float x[6] = {-0.3, 0, 0.25};
```

The results, on an element-by-element basis, are as follows.

digits[0] = 3	x[0] = -0.3
digits[1] = 3	x[1] = 0
digits[2] = 3	x[2] = 0.25
digits[3] = 0	x[3] = 0
digits[4] = 0	x[4] = 0
digits[5] = 0	x[5] = 0
digits[6] = 0	
digits[7] = 0	
digits[8] = 0	
digits[9] = 0	

In each case, all the array elements are automatically set to zero except those that have been explicitly initialized within the array definitions. Note that the repeated values (i.e., **3, 3, 3**) must be shown individually.

The array size need not be specified explicitly when initial values are included as a part of an array definition. With a numerical array, the array size will automatically be set equal to the number of initial values included within the definition.

Consider the following array definitions, which are variations of the definitions shown in the above Examples.

```
int digits[] = {1, 2, 3, 4, 5, 6};
static float x[] = {0, 0.25, 0, -0.5};
```

Thus, **digits** will be a *six-element* integer array, and **x** will be a static, four-element floating-point array. The individual elements will be assigned the following values. (*Note the empty brackets in the array declarations.*)

digits[0]	= 1	x[0]	= 0
digits[1]	= 2	x[1]	= 0.25
digits[2]	= 3	x[2]	= 0
digits[3]	= 4	x[3]	= -0.5
digits[4]	= 5		
digits[5]	= 6		

Strings (i.e., character arrays) are handled somewhat differently. When a string constant is assigned to an external or a static character array as a part of the array definition, the array size specification is usually omitted. The proper array size will be assigned automatically. This will include a provision for the *null character* `\0`, which is automatically added at the end of every string.

Consider the following two-character array definitions. Each includes the initial assignment of the string constant. However, the first array is defined as a three-element array, whereas the size of the second array is unspecified.

```
char color[3] = "RED";
char color[] = "RED";
```

The results of these initial assignments are not the same because of the null character, `\0`, which is automatically added at the end of the second string. Thus, the elements of the first array are

```
color[0] = 'R'
color[1] = 'E'
color[2] = 'D'
```

whereas the elements of the second array are

```
color[0] = 'R'
color[1] = 'E'
color[2] = 'D'
color[3] = '\0'
```

Thus, the first form is *incorrect*, since the null character `\0` is not included in the array. The array definition could also have been written as

```
char color[4] = "RED" ;
```

This definition is correct, since we are now defining a four-element array which includes an element for the null character.

However, many programmers prefer the earlier form, which omits the size specifier.

Processing an Array

Single operations which involve entire arrays are not permitted in C. Thus, if **a** and **b** are similar arrays (i.e., same data type, same dimensionality and same size), assignment operations, comparison operations, etc. must be carried out on an *element-by-element* basis. This is usually accomplished within a loop, where each pass through the loop is used to

process one array element. The number of passes through the loop will therefore equal the number of array elements to be processed.

Deviations About an Average Suppose we want to read a list of n floating-point quantities and then calculate their average, as in the earlier Example. In addition to simply calculating the average, however, we will also compute the *deviation* of each numerical quantity about the average, using the formula

$$d = x_i - \text{avg}$$

where x_i represents each of the given quantities, $i = 1, 2, \dots, n$, and **avg** represents the calculated average.

In order to solve this problem, we must store each of the given quantities in a one-dimensional, floating-point array.

This is an essential part of the program. The reason, which must be clearly understood, is as follows.

In all the earlier examples where we calculated the average of a list of numbers. Hence, each individual number was no longer available for subsequent calculations once the next number had been entered. Now, however, these individual quantities must be retained within the computer in order to calculate their corresponding deviations after the average has been determined. We therefore store them in a one-dimensional array, which we shall call *list*.

Let us define *list* to be a *100-element*, floating-point array. However, we need not make use of all 100 elements. Rather, we shall specify the actual number of elements by entering a positive integer quantity (not exceeding 100) for the integer variable n .

Here is the complete C program.

```
/* calculate the average of n numbers, then compute the
   deviation of each number about the average*/
#include <stdio.h>
```

```
main()
{
    int n, count;
    float avg, d, sum = 0;
    float list[100];

    /*read a value for n */
    printf("\nHow many numbers will be averaged? ");
    scanf(" %d", &n);

    /* read the numbers and calculate their sum */
    for (count = 0; count < n; ++count)
    {
        printf("\ni = %d    x = ", count + 1);
        scanf(" %f", &list[count]);
        sum += list[count];
    }

    /* calculate and display the average */
    avg = sum / n;
    printf("\nThe average is %5.2f\n", avg);

    /*calculate and display the deviation about the average */
```

```

    for (count = 0; count < n; ++count)
    {
        d = list[count] - avg;
        printf("\ni = %d    x = %5.2f    d = %5.2f", count+1,
list[count], d);
    }
}

```

Note that the second *scanf* function (within the first *for* loop) includes an *ampersand (&)* in front of *list[count]*, since we are entering a single array element rather than an entire array. The interactive session, including the data entry and the calculated results, is shown below. The user's responses are underlined.

How many numbers will be averaged? 5

```

i = 1    x = 3
i = 2    x = -2
i = 3    x = 12
i = 4    x = 4.4
i = 5    x = 3.5

```

The average is 4.18

```

i = 1    x = 3.00    d = -1.18
i = 2    x = -2.00    d = -6.18
i = 3    x = 12.00    d = 7.82
i = 4    x = 4.40    d = 0.22
i = 5    x = 3.50    d = -0.68

```

In some applications it may be desirable to assign initial values to the elements of an array. This requires that the array either be defined globally, or locally (within the function) as a static array. The next example illustrates the use of a global array definition.

Deviations About an Average Revisited Let us again calculate the average of a given set of numbers and then compute the deviation of each number about the average, as in the last Example. Now, however, let us assign the given numbers to the array within the array definition. To do so, let us move the definition of the array *list* outside of the main portion of the program. Thus, *list* will become an external array. Moreover, we will remove the explicit size specification from the definition, since the number of initial values will now determine the array size.

The initial values included in the following program are the same five values that were specified as input data for the previous example. To be consistent, we will also assign an initial value for *n*. This can be accomplished by defining *n* as either an automatic variable within main, or as an external variable. We have chosen the latter method, so that all the initial assignments that might otherwise be entered as input data are grouped together. Here is the complete program.

```

/* calculate the average of n numbers, then compute the
   deviation of each number about the average*/
#include <stdio.h>

int n = 5;
float list[] = {3, -2, 12, 4.4, 3.5};
main()

```

```

{
    int count;
    float avg, d, sum = 0;

    /* read the numbers and calculate their sum */
    for (count = 0; count < n; ++count)
        sum += list[count];

    /* calculate and display the average */
    avg = sum / n;
    printf("\nThe average is %5.2f\n", avg);

    /* calculate and display the deviation about the average */
    for (count = 0; count < n; ++count)
    {
        d = list[count] - avg;
        printf("\ni = %d      x = %5.2f      d = %5.2f", count+1,
list[count], d);
    }
}

```

Note that this version of the program does not require any input data. Execution of this program will generate the following output.

The average is 4.08

i = 1	x = 3.00	d = -1.08
i = 2	x = -2.00	d = -6.08
i = 3	x = 12.00	d = 7.92
i = 4	x = 4.40	d = 0.32
i = 5	x = 3.00	d = -1.08

Passing Arrays to Functions

An entire array can be passed to a function as an argument. The way the array is passed differs markedly, however, from that of an ordinary variable.

To pass an array to a function, the array name must appear by itself, without brackets or subscripts, as an actual argument within the function call. The corresponding formal argument is written in the same manner, though it must be declared as an array within the formal argument declarations. When declaring a one-dimensional array as a formal argument, the array name is written with a pair of empty square brackets. The size of the array is *not specified* within the formal argument declaration.

Some care is required when writing function prototypes that include array arguments. An empty pair of square brackets must follow the name of each array argument, thus indicating that the argument is an array. If argument names are not included in a function declaration, then an empty pair of square brackets must follow the array argument data type.

The following program outline illustrates the passing of an array from the main portion of the program to a function.

```

float average(int a, float x[]); /* function prototype */

main()

```

```

{
    int n;
    float avg;
    float list[100];
    ...
    avg = average(n, list);
    ...
}
float average(int a, float x[])    /* function definition */
{
    ...
}

```

Within **main** we see a call to the function **average**. This function call contains two actual arguments -the integer variable **n**, and the one-dimensional, floating-point array **list**. Notice that **list** appears as an ordinary variable within the function call; i.e., the square brackets are not included.

The first line of the function definition includes two formal arguments, **a** and **x**. The formal argument declarations establish **a** as an integer variable and **x** as a one-dimensional, floating-point array. Thus, there is a correspondence between the actual argument **n** and the formal argument **a**. Similarly, there is a correspondence between the actual argument **list** and the formal argument **x**. Note that the size of **x** is not specified within the formal argument declaration.

Note that the function prototype could have been written without argument names, as

```
float average(int, float[]); /* function declaration */
```

Either form is valid.

We have already discussed the fact that arguments are passed to a function by value when the arguments are ordinary variables. When an array is passed to a function, however, the values of the array elements *are not* passed to the function. Rather, the array name is interpreted as the **address** of the first array element (i.e., the address of the memory location containing the first array element). This address is assigned to the corresponding formal argument when the function is called. The formal argument therefore becomes a **pointer** to the first array element (more about this in the next chapter). Arguments that are passed in this manner are said to be passed **by reference** rather than passed *by value*.

When a reference is made to an array element within the function, the value of the element's subscript is added to the value of the pointer to indicate the address of the specified array element. Therefore, any array element can be accessed from within the function. Moreover, *if an array element is altered within the function, the alteration will be recognized in the calling portion of the program* (actually, throughout the entire scope of the array).

Here is a simple C program that passes a three-element integer array to a function, where the array elements are altered. The values of the array elements are displayed at three different places in the program, thus illustrating the effects of the alterations.

```

#include <stdio.h>
void modify(int a[]); /* function prototype */

main()
{
    int count, a[3];

```

```

printf("\nEnter any three numbers:\n");
for (count = 0; count < 3; ++count)
{
    printf("\na[%d] = ", count);
    scanf(" %d", &a[count]);
}

printf("From main(), before modification:\n");
for (count = 0; count < 3; ++count)
{
    printf("\na[%d] = %d", count, a[count]);
}

modify(a); /* function call */

printf("From main(), after modification:\n");
for (count = 0; count < 3; ++count)
{
    printf("\na[%d] = %d", count, a[count]);
}
}

void modify(int a[]) /* function definition */
{
    int count;

    printf("\nFrom the function, after modification the values:\n");
    for (count = 0; count < 3; ++count)
    {
        a[count] = -9;
        printf("a[%d] = %d\n", count, a[count]);
    }
    return;
}

```

The array elements are read from the user within the first loop appearing in *main*. These values are displayed as soon as they are assigned. The array is then passed to the function **modify**, when each array element is assigned the value **-9**. These new values are then displayed from within the function. Finally, the values of array elements are again displayed from **main()**, after control has been transferred back to *main* from *modify*. When the program is executed, the following output is generated.

```

Enter any three numbers:
a[0] = 1
a[1] = 2
a[2] = 3
From main(), before modification:

a[0] = 1
a[1] = 2
a[2] = 3
From the function, after modification the values:
a[0] = -9

```



```
a[1] = -9
a[2] = -9
From main(), after modification:
```

```
a[0] = -9
a[1] = -9
a[2] = -9
```

These results show that the elements of *a* are altered within *main* as a result of the changes that were made within *modify*.

We now consider a variation of the previous program. The present program includes the use of a global variable, and the transfer of both a local variable and an array to the function.

```
#include <stdio.h>
int a = 1;          /* global variable */
void modify(int b, int c[]); /* function prototype */

main()
{
    int b = 2, count;
    int c[3];

    printf("\nEnter any three numbers:\n");
    for (count = 0; count < 3; ++count)
    {
        printf("\na[%d] = ", count);
        scanf(" %d", &c[count]);
    }

    printf("From main(), before modification:\n");
    printf("\na = %d    b = %d\n", a, b);
    for (count = 0; count < 3; ++count)
    {
        printf("\nc[%d] = %d", count, c[count]);
    }

    modify(b, c); /*function access */

    printf("From main(), after modification:\n");
    printf("\na = %d    b = %d\n", a, b);
    for (count = 0; count < 3; ++count)
    {
        printf("\nc[%d] = %d", count, c[count]);
    }
}

void modify(int b, int c[]) /* function definition */
{
    int count;
```

```

        printf("\nFrom the function, after modification the
values:\n");
        a = -999;
        b = -999;
        printf("\na = %d    b = %d\n", a, b);
        for (count = 0; count < 3; ++count)
        {
            c[count] = -9;
            printf("c[%d] = %d\n", count, c[count]);
        }
        return;
    }
}

```

When the program is executed, the following output is generated.

Enter any three numbers:

```

a[0] = 10
a[1] = 20
a[2] = 30

```

From main(), before modification:

```

a = 1    b = 2
c[0] = 10
c[1] = 20
c[2] = 30

```

From the function, after modification the values:

```

a = -999    b = -999
c[0] = -9
c[1] = -9
c[2] = -9

```

From main(), after modification:

```

a = -999    b = 2
c[0] = -9
c[1] = -9
c[2] = -9

```

We now see that the value of *a* and the elements of the array *c* are altered within *main* as a result of the changes that were made in *modify*. However, the change made to *b* is confined to the function.

The ability to alter an array globally within a function provides a convenient mechanism for moving *multiple data items back and forth between the function and the calling portion of the program*. Simply pass the array to the function and then alter its elements within the function. Or, if the original array must be preserved, copy the array (element-by-element) within the calling portion of the program, pass the copy to the function, and perform the alterations. You should exercise some caution in altering an array within a function, however, since it is very easy to unintentionally alter the array outside of the function.

Reordering a List of Numbers Consider the well-known problem of rearranging (i.e., *sorting*) a list of n integer quantities into a sequence of increasing values. Let us write a sorting program in such a manner that unnecessary storage will not be used. Therefore, the program will contain only one array—a one-dimensional, integer array called x , which will be rearranged one element at a time.

The rearrangement will begin by scanning the entire array for the smallest number. This number will then be interchanged with the first number in the array, thus placing the smallest number at the top of the list. Next the remaining $n - 1$ numbers will be scanned for the smallest, which will be exchanged with the second number. The remaining $n - 2$ numbers will then be scanned for the smallest, which will be interchanged with the third number, and so on, until the entire array has been rearranged. The complete rearrangement will require a total of $n - 1$ passes through the array, though the length of each scan will become progressively smaller with each pass.

In order to find the smallest number within each pass, we sequentially compare each number in the array, $x[i]$, with the starting number, $x[item]$, where *item* is an integer variable that is used to identify a particular array element. If $x[i]$ is smaller than $x[item]$, then we interchange the two numbers; otherwise we leave the two numbers in their original positions. Once this procedure has been applied to the entire array, the first number in the array will be the smallest. We then repeat the entire procedure $n-2$ times, for a total of $n-1$ passes (item 0, 1, ..., $n-2$).

The only remaining question is how the two numbers are interchanged. To carry out the interchange, we first temporarily save the value of $x[item]$ for future reference. Then we assign the current value of $x[i]$ to $x[item]$. Finally, we assign the *original* value of $x[item]$, which has temporarily been saved, to $x[i]$. The interchange is now complete.

The strategy described above can be written in C as follows.

```
/* reorder all array elements */
for (item = 0; item < n - 1; ++item)
    /* find the smallest of all remaining elements */
    for (i = item + 1; i < n; ++i)
    {
        /*interchange two elements */
        temp = x[item];
        x[item] = x[i];
        x[i] = temp;
    }
```

We are assuming that *item* and *i* are integer variables that are used as counters, and that *temp* is an integer variable that is used to temporarily store the value of $x[item]$.

It is now a simple matter to add the required variable and array definitions, and the required input/output statements. Here is a complete C program.

```
/* reorder a one-dimensional, integer
array from smallest to largest */
#include <stdio.h>

#define SIZE 100

void reorder(int n, int x[]); /* function prototype */
```

```

main()
{
    int i, n, x[SIZE];

    /* read in a value for n */
    printf("How many numbers will be entered? ");
    scanf(" %d", &n);
    /*read the list of numbers */
    for (i = 0; i < n; ++i)
    {
        printf("i = %d    x = ", i+1);
        scanf(" %d", &x[i]);
    }

    /* reorder all array elements */
    reorder(n, x);

    /*display the reordered list of number */
    printf("\nReordered list of numbers:\n");
    for(i = 0; i < n; ++i)
        printf("i = %d    x = %d\n", i+1, x[i]);
}

void reorder(int n, int x[]) /*rearrange the list of numbers*/
{
    int i, item, temp;
    for (item = 0; item < n - 1; ++item)
    {
        /* find the smallest of all remaining elements */
        for (i = item + 1; i < n; ++i)
        {
            if (x[i] < x[item])
            {
                /* interchange two elements */
                temp = x[item];
                x[item] = x[i];
                x[i] = temp;
            }
        }
    }
    return;
}

```

In this program **x** is defined initially as a 100-element integer array. (Notice the use of the symbolic constant **SIZE** to define the size of **x**.) A value for **n** is first read into the computer, followed by numerical values for the first **n** elements of **x** (i.e., **x[0]**, **x[1]**, ..., **x[n-1]**). Following the data input, **n** and **x** are passed to the function **reorder**, where the first **n** elements of **x** are rearranged into ascending order. The reordered elements of **x** are then displayed from **main** at the conclusion of the program.

The declaration for `reorder` appearing in *main* is written as a function prototype, as a matter of good programming practice. Notice the way the function arguments are written. In particular, note that the second argument is identified as an integer array by the empty square brackets that follow the array name, i.e., `int x[ ]`. The square brackets are a required part of this argument specification.

The program will generate the following interactive dialog.

How many numbers will be entered? 6

```
i = 1    x = 595
i = 2    x = 78
i = 3    x = -1505
i = 4    x = 891
i = 5    x = -29
i = 6    x = -7
```

Reordered list of numbers:

```
i = 1    x = -1505
i = 2    x = -29
i = 3    x = -7
i = 4    x = 78
i = 5    x = 595
i = 6    x = 891
```

It should be mentioned that *the **return** statement cannot be used to return an array*, since **return** can pass only a *single-valued* expression back to the calling portion of the program. Therefore, if the elements of an array are to be passed back to the calling portion of the program, the array must either be defined as an external array whose scope includes both the function and the calling portion of the program, or it must be passed to the function as a formal argument.

Multidimensional Arrays

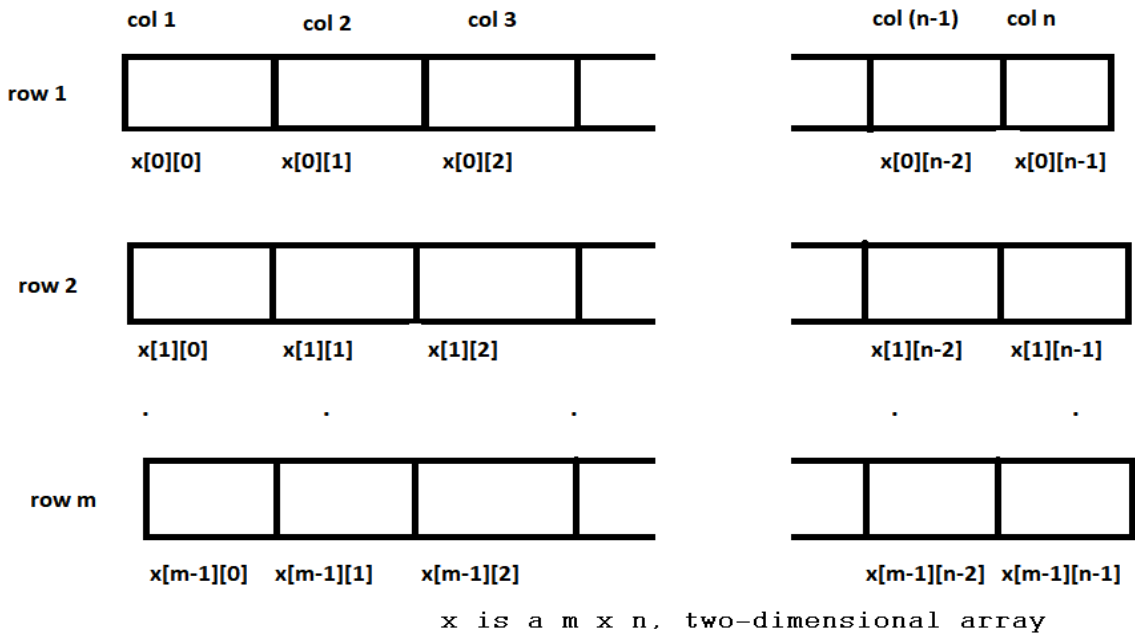
Multidimensional arrays are defined in much the same manner as one-dimensional arrays, except that a separate pair of square brackets is required for each subscript. Thus, a two-dimensional array will require two pairs of square brackets, a three-dimensional array will require three pairs of square brackets, and so on.

In general terms, a multidimensional array definition can be written as

```
storage-class data-type array[expression 1] [expression 2]...[expression n];
```

where *storage-class* refers to the storage class of the array, *data-type* is its data type, *array* is the array name, and *expression 1*, *expression 2*, ..., *expression n* are positive-valued integer expressions that indicate the number of array elements associated with each subscript. Remember that the *storage-class* is optional; the default values are *automatic* for arrays that are defined inside of a function, and *external* for arrays defined outside of a function.

A ***m x n***, two-dimensional array can be thought of as a *table* of values having *m* rows and *n* columns, as illustrated in the following figure. Extending this idea, a three-dimensional array can be visualized as a *set of tables* (e.g., a **book** in which each page is a table), and so on.



Several typical multidimensional array definitions are shown below.

```
float table[50][50];
char page[24][80];
static double records[100][66][255];
static double records[L][M][N];
```

The first line defines *table* as a floating-point array having 50 rows and 50 columns (hence $50 \times 50 = 2500$ elements), and the second line establishes *page* as a character array with 24 rows and 80 columns ($24 \times 80 = 1920$ elements). The third array can be thought of as a set of 100 static, double-precision tables, each having 66 lines and 255 columns (hence $100 \times 66 \times 255 = 1,683,000$ elements).

The last definition is like the preceding definition except that the array size is defined by the symbolic constants L, M and N. Thus, the values assigned to these symbolic constants will determine the actual size of the array.

Some care must be given to the order in which initial values are assigned to multidimensional array elements. (Remember, *only external and static arrays can be initialized.*) The rule is that the last (rightmost) subscript increases most rapidly, and the first (leftmost) subscript increases least rapidly. Thus, the elements of a two-dimensional array will be assigned by rows; i.e., the elements of the first row will be assigned, then the elements of the second row, and so on.

Consider the following two-dimensional array definition.

```
int values[3][4] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12};
```

Note that **values** can be thought of as a *table* having 3 rows and 4 columns (4 elements per row). Since the initial values are assigned by rows (i.e., last subscript increasing most rapidly), the results of this initial assignment are as follows.

```
values[0][0] = 1  values[0][1] = 2  values[0][2] = 3  values[0][3] = 4
values[1][0] = 5  values[1][1] = 6  values[1][2] = 7  values[1][3] = 8
values[2][0] = 9  values[2][1] = 10 values[2][2] = 11 values[2][3] = 12
```

Remember that the first subscript ranges from 0 to 2, and the second subscript ranges from 0 to 3.

The natural order in which the initial values are assigned can be altered by forming groups of initial values enclosed within braces (i.e., { . . . }). The values within each innermost pair of braces will be assigned to those array elements whose last subscript changes most rapidly. In a two-dimensional array, for example, the values within an inner pair of braces will be assigned to the elements of a row, since the second (column) subscript increases most rapidly. If there are too few values within a pair of braces, the remaining elements of that row will be assigned *zeros*. However, the number of values within each pair of braces cannot exceed the defined row size.

Here is a variation of the two-dimensional array definition presented in the last example.

```
int values[3][4] = {
    {1, 2, 3, 4},
    {5, 6, 7, 8},
    {9, 10, 11, 12}
};
```

This definition results in the same initial assignments as in the last example. Thus, the four values in the first inner pair of braces are assigned to the array elements in the first row, the values in the second inner pair of braces are assigned to the array elements in the second row, etc. Note that an outer pair of braces is required, containing the inner pairs.

Now consider the following two-dimensional array definition.

```
int values[3][4] = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};
```

This definition assigns values only to the first three elements in each row. Therefore, the array elements will have the following initial values.

```
values[0][0] = 1  values[0][1] = 2  values[0][2] = 3  values[0][3] = 0
values[1][0] = 4  values[1][1] = 5  values[1][2] = 6  values[1][3] = 0
values[2][0] = 7  values[2][1] = 8  values[2][2] = 9  values[2][3] = 0
```

Notice that the last element in each row is assigned a value of *zero*. If the preceding array definition is written as

```
int values[3][4] = {1, 2, 3, 4, 5, 6, 7, 8, 9};
```

then three of the array elements will again be assigned zeros, though the order of the assignments will be different. In particular, the array elements will have the following initial values.

```
values[0][0] = 1  values[0][1] = 2  values[0][2] = 3  values[0][3] = 4
values[1][0] = 5  values[1][1] = 6  values[1][2] = 7  values[1][3] = 8
values[2][0] = 9  values[2][1] = 0  values[2][2] = 0  values[2][3] = 0
```

Now the initial values are assigned with the last subscript increasing most rapidly, on a *row-by-row* basis, until all the initial values have been assigned. Without the inner pairs of braces, however, the initial values cannot be grouped for assignment to specific rows. Finally, consider the array definition

```
int values[3][4] = {
    {1, 2, 3, 4, 5},
    {6, 7, 8, 9, 10},
    {11, 12, 13, 14, 15}
};
```

This will result in a *compilation error*, since the number of values in each inner pair of braces (*five values* in each pair) exceeds the defined array size (*four elements* in each row).

The use of embedded groups of initial values can be generalized to higher dimensional arrays. Consider the following three-dimensional array definition.

```
int t[10][20][30] = {
    {
        /*table 1*/
        {1, 2, 3, 4}, /* row 1 */
        {5, 6, 7, 8}, /* row 2 */
        {9, 10, 11, 12} /* row 3 */
    },
    {
        /* table 2 */
        {21, 22, 23, 24}, /* row 1 */
        {25, 26, 27, 28}, /* row 2 */
        {29, 30, 31, 32} /* row 3 */
    }
};
```

Think of this array as a collection of **10 tables**, each having **20 rows** and **30 columns**. The groups of initial values will result in the assignment of the following *nonzero* values in the *first two tables*.

```
t[0][0][0] = 1    t[0][0][1] = 2    t[0][0][2] = 3    t[0][0][3] = 4
t[0][1][0] = 5    t[0][1][1] = 6    t[0][1][2] = 7    t[0][1][3] = 8
t[0][2][0] = 9    t[0][2][1] = 10   t[0][2][2] = 11   t[0][2][3] = 12
```

```
t[1][0][0] = 21   t[1][0][1] = 22   t[1][0][2] = 23   t[1][0][3] = 24
t[1][1][0] = 25   t[1][1][1] = 26   t[1][1][2] = 27   t[1][1][3] = 28
t[1][2][0] = 29   t[1][2][1] = 30   t[1][2][2] = 31   t[1][2][3] = 32
```

all the remaining array elements will be assigned zeros.

Multidimensional arrays are processed in the same manner as one-dimensional arrays, on an element-by-element basis. However, some care is required when passing multidimensional arrays to a function. In particular, the formal argument declarations within the function definition must include explicit size specifications in all the subscript positions *except the first*. These size specifications must be consistent with the corresponding size specifications in the calling program. The *first subscript* position may be written as an *empty pair* of square brackets, as with a one-dimensional array. The corresponding function prototypes must be written in the same manner.

Adding Two Tables of Numbers (sum of two matrices) Suppose we want to read two tables of integers into the computer, calculate the sums of the corresponding elements, i.e.,

```
c[i][j] = a[i][j] + b[i][j]
```

and then display the *new table* containing these sums. We will assume that all the tables contain the same number of rows and columns, not exceeding *20 rows* and *30 columns*.

Let us make use of the following variable and array definitions.

a, b, c = two-dimensional arrays, each having the same number of rows and the same number of columns, not exceeding 20 rows and 30 columns.

nrows = an integer variable indicating the actual number of rows in each table

ncols = an integer variable indicating the actual number of columns in each table

row = an integer counter that indicates the row number

col = an integer counter that indicates the column number

The program will be modularized by writing separate functions to read in an array, calculate the sum of the array elements, and display an array. Let us call these functions *readinput*, *computesums* and *writeoutput*, respectively.

The logic within each function is quite straightforward. Here is a complete C program for carrying out the computation.

```
/* calculate the sum of the elements in two tables of integers */
#include <stdio.h>

#define MAXROWS 20
#define MAXCOLS 30

void readinput(int a[][MAXCOLS], int nrows, int ncols);
void writeoutput(int c[][MAXCOLS], int nrows, int ncols);
void computesums(int a[][MAXCOLS], int b[][MAXCOLS], int
c[][MAXCOLS], int nrows, int ncols);

main()
{
    int nrows, ncols;
    /* array definitions */
    int a[MAXROWS][MAXCOLS], b[MAXROWS][MAXCOLS],
c[MAXROWS][MAXCOLS];

    printf("How many rows?");
    scanf(" %d", &nrows);
    printf("How many columns?");
    scanf(" %d", &ncols);

    printf("\nFirst Table:\n");
    readinput(a, nrows, ncols);

    printf("\nSecond Table:\n");
    readinput(b, nrows, ncols);

    computesums(a, b, c, nrows, ncols);
    printf("\nSum of the elements:\n");
    writeoutput(c, nrows, ncols);
}

/* read in a table of integers */
void readinput(int a[][MAXCOLS], int m, int n)
{
    int row, col;

    for (row = 0; row < m; ++row)
    {
        printf("\nEnter data for row no. %2d\n", row + 1);
        for (col = 0; col < n; ++col)
        {
            scanf(" %d", &a[row][col]);
        }
    }
    return;
}
```

```

}

/* add the elements of two integer tables */
void computesums(int a[][MAXCOLS], int b[][MAXCOLS], int
c[][MAXCOLS], int m, int n)
{
    int row, col;
    for (row = 0; row < m; ++row)
    {
        for(col = 0; col < n; ++col)
        {
            c[row][col] = a[row][col] + b[row][col];
        }
    }
    return;
}

/*display a table of integers */
void writeoutput(int a[][MAXCOLS], int m, int n)
{
    int row, col;
    for (row = 0; row < m; ++row)
    {
        for (col = 0; col < n; ++col)
        {
            printf("%4d", a[row][col]);
        }
        printf("\n");
    }
    //return;
}

```

The array definitions are expressed in terms of the symbolic constants MAXROWS and MAXCOLS, whose values are specified as 20 and 30, respectively, at the beginning of the program. Notice the manner in which the formal argument declarations are written within each function definition. For example, the first line of function *readinput* is written as

```
void readinput(int a[][MAXCOLS], int m, int n)
```

The array name, *a*, is followed by two pairs of square brackets. The first pair is empty, because the number of rows need not be specified explicitly. However, the second pair contains the symbolic constant MAXCOLS, which provides an explicit size specification for the number of columns. The array names appearing in the other function definitions (i.e., in functions *computesums* and *writeoutput*) are written in the same manner.

Also, notice the function prototypes at the beginning of the program. Each prototype is analogous to the first line of the corresponding function definition. Each array name is followed by two pairs of brackets, the first of which is empty. The second pair of brackets contains the size specification for the number of columns, as required.

Execution of the program will generate the following dialog.

```

How many rows?3
How many columns?4

```

First Table:

Enter data for row no. 1

1 2 3 4

Enter data for row no. 2

5 6 7 8

Enter data for row no. 3

9 10 11 12

Second Table:

Enter data for row no. 1

10 11 12 13

Enter data for row no. 2

14 15 16 17

Enter data for row no. 3

18 19 20 21

Sum of the elements:

11 13 15 17

19 21 23 25

27 29 31 33

Some C compilers are unable to pass sizeable multidimensional arrays to functions. In such situations it may be possible to redesign the program so that the multidimensional arrays are defined as *external (global) arrays*. Hence, the arrays need not be passed to functions as arguments. This strategy will not always work, however, because some programs (such as the program shown in the last example) use the same function to process different arrays. Problems of this type can usually be circumvented using pointers, as discussed in the next chapter.

Array and Strings

We have already seen that a string can be represented as a one-dimensional character-type array. Each character within the string will be stored within one element of the array. Some problems require that the characters within a string be processed individually. However, there are many other problems which require that strings be processed as complete entities. Such problems can be simplified considerably through the use special string-oriented library functions.

For example, most C compilers include library functions that allow strings to be *compared*, *copied* or *concatenated* (i.e., combined, one behind another). Other library functions permit operations on individual characters within strings; e.g., they allow individual characters to be located within strings, and so on. The following example illustrates the use of some of these library functions.

Reordering a List of Strings Suppose we wish to enter a list of strings into the computer, rearrange them into alphabetical order, and then display the rearranged list. The strategy for doing this is very similar to that shown in the earlier Example, where we rearranged a list of numbers into ascending order. Now, however, there is the additional complication of

comparing entire strings, rather than single numerical values. We will therefore store the strings within a two-dimensional character array. Each string will be stored in a separate row within the array.

To simplify, the computation, let us make use of the library functions **strcmp** and **strcpy**. These functions are used to *compare two strings* and to *copy one string to another*, respectively. (Some compilers also include the **strcmpi** function, which is a variation of the more common **strcmp**. The use of **strcmpi** is sometimes more convenient, since it does not distinguish between upper- and lowercase. However, it is not supported by the ANSI standard.) The **strcmp** function accepts two strings as arguments and returns an integer value, depending upon the relative order of the two strings, as follows:

- i. A negative value if the first string precedes the second string alphabetically.
- ii. A value of zero if the first string and the second string are identical (disregarding case).
- iii. A positive value if the second string precedes the first string alphabetically.

Therefore, if **strcmp**(string 1, string 2) returns a *positive* value, it would indicate that *string 2* must be moved, placing it ahead of *string 1* in order to *alphabetize* the two strings properly. The **strcpy** function also accepts two strings as arguments. Its first argument is generally an identifier that represents a string. The second argument can be a string constant or an identifier representing another string. The function copies the value of string 2 to string 1. Hence, it effectively causes one string to be assigned to another. Now, we will allow the program to accept an unspecified number of strings, until a string is entered whose first three characters are **END** (in either upper- or lowercase). The program will count the strings as they are entered, ignoring the last string, which contains **END**. Here is the entire program.

```
/* sort a list of strings alphabetically
using a two-dimensional character array */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

void reorder(int n, char x[][12]); /* function prototype */

main()
{
    int i, n = 0;
    char x[10][12]; /* 10 strings having 12 characters each */

    printf("Enter each string on a separate line below\n");
    printf("Type \"END\" when finished.\n");
    /*read in the list of strings */
    do
    {
        printf("string %d: ", n + 1);
        scanf(" %s", x[n]);
    } while (strcmp(x[n++], "END"));

    /*adjust the value of strings */
    reorder(--n, x); /*total n-1 strings since "END" is excluded */

    /*display the reordered list of strings */
    printf("\nReordered list of strings:\n");
    for (i = 0; i < n; ++i)
        printf("\nString %d: %s", i + 1, x[i]);
}
```

```

void reorder(int n, char x[][12])
{
    char temp[12];
    int i, item;

    for (item = 0; item < n-1; ++item)
    {
        /*find the lowest of all remaining strings */
        for(i = item+1; i < n; ++i)
        {
            if (strcmp(x[item], x[i]) > 0)
            {
                /*interchange the two strings */
                strcpy(temp, x[item]);
                strcpy(x[item], x[i]);
                strcpy(x[i], temp);
            }
        }
    }
    //return;
}

```

The ***strcmp*** function appears in two different places within this program: in *main*, when testing for a stopping condition, and in *rearrange*, when testing for the need to interchange two strings. The actual string interchange is carried out using ***strcpy***.

The dialog resulting from a typical execution of the program is shown below. The user's responses are underlined, as usual.

Enter each string on a separate line below
Type "END" when finished.

```

string 1: Pacific
string 2: Atlantic
string 3: Indian
string 4: Caribbean
string 5: Black
string 6: Red
string 7: Caspian
string 8: Baltic
string 9: END

```

Reordered list of strings:

```

String 1: Atlantic
String 2: Baltic
String 3: Black
String 4: Caribbean
String 5: Caspian
String 6: Indian
String 7: Pacific
String 8: Red
String 9: END

```

4. POINTERS

A *pointer* is a variable that represents the *location* (rather than the *value*) of a data item, such as a variable or an array element. Pointers are used frequently in C, as they have several useful applications. For example, pointers can be used to pass information *back and forth* between a function and its reference point. Pointers provide a way to return multiple data items from a function via function arguments. Pointers also permit references to other functions to be specified as arguments to a given function. This has the effect of passing functions as arguments to the given function.

Pointers are also closely associated with arrays and therefore provide an alternate way to access individual array elements. Moreover, pointers provide a convenient way to represent multidimensional arrays, allowing a single multidimensional array to be replaced by a lower-dimensional array of pointers. This feature permits a group of strings to be represented within a single array, though the individual strings may differ in length.

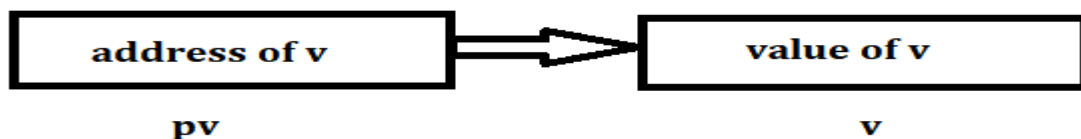
Fundamentals

Within the computer's memory, every stored data item occupies one or more contiguous memory cells (i.e., adjacent words or bytes). The number of memory cells required to store a data item depends on the type of data item. For example, a single character will typically be stored in one byte (8 bits) of memory; an integer usually requires two contiguous bytes; a floating-point number may require four contiguous bytes; and a double-precision quantity may require eight contiguous bytes.

Suppose v is a variable that represents some data item. The compiler will automatically assign memory cells for this data item. The data item can then be accessed if we know the location (i.e., the *address*) of the first memory cell. The address of v 's memory location can be determined by the expression $\&v$, where $\&$ is a unary operator, called the *address operator*, that evaluates the address of its operand. Now let us assign the address of v to another variable, pv . Thus,

$pv = \&v$

This new variable is called a *pointer* to v , since it points to the location where v is stored in memory. Remember, however, that pv represents v 's *address*, not its value. Thus, pv is referred to as a *pointer variable*. The relationship between pv and v is illustrated in Figure below.



*fig: Relationship between pv and v (where $pv = \&v$ and $v = *pv$)*

The data item represented by v (i.e., the data item stored in v 's memory cells) can be accessed by the expression $*pv$, where $*$ is a unary operator, called the *indirection operator*, that operates only on a pointer variable. Therefore, $*pv$ and v both represent the same data item (i.e., the contents of the same memory cells). Furthermore, if we write $pv = \&v$ and $u = *pv$, then u and v will both represent the same value; i.e., the value of v will indirectly be assigned to u . (It is assumed that u and v are of the same data type.)

Note: Adjacent memory cells within a computer are numbered consecutively, from the beginning to the end of the memory area. The number associated with each memory cell is known as the memory cell's address. Most computers use a hexadecimal numbering system to designate the addresses of consecutive memory cells, though some computers use an octal numbering system.

Shown below is a simple program that illustrates the relationship between two integer variables, their corresponding addresses and their associated pointers.

```
#include <stdio.h>

main()
{
    int u = 3;
    int v;
    int *pu; /*pointer to an integer */
    int *pv; /*pointer to an integer */

    pu = &u; /* assign address of u to pu */
    v = *pu; /* assign value of u to v */
    pv = &v; /* assign address of v to pv */

    printf("\nu=%d  &u=%X  pu=%X  *pu=%d", u, &u, pu, *pu);
    printf("\nv=%d  &v=%X  pv=%X  *pv=%d", v, &v, pv, *pv);
}
```

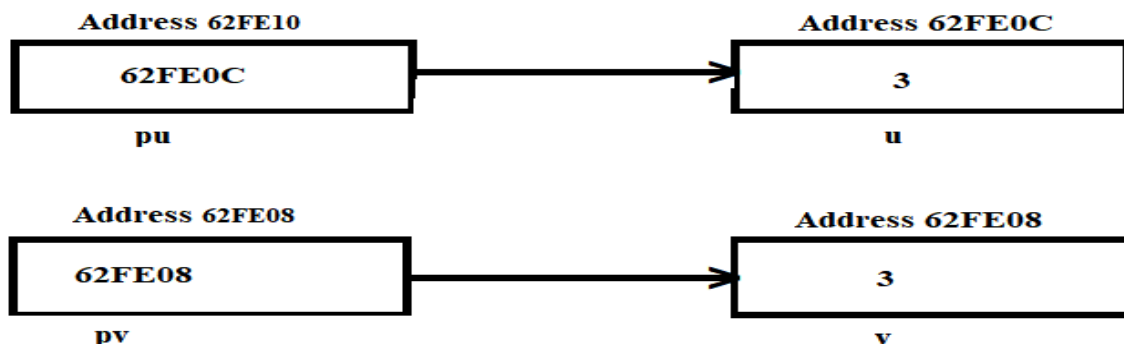
Note that *pu* is a pointer to *u*, and *pv* is a pointer to *v*. Therefore, *pu* represents the address of *u*, and *pv* represents the address of *v*. (Pointer declarations will be discussed in the next section.) Execution of this program results in the following output.

```
u=3  &u=62FE0C  pu=62FE0C  *pu=3
v=3  &v=62FE08  pv=62FE08  *pv=3
```

In the first line, we see that *u* represents the value 3, as specified in the declaration statement. The address of *u* is determined automatically by the compiler as 62FE0C (hexadecimal). The pointer *pu* is assigned this value; hence, *pu* also represents the (hexadecimal) address 62FE0C. Finally, the value to which *pu* points (i.e., the value stored in the memory cell whose address is 62FE0C) is 3, as expected.

Similarly, the second line shows that *v* also represents the value 3. This is expected, since we have assigned the value **pu* to *v*. The address of *v*, and hence the value of *pv*, is F8C. Notice that *u* and *v* have different addresses. And finally, we see that the value to which 62FE08 points is 3, as expected.

The relationships between *pu* and *u*, and *pv* and *v*, are shown in Figure below. Note that the memory locations of the pointer variables (i.e., address 62FE10 for *pu*, and 62FE08 for *pv*) are not displayed by the program.



The unary operators **&** and ***** are members of the same precedence group as the other unary operators, i.e., **-**, **++**, **--**, **!**, **sizeof** and **(type)**, which were presented in earlier Chapter. Remember that this group of operators has a higher precedence than the groups containing the arithmetic operators, and that the associativity of the unary operators is *right to left*.

The address operator (&) must act upon operands that are associated with unique addresses, such as ordinary variables or single array elements. Thus, *the address operator cannot act upon arithmetic expressions*, such as $2 * (u + v)$.

The indirection operator (*) can only act upon operands that are pointers (e.g., pointer variables). However, if ***pv*** points to ***v*** (i.e., ***pv*** = &***v***), then an expression such as ****pv*** can be used interchangeably with its corresponding variable ***v***. Thus, an indirect reference (e.g., ****pv***) can appear in place of an ordinary variable (e.g., ***v***) within a more complicated expression. Consider the simple C program shown below.

```
#include <stdio.h>

main()
{
    int u1, u2;
    int v = 3;
    int *pv;

    u1 = 2 * (v + 5); /* ordinary expression */

    pv = &v; /* pv points to v */
    u2 = 2 * (*pv + 5); /* equivalent expression */
    printf("\nu1 = %d  u2 = %d", u1, u2);
}
```

This program involves the use of two integer expressions. The first, $2 * (v + 5)$, is an ordinary arithmetic expression whereas the second, $2 * (*pv + 5)$, involves the use of a pointer. The expressions are equivalent, since ***v*** and ****pv*** each represent the same integer value. The following output is generated when the program is executed.

```
u1 = 16  u2 = 16
```

An *indirect reference* can also appear on the left side of an assignment statement. This provides another method for assigning a value to a variable or an array element. A simple C program is shown below.

```
#include <stdio.h>

main()
{
    int v = 3;
    int *pv;

    pv = &v; /* pv points to v */
    printf("\n*pv = %d  v = %d", *pv, v);

    *pv = 0; /* reset v indirectly */
    printf("\n*pv = %d  v = %d", *pv, v);
}
```

The program begins by assigning an initial value of 3 to the integer variable ***v***, and then assigns the address of ***v*** to the pointer variable ***pv***. Thus, ***pv*** becomes a pointer to ***v***. The expression ****pv*** therefore represents the value 3. The first *printf* statement is intended to illustrate this by displaying the current values of ****pv*** and ***v***.

Following the first *printf* statement, the value of **pv* is reset to 0. Therefore, *v* will be reassigned the value 0.

This is illustrated by the second *printf* statement, which causes the new values of **pv* and *v* to be displayed. When the program is executed, the following output is generated.

```
*pv = 3    v = 3
*pv = 0    v = 0
```

Pointer Declarations

Pointer variables, like all other variables, must be declared before they may be used in a C program. The interpretation of a pointer declaration differs, however, from the interpretation of other variable declarations. When a pointer variable is declared, the variable name must be preceded by an **asterisk (*)**. This identifies the fact that the variable is a pointer. The data type that appears in the declaration refers to the **object** of the pointer, i.e., the data item that is stored in the address represented by the pointer, rather than the pointer itself.

Thus, a pointer declaration may be written in general terms as

```
data-type *ptvar;
```

where **ptvar** is the name of the pointer variable, and **data-type** refers to the data type of the pointer's object. Remember that an asterisk must precede **ptvar**.

A C program contains the following declarations.

```
float u, v;
float *pv;
```

The first line declares **u** and **v** to be floating-point variables. The second line declares **pv** to be a pointer variable whose object is a floating-point quantity; i.e., **pv** points to a floating-point quantity. Note that **pv** represents an address, not a floating-point quantity. (Some additional pointer declarations are shown in the earlier Examples.

Within a variable declaration, a pointer variable can be initialized by assigning it the address of another variable. Remember that the variable whose *address* is assigned to the pointer variable must have been declared earlier in the program.

A C program contains the following declarations.

```
float u, v;
float *pv = &v
```

The variables **u** and **v** are declared to be floating-point variables and **pv** is declared as a pointer variable that points to a floating-point quantity, as in Example. In addition, the address of **v** is initially assigned to **pv**.

This terminology can be confusing. Remember that these declarations are equivalent to writing

```
float u, v;
float *pv;
...
pv = &v; /* assigns v's address to pv */
```

In general, it does not make sense to assign an integer value to a pointer variable. An exception, however, is an assignment of 0, which is sometimes used to indicate some special condition. In such situations the recommended programming practice is to define a symbolic constant **NULL** which represents 0, and to use **NULL** in the pointer initialization. This practice emphasizes the fact that the zero assignment represents a special situation.

A C program contains the following symbolic constant definitions and array declarations.

```
#define NULL 0

float u, v;

float *pv = NULL;
```

The variables **u** and **v** are declared to be floating-point variables and **pv** is declared as a pointer variable that points to a floating-point quantity. In addition, **pv** is initially assigned a value of 0 to indicate some special condition dictated by the logic of the program (which is not shown in this example). The use of the symbolic constant **NULL** suggests that this initial assignment is something other than the assignment of an ordinary integer value.

[We will see other kinds of pointer declarations later in this chapter.]

Passing Pointer to a Function

Pointers are often passed to a function as arguments. This allows data items within the calling portion of the program to be accessed by the function, altered within the function, and then returned to the calling portion of the program in altered form. We refer to this use of pointers as passing arguments by **reference** (or by **address** or by **location**), in contrast to passing arguments by **value** as discussed in *Function chapter*.

When an argument is passed by value, the data item is **copied** to the function. Thus, any alteration made to the data item within the function is not carried over into the calling routine. When an argument is passed by reference, however (i.e., when a pointer is passed to a function), the **address** of a data item is passed to the function. The contents of that address can be accessed freely, either within the function or within the calling routine. Moreover, any change that is made to the data item (i.e., to the contents of the address) will be recognized in both the function and the calling routine. Thus, the use of a pointer as a function argument permits the corresponding data item to be altered globally from within the function.

When pointers are used as arguments to a function, some care is required with the formal argument declarations within the function. Specifically, formal pointer arguments that must each be preceded by an asterisk. Function prototypes are written in the same manner. If a function declaration does not include variable names, the data type of each pointer argument must be followed by an asterisk. The use of pointer arguments is illustrated in the following example.

Here is a simple C program that illustrates the difference between ordinary arguments, which are passed by value, and pointer arguments, which are passed by reference.

```
#include <stdio.h>

void fun1(int u, int v);    /*function prototype*/
void fun2(int *pu, int *pv);/*function prototype*/

main()
{
    int u = 1;
    int v = 3;

    printf("\nBefore calling fun1:   u=%d    v=%d", u, v);
```

```

    fun1(u, v);
    printf("\nAfter calling fun1:      u=%d      v=%d", u, v);

    printf("\n\nBefore calling fun2: u=%d      v=%d", u, v);
    fun2(&u, &v);
    printf("\nAfter calling fun2:      u=%d      v=%d", u, v);
}

void fun1(int u, int v)
{
    u = 0;
    v = 0;
    printf("\nWithin fun1:              u=%d      v=%d", u, v);
    return;
}

void fun2(int *pu, int *pv)
{
    *pu = 0;
    *pv = 0;
    printf("\nWithin fun2:              *pu=%d      *pv=%d", *pu, *pv);
    return;
}

```

This program contains two functions, called *fun1* and *fun2*. This first function, *fun1*, receives two integers variables as arguments. These variables are originally assigned the values **1** and **3**, respectively. The values are then changed, to **0, 0** within *fun1*. The new values are not recognized in main, however, because the arguments were passed by value, and any changes to the arguments are local to the function in which the changes occur.

Now consider the second function, *fun2*. This function receives two **pointers** to integer variables as its arguments. The arguments are identified as pointers by the *indirection* operators (i.e., the *asterisks*) that appear in the argument declaration. In addition, the argument declaration indicates that the pointers contain the addresses of **integer** quantities.

Within *fun2*, the contents of the pointer addresses are reassigned the values **0, 0**. Since the addresses are recognized in both *fun2* and main, the reassigned values will be recognized within main after the call to *fun2*. Therefore, the integer variables **u** and **v** will have their values changed from **1, 3** to **0, 0**.

The six printf statements illustrate the values of **u** and **v**, and their associated values ***pu** and ***pv**, within main and within the two functions. Hence, the following output is generated when the program is executed.

```

Before calling fun1:      u=1      v=3
Within fun1:              u=0      v=0
After calling fun1:      u=1      v=3

Before calling fun2:      u=1      v=3
Within fun2:              *pu=0      *pv=0
After calling fun2:      u=0      v=0

```

Notice that the values of **u** and **v** are unchanged within **main** after the call to **fun1**, though the values of these variables are changed within **main** after the call to **fun2**. Thus, the output

illustrates the local nature of the alterations within **fun1**, and the global nature of the alterations within **fun2**.

This example contains some additional features that should be pointed out. Notice, for example, the function prototype

```
void fun2(int *pu,int *pv);
```

The items in parentheses identify the arguments as pointers to integer quantities. The pointer variables, **pu** and **pv**, have not been declared elsewhere in **main**. This is permitted in the function prototype, however, because **pu** and **pv** are dummy arguments rather than actual arguments. The function declaration could also have been written without any variable names, as

```
void fun2(int *,int *);
```

Now consider the declaration of the formal arguments within the first line of **fun2**, i.e.,

```
void fun2(int *pu,int *pv)
```

The formal arguments **pu** and **pv** are consistent with the dummy arguments in the function prototype. In this example the corresponding variable names are the same, though this is generally not required.

Finally, notice the way **u** and **v** are accessed within **fun2**, i.e.,

```
*pu = 0;
```

```
*pv = 0;
```

Thus, **u** and **v** are accessed indirectly, by referencing the contents of the addresses represented by the pointers **pu** and **pv**.

This is necessary because the variables **u** and **v** are not recognized as such within **fun2**.

We have already mentioned the fact that an array name is a pointer to the array; i.e., the array name represents the address of the first element in the array. Therefore, an array name is treated as a pointer when it is passed to a function. However, it is not necessary to precede the array name with an ampersand within the function call.

An array name that appears as a formal argument within a function definition can be declared either as a pointer or as an array of unspecified size. The choice is a matter of personal preference, though it will often be determined by the way the individual array elements are accessed within the function (more about this in the next section).

Analyzing a Line of Text Suppose we wish to analyze a line of text by examining each of the characters and determining into which of several different categories it falls. Suppose we count the number of vowels, consonants, digits, whitespace characters and “other” characters (punctuation, operators, brackets, etc.) This can easily be accomplished by reading in a line of text, storing it in a one-dimensional character array, and then analyzing the individual array elements. An appropriate counter will be incremented for each character. The value of each counter (number of vowels, number of consonants, etc.) can then be written out after all the characters have been analyzed.

Let us write a complete C program that will carry out such an analysis. To do so, we first define the following symbols.

line = an 80-element character array containing the line of text

vowels = an integer counter indicating the number of vowels

consonants = an integer counter indicating the number of consonants

digits = an integer counter indicating the number of digits

whitespace = an integer counter indicating the number of whitespace characters (blank spaces or tabs)

other = an integer counter indicating the number of characters that do not fall into any of the preceding categories

Notice that *newline* characters are not included in the "whitespace" category, because there can be no newline characters within a single line of text.

We will structure the program so that the line of text is read into the main portion of the program, and then passed to a function where it will be analyzed. The function will return the value of each counter after all the characters have been analyzed. The results of the analysis (i.e., the value of each counter) will then be displayed from the main portion of the program.

The actual analysis can be carried out by creating a loop to examine each of the characters. Within the loop we first *convert* each character that is a letter to *uppercase*. This avoids the need to distinguish between uppercase and lowercase letters. We can then categorize the character using a nest of *if-else* statements. Once the proper category has been identified, the corresponding counter is incremented. The entire process is repeated until the string termination character (**\0**) has been found. The complete C program is shown below.

```
/* count the number of vowels, consonants, digits, whitespace
   characters, and other characters in a line of text */
#include <stdio.h>
#include <ctype.h>

/* function prototype */
void scan_line(char line[], int *pv, int *pc, int *pd, int *pw, int *po);

main()
{
    char line[80];          /* line of text */
    int vowels = 0;         /* vowel counter */
    int consonants = 0;     /* consonant counter */
    int digits = 0;         /* digit counter */
    int whitespc = 0;       /* whitespace counter */
    int other = 0;          /* remaining character counter */

    printf("Enter a line of text:\n");
    scanf(" %[^\\n]", line);

    scan_line(line, &vowels, &consonants, &digits, &whitespc, &other);

    printf("\nNo. of vowels: %d", vowels);
    printf("\nNo. of consonants: %d", consonants);
    printf("\nNo. of digits: %d", digits);
    printf("\nNo. of whitespace characters: %d", whitespc);
    printf("\nNo. of other characters: %d", other);
}

void scan_line(char line[], int *pv, int *pc, int *pd, int *pw, int *po)
{
    /* analyze the characters in a line of text */
    char c;
    int count = 0;

    while ((c = toupper(line[count])) != '\\0')
    {
        if (c == 'A' || c == 'E' || c == 'I' || c == 'O' || c == 'U')
            ++ *pv;
    }
}
```

```

        else if (c >= 'A' && c <= 'Z')
            ++ *pc;
        else if (c >= '0' && c <= '9')
            ++ *pd;
        else if (c == ' ' || c == '\t')
            ++ *pw;
        else
            ++ *po;

        ++count;
    }
    //return;
}

```

Notice the function prototype for **scan-line** that appears at the beginning of the program. Notice the use of the **void** data type and notice the manner in which the argument data types are specified. Note the distinction between the array argument and the remaining pointer arguments.

Also, observe the way the actual arguments are written in the call to **scan-line**. The array argument, **line**, is not preceded by an *ampersand*, since arrays are, by definition, pointers. Each of the remaining arguments must be preceded by an ampersand so that its address, rather than its value, is passed to the function.

Now consider the function **scan-line**. All of the formal arguments, including **line**, are pointers. However, **line** is declared as an array whose size is unspecified, whereas the remaining arguments are specifically declared as pointers. It is possible (and quite common) to declare **line** as a pointer rather than an array. Thus, the first line of **scan-line** could have been written as

```
void scan_line(char line[], int *pv, int *pc, int *pd, int *pw, int *po).
```

To be consistent, the corresponding function prototype would then be written in a similar manner.

Incrementing the various counters also requires some explanation. First, note that it is the content of each address (i.e., the object of each pointer) that is incremented. Second, note that each indirection expression (e.g., ***pv**) is *preceded* by the unary operator **++**. Since the unary operators are evaluated from *right to left*, we are assured that the content of each address, rather than the address itself, is increased in value.

Here is a typical dialog that might be encountered when the program is executed. (The line of text entered by the user is underlined.)

```

Enter a line of text:
Personal Computers with memories in excess of 1024 MB are now quite common.
No. of vowels: 23
No. of consonants: 35
No. of digits: 4
No. of whitespace characters: 12
No. of other characters: 1

```

Recall that the **scanf** function requires those arguments that are ordinary variables to be preceded by *ampersands*. However, array names are exempted from this requirement. We now know about array names and addresses. Thus, the **scanf** function requires that the addresses of the data items being entered into the computer's memory be specified. The *ampersands* provide a means for accessing the addresses of ordinary single-valued variables. Ampersands are not required with array names, since array names themselves represent addresses.

It is possible to pass a *portion* of an array, rather than an entire array, to a function. To do so, the address of the first array element to be passed must be specified as an argument. The remainder of the array, starting with the specified array element, will then be passed to the function. The skeletal structure of a C program is shown below.

```
#include <stdio.h>
void process(float z[]);

main()
{
    float z[100];
    ...
    /* enter the values for elements of z */
    ...
    process(&z[50]);
    ...
}
void process(float f[])
{
    ...
    /* process elements of f */
    ...
    return;
}
```

Within **main**, **z** is declared to be a 100-element, floating-point array. After the elements of **z** are entered into the computer, the address of **z[50]** (i.e., **&z[50]**) is passed to the function **process**. Hence, the last 50 elements of **z** (i.e., the elements **z [50]** through **z [99]**) will be available to **process**.

Within **process**, the corresponding array is referred to as **f**. This array is declared to be a floating-point array whose size is unspecified. Thus, the fact that the function receives only a portion of **z** is immaterial; if all the array elements are altered within **process**, only the last 50 elements will be affected within **main**.

Within **process**, it may be desirable to declare the formal argument **f** as a *pointer* to a floating-point quantity rather than an array name. Thus, the outline of **process** may be written as

```
void process(float *f)
{
    ...
    /* process elements of f */
    ...
    return;
}
```

Notice the difference between the formal argument declarations in the two function outlines. Both declarations are valid.

A function can also *return a pointer* to the calling portion of the program. To do so, the function definition and any corresponding function declarations must indicate that the function will return a pointer. This is accomplished by *preceding* the function name by an *asterisk*. The asterisk must appear in both the function definition and the function

declarations. Shown below is the skeletal structure of a C program that transfers a double-precision array to a function and returns a pointer to one of the array elements.

```
#include <stdio.h>

double *scan(double z[]);

main()
{
    double z[100];
    double *pz;

    /* enter values for elements of z */
    ...
    pz = scan(z);
    ...
}

double *scan(double f[])
{
    double *pf;
    ...
    /* process elements of f */
    pf = ...;
    return(pf);
}
```

Within **main** we see that **z** is declared to be a 100-element, double-precision array, and **pz** is a pointer to a double-precision quantity. We also see a declaration for the function **scan**.

Note that **scan** will accept a double-precision array as an argument, and it will return a pointer to (i.e., the address of) a double-precision quantity. The asterisk preceding the function name (***scan**) indicates that the function will return a pointer.

Within the function definition, the first line indicates that **scan** accepts one formal parameter (**f []**) and returns a pointer to a double-precision quantity. The formal parameter will be a one-dimensional, double-precision array. The outline suggests that the address of one of the array elements is assigned to the pointer **pf** during or after the processing of the array elements. This address is then returned to **main**, where it is assigned to the pointer variable **pz**.

Pointers and One-Dimensional Arrays

Recall that an array name is really a pointer to the first element in the array. Therefore, if **x** is a one-dimensional array, then the address of the first array element can be expressed as either **&x[0]** or simply as **x**. Moreover, the address of the second array element can be written as either **&x[1]** or as **(x + 1)**, and so on. In general, the address of array element **(i + 1)** can be expressed as either **&x[i]** or as **(x + i)**. Thus, we have two different ways to write the address of any array element: We can write the actual array element, preceded by an ampersand; or we can write an expression in which the subscript is added to the array name.

In the latter case, we are dealing with a very special and unusual type of expression. In the expression **(x + i)**, for example, **x** represents an address, whereas **i** represents an integer quantity. Moreover, **x** is the name of an array whose elements may be characters, integers, floating-point quantities, etc. (though all the array elements must be of the same data type).

Thus, we are not simply adding numerical values. Rather, we are specifying an address that is a certain number of memory cells beyond the address of the first array element. Or, in simpler terms, we are specifying a location that is i array elements beyond the first. Hence, the expression $(x + i)$ is a symbolic representation for an address specification rather than an arithmetic expression.

Recall that the number of memory cells associated with an array element will depend upon the data type of the array as well as the particular computer's architecture. With some computers, for example, an integer quantity occupies two bytes (two memory cells), a floating-point quantity requires four bytes, and a double-precision quantity requires eight bytes of memory. With other computers, an integer quantity may require four bytes, and floating-point and double-precision quantities may each require eight bytes. And so on.

When writing the address of an array element in the form $(x + i)$, however, you need not be concerned with the number of memory cells associated with each type of array element; the C compiler adjusts for this automatically. You must specify only the address of the first array element (i.e., the name of the array) and the number of array elements beyond the first (i.e., a value for the subscript). The value of i is sometimes referred to as an **offset** when used in this manner.

Since $\&x[i]$ and $(x + i)$ both represent the address of the i^{th} element of x , it would seem reasonable that $x[i]$ and $*(x + i)$ both represent the contents of that address, i.e., the *value* of the i^{th} element of x . This is indeed the case. The two terms are interchangeable. Hence, either term can be used in any application. The choice depends upon your individual preferences. Here is a simple program that illustrates the relationship between array elements and their addresses.

```
#include <stdio.h>

main()
{
    static int x[10] = {10, 11, 12, 13, 14, 15, 16, 17, 18, 19};
    int i;

    for (i = 0; i <= 9; ++i)
    {
        /*display an array element*/
        printf("\ni = %d      x[i] = %d      *(x+i) = %d", i, x[i], *(x + i));

        /*display the corresponding array address*/
        printf("      &x[i] = %X      x+i = %X", &x[i], (x + i));
    }
}
```

This program defines a one-dimensional, 10-element integer array x , whose elements are assigned the values 10, 11, ..., 19. The action portion of the program consists of a loop that displays the value and the corresponding address of each array element. Note that the value of each array element is specified in two different ways, as $x[i]$ and as $*(x+i)$, in order to illustrate their equivalence. Similarly, the address of each array element is specified in two different ways, as $\&x[i]$ and as $(x + i)$, for the same reason. Therefore, the value and the address of each array element should appear twice. Execution of this program results in the following output.

$i = 0$	$x[i] = 10$	$*(x+i) = 10$	$\&x[i] = 403020$	$x+i = 403020$
$i = 1$	$x[i] = 11$	$*(x+i) = 11$	$\&x[i] = 403024$	$x+i = 403024$
$i = 2$	$x[i] = 12$	$*(x+i) = 12$	$\&x[i] = 403028$	$x+i = 403028$
$i = 3$	$x[i] = 13$	$*(x+i) = 13$	$\&x[i] = 40302C$	$x+i = 40302C$
$i = 4$	$x[i] = 14$	$*(x+i) = 14$	$\&x[i] = 403030$	$x+i = 403030$

i = 5	x[i] = 15	*(x+i) = 15	&x[i] = 403034	x+i = 403034
i = 6	x[i] = 16	*(x+i) = 16	&x[i] = 403038	x+i = 403038
i = 7	x[i] = 17	*(x+i) = 17	&x[i] = 40303C	x+i = 40303C
i = 8	x[i] = 18	*(x+i) = 18	&x[i] = 403040	x+i = 403040
i = 9	x[i] = 19	*(x+i) = 19	&x[i] = 403044	x+i = 403044

The output clearly illustrates the distinction between **x[i]**, which represents the value of the i^{th} array element, and **&x[i]**, which represents its address. Moreover, we see that the value of the i^{th} array element can be represented by either **x[i]** or **(x+i)**, and the address of the i^{th} element can be represented by either **&x[i]** or **x+i**. Thus, we see another comparison, between ***(x+i)**, which also represents the value of the i^{th} element, and **x+i**, which also represents its address.

Notice, for example, that the first array element (corresponding to **i = 0**) has been assigned a value of 10 and a (hexadecimal) address of 403020. The second array element has a value of 11 and an address of 403024, etc. Thus, memory location 403020 will contain the integer value 10, location 403024 will contain 11, and so on. You should understand that these addresses are assigned automatically by the compiler.

When assigning a value to an array element such as **x[i]**, the left side of the assignment statement may be written as either **x[i]** or as ***(x+i)**. Thus, a value may be assigned directly to an array element, or it may be assigned to the memory area whose address is that of the array element.

On the other hand, it is sometimes necessary to assign an address to an identifier. In such situations, a pointer variable must appear on the left side of the assignment statement. It is not possible to assign an arbitrary address to an array name or to an array element. Thus, expressions such as **x**, **(x+i)** and **&x[i]** cannot appear on the left side of an assignment statement. Moreover, the address of an array cannot arbitrarily be altered, so that expressions such as **++x** are not permitted. Consider the skeletal structure of the C program shown below.

```
#include <stdio.h>

main()
{
    int line[80];
    int *pl;
    ...
    /* assign values */
    line[2] = line[1];
    line[2] = *(line + 1);
    *(line + 2) = line[1];
    *(line + 2) = *(line + 1);
    /* assign addresses */
    pl = &line[1];
    pl = line + 1;
}
```

Each of the first four assignment statements assigns the value of the second array element (i.e., **line[1]**) to the third array element (**line[2]**). Thus, the four statements are all equivalent. An experienced programmer would probably choose either the first or the fourth, however, in order that the notation be consistent.

The last two assignment statements each assigns the address of the second array element to the pointer **pl**. We might choose to do this in an actual program if it were necessary to “tag” the address of **line[1]** for some reason.

Note that *the address of one array element cannot be assigned to some other array element*. Thus, we cannot write a statement such as

```
&line[2] = &line[1];
```

On the other hand, we can assign the value of one array element to another through a pointer if we wish, e.g.,

```
pl = &line[1];
```

```
line[2] = *pl;
```

or

```
pl = line + 1;
```

```
*(line + 2) = *pl;
```

If a numerical array is defined as a pointer variable, the array elements cannot be assigned initial values. Therefore, a conventional array definition is required if initial values will be assigned to the elements of a numerical array. However, a **character-type** pointer variable can be assigned an entire *string* as a part of the variable declaration. Thus, a string can conveniently be represented by either a one-dimensional character array or a character pointer. Shown below is a simple C program in which two strings are represented as one-dimensional character arrays.

```
#include <stdio.h>
char x[] = "This string is declared externally.\n";
main()
{
    static char y[] = "This string is declared within main.\n";

    printf("%s", x);
    printf("%s", y);
}
```

The first string is assigned to the external array **x[]**. The second string is assigned to the static array **y[]**, which is defined within **main**. This second definition occurs within a function; therefore **y[]** must be defined as a **static** array so that it can be initialized.

Here is a different version of the same program. The strings are now assigned to pointer variables rather than to one-dimensional arrays.

```
#include <stdio.h>
char *x = "This string is declared externally.\n";
main()
{
    char *y = "This string is declared within main.\n";

    printf("%s", x);
    printf("%s", y);
}
```

The external pointer variable **x** points to the beginning of the first string, whereas the pointer variable **y**, declared within **main**, points to the beginning of the second string. Note that **y** can now be initialized without being declared **static**. Execution of either program produces the following output.

This string is declared externally.
This string is declared within main.

Syntactically, of course, it is possible to declare a pointer variable **static**. However, there is no reason to do so in this example.

Dynamic Memory Allocation

Since an array name is a pointer to the first element within the array, it should be possible to define the array as a pointer variable rather than as a conventional array. Syntactically, the two definitions are equivalent. However, a conventional array definition results in a fixed block of memory being reserved at the beginning of program execution, whereas this does not occur if the array is represented in terms of a pointer variable. Therefore, the use of a pointer variable to represent an array requires some type of initial memory assignment before the array elements are processed. This is known as **dynamic memory allocation**. Generally, the **malloc** library function is used for this purpose, as illustrated in the next example. Suppose **x** is a one-dimensional, 10-element array of integers. It is possible to define **x** as a pointer variable rather than an array. Thus, we can write

```
int *x;
rather than
```

```
int x[10];
```

However, **x** is not automatically assigned a memory block when it is defined as a pointer variable, though a block of memory large enough to store 10 integer quantities will be reserved when **x** is defined as an array.

To assign enough memory for **x**, we can make use of the library function **malloc**, as follows.

```
x = (int *) malloc(10 * sizeof(int));
```

This function reserves a block of memory whose size (in bytes) is equivalent to 10 integer quantities. As written, the function returns a pointer to an integer. This pointer indicates the beginning of the memory block. In general, the type cast preceding **malloc** must be consistent with the data type of the pointer variable. Thus, if **y** were defined as a pointer to a double-precision quantity and we wanted enough memory to store 10 double-precision quantities, we would write

```
y = (double *) malloc(10 * sizeof(double));
```

*If the declaration is to include the assignment of initial values, however, then **x** must be defined as an array rather than a pointer variable.* For example,

```
int x[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
or
```

```
int x[] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
```

When programming in C, it is not unusual to use pointer expressions rather than references to individual array elements. The resulting programs may appear strange at first, though they are straightforward once you become comfortable accessing values that are stored in specified addresses. Generally, a small amount of practice is all that is required.

Reordering a List of Numbers To illustrate the use of pointers with dynamic memory allocation, let us once again consider the problem of reordering a list of integers, as described in earlier Example. Now, however, we will utilize pointer expressions to access numerical

values rather than refer explicitly to individual array elements. Here is the complete C program.

```
/* reordering a one-dimensional, integer array from
smallest to largest, using pointer notation */
#include <stdio.h>
#include <stdlib.h>

void reorder(int n, int *x);

main()
{
    int i, n, *x;

    /* read in a value for n */
    printf("\nHow many numbers will be entered? ");
    scanf(" %d", &n);

    /* dynamically allocate memory for n numbers */
    x = (int *) malloc(n * sizeof(int));

    /*read in the list of numbers */
    for (i = 0; i < n; ++i)
    {
        printf("i = %d x = ", i + 1);
        scanf(" %d", x + i);
    }

    /* reorder all array elements */
    reorder(n, x);

    /* display the reordered list of numbers */
    printf("\nReordered list of numbers:\n");
    for (i = 0; i < n; ++i)
        printf("i = %d x = %d\n", i+1, *(x + i));
    free(x);
}

void reorder(int n, int *x)
{
    int i, item, temp;
    for (item = 0; item < n - 1; ++item)
        for (i = item + 1; i < n; ++i)
            if(*(x + i) < *(x + item))
            {
                temp = *(x + item);
                *(x + item) = *(x + i);
                *(x + i) = temp;
            }

    return;
}
```

```
}
```

In this program, the integer array is defined as a pointer to an integer. Memory is initially assigned to the pointer variable via the `malloc` library function. Elsewhere in the program, pointer notation is used to process the individual array elements. For example, the function prototype now specifies that the second argument is a pointer to an integer quantity rather than an integer array. This pointer will identify the beginning of the integer array.

We also see that the `scanf` function now specifies the address of the i^{th} element as `x + i` rather than `&x[i]`. Similarly, the `printf` function now represents the value of the i^{th} element as `*(x+i)` rather than `x[i]`. The call to `reorder`, however, is the same as in the earlier program—namely, **`reorder(n,x);`**.

Within the function `reorder`, we see that the second formal argument is now defined as a pointer variable rather than an integer array. This is consistent with the function prototype. Even more pronounced, however, are the differences in the **`if`** statement. Notice that each reference to an array element is now written as the contents of an address. Thus, `x[i]` is now written as `*(x+i)`, and `x[item]` is now written as `*(x+item)`. This compound **`if`** statement can be viewed as a conditional interchange involving the contents of two different addresses, rather than an interchange of two different elements within a conventional array.

You should compare this program with that shown in the earlier Example in order to appreciate the differences. Both programs will generate identical results with the same input data. However, you should understand the syntactic differences between the two programs.

How many numbers will be entered? 10

```
i = 1    x = 20
i = 2    x = 30
i = 3    x = 10
i = 4    x = 5
i = 5    x = -15
i = 6    x = 15
i = 7    x = 25
i = 8    x = 35
i = 9    x = 22
i = 10   x = 40
```

Reordered list of numbers:

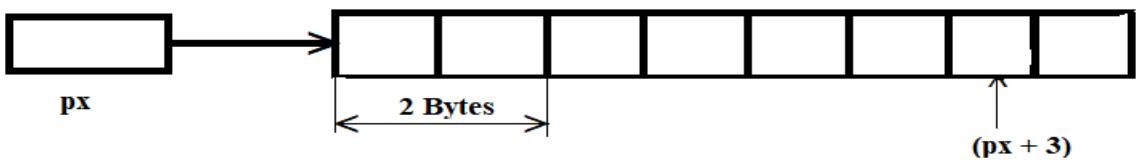
```
i = 1    x = -15
i = 2    x = 5
i = 3    x = 10
i = 4    x = 15
i = 5    x = 20
i = 6    x = 22
i = 7    x = 25
i = 8    x = 30
i = 9    x = 35
i = 10   x = 40
```

An important advantage of dynamic memory allocation is the ability to reserve as much memory as may be required during program execution, and then release this memory when it is no longer needed. Moreover, this process may be repeated many times during execution of a program. The library functions **`malloc`** and **`free`** are used for these purposes.

Operations on Pointers

An integer value can be added to or subtracted from a pointer variable, though the resulting expression must be interpreted very carefully. Suppose, for example, that **px** is a pointer variable that represents the address of some variable **x**. We can write expressions such as **++px**, **--px**, **(px + 3)**, **(px + i)**, and **(px - i)**, where **i** is an integer variable.

Each expression will represent an address that is located some distance from the original address represented by **px**. The exact distance will be the *product* of the integer quantity and the number of bytes or words associated with the data item to which **px** points. Suppose, for example, that **px** points to an integer quantity, and each integer quantity requires two bytes of memory. Then the expression **(px+3)** will result in an address that is *six bytes* beyond the integer to which **px** points, as illustrated in Figure below. It should be understood, however, that this new address will not necessarily represent the address of another data item, particularly if the data items stored between the two addresses involve different data types.



Consider the simple C program shown below.

```
#include <stdio.h>

main()
{
    int *px;
    int i = 1;
    float f = 0.3;
    double d = 0.005;
    char c = '*';

    px = &i;
    printf("Values: i=%i  f=%f  d=%lf  c=%c\n", i, f, d, c);
    printf("Addresses: &i=%X  &f=%X  &d=%X  &c=%X\n", &i, &f, &d, &c);
    printf("Pointer values: px=%X  px + 1=%X  px + 2=%X  px + 3=%X", px,
    px + 1, px + 2, px + 3);
}
```

This program displays the values and addresses associated with four different types of variables: **i**, an integer variable; **f**, a floating-point variable; **d**, a double-precision variable; and **c**, a character variable. The program also makes use of a pointer variable, **px**, which represents the address of **i**. The values of **px**, **px + 1**, **px + 2** and **px + 3** are also displayed, so that they may be compared with the addresses of the different variables. Execution of the program results in the following output.

```
Values: i=1  f=0.300000  d=0.005000  c=*
Addresses: &i=62FE14  &f=62FE10  &d=62FE08  &c=62FE07
Pointer values: px=62FE14  px + 1=62FE18  px + 2=62FE1C  px + 3=62FE20
```

The first line simply displays the values of the variables, and the second line displays their addresses, as assigned by the compiler. Notice that the number of bytes associated with each data item is different.

If **px** is defined as a pointer to a different type of object (e.g., a character or a floating-point quantity), then any integer constant that is added to or subtracted from the pointer will be interpreted differently. Each integer value will represent an equivalent number of individual

bytes if **px** points to a character, or a corresponding number of four-byte multiples if **px** points to a floating-point quantity. You are encouraged to verify this on your own.

One pointer variable can be subtracted from another provided both variables point to elements of the same array. The resulting value indicates the number of words or bytes separating the corresponding array elements.

In the program shown below, two different pointer variables point to the first and the last elements of an integer array.

```
#include <stdio.h>

main()
{
    int *px, *py;
    static int a[6] = {1, 2, 3, 4, 5, 6};

    px = &a[0];
    py = &a[5];

    printf("px = %X    py = %X", px, py);
    printf("\npy - px = %X", py - px);
}
```

In particular, the pointer variable **px** points to **a[0]**, and **py** points to **a[5]**. The difference, **py - px**, should be **5**, since **a[5]** is the fifth element beyond **a[0]**. Execution of the program results in the following output.

```
px = 403010    py = 403024
py - px = 5
```

The first line indicates that the address of **a[0]** is 403010, and the address of **a[5]** is 403024. The difference between these two hexadecimal numbers is 20 (when converted to decimal). Thus, **a[5]** is stored at an address which is 20 bytes beyond the address of **a[0]**. Since each integer quantity occupies four bytes, we would expect the difference between **py** and **px** to be **20/4 = 5**. The second line of output confirms this value.

Pointer variables can be compared provided both variables are of the same data type. Such comparisons can be useful when both pointer variables point to elements of the same array. The comparisons can test for either equality or inequality. Moreover, a pointer variable can be compared with zero (which is usually expressed as **NULL** when used in this manner).

Suppose **px** and **py** are pointer variables that point to elements of the same array. Several logical expressions involving these two variables are shown below. All the expressions are syntactically correct.

```
(px < py)
(px >= py)
(px == py)
(px != py)
(px == NULL)
```

These expressions can be used as any other logical expression. For example,


```
#include <stdio.h>

main()
{
    int *px, *py;
    static int a[6] = {10, 2, 3, 4, 5, 6};

    px = &a[0];
    py = &a[5];
    if (px > py)
        printf("px > py");
    else
        printf("\npx < py");

    if (*px > *py)
        printf("\n*px > *py");
    else
        printf("\n*px < *py");
}
```

Expressions such as **(px < py)** indicate whether the element associated with **px** is ranked ahead of the element associated with **py** (i.e., whether the subscript associated with ***px** is less than the subscript associated with ***py**).

```
px < py
*px > *py
```

You should understand that the operations discussed previously are the only operations that can be carried out on pointers. These permissible operations are summarized below.

- i. A pointer variable can be assigned the address of an ordinary variable (e.g., **pv = &v**).
- ii. A pointer variable can be assigned the value of another pointer variable (e.g., **pv = px**) provided both pointers point to objects of the same data type.
- iii. A pointer variable can be assigned a null (zero) value (e.g., **pv = NULL**, where **NULL** is a symbolic constant that represents the value **0**).
- iv. An integer quantity can be added to or subtracted from a pointer variable (e.g., **pv + 3**, **++pv**, **--pv**, etc.)
- v. One pointer variable can be subtracted from another provided both pointers point to elements of the same array.
- vi. Two pointer variables can be compared provided both pointers point to objects of the same data type.

Other arithmetic operations on pointers are not allowed. Thus, a pointer variable cannot be multiplied by a constant; two pointer variables cannot be added; and so on. Also, you are again reminded that an ordinary variable cannot be assigned an arbitrary address (i.e., an expression such as **&x** cannot appear on the left side of an assignment statement).

Pointers and Multidimensional Arrays

A two-dimensional array, for example, is a collection of one-dimensional arrays. Therefore, we can define a two-dimensional array as a pointer to a group of contiguous one-dimensional arrays. Thus, a two-dimensional array declaration can be written as

```
data-type (*ptvar) [expression 2];
```

rather than

```
data-type array[expression 1][expression 2];
```

This concept can be generalized to higher-dimensional arrays; that is,

```
data-type (*ptvar) [expression 2][expression 3]...[expression n];
```

replaces

```
data-type array[expression 1][expression 2]...[expression n];
```

In these declarations **data-type** refers to the data type of the array, **ptvar** is the name of the pointer variable, **array** is the corresponding array name, and **expression 1**, **expression 2**, ..., **expression n** are positive-valued integer expressions that indicate the maximum number of array elements associated with each subscript.

Notice the parentheses that surround the array name and the preceding asterisk in the pointer version of each declaration. These parentheses must be present. Without them we would be defining an array of pointers rather than a pointer to a group of arrays, since these symbols (i.e., the **square brackets** and the **asterisk**) would normally be evaluated *right to left*.

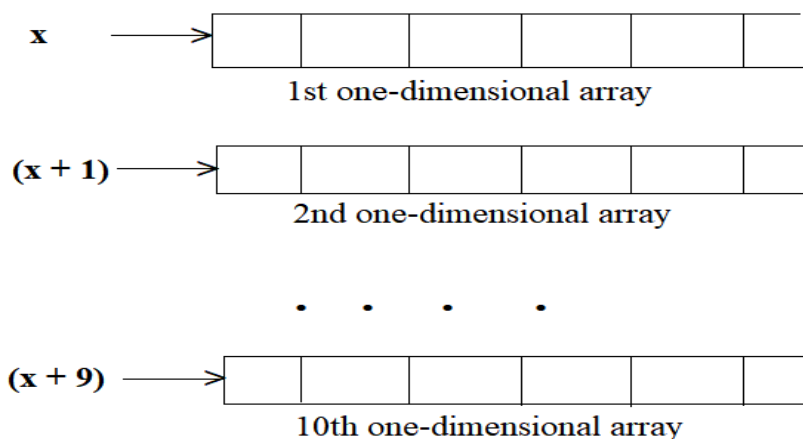
EXAMPLE 10.20 Suppose **x** is a two-dimensional integer array having **10** rows and **20** columns. We can declare **x** as

```
int (*x) [20];
```

rather than

```
int x[10][20];
```

In the first declaration, **x** is defined to be a *pointer* to a group of contiguous, one-dimensional, *20-element integer arrays*. Thus, **x** points to the first *20-element array*, which is the *first row* (i.e., **row 0**) of the original two-dimensional array. Similarly, **(x + 1)** points to the second *20-element array*, which is the second row (row 1) of the original two-dimensional array, and so on, as illustrated in figure below.



Now consider a three-dimensional floating-point array **t**. This array can be defined as

```
float (*t)[20][30];
rather than
```

```
float t[10][20][30];
```

In the first declaration, **t** is defined as a pointer to a group of contiguous two-dimensional, 20 x 30 floating-point arrays.

Hence, **t** points to the first 20 x 30 array, (**t + 1**) points to the second 20 x 30 array, and so on.

An individual array element within a multidimensional array can be accessed by the repeated use of the indirection operator. Usually, however, this procedure is more awkward than the conventional method for accessing an array element. The following example illustrates the use of the indirection operator.

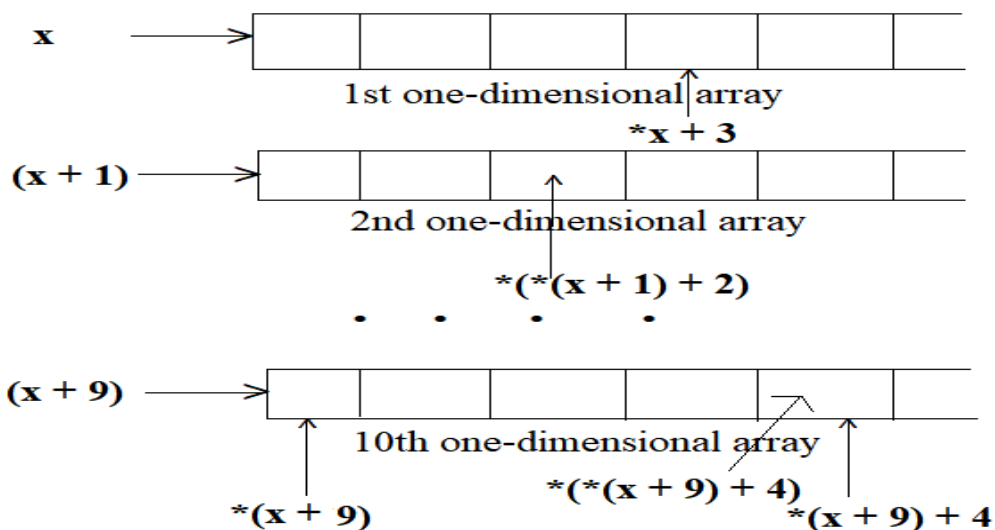
Suppose **x** is a two-dimensional integer array having **10** rows and **20** columns, as declared in the previous example. The item in row **2**, column **5** can be accessed by writing either

```
x[2][5]
```

or

```
*(* (x + 2) + 5)
```

The second form requires some explanation. First, note that **(x + 2)** is a pointer to row **2**. Therefore, the object of this pointer, *** (x + 2)**, refers to the entire row. Since row **2** is a one-dimensional array, *** (x + 2)** is a pointer to the first element in row **2**. We now add **5** to this pointer. Hence, **(* (x + 2) + 5)** is a pointer to element **5** (i.e., the sixth element) in row **2**. The object of this pointer, *** (* (x + 2) + 5)**, therefore refers to the item in column **5** of row **2**, which is **x[2][5]**. These relationships are illustrated in Figure below.



Adding Two Tables of Numbers In the earlier Example, we developed a C program to calculate the sum of the corresponding elements of two tables of integers. That program required three separate two-dimensional arrays, which were defined and processed in the conventional manner.

```
/* calculate the sum of the elements in two
tables of integers using pointer*/
```

```
#include <stdio.h>
#include <stdlib.h>
```

```

main()
{
    int row, col, nrows, ncols;
    int a[10][10], b[10][10], c[10][10];
    printf("How many rows? ");
    scanf(" %d", &nrows);
    printf("\nHow many columns? ");
    scanf(" %d", &ncols);

    printf("\nFirst Table:\n");
    for (row = 0; row < nrows; ++row)
    {
        printf("\nrow no. %2d\n", row + 1);
        for (col = 0; col < ncols; ++col)
            scanf(" %d", (*(a + row) + col));
    }

    printf("\nSecond Table:\n");
    for (row = 0; row < nrows; ++row)
    {
        printf("\nrow no. %2d\n", row + 1);
        for (col = 0; col < ncols; ++col)
            scanf(" %d", (*(b + row) + col));
    }
    /* calculating sum */
    for (row = 0; row < nrows; ++row)
        for (col = 0; col < ncols; ++col)
            *(*(c + row) + col) = *(*(a + row) + col) +
            *(*(b + row) + col);

    printf("\nSum of the elements:\n");
    for (row = 0; row < nrows; ++row)
    {
        for (col = 0; col < ncols; ++col)
            printf("%d\t", *(*(c + row) + col));
        printf("\n");
    }
}

```

Output:

How many rows? 3

How many columns? 3

First Table:

row no. 1
1 2 3

row no. 2
4 5 6

```
row no.   3
7 8 9
```

Second Table:

```
row no.   1
9 8 7
```

```
row no.   2
6 5 4
```

```
row no.   3
3 2 1
```

Sum of the elements:

```
10      10      10
10      10      10
10      10      10
```

Arrays of Pointers

A multidimensional array can be expressed in terms of an array of pointers rather than a pointer to a group of contiguous arrays. In such situations the newly defined array will have one less dimension than the original multidimensional array. Each pointer will indicate the beginning of a separate **(n - 1)**-dimensional array.

In general terms, a two-dimensional array can be defined as a one-dimensional array of pointers by writing

```
data-type *array[expression 1];
```

rather than the conventional array definition,

```
data-type array[expression 1][expression 2];
```

Similarly, an n-dimensional array can be defined as an (n - 1)-dimensional array of pointers by writing

```
data-type *array[expression 1][expression 2]...[expression n-1];
```

rather than

```
data-type array[expression 1][expression 2]...[expression n];
```

In these declarations **data-type** refers to the data type of the original n-dimensional array, **array** is the array name, and **expression 1, expression 2, . . . , expression n** are positive-valued integer expressions that indicate the maximum number of elements associated with each subscript.

Notice that the array name and its preceding asterisk are not enclosed in parentheses in this type of declaration. Thus, a *right-to-left* rule first associates the pairs of square brackets with **array**, defining the named object as an array. The preceding asterisk then establishes that the array will contain pointers.

Moreover, note that the **last** (i.e., the rightmost) expression is omitted when defining an *array of pointers*, whereas the **first** (i.e., the leftmost) expression is omitted when defining a *pointer to a group of arrays*. (Again, compare carefully with the declarations presented in the

last section.) You should understand the distinction between this type of declaration and that presented in the last section.

When an n -dimensional array is expressed in this manner, an individual array element within the n -dimensional array can be accessed by a *single* use of the *indirection* operator. The following example illustrates how this is done.

Suppose x is a two-dimensional integer array having 10 rows and 20 columns. We can define x as a one-dimensional array of pointers by writing

```
int *x[10];
```

Hence, $x[0]$ points to the beginning of the first row, $x[1]$ points to the beginning of the second row, and so on. Note that the number of elements within each row is not explicitly specified.

An individual array element, such as $x[2][5]$, can be accessed by writing

```
*(x[2] + 5)
```

In this expression, $x[2]$ is a pointer to the first element in row 2, so that $(x[2] + 5)$ points to element 5 (actually, the sixth element) within row 2. The object of this pointer, $*(x[2] + 5)$, therefore refers to $x[2][5]$. These relationships are illustrated in Figure below.

Now consider a three-dimensional floating-point array t . Suppose the dimensionality of t is $10 \times 20 \times 30$. This array can be expressed as a two-dimensional array of pointers by writing

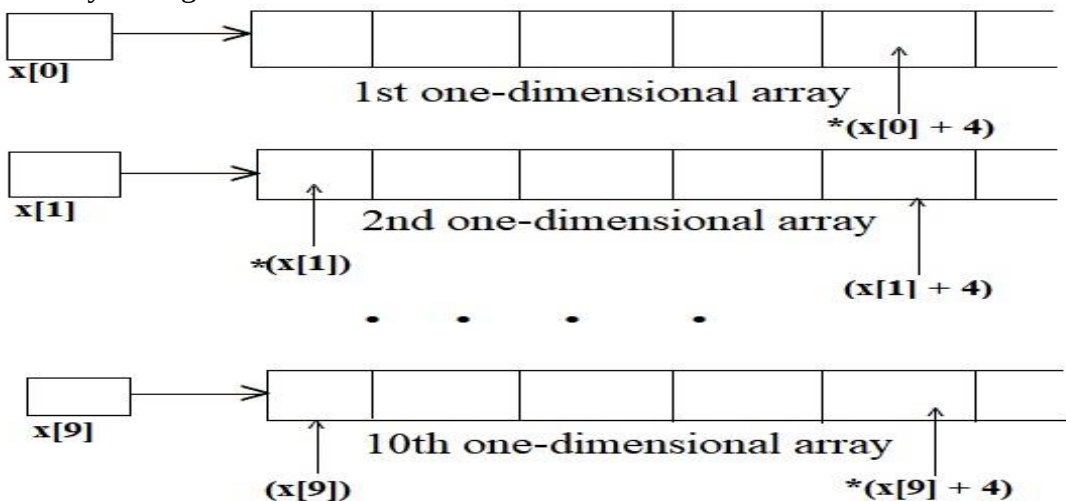
```
float *t[10][20];
```

Therefore, we have 200 pointers (10 rows, 20 columns), each pointing to a one-dimensional array.

An individual array element, such as $t[2][3][5]$, can be accessed by writing

```
*(t[2][3] + 5)
```

In this expression, $t[2][3]$ is a pointer to the first element in the one-dimensional array represented by $t[2][3]$. Hence, $(t[2][3] + 5)$ points to element 5 (the sixth element) within this array. The object of this pointer, $*(t[2][3] + 5)$, therefore represents $t[2][3][5]$. This situation is directly analogous to the two-dimensional case described above.



Adding Two Tables of Numbers Here is yet another version of the programs presented in the earlier Examples, which calculate the sum of the corresponding elements of two tables of integers. Now each two-dimensional array is represented as an array of pointers to one-dimensional arrays. Each one-dimensional array will correspond to one row within the original two-dimensional array.

```

/* calculate the sum of the elements in two tables of integers */
/* each two-dimensional array is represented as an array of pointers each
Pointer indicates a row in the original 2-dimensional array */
#include <stdio.h>
#include <stdlib.h>

#define MAXROWS 20

/* function prototypes */
void readinput(int *a[MAXROWS], int nrows, int ncols);
void computesum(int *a[MAXROWS], int *b[MAXROWS], int *c[MAXROWS],
int nrows, int ncols);
void writeoutput(int *c[MAXROWS], int nrows, int ncols);

main()
{
    int row, nrows, ncols;
    /* array of pointer definitions */
    int *a[MAXROWS], *b[MAXROWS], *c[MAXROWS];

    printf("How many rows? ");
    scanf(" %d", &nrows);
    printf("\nHow many columns? ");
    scanf(" %d", &ncols);

    /* allocate initial memory */
    for (row = 0; row < nrows; ++row)
    {
        a[row] = (int *) malloc(ncols * sizeof(int));
        b[row] = (int *) malloc(ncols * sizeof(int));
        c[row] = (int *) malloc(ncols * sizeof(int));
    }

    printf("\nFirst Table:\n");
    readinput(a, nrows, ncols);

    printf("\nSecond Table:\n");
    readinput(b, nrows, ncols);

    computesum(a, b, c, nrows, ncols);

    printf("\nSum of the elements:\n");
    writeoutput(c, nrows, ncols);
}

void readinput(int *a[MAXROWS], int m, int n)
{ /* read in a table of integers */
    int row, col;

    for (row = 0; row < m; ++row)
    {
        printf("\nrow no. %2d\n", row + 1);
        for (col = 0; col < n; ++col)
            scanf(" %d", (a[row] + col));
    }
    return;
}

void writeoutput(int *a[MAXROWS], int m, int n)
{ /* write out a table of integers */

```

```

    int row, col;

    for (row = 0; row < m; ++row)
    {
        for (col = 0; col < n; ++col)
            printf("%d\t", *(a[row] + col));
        printf("\n");
    }
    return;
}

void computesum(int *a[MAXROWS], int *b[MAXROWS], int *c[MAXROWS], int m,
int n)
{/* add the elements of two integer tables */
    int row, col;

    for (row = 0; row < m; ++row)
        for (col = 0; col < n; ++col)
            *(c[row] + col) = *(a[row] + col) + *(b[row] + col);
    return;
}

```

Notice that **a**, **b** and **c** are now defined as one-dimensional arrays of pointers. Each array will contain MAXROWS elements (i.e., MAXROWS pointers). Each array element will point to a one-dimensional array of integers. The function prototypes and the formal argument declarations within the subordinate functions also represent the arrays in this manner. Each one-dimensional array that is the object of a pointer (i.e., each row within each of the tables) must be allocated an initial block of memory. The **malloc** function accomplishes this. For example, each row within the first table is allocated an initial block of memory in the following manner.

```
a[row] = (int *)malloc(ncols * sizeof(int));
```

This statement associates a block of memory large enough to store **ncols** integer quantities with each pointer (i.e., with each element of **a**). Similar memory allocations are written for **b** and **c**. These **malloc** statements are placed within a **for** loop in order to allocate a block of memory for each of the nonzero rows within the three tables.

Notice the way the individual array elements are processed, using a combination of array and pointer notation. For example, in **readinput** each array element is now referenced as

```
scanf(" %d", (a[row] + col));
```

Similarly, in **computesum** and **writeoutput** the individual array elements are referenced as

```
*(c[row] + col) = *(a[row] + col) + *(b[row] + col);
```

and

```
printf("%d", *(a[row] + col));
```

respectively. These statements could also have been written using conventional two-dimensional array notation.

Also, we can do this job using pointer to an array notation as shown below.

```

/* calculate the sum of the elements in two tables of integers */
/* each 2-dimensional array is processed as an array of pointers
   to a set of 1-dimensional integer arrays */
#include <stdio.h>
#include <stdlib.h>

#define MAXROWS 20

```



```

/* function prototypes */
void readinput(int *a[MAXROWS], int nrows, int ncols);
void computesum(int *a[MAXROWS], int *b[MAXROWS], int *c[MAXROWS],
int nrows, int ncols);
void writeoutput(int *c[MAXROWS], int nrows, int ncols);

main()
{
    int row, nrows, ncols;
    /* array of pointer definitions */
    int *a[MAXROWS], *b[MAXROWS], *c[MAXROWS];

    printf("How many rows? ");
    scanf(" %d", &nrows);
    printf("\nHow many columns? ");
    scanf(" %d", &ncols);

    /* allocate initial memory */
    for (row = 0; row < nrows; ++row)
    {
        a[row] = (int *) malloc(ncols * sizeof(int));
        b[row] = (int *) malloc(ncols * sizeof(int));
        c[row] = (int *) malloc(ncols * sizeof(int));
    }

    printf("\nFirst Table:\n");
    readinput(a, nrows, ncols);

    printf("\nSecond Table:\n")    ;
    readinput(b, nrows, ncols);

    computesum(a, b, c, nrows, ncols);

    printf("\nSum of the elements:\n");
    writeoutput(c, nrows, ncols);
}

void readinput(int *a[MAXROWS], int m, int n)
{
    /* read in a table of integers */
    int row, col;

    for (row = 0; row < m; ++row)
    {
        printf("\nrow no. %2d\n", row + 1);
        for (col = 0; col < n; ++col)
            scanf(" %d", (*(a + row) + col));
    }
    return;
}

void writeoutput(int *a[MAXROWS], int m, int n)
{
    /* write out a table of integers */
    int row, col;

    for (row = 0; row < m; ++row)
    {
        for (col = 0; col < n; ++col)
            printf("%d\t", (*(a + row) + col));
        printf("\n");
    }
}

```

```

        return;
    }

void computesum(int *a[MAXROWS], int *b[MAXROWS], int *c[MAXROWS],
int m, int n)
{ /* add the elements of two integer tables */
    int row, col;

    for (row = 0; row < m; ++row)
        for (col = 0; col < n; ++col)
            *(*(c + row) + col) = *(*(a + row) + col) +
*(*(b + row) + col);
    return;
}

```

In this program **a**, **b** and **c** are each defined as an array of pointers to integers. Each array has a maximum of MAXROWS elements. The function prototypes and the formal argument declarations within the subordinate functions also represent the arrays in this manner.

Since each element of **a**, **b** and **c** is a pointer, we must provide each pointer with enough memory for each row of integer quantities, using the **malloc** function. These memory allocations appear in main, after the values for nrows and ncols have been entered. Consider the first memory allocation; i.e.,

```
a[row] = (int *)malloc(ncols * sizeof(int));
```

In this statement **a[0]** points to the first row. Similarly, **a[1]** points to the second row, **a[2]** points to the third row, and so on. Thus, each array element points to a block of memory large enough to store one row of integer quantities (ncols integer quantities). Similar memory allocations are written for the other two arrays.

The individual array elements are processed by repeated use of the indirection operator. In **readinput**, for example, each array element is referenced as

```
scanf(" %d", (*(a + row) + col));
```

Similarly, the addition of the array elements within **computesum** is written as

```
*(*(c + row) + col) = *(*(a + row) + col) + *(*(b + row) + col);
```

And the first printf statement within **writeoutput** is written as

```
printf("%d", *(*(a + row) + col));
```

We could, of course, have used the more conventional notation within the functions. Thus, in **readinput** we could have written

```
scanf(" %d", &a[row][col]);
```

instead of

```
scanf(" %d" , (*(a + row) + col));
```

Similarly, in **computesums** we could have written

```
c[row][col] = a[row][col] + b[row][col];
```

instead of

```
*(*(c + row) + col) = *(*(a + row) + col) + *(*(b + row) + col);
```

and in **writeoutput** we could have written

```
printf("%d", a[row][col]);
```

rather than

```
printf(" %d", *(*(a + row) + col));
```

This program will generate output identical to that shown in the previous Example when executed with the same input data.

Pointer arrays offer a particularly convenient method for storing strings. In this situation, each array element is a character-type pointer that indicates the beginning of a separate string. Thus, an *n*-element array can point to *n* different strings. Each individual string can be accessed by referring to its corresponding pointer.

Suppose the following strings are to be stored in a character-type array

```
PACIFIC
ATLANTIC
INDIAN
CARIBBEAN
BERING
BLACK
RED
NORTH
BALTIC
CASPIAN
```

These strings can be stored in a two-dimensional character-type array; e.g.,

```
char name[10][12];
```

Note that **name** contains 10 rows, to accommodate the 10 strings. Each row must be large enough to store at least 10 characters, since CARIBBEAN contains 9 letters plus the null character (**\0**) at the end. To provide for larger strings, we are allowing each row to contain as many as 12 characters.

A better way to do this is to define a 10-element array of pointers; i.e.

```
char *name[10];
```

Thus, name[0] will point to PACIFIC, name[1] will point to ATLANTIC, and so on. Note that it is not necessary to include a maximum string size within the array declaration. However, a specified amount of memory will have to be allocated for each string later in the program, e.g.,

```
name[i] = (char *) malloc(12 * sizeof(char));
```

Just as individual strings can be accessed by referring to the corresponding pointer (i.e., the corresponding array element), so individual string elements can be accessed using the indirection operator. For example, ***(*(name + 2) + 3)** refers to the fourth character (i.e., character number 3) in the third string (row number 2) of the array **name**, as defined in the preceding example.

Rearrangement of the strings can be accomplished simply by reassigning the pointers (i.e., by reassigning the elements in an array of pointers). The strings themselves need not be moved.

Reordering a List of Strings Consider once again the problem of entering a list of strings into the computer and rearranging them into alphabetical order. We saw one approach to this problem in the earlier Example, where the list of strings was stored in a two-dimensional array. Let us now approach this problem using a one-dimensional array of pointers, where each pointer indicates the beginning of a string. The string interchanges can now be carried

out simply by reassigning the pointers, as required. The complete program is presented below.

```

/* sort a list of strings into alphabetical order using an array of
pointers */
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

void reorder(int n, char *x[]);

main()
{
    int i, n = 0;
    char *x[10];

    printf("Enter each string on a separate line below\n");
    printf("Type \"END\" when finished\n");

    /* read in the list of string */
    do
    {
        /*allocate memory */
        x[n] = (char *) malloc(12 * sizeof(char));
        printf("string %d: ", n + 1);
        scanf(" %s", x[n]);
    } while(strcmp(x[n++], "END"));

    /*reorder the list of string */
    reorder(--n, x);

    /*display the reordered list of strings */
    printf("\nReordered list of strings:\n");
    for(i = 0; i < n; ++i)
        printf("\nstring %d: %s", i + 1, x[i]);
}

void reorder(int n, char *x[])
{
    char *temp;
    int i, item;

    for (item = 0; item < n - 1; ++item)
    {
        for (i = item + 1; i < n; ++i)
        {
            if(strcmp(x[item], x[i]) > 0)
            {
                temp = x[item];
                x[item] = x[i];
                x[i] = temp;
            }
        }
    }
    return;
}

```

The logic is essentially the same as that shown in the earlier Example, though the array containing the strings is now defined as an *array of pointers*. Notice that the second formal argument in the function **reorder** is declared in the same manner. Also, notice the string interchange routine (i.e., the **if** statement) within **reorder**. It is now the pointers, not the actual strings, that are interchanged. Hence, the library function **strcpy**, which was used in the earlier Example, is not required. The program will therefore run somewhat faster than the earlier version.

Execution of this program will generate the same dialog as that shown in the Example.

More about Pointer Declarations

Several Declarations involving Pointer

- ✓ **int *p;** /* p is a pointer to an integer quantity*/
- ✓ **int *p[10];** /* p is a 10-element array of pointers to integer quantities*/
- ✓ **int (*p)[10];** /* p is a pointer to 10-element integer array.*/
- ✓ **int *p(void);** /* p is a function that returns a pointer to an integer quantity*/
- ✓ **int p(char *a);** /* p is a function that accepts an argument which is a pointer to a character and returns an integer quantity*/
- ✓ **int *p(char *a);** /* p is a function that accepts an argument which is a pointer to character and returns a pointer to an integer quantity*/
- ✓ **int (*p)(char *a);** /* p is pointer to a function that accepts an argument which is a pointer to a character returns an integer quantity*/
- ✓ **int (*p(char *a))[10];** /* p is a function that accepts an argument which is a pointer to character returns a pointer to a 10-element integer array*/

5. STRUCTURE AND UNION

In the array, which is a data structure whose elements are all the *same data type*. We now turn our attention to the *structure*, in which the individual elements can *differ in type*. Thus, a single structure might contain integer elements, floating-point elements and character elements. Pointers, arrays and other structures can also be included as elements within a structure. The individual structure elements are referred to as **members**.

This chapter is concerned with the use of structures within a C program. We will see how structures are defined, and how their individual members are accessed and processed within a program. The relationships between structures and pointers, arrays and functions will also be examined.

Closely associated with the structure is the *union*, which also contains multiple members. Unlike a structure, however, the members of a union share the *same storage area*, even though the individual members may differ in type. Thus, a union permits several different data items to be stored in the same portion of the computer's memory at different times. We will see how unions are defined and utilized within a C program.

Defining a Structure

Structure declarations are somewhat more complicated than array declarations, since a structure must be defined in terms of its individual members. In general terms, the composition of a structure may be defined as

struct tag

```

{
    member 1;
    member 2;
    ...
    member m;
};

```

In this declaration, **struct** is a required keyword; **tag** is a name that identifies structure of this type (i.e.; structure having this composition); and **member 1, member 2, ..., member m** are individual member declarations. (Note: There is no formal distinction between a structure *definition* and a structure *declaration*; the terms are used interchangeably.)

The individual members can be ordinary variable, pointers, arrays, or other structures. The member names within a structure must be distinct from one another, though a member name can be the same as the name of a variable that is defined outside the structure. A storage class, however, cannot be assigned to an individual member, and individual members cannot be initialized within a structure type declaration.

Once the composition of structure has been defined, individual structure-type variables can be declared as follows:

```
storage-class struct tag variable 1, variable 2, ..., variable n;
```

Where **storage-class** is an optional storage class specifier, **struct** is a required keyword, **tag** is the name that appeared in the structure declaration, and **variable 1, variable 2, ..., variable n** are **struct** variables of type **tag**.

A typical structure declaration is shown below.

```

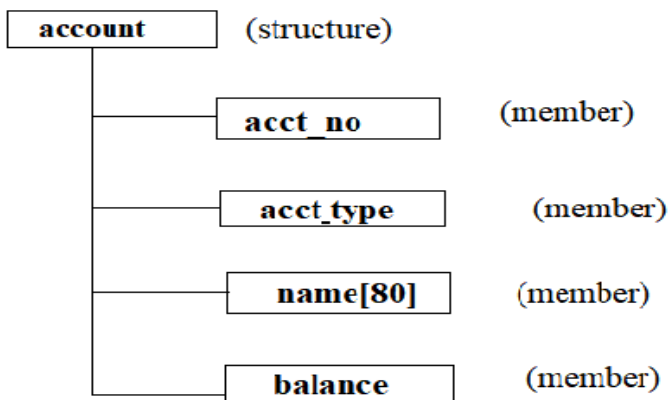
struct account
{
    int acct_no;
    char acct_type;
    char name[80];
    float balance;
};

```

This structure is named **account**. It contains four members: an integer quantity (*acct_no*), a

single character (*acct_type*), an 80-element character array (*name[80]*), and a floating-point quantity (*balance*). The composition of this account is illustrated schematically in figure below.

We can now declare the structure variables **oldcustomer** and **newcustomer** as follows.
struct account oldcustomer,
newcustomer;



Thus, **oldcustomer** and **newcustomer** are variables of type **account**. It is possible to combine the declaration of the structure composition with that of the structure variables, as shown below.

```
storage-class struct tag
```

```

{
    member 1;
    member 2;
    ...
    member m;

} variable 1, variable 2, ..., variable n;

```

The *tag* is optional in this situation.

The following single declaration is equivalent to the two declarations presented in the previous example.

```

struct account
{
    int acct_no;
    char acct_type;
    char name[80];
    float balance;
} oldcustomer, newcustomer;

```

Thus, **oldcustomer** and **newcustomer** are structure variables of type **account**.

Since the variable declarations are now combined with the declaration of the structure type, the tag (i.e., **account**) need not be included. Thus, the above declaration can also be written as

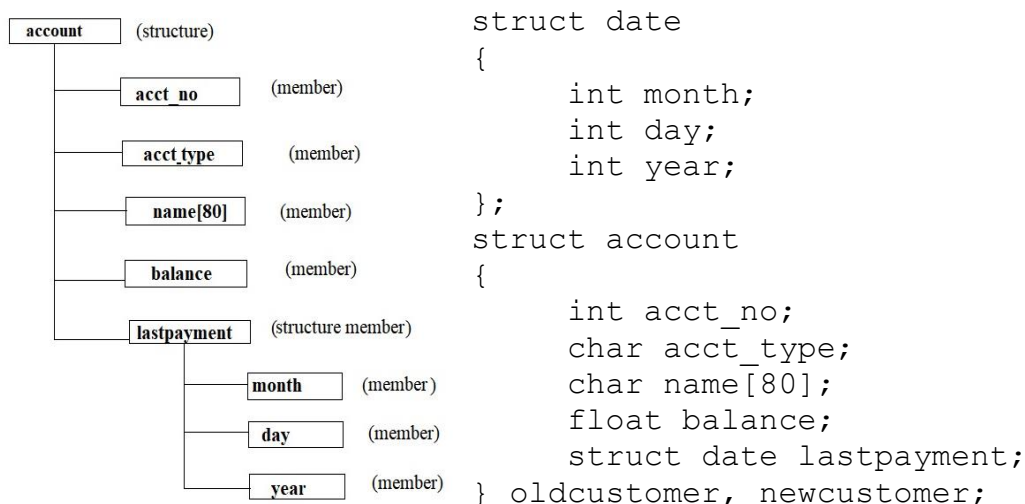
```

struct
{
    int acct_no;
    char acct_type;
    char name[80];
    float balance;
} oldcustomer, newcustomer;

```

A structure variable may be defined as a member of another structure. In such situations, the declaration of the embedded structure must appear before the declaration of the outer structure.

A C program contains the following structure declarations.



The second structure (***account***) now contains another structure (***date***) as one of its members. Note that the declaration of ***date*** precedes the declaration of ***account***. The composition of ***account*** is shown schematically in the Figure.

The members of a structure variable can be assigned initial values in much the same manner as the elements of an array. The initial values must appear in the order in which they will be assigned to their corresponding structure members, enclosed in braces and separated by commas. The general form is

storage-class struct tag variable = {value 1, value 2, ..., value m}

where **value 1**, refers to the value of the first member, **value 2** refers to the second member, and so on. A structure variable, like an array, can be initialized only if its storage class is either *external* or *static*.

This example illustrates the assignment of initial values to the members of a structure variable.

```
struct date
{
    int month;
    int day;
    int year;
};
struct account
{
    int acct_no;
    char acct_type;
    char name[80];
    float balance;
    struct date lastpayment;
};
static struct account customer = {12345, 'c', "Binod Poudel",
586.30, 5, 24, 90};
```

Thus, ***customer*** is a static structure variable of type ***account***, whose members are assigned initial values. The first member (***acct-no***) is assigned the integer value **12345**, the second member (***acct-type***) is assigned the character '**C**', the third member (***name [80]***) is assigned the string "**Binod Poudel**" and the fourth member (***balance***) is assigned the floating-point value **586.30**. The last member is itself a structure that contains three integer members (***month***, ***day*** and ***year***). Therefore, the last member of ***customer*** is assigned the integer values **5**, **24** and **90**.

It is also possible to define an array of structures; i.e., an array in which each element is a structure. The procedure is illustrated in the following example.

```
struct date
{
    int month;
    int day;
    int year;
};
struct account
{
```



```

    int acct_no;
    char acct_type;
    char name[80];
    float balance;
    struct date lastpayment;
};
struct account customer[100];

```

In this declaration **customer** is a 100-element array of structures. Hence, each element of customer is a separate structure of type account (i.e., each element of customer represents an individual customer record).

Note that each structure of type account includes an array (**name[80]**) and another structure (**date**) as members. Thus, we have an array and a structure embedded within another structure, which is itself an element of an array.

An array of structures can be assigned initial values just as any other array. Remember that each array element is a structure that must be assigned a corresponding set of initial values, as illustrated below. A C program contains the following declarations.

```

struct date
{
    char name[80];
    int month;
    int day;
    int year;
};
static struct data birthday[] = {"Amy", 12, 30, 73,
                                "Gail", 5, 13, 66,
                                "Marc", 7, 15, 72,
                                "Marla", 11, 29, 70,
                                "Megan", 2, 4, 77,
                                "Sharon", 12, 29, 63,
                                "Susan", 4, 12, 69};

```

In this example **birthday** is an array of structures whose size is unspecified. The initial values will define the size of the array, and the amount of memory required to store the array.

Notice that each row in the variable declaration contains *four constants*. These constants represent the initial values, i.e., the *name*, *month*, *day* and *year*, for one array element. Since there are 7 rows (7 sets of constants), the array will contain 7 elements, numbered **0** to **6**.

Some programmers may prefer to embed each set of constants within a separate pair of braces, in order to delineate the individual array elements more clearly. This is entirely permissible. Thus, the array declaration can be written

```

static struct data birthday[] = {
                                {"Amy", 12, 30, 73},
                                {"Gail", 5, 13, 66},
                                {"Marc", 7, 15, 72},
                                {"Marla", 11, 29, 70},
                                {"Megan", 2, 4, 77},
                                {"Sharon", 12, 29, 63},
                                {"Susan", 4, 12, 69}
};

```

Remember that each structure is a self-contained entity with respect to member definitions. Thus, the same member name can be used in different structures to represent different data. In other words, the scope of a member name is confined to the structure within which it is defined. Two different structures, called **first** and **second**, are declared below.

```
struct first
{
    float a;
    int b;
    char c;
};
struct second
{
    char a;
    float b, c;
};
```

Notice that the individual member names **a**, **b** and **c** appear in both structure declarations, but the associated data types are different. Thus, **a** represents a floating-point quantity in **first** and a character in **second**. Similarly, **b** represents an integer quantity in **first** and a floating-point quantity in **second**, whereas **c** represents a character in **first** and a floating-point quantity in **second**. This duplication of member names is permissible, since the scope of each set of member definitions is confined to its respective structure. Within each structure the member names are distinct, as required.

Processing a Structure

The members of a structure are usually processed individually, as separate entities. Therefore, we must be able to access the individual structure members. A structure member can be accessed by writing

variable.member

where **variable** refers to the name of a structure-type variable, and **member** refers to the name of a member within the structure. Notice the *period* (.) that separates the variable name from the member name. This period is an *operator*; it is a member of the highest precedence group, and its associativity is *left to right*. Consider the following structure declarations.

```
struct date
{
    int month;
    int day;
    int year;
};
struct account
{
    int acct_no;
    char acct_type;
    char name[80];
    float balance;
    struct date lastpayment;
}customer;
```

In this example, **customer** is a structure variable of type **account**. If we wanted to access the customer's account number, we would write

```
customer.acct_no
```

Similarly, the customer's name and the customer's balance can be accessed by writing

```
customer.name
```

and

```
customer.balance
```

Since the *period operator* is a member of the highest precedence group, this operator will take precedence over the unary operators as well as the various arithmetic, relational, logical and assignment operators. Thus, an expression of the form **++variable.member** is equivalent to **++(variable.member)**; i.e., the **++operator** will apply to the structure member, not the entire structure variable. Similarly, the expression **&variable.member** is equivalent to **&(variable.member)**; thus, the expression accesses the address of the structure member, not the starting address of the structure variable.

Consider the structure declarations given in the last Example.

```
struct date
{
    int month;
    int day;
    int year;
};
struct account
{
    int acct_no;
    char acct_type;
    char name[80];
    float balance;
    struct date lastpayment;
}customer;
```

Several expressions involving the structure variable customer and its members are given below.

Expression	Interpretation
++customer.balance	Increment the value of customer.balance
customer.balance++	Increment the value of customer.balance after accessing its value
--customer.acct_no	Decrement the value of customer.acct_no
&customer	Access the beginning address of customer
&customer.acct_no	Access the address of customer.acct_no

More complex expressions involving the repeated use of the period operator may also be written. For example, if a structure member is itself a structure, then a member of the embedded structure can be accessed by writing

variable.member.sub_member

where **member** refers to the name of the member within the outer structure, and **sub_member** refers to the name of the member within the embedded structure. Similarly, if a structure member is an array, then an individual array element can be accessed by writing

variable.member[expression]

where *expression* is a nonnegative value that indicates the array element. Consider once again the structure declarations presented in the last Example.

```
struct date
{
    int month;
    int day;
    int year;
};
struct account
{
    int acct_no;
    char acct_type;
    char name[80];
    float balance;
    struct date lastpayment;
}customer;
```

The last member of **customer** is **customer.lastpayment**, which is itself a structure of type **date**. To access the month of the last payment, we would therefore write

```
customer.lastpayment.month
```

Moreover, this value can be incremented by writing

```
++customer.lastpayment.month
```

Similarly, the third member of **customer** is the character array **customer.name**. The third character within this array can be accessed by writing

```
customer.name[2]
```

This character's address can be obtained as

```
&customer.name[2]
```

The use of the period operator can be extended to arrays of structures, by writing

```
array[expression].member
```

where *array* refers to the array name, and *array[expression]* is an individual array element (a structure variable). Therefore *array[expression].member* will refer to a specific member within a structure.

Consider the following structure declarations, which were presented in the previous example.

```
struct date
{
    int month;
```

```

    int day;
    int year;
};
struct account
{
    int acct_no;
    char acct_type;
    char name[80];
    float balance;
    struct date lastpayment;
};
struct account customer[100];

```

In this example **customer** is an array that may contain as many as 100 elements. Each element is a structure of type **account**. Thus, if we wanted to access the account number for the 14th customer (i.e., *customer[13]*, since the subscripts begin with 0), we would write *customer[13].acct_no*. Similarly, this customer's balance can be accessed by writing *customer[13].balance*. The corresponding address can be obtained as *&customer[13].balance*.

The 14th customer's name can be accessed by writing *customer[13].name*. Moreover, we can access an individual character within the name by specifying a subscript. For example, the 8th character within the customer's name can be accessed by writing *customer[13].name[7]*. In a similar manner we can access the month, day and year of the 14th customer's last payment by specifying the individual members of *customer[13].lastpayment*, i.e., *customer[13].lastpayment*, *customer[13].lastpayment.day*, *customer[13].lastpayment.year*. Moreover, the expression *++customer[13].lastpayment.day* causes the value of the day to be incremented.

Structure members can be processed in the same manner as ordinary variables of the same data type. Single-valued structure members can appear in expressions, they can be passed to functions, and they can be returned from functions, as though they were ordinary single-valued variables. Complex structure members are processed in the same way as ordinary data items of that same type. For example, a structure member that is an array can be processed in the same manner as an ordinary array, and with the same restrictions. Similarly, a structure member that is itself a structure can be processed on a member-by-member basis (the members here refer to the embedded structure), the same as any other structure.

Several statements or groups of statements that access individual structure members are shown below. All the structure members conform to the declarations given in Example

```

customer.balance = 0;
customer.balance -= payment;
customer.lastpayment.month = 12;
printf("name: %s\n", customer.name);
if (customer.acct_type == 'P')
    printf("Preferred account no.: %d\n", customer.acct_no);
else
    printf("Regular account no.: %d\n", customer.acct_no);

```

The first statement assigns a value of 0 to *customer.balance*, whereas the second statement causes the value of *customer.balance* to be decreased by the value of *payment*. The third

statement causes the value 12 to be assigned to *customer.lastpayment.month*. Note that *customer.lastpayment.month* is a member of the embedded structure *customer.lastpayment*. The fourth statement passes the array *customer.name* to the *printf* function, causing the customer name to be displayed. Finally, the last example illustrates the use of structure members in an *if - else* statement. Also, we see a situation in which the structure member *customer.acct_no* is passed to a function as an argument.

The current ANSI standard permits entire structures to be assigned to one another provided the structures have the same composition. Suppose *oldcustomer* and *newcustomer* are structure variables having the same composition; i.e.,

```
struct date
{
    int month;
    int day;
    int year;
};
struct account
{
    int acct_no;
    char acct_type;
    char name[80];
    float balance;
    struct date lastpayment;
} oldcustomer, newcustomer;
```

As declared in earlier example. Let us assume that all the members of *oldcustomer* have been assigned individual values. In most newer versions of C, it is possible to copy these values to *newcustomer* simply by writing

```
newcustomer = oldcustomer;
```

On the other hand, some older versions of C may require that the values be copied individually, member by member; for example,

```
newcustomer.acct_no = oldcustomer.acct_no;
newcustomer.acct_type = oldcustomer.acct_type;
...
newcustomer.lastpayment.year = oldcustomer.lastpayment.year;
```

```
#include <stdio.h>
```

```
struct date
{
    int month;
    int day;
    int year;
};
```

```
struct account
{
    int acct_no;
    char acct_type;
    char name[80];
```

```

    float balance;
    struct date lastpayment;
} customer[100];

main()
{
    int i, n;
    printf("How many customers are there? ");
    scanf(" %d", &n);
    for (i = 0; i < n; ++i)
    {
        printf("\nCustomer no. %d\n", i + 1);
        printf("    Account number: ");
        scanf(" %d", &customer[i].acct_no);
        printf("    Account type: ");
        scanf(" %c", &customer[i].acct_type);
        printf("    Name: ");
        scanf(" %[^\n]", customer[i].name);
        printf("    Balance: ");
        scanf(" %f", &customer[i].balance);
        printf("    Payment date (mm/dd/yyyy): ");
        scanf(" %d/%d/%d", &customer[i].lastpayment.month,
&customer[i].lastpayment.day, &customer[i].lastpayment.year);
    }
    printf("\nDisplaying current information for each
customer\n");
    printf("Account Number\tAccount Type\tCustomer
Name\tBalance\tPayment date(mm/dd/yyyy)");
    for (i = 0; i < n; ++i)
    {
        printf("\n%d\t%c\t\t%s\t%.2f\t%d/%d/%d",
customer[i].acct_no, customer[i].acct_type, customer[i].name,
customer[i].balance, [i].lastpayment.month,
customer[i].lastpayment.day, customer[i].lastpayment.year);
    }
}

```

Output

How many customers are there? 4

Customer no. 1

```

    Account number: 4208
    Account type: C
    Name: Steve Johnson
    Balance: 247.88
    Payment date (mm/dd/yyyy): 6/14/1998

```

Customer no. 2

```

    Account number: 2219
    Account type: D
    Name: Susan Richards
    Balance: 135.0
    Payment date (mm/dd/yyyy): 8/10/2000

```

Customer no. 3

Account number: 8452
 Account type: O
 Name: Martin Peterson
 Balance: 387.42
 Payment date (mm/dd/yyyy): 9/22/1999

Customer no. 4

Account number: 711
 Account type: D
 Name: Phyllis Smith
 Balance: 260.0
 Payment date (mm/dd/yyyy): 11/27/2001

Displaying current information for each customer

Account#	Account Type	Customer Name	Balance	Payment date (mm/dd/yyyy)
4208	C	Steve Johnson	247.88	6/14/1998
2219	D	Susan Richards	135.00	8/10/2000
8452	O	Martin Peterson	387.42	9/22/1999
711	D	Phyllis Smith	260.00	11/27/2001

User Defined Data Types (*typedef*)

The ***typedef*** feature allows users to define new datatypes that are equivalent to existing data types. Once a user-defined data type has been established, then new variables, arrays, structures, etc. can be declared in terms of this new data type. In general terms, a new data type is defined as

typedef **type** **new-type**;

where ***type*** refers to an existing data type (either a standard data type, or previous user-defined data type), and ***new-type*** refers to the new user-defined data type. It should be understood, however, that the new data type will be new in name only. In reality, this new data type will not be fundamentally different from one of the standard data types.

Here is a simple declaration involving the use of ***typedef***.

```
typedef int age;
```

In this declaration ***age*** is a user-defined data type, which is equivalent to type ***int***. Hence, the variable declaration

```
age male, female;
```

is equivalent to writing

```
int male, female;
```

In other words, ***male*** and ***female*** are regarded as variables of type ***age***, though they are actually integer-type variables.

Similarly, the declarations

```
typedef float height[100];
```

```
height men, women;
```

define ***height*** as a 100-element, floating-point array type-hence, ***men*** and ***women*** are 100-element, floating-point **arrays**. Another way to express this is

```
typedef float height;
```

```
height men[100], women[100];
```

The ***typedef*** feature is particularly convenient when defining structures, since it eliminates the need to repeatedly write ***struct tag*** whenever a structure is referenced. Hence, the

structure can be referenced more concisely. In addition, the name given to a user-defined structure type often suggests the purpose of the structure within the program.

In general terms, a user-defined structure type can be written as

```
typedef struct
{
    member 1;
    member 2;
    ...
    member m;
} new-type;
```

where **new-type** is the user-defined structure type. Structure variables can then be defined in terms of the new data type.

The following declarations are comparable to the structure declarations presented in the previous Examples. Now, however, we introduce a user-defined data type to describe the structure.

```
typedef struct
{
    int acct_no;
    char acct_type;
    char name[80];
    float balance;
} record;
record oldcustomer, newcustomer;
```

The first declaration defines **record** as a user-defined data type. The second declaration defines **oldcustomer** and **newcustomer** as structure variables of type **record**.

The **typedef** feature can be used repeatedly, to define one data type in terms of other user-defined data types.

Here are some variations of the structure declarations presented in the previous example.

```
typedef struct
{
    int month;
    int day;
    int year;
} date;
typedef struct
{
    int acct_no;
    char acct_type;
    char name[80];
    float balance;
    date lastpayment;
} record;
record customer[100];
```

In this example **date** and **record** are user-defined structure types, and **customer** is a 100-element array whose elements are structures of type **record**. (Recall that **date** was a tag rather than an actual data type in the earlier Example.) The individual members within the i^{th}

element of customer can be written as **customer[i].acct_no**, **customer[i].name**, **customer[i].lastpayment.month**, etc., as before. There are, of course, variations on this theme. Thus, an alternate declaration can be written as

```
typedef struct
{
    int month;
    int day;
    int year;
} date;
typedef struct
{
    int acct_no;
    char acct_type;
    char name[80];
    float balance;
    date lastpayment;
} record[100];
record customer;
```

or simply

```
typedef struct
{
    int month;
    int day;
    int year;
} date;
struct
{
    int acct_no;
    char acct_type;
    char name[80];
    float balance;
    date lastpayment;
} customer[100];
```

All three sets of declarations are equivalent.

Structure and Pointers

The beginning address of a structure can be accessed in the same manner as any other address, using the address (**&**) operator. Thus, if *variable* represents a structure-type variable, then **&variable** represents the starting address of that variable. Moreover, we can declare a pointer variable for a structure by writing

```
type *ptvar;
```

where **type** is a data type that identifies the composition of the structure, and **ptvar** represents the name of the pointer variable. We can then assign the beginning address of a structure variable to this pointer by writing

```
ptvar = &variable;
```

Consider the following structure declaration, which is a variation of the declaration presented in the previous example.

```
typedef struct
{
    int acct_no;
    char acct_type;
    char name[80];
    float balance;
    date lastpayment;
} account;
account customer, *pc;
```

In this example **customer** is a structure variable of type *account*, and **pc** is a pointer variable whose object is a structure variable of type *account*. Thus, the beginning address of *customer* can be assigned to *pc* by writing

```
pc = &customer;
```

The variable and pointer declarations can be combined with the structure declaration by writing

```
struct
{
    member 1;
    member 2;
    ...
    member m;
} variable, *ptvar;
```

where **variable** again represents a structure-type variable, and **ptvar** represents the name of a pointer variable.

The following single declaration is equivalent to the two declarations presented in the previous example.

```
struct
{
    int acct_no;
    char acct_type;
    char name[80];
    float balance;
} customer, *pc;
```

The beginning address of **customer** can be assigned to *pc* by writing

```
pc = &customer;
```

As in the previous example.

An individual structure member can be accessed in terms of its corresponding pointer variable by writing

```
ptvar->member
```

where **ptvar** refers to a structure-type pointer variable and the operator **->** is comparable to the period (.) operator discussed in earlier Sec. Thus, the expression

```
ptvar->member
```

is equivalent to writing

```
variable.member
```

where **variable** is a structure-type variable. The operator `->` falls into the highest precedence group, like the period operator (`.`). Its associativity is *left to right*.

The `->` operator can be combined with the period operator to access a sub member within a structure (i.e., to access a member of a structure that is itself a member of another structure).

Hence, a sub member can be accessed by writing

```
ptvar->member.submember
```

Similarly, the `->` operator can be used to access an element of an array that is a member of a structure. This is accomplished by writing

```
ptvar->member[expression]
```

where **expression** is a nonnegative integer that indicates the array element.

Here is a variation of the declarations shown in Example previous.

```
typedef struct
{
    int month;
    int day;
    int year;
} date;
struct
{
    int acct_no;
    char acct_type;
    char name[80];
    float balance;
    date lastpayment;
} customer, *pc = &customer;
```

Notice that the pointer variable **pc** is initialized by assigning it the beginning address of the structure variable **customer**. In other words, **pc** will point to **customer**.

If we wanted to access the customer's account number, we could write any of the following:

```
customer.acct_no    pc->acct_no    (*pc).acct_no
```

The parentheses are required in the last expression because the period operator has a higher precedence than the indirection operator (`*`). Without the parentheses the compiler would generate an error, because **pc** (a pointer) is not directly compatible with the dot operator.

Similarly, the customer's balance can be accessed by writing any of the following:

```
customer.balance    pc->balance    (*pc).balance
```

and the month of the last payment can be accessed by writing any of the following:

```
customer.lastpayment.month  pc->lastpayment.month  (*pc).lastpayment.month
```

Finally, the customer's name can be accessed by writing any of the following:

```
customer.name    pc->name    (*pc).name
```

Therefore, the third character of the customer's name can be accessed by writing any of the following.

```
customer.name[2]    pc->name[2]    (*pc).name[2]
*(customer.name + 2)    pc->(name + 2)  *((*pc).name + 2)
```

Passing Structure to Functions

There are several different ways to pass structure-type information to or from a function. Structure members can be transferred individually, or entire structures can be transferred. Individual structure members can be passed to a function as arguments in the function call, and a single structure member can be returned via the **return** statement. To do so, each structure member is treated the same as an ordinary single-valued variable. The skeletal outline of a C program is shown below. This outline makes use of the structure declarations presented earlier.

```
float adjust(char name[], int acct_no, float balance);

main()
{
    typedef struct
    {
        int month;
        int day;
        int year;
    } date;
    struct
    {
        int acct_no;
        char acct_type;
        char name[80];
        float balance;
        date lastpayment;
    } customer;
    ...
    customer.balance = adjust(customer.name,
customer.acct_no, customer.balance);
}
float adjust(char name[], int acct_no, float balance)
{
    float newbalance;
    ...
    newbalance = ...;
    return(newbalance);
}
```

This program outline illustrates the way structure members can be passed to a function. In particular, **customer.name**, **customer.acct_no** and **customer.balance** are passed to the function **adjust**. Within **adjust**, the value assigned to **newbalance** presumably makes use of the information passed to the function. This value is then returned to main, where it is assigned to the structure member **customer.balance**.

Notice the function declaration in main. This declaration could also have been written without the argument names, as follows:

```
float adjust(char [], int, float);
```

Some programmers prefer this form, since it avoids the specification of dummy argument names for data items that are structure members.

A complete structure can be transferred to a function by passing a structure-type pointer as an argument. In principle, this is like the procedure used to transfer an array to a function. However, we must use explicit pointer notation to represent a structure that is passed as an argument.

You should understand that a structure passed in this manner will be passed by *reference* rather than by *value*. Hence, if any of the structure members are altered within the function, the alterations will be recognized outside of the function. Again, we see a direct analogy with the transfer of arrays to a function. Consider the simple C program shown below.

```
#include <stdio.h>

typedef struct
{
    char *name;
    int acct_no;
    char acct_type;
    float balance;
} record;

void adjust(record *pt);

main()
{
    static record customer = {"Smith", 3333, 'C', 33.33};
    printf("%s %d %c %.2f\n", customer.name, customer.acct_no,
customer.acct_type, customer.balance);
    adjust(&customer);
    printf("%s %d %c %.2f\n", customer.name, customer.acct_no,
customer.acct_type, customer.balance);
}

void adjust(record *pt)
{
    pt->name = "John";
    pt->acct_no = 9999;
    pt->acct_type = 'R';
    pt->balance = 99.99;
    return;
}
```

This program illustrates the transfer of a structure to a function by passing the structure's address (a pointer) to the function. In particular, *customer* is a static structure of type *record*, whose members are assigned an initial set of values. These initial values are displayed when the program begins to execute. The structure's address is then passed to the function ***adjust***, where different values are assigned to the members of the structure.

Within ***adjust***, the formal argument declaration defines ***pt*** as a pointer to a structure of type *record*. Also, notice the empty *return* statement; i.e., nothing is explicitly returned from ***adjust*** to *main*.

Within *main*, we see that the current values assigned to the members of *customer* are displayed a second time, after ***adjust*** has been accessed. Thus, the program illustrates whether the changes made in ***adjust*** carry over to the calling portion of the program. Execution of the program results in the following output.

Smith 3333 C 33.33

John 9999 R 99.99

Thus, the values assigned to the members of *customer* within ***adjust*** are recognized within ***main***, as expected.

A pointer to a structure can be returned from a function to the calling portion of the program. This feature may be useful when several structures are passed to a function, but only one structure is returned.

Locating Customer Records Here is a simple C program that illustrates how an *array of structures* is passed to a function, and how a pointer to a particular structure is returned.

Suppose we specify an account number for a particular customer and then *locate and display* the complete record for that customer. Each customer record will be maintained in a structure, as in the last example. Now, however, the entire set of customer records will be stored in an array called *customer*. Each element of *customer* will be an independent structure.

The basic strategy will be to enter an account number, and then transfer both the account number and the array of records to a function called ***search***. Within ***search***, the specified account number will be compared with the account number that is stored within each customer record until a match is found, or until the entire list of records has been searched. If a match is found, a pointer to that array element (the structure containing the desired customer record) is returned to ***main***, and the contents of the record are displayed.

If a match is not found after searching the entire array, then the function returns a value of NULL (zero) to ***main***. The program then displays an error message requesting that the user reenter the account number. This overall search procedure will continue until a value of *zero* is entered for the account number.

The complete program is shown below. Within this program, *customer* is an array of structures of type *record*, and ***pt*** is a pointer to a structure of this same type. Also, ***search*** is a function that accepts two arguments and returns a pointer to a structure of type *record*. The arguments are an array of structures of type *record* and an integer quantity, respectively. Within ***search***, the quantity returned is either the address of an array element, or NULL (zero).

```
#include <stdio.h>
#define N 3
//#define NULL 0

typedef struct
{
    char *name;
    int acct_no;
    char acct_type;
    float balance;
} record;
record *search(record table[], int acctn);

main()
{
    static record customer[N] = {
```

```

{"Smith", 3333, 'C', 33.33},
{"Jones", 6666, 'O', 66.66},
{"Brown", 9999, 'D', 99.99}

```

```
};
```

```

int acctn;
record *pt;

```

```

printf("Customer Account Locator\n");
printf("To END, enter 0 for the account number\n");
printf("\nAccount no.: ");
scanf(" %d", &acctn);

```

```

while (acctn != 0)
{
    pt = search(customer, acctn);

    if (pt != NULL)
    {
        printf("\nName: %s\n", pt->name);
        printf("Account no.: %d\n", pt->acct_no);
        printf("Account type: %c\n", pt->acct_type);
        printf("Balance: %.2f\n", pt->balance);
    }
    else
        printf("\nERROR- Please try again\n");
    printf("\nAccount no.: ");
    scanf(" %d", &acctn);
}
}

```

```

record *search(record table[], int acctn)
{
    int count;

    for (count = 0; count < N; ++count)
    {
        if (table[count].acct_no == acctn)
            return(&table[count]);
    }
    return(NULL);
}

```

The array size is expressed in terms of the symbolic constant **N**. For this simple example we have selected a value of **N = 3**. That is, we are storing only three sample records within the array. In a more realistic example, **N** would have a much greater value.

Finally, it should be mentioned that there are much better ways to search through a set of records than examining each record sequentially. We have selected this simple though inefficient procedure in order to concentrate on the mechanics of transferring structures between **main** and its subordinate function **search**.

Shown below is a typical dialog that might result from execution of the program. The user's responses are underlined, as usual.

```

Customer Account Locator
To END, enter 0 for the account number

```


Account no.: 3333

Name: Smith
 Account no.: 3333
 Account type: C
 Balance: 33.33

Account no.: 9999

Name: Brown
 Account no.: 9999
 Account type: D
 Balance: 99.99

Account no.: 444

ERROR- Please try again

Account no.: 6666

Name: Jones
 Account no.: 6666
 Account type: O
 Balance: 66.66

Account no.: 0

Newer versions of **C** permit an entire structure to be transferred directly to a function as an argument and returned directly from a function via the ***return*** statement. (Notice the contrast with *mays*, which cannot be returned via the *return* statement.) These features are included in the current ANSI standard.

When a structure is passed directly to a function, the transfer is by ***value*** rather than by *reference*. This is consistent with other direct (nonpointer) transfers in C. Therefore, if any of the structure members are altered within the function, the alterations will not be recognized outside of the function. However, if the altered structure is returned to the calling portion of the program, then the changes will be recognized within this broader scope.

```
#include <stdio.h>
```

```
typedef struct
{
    char *name;
    int acct_no;
    char acct_type;
    float balance;
} record;
record adjust(record customer);
```

```
main()
{
    static record customer = {"Smith", 3333, 'C', 33.33};
```

```

    printf("%s %d %c %.2f\n", customer.name, customer.acct_no,
customer.acct_type, customer.balance);

    customer = adjust(customer);

    printf("%s %d %c %.2f\n", customer.name, customer.acct_no,
customer.acct_type, customer.balance);
}

record adjust(record cust)
{
    cust.name = "John";
    cust.acct_no = 9999;
    cust.acct_type = 'R';
    cust.balance = 99.99;
    return(cust);
}

```

Notice that ***adjust*** now returns a structure of type *record* to ***main***. The *return* statement is modified accordingly. Execution of this program results in the following output.

```

Smith 3333 C 33.33
John 9999 R 99.99

```

Thus, the alterations that were made within ***adjust*** are now recognized within ***main***. This is expected, since the altered structure is now returned directly to the calling portion of the program.

Self-Referential Structure

It is sometimes desirable to include within a structure one member that is a pointer to the parent structure type. In general terms, this can be expressed as

```

struct tag
{
    member 1;
    member 2;
    ...
    struct tag *name;
};

```

where ***name*** refers to the name of a pointer variable. Thus, the structure of type ***tag*** will contain a member that points to another structure of type ***tag***. Such structures are known as ***self-referential*** structures.

A C program contains the following structure declaration.

```

struct list_element
{
    char item[40];
    struct list_element *next;
};

```

This is a structure of type ***list-element***. The structure contains two members: a 40-element character array, called ***item***, and a pointer to a structure of the same type (i.e., a pointer to a structure of type ***list-element***), called ***next***. Therefore, this is a self-referential structure.

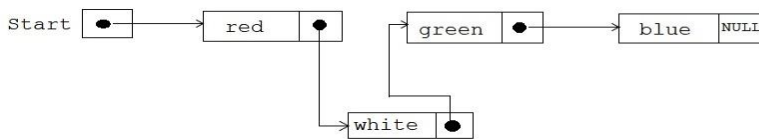
Self-referential structures are very useful in applications that involve *linked data structures*, such as **lists** and **trees**. First, however, we present a brief summary of linked data structures.

The basic idea of a linked data structure is that each component within the structure includes a pointer indicating where the next component can be found. Therefore, the relative order of the components can easily be changed simply by altering the pointers. In addition, individual components can easily be added or deleted, again by altering the pointers. As a result, a linked data structure is not confined to some maximum number of components. Rather, the data structure can expand or contract in size as required.

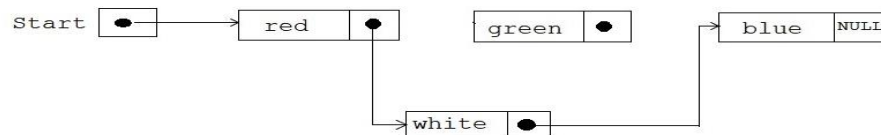
Figure below illustrates a linked list containing three components. Each component consists of two data items: a string, and a pointer that references the next component within the list. Thus, the first component contains the string **red**, the second contains **green** and the third contains **blue**. The beginning of the list is indicated by a separate pointer, which is labeled start. Also, the end of the list is indicated by a special pointer, called NULL.



Now let us add another component, whose value is **white**, between **red** and **green**. To do so we merely change the pointers, as illustrated in Figure below.



Similarly, if we choose to delete the component whose value is **green**, we simply change the pointer associated with the second component, as shown in Figure below.



Finally, in the Figure shown below we see an example of a **tree**. Trees consist of *nodes* and *branches*, arranged in some hierarchical manner which indicates a corresponding hierarchical structure within the data. (A **binary tree** is a tree in which every node has no more than two branches.)

In the Figure (a) the *root node* has the value **screen**, and the associated branches lead to the nodes whose values are **foreground** and **background**, respectively. Similarly, the branches associated with **foreground** lead to the nodes whose values are **white**, **green** and **amber**, and the branches associated with **background** lead to the nodes whose values are **black**, **blue** and **white**.

Figure (b) illustrates the manner in which pointers are used to construct the tree.

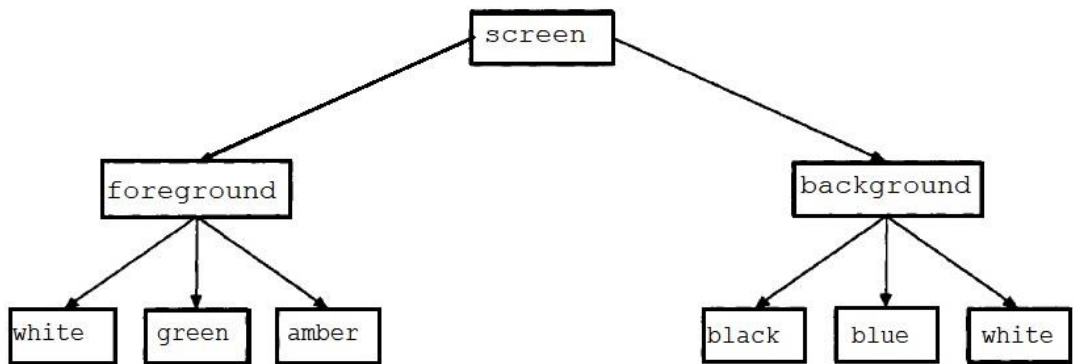


figure (a)

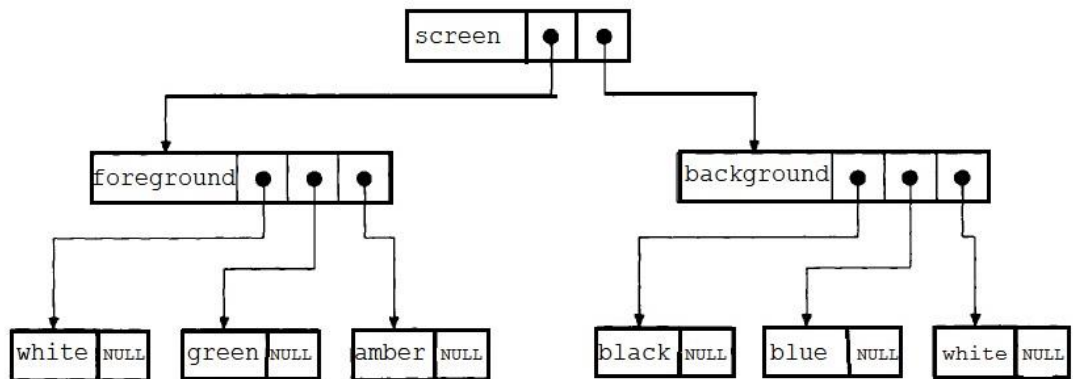


figure (b)

Self-referential structures are ideally suited for applications involving *linked data structures*. Each structure will represent a single component (i.e., one node) within the linked data structure. The self-referential pointer will indicate the location of the next component.

Union

Unions, like structures, contain members whose individual data types may differ from one another. However, the members within a union *all share the same storage area* within the computer's memory, whereas each member within a structure is assigned its own unique storage area. Thus, unions are used to conserve memory. They are useful for applications involving multiple members, where values need not be assigned to all of the members at any one time.

Within a union, the bookkeeping required to store members whose data types are different (having different memory requirements) is handled automatically by the compiler. However, the user must keep track of what type of information is stored at any given time. An attempt to access the wrong type of information will produce meaningless results. In general terms, the composition of a union may be defined as

```

union tag
{
    member 1;
    member 2;
    . . . . .
    member m;
  
```

```
};
```

where **union** is a required keyword and the other terms have the same meaning as in a structure definition. Individual union variables can then be declared as

storage-class union tag variable 1, variable 2, ..., variable n;
 where **storage-class** is an optional storage class specifier, **union** is a required keyword, **tag** is the name that appeared in the union definition, and **variable1, variable 2, ..., variable n** are union variables of type **tag**.

The two declarations may be combined, just as we did with structures. Thus, we can write

```
storage-class union tag
{
    member 1;
    member 2;
    ...
    member m;
} variable 1, variable 2, ..., variable n;
```

The **tag** is optional in this type of declaration. A C program contains the following union declaration.

```
union id
{
    char color[12];
    int size;
} shirt, blouse;
```

Here we have two union variables, *shirt* and *blouse*, of type **id**. Each variable can represent either a 12-character string (**color**) or an integer quantity (**size**) at any one time.

The 12-character string will require more storage area within the computer's memory than the integer quantity. Therefore, a block of memory large enough for the 12-character string will be allocated to each union variable. The compiler will automatically distinguish between the 12-character array and the integer quantity within the given block of memory, as required.

A union may be a member of a structure, and a structure may be a member of a union. Moreover, structures and unions may be freely mixed with arrays. A C program contains the following declarations.

```
union id
{
    char color[12];
    int size;
};
struct clothes
{
    char manufacturer[20];
    float cost;
    union id description;
} shirt, blouse;
```

Now **shirt** and **blouse** are structure variables of type *clothes*. Each variable will contain the following members: a string (**manufacturer**), a floating-point quantity (**cost**), and a union

(*description*). The union may represent either a string (*color*) or an integer quantity (*size*). Another way to declare the structure variables **shirt** and **blouse** is to combine the preceding two declarations, as follows.

```
struct clothes
{
    char manufacturer[20];
    float cost;
    union
    {
        char color[12];
        int size;
    } description;
} shirt, blouse;
```

An individual union member can be accessed in the same manner as an individual structure member, using the operators `.` and `->`. Thus, if **variable** is a union variable, then **variable.member** refers to a member of the union. Similarly, if **ptvar** is a pointer variable that points to a union, then **ptvar->member** refers to a member of that union. Consider the simple C program shown below.

```
#include <stdio.h>

main()
{
    union id
    {
        char color;
        int size;
    };

    struct
    {
        char manufacturer[20];
        float cost;
        union id description;
    } shirt, blouse;
    printf("%d\n", sizeof(union id));

    shirt.description.color = 'w';
    printf("%c %d\n", shirt.description.color,
shirt.description.size);

    shirt.description.size = 12;
    printf("%c %d\n", shirt.description.color,
shirt.description.size);
}
```

The first member of the union is a single character rather than the 12-character array shown in the previous example. This change is made to simplify the assignment of appropriate values to the union members.

Following the declarations and the initial *printf* statement, we see that the character ‘**w**’ is assigned to the union member *shirt.description.color*. Note that the other union member,

shirt.description.size, will not have a meaningful value. The values of both union members are then displayed.

We then assign the value **12** to *shirt.description.size*, thus overwriting the single character previously assigned to *shirt.description.color*. The values of both union members are then displayed once more. Execution of the program results in the following output.

```
4
w      119
♫      12
```

The first line indicates that the union is allocated four bytes of memory, to accommodate an integer quantity. In line 2, the first data item (**w**) is meaningful, but the second (**119**) is not. In line 3, the first data item (**♫**) is meaningless, but the second data item (**12**) has meaning. Thus, each line of output contains one meaningful value, in accordance with the assignment statement preceding each *printf* statement.

9. FILES AND FILE HANDLING

Many applications require that information be written to or read from an auxiliary memory device. Such information is stored on the memory device in the form of a **data file**. Thus, data files allow us to store information permanently, and to access and alter that information whenever necessary.

In C, an extensive set of library functions is available for creating and processing data files. Unlike other programming languages, C does not distinguish between sequential and direct access (random access) data files. However, there are two different types of data files, called **stream-oriented** (or **standard**) data files, and **system-oriented** (or **low-level**) data files. Stream-oriented data files are generally easier to work with and are therefore more commonly used.

Stream-oriented data files can be subdivided into two categories. In the first category are **text** files, consisting of consecutive characters. These characters can be interpreted as individual data items, or as components of strings or numbers. The manner in which these characters are interpreted is determined either by the particular library functions used to transfer the information, or by format specifications within the library functions, as in the *scanf* and *printf* functions.

The second category of stream-oriented data files, often referred to as **unformatted** data files, organizes data into blocks containing contiguous bytes of information. These blocks represent more complex data structures, such as arrays and structures. A separate set of library functions is available for processing stream-oriented data files of this type. These library functions provide single instructions that can transfer entire arrays or structures to or from data files.

System-oriented data files are more closely related to the computer's operating system than stream-oriented data files. They are somewhat more complicated to work with, though their use may be more efficient for certain kinds of applications. A separate set of procedures, with accompanying library functions, is required to process system-oriented data files.

This chapter is concerned only with stream-oriented data files. The overall approach is relatively standardized, though the details may vary from one version of C to another. Thus, the examples presented in this chapter may not apply to all versions of the language in exactly the manner shown.

Why Files

There can be many reasons to use files. The first and the foremost reason is need of *permanency* of storage of data. If we want to preserve the output of any program for future use, it is not possible without files. All messages or values printed with help of any output statements like `printf`, `putchar`, etc. are never available for future use and the only solution in that case is to write the output in files. Similarly, if there is the large amount of data generated as output by a program, storing that output in file will help in easy handling/analysis of the output, as user can see the whole output at any time even after complete execution of the program.

The input files are also equally important and useful. If a program needs a lot of data to be inputted, user cannot keep on typing that again and again for repeated execution of the program. In that case, all input data can be once written in a file and then that file can be easily used as the input file. Moreover, the transfer of input-data and/or output-data from one computer to another can be easily done by using files.

Opening and Closing a Data File

When working with a *stream-oriented* data file, the first step is to establish a *buffer area* where information is temporarily stored while being transferred between the computer's memory and data file. The buffer area is established by writing

```
FILE *ptvar;
```

Where **FILE** is special structure type that establishes the buffer area, and **ptvar** is a pointer variable that indicates the beginning of the buffer area. The structure type **FILE** is defined within a system include file, typically **stdio.h**. The pointer **ptvar** is often referred to as a *stream pointer*, or simply a *stream*.

The data file must be *opened* before it can be created or processed. This associates the file name with the buffer area (i.e., with the stream). It also specifies how the data file will be utilized, i.e, as a read-only file, or a read/write file, in which both operations are permitted. The library function `fopen()` is used to open a file.

```
ptvar = fopen(file_name, file_type);
```

where *file_name* and *file_type* are strings that represent the name of the data file and the manner in which the data file will be utilized. The name chosen for the *file_name* must be consistent with the rules for naming, as determined by the computer's operating system. The *file_type* must be one of the strings shown below.

File-Type	Meaning
“r”	Open an existing file for reading only.
“w”	Open a new file for writing only. If a file with the specified <i>file_name</i> currently exists, it will be destroyed and a <i>new_file</i> created in its place.
“a”	Open an existing file for appending. A new file will be created if the file with the specified <i>file_name</i> does not exist.
“r+”	Open an existing file for both reading and writing.

“w+”	Open a new file for both writing and reading. If a file with the specified <i>file_name</i> currently exists, it will be destroyed, and a new file created in its place.
“a+”	Open an existing file for both reading and appending. A new file will be created if the file with the specified <i>file_name</i> does not exist.

The **fopen()** returns a pointer to the beginning of the buffer area associated with the file. A NULL value is returned if the file cannot be opened as, for example, when an existing data file cannot be found.

Finally, the data file must be closed at the end of the program. This can be accomplished with the library function **fclose()**.

fclose(ptvar) ;

It is good programming practice to close a data file explicitly using the **fclose()**, though most C compilers will automatically close a data file at the end of program execution if a call to **fclose()** is not present.

File Pointer

A file pointer is a pointer to a structure, which contains information about the file, including its name, current position of the file, whether the file is being read or written, and whether errors or end of the file have occurred. The user does not need to know the details, because the definitions obtained from *stdio.h* include a structure declaration called FILE. The only declaration needed for a file pointer is symbolized by

FILE *fp;

This says that **fp** is the file pointer that points to a FILE structure.

A C program contains the following statements.

```
#include <stdio.h>
FILE *fpt;
fpt = fopen("sample.dat", "w");
...
fclose(fpt);
```

The first statement causes the header file *stdio.h* to be included in the program. The second statement defines a pointer called **fpt** which will point to a structure of type FILE, indicating the beginning of the data-file buffer area. Note that FILE is defined in *stdio.h*.

The third statement opens a new data file called **sample.dat** as a write-only file. Moreover, the **fopen()** returns a pointer to the *beginning* of the buffer area and assigns it to the pointer variable **fpt**. Thus, **fpt** points to the buffer area associated with the data file *sample.dat*. All subsequent file processing statements (which are not shown explicitly in this example) will access the data file via the pointer variable **fpt** rather than by the file name.

Finally, the last statement closes the data file. Note that the argument is the pointer variable **fpt**, not the file name *sample.dat*.

The value returned by the **fopen()** can be used to generate an error message if a data file cannot be opened, as illustrated in the next example. A C program contains the following statements.

```
#include <stdio.h>

main()
{
```

```

FILE *fpt;
fpt = fopen("sample.dat", "r+");
if (fpt == null)
    printf("\nerror- cannot open the designated file.\n");
else
{
    ...
    fclose(fpt);
}
}

```

This program attempts to open an existing data file called *sample.dat* for both reading and writing. An error message will be generated if this data file cannot be found. Otherwise the data file will be opened and processed, as indicated. The *fopen()* and the **if** statement are often combined, as follows.

```

if ((fpt = fopen("sample.dat", "r+")) == NULL)
    printf("\nERROR - cannot open the designated file.\n");

```

Either method is acceptable.

Reading and Writing a Data File

Normally, files are used for one of the two purposes- reading or writing. Whenever we want to store something into a file, the file needs to be opened in write mode. If the contents to be stored in output file are already known, then the file can also be created directly using a text editor or word processor. In that case, the file is obviously a formatted stream-oriented data file. However, if the contents to be stored are not pre-defined and are computed/generated with help of a program itself, then the output file can be created with help of a program only.

Writing to a data file: This is a program that writes A to Z each in a separate line in an output file. The program is very similar to the one which displays A to Z on output screen (each in separate line). The only difference in this program is use of *fprintf* instead of *printf*. The program begins by defining the stream pointer *fp* indicating the beginning of the buffer area for the data file. A new data file called output.dat is then opened for writing only. Next a for loop writes character A to Z followed by *\n* in each case with help of *fprintf* statement to the output file. The output file is closed at last.

```

#include <stdio.h>

main()
{
    int k;
    FILE *fp;
    fp = fopen("output.dat", "w");
    for (k = 65; k <= 90; k++)
        fprintf(fp, "%c\n", k);
    fclose(fp);
}

```

The *fprintf* statement has same functionality as that of *printf* with the only difference that *fprintf* stores the output in the file. After storing the output of every *fprintf* statement, the internal file pointer automatically advances to next location. It means that the output contents of second execution of *fprintf* statement will always be stored after the output contents of

first *fprintf* statement and so on. In fact, this is true for all file related input-output statements. The internal file pointer is always updated automatically to next location after every file related input-output statement. So, after the file has been executed, the data file **output.dat** will contain A in first line followed by B in second line and son on up to Z in last line. Here *putc* or *fputc* statements can also be used instead of *fprintf* statement to write a character value to the file.

The way of handling input files is equally easy. If the stream oriented formatted files are to be used for reading purposes, *fscanf* statement can be used instead of *scanf* statement.

Reading a Data File The following program reads the contents of a data file on a character-by-character basis, and displays them on the screen. In order to add more flexibility to the program, the filename is also taken as input from the user.

```
/* reading contents of a data file and display them on the screen */
#include <stdio.h>
```

```
main()
{
    FILE *fp;
    char ch, fname[50];
    printf("Enter the input file name with extension");
    gets(fname);

    if ((fp = fopen(fname, "r")) == NULL)
        printf("\nERROR! cannot open the designated file.\n");
    else
    {
        do
        {
            fscanf(fp, "%c", &ch);
            printf("%c", ch);
        } while (!feof(fp));
    }
    fclose(fp);
}
```

The **fscanf()** works exactly in the same way of *scanf* except for the fact that **fscanf** reads from the file, not from the keyboard. The **feof** function (more correctly a macro) is used to check for end of file. Actually, every file stored inside the computer is always terminated by internally placing a special character at the end of the file. This special character is known as **eof** (end of file) character. The **feof** function checks whether **eof** character has been reached or not. As already mentioned, the internal file pointer gets updated automatically after every **fscanf** statement. So **feof** returns *true* whenever internal file pointer reaches to **eof** character otherwise it returns *false*. Note that **feof** does not cause any change in position of internal file pointer.

This program can be used to display contents of any data file by providing its complete path and name. If input file name is supplied as **c:\turbo\add.c**, the contents of this file will be displayed on the screen.

If character inputs are to be read from the file, *getc* and *fgetc* can also be used instead of *fscanf* statement. However, the *fscanf* statement can be used to read all types of data i.e. integer, float, char, strings, structures, etc. For example, if an input data file named as **values.dat** contains some float values, then the following program can be used to find the

average of all of these numbers. The program also stores the output in another file and an example of using input as well as output files in the same program.

Average of numbers stored in a Data File

The following program read all numbers from the input file **values.dat** and stores the average of these numbers in an output file named as **average.txt**.

```
#include <stdio.h>

main()
{
    FILE *fpin, *fpout;
    float val, avg, sum = 0;
    int count = 0;
    if ((fpin = fopen("values.dat", "r")) == NULL)
        /* open the data file for reading purposes */
        printf("\nERROR! Cannot open the designated file.\n");
    else
    {
        while (!feof(fpin))
        {
            fscanf(fpin, "%f", &val);
            sum += val;
            ++count;
        }
        avg = sum / count;
        if ((fpout = fopen("average.txt", "w")) == NULL)
            printf("\nERROR! Cannot open the designated file.\n");
        else
            fprintf(fpout, "The average of numbers of file values.dat
is %.2f\n", avg);

        fclose (fpin);
        fclose (fpout);
    }
}
```

If average value is computed say 3.50, then the output file will have one following line.

The average of numbers of file values.dat is 3.50

The concept of **eof** character is used internally by operating systems to handle data files. In fact, it is the duty of an operating system to append eof character at the end of every file being written, whenever the file is closed. So whenever, **fclose** executes for any data file opened in write mode, the operating system appends **eof** character at the end of all contents and then stores the file into permanent storage (although this **eof** character is not counted while calculating the size of file). So, a file can be copied onto another file by reading and storing character-by-character till **eof** is not reached.

Copying a Data File Here is a simple implementation of copy command of any operating system, which can be used to copy contents of a given source data file to a new destination file.

```
/* copy a file */
#include <stdio.h>
```

```

#include <stdlib.h>

main()
{
    char ch, inpfiler[50], outfile[50];
    FILE *fpin, *fpout;

    printf("Enter the source file name ");
    gets(inpfiler);
    if ((fpin = fopen(inpfiler, "r")) == NULL)
    {
        printf("\nERROR- Cannot open the designated file\n");
        exit(1);
    }
    printf("Enter the destination file name ");
    gets(outfile);
    if ((fpout = fopen(outfile, "w")) == NULL)
    {
        printf("\nERROR- Cannot create the destination file\n");
        exit(1);
    }
    while (!feof(fpin))
    {
        ch = getc(fpin);
        putc(ch, fpout);
    }
    fclose(fpin);
    fclose(fpout);
}

```

Data files consisting entirely of strings can often be created and read more easily with programs that utilize special string-oriented library functions. Some commonly used functions of this type are *gets*, *puts*, *fgets* and *fputs*. The functions **gets** and **puts** read or write strings to or from the standard output devices, whereas, *fgets* and *fputs* exchange strings with data files.

Many data files consist of complex data structures, such as structures that contain various combinations of numeric and character information. Such data files can be processed using the library functions **fscanf()** and **fprintf()**.

Creating a File Containing Customer Records The last chapter presented program that supposedly were used to create and update customer record. That program was unrealistic, because data files should be used for applications of this type.

```

/* create a data file containing customer records */
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

typedef struct
{
    int month;
    int day;
    int year;
}

```

```

} date;

typedef struct
{
    int acct_no;
    char acct_type;
    char name[80];
    float balance;
    date lastpayment;
} record;

record readscreen(record customer);
void writefile(record customer);

FILE *fpt;

main()
{
    int n, i;
    record customer;

    if ((fpt = fopen("records.dat", "w")) == NULL)
    {
        printf("\nERROR! File cannot be created.");
        exit(1);
    }

    printf("\nHow many records? ");
    scanf(" %d", &n);
    printf("\nEnter the records of %d customers.", n);
    for(i = 0; i < n; ++i)
    {
        customer = readscreen(customer);
        writefile(customer);
    }
    fclose(fpt);
}

record readscreen(record customer)
{
    printf("Account Number: ");
    scanf(" %d", &customer.acct_no);
    printf("Account Type: ");
    scanf(" %c", &customer.acct_type);
    printf("Name: ");
    scanf(" %[^\n]", customer.name);
    printf("Balance: ");
    scanf(" %f", &customer.balance);
    printf("Payment Date: (mm/dd/yyyy) ");
    scanf(" %d/%d/%d", &customer.lastpayment.month,
&customer.lastpayment.day, &customer.lastpayment.year);
    return(customer);
}

void writefile(record customer)

```

```

{
    fprintf(fpt, "%d\\n%c\\n%s\\n%.2f\\n%d/%d/%d", customer.acct_no,
customer.acct_type, customer.name, customer.balance,
customer.lastpayment.month, customer.lastpayment.day,
customer.lastpayment.year);
    return;
}

```

The program begins by defining the composition of each customer record and the stream pointer ***fpt***. Within main, a new data file, called ***records.dat***, is then opened for writing only. The information for the current customer is entered via function *readscreen*, and then written to the data file via function *writefile*.

Within *readscreen* we see that the various data items are entered interactively, using the familiar formatted *printf* and *scanf* functions. On the other hand, within *main* and *writefile* the data are written to the data file via the *fprintf* function. The syntax governing the use of this function is the same as the syntax used with *printf*, except that a stream pointer must be included as an additional argument. Notice that the control string makes use of the same character groups (i.e., the same formatting features) as the *printf* function.

When the program is executed, the information for each customer record is entered interactively, as shown below for three fictitious customers.

```
How many records? 3
Enter the records of 3 customers.
Account Number: 123
Account Type: C
Name: Steve Johnson
Balance: 247.88
Payment Date:(mm/dd/yyyy) 5/25/1998
Account Number: 234
Account Type: C
Name: Susan Richards
Balance: 135
Payment Date:(mm/dd/yyyy) 10/12/2000
Account Number: 345
Account Type: C
Name: Phyllis Smith
Balance: 260.00
Payment Date:(mm/dd/yyyy) 8/4/2010
```

After the program has been executed, the data file **records.dat** will have been created, containing the following information.

```
123
C
Steve Johnson
247.88
5/25/1998234
C
Susan Richards
135.00
10/12/2000345
C
Phyllis Smith
260.00
8/4/2010
```

Unformatted Data Files

Some applications involve the use of data files to store blocks of data, where each block consists of a fixed number of contiguous bytes. Each block will generally represent a complex data structure, such as a *structure* or an *array*. For example, a data file may consist of multiple structures having the same composition, or it may contain multiple arrays of the same type and size. For such applications it may be desirable to *read the entire block* from the

data file, or *write the entire block* to the data file, rather than reading or writing the individual components (i.e., structure members or array elements) within each block separately.

The library functions **fread** and **fwrite** are intended to be used in situations of this type. These functions are often referred to as unformatted read and write functions. Similarly, data files of this type are often referred to as unformatted data files.

Each of these functions requires **four arguments**: *a pointer to the data block, the size of the data block, the number of data blocks being transferred, and the stream pointer*. Thus, a typical **fwrite** function might be written as

```
fwrite(&customer, sizeof(record), 1, fpt);
```

where *customer* is a structure variable of type *record*, and *fpt* is the stream pointer associated with a data file that has been opened for output.

Creating an Unformatted Data File Containing Customer Records Consider a variation of the program presented in the earlier Example, for creating a data file containing customer records. Now, however, we will write each customer record to the data file **data.bin** as a single, unformatted block of information. This is in contrast to the earlier program, where we wrote the items within each record (i.e., the individual structure members) as separate, formatted data items. Here is the complete program.

```
/* create a data file containing customer records */
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

typedef struct
{
    int month;
    int day;
    int year;
} date;

typedef struct
{
    int acct_no;
    char acct_type;
    char name[80];
    float balance;
    date lastpayment;
} record;

record readscreen(record customer);

FILE *fpt;

main()
{
    int n, i;
    record customer;

    if ((fpt = fopen("data.bin", "w")) == NULL)
    {
```

```

        printf("\nERROR! File cannot be created.");
        exit(1);
    }

    printf("\nHow many records? ");
    scanf(" %d", &n);
    printf("\nEnter the records of %d customers.", n);
    for(i = 0; i < n; ++i)
    {
        customer = readscreen(customer);
        fwrite(&customer, sizeof(record), 1, fpt);
    }

    fclose(fpt);
}

record readscreen(record customer)
{
    printf("\nAccount Number: ");
    scanf(" %d", &customer.acct_no);
    printf("\nAccount Type: ");
    scanf(" %c", &customer.acct_type);
    printf("\nName: ");
    scanf(" %[^\n]", customer.name);
    printf("\nBalance: ");
    scanf(" %f", &customer.balance);
    printf("\nPayment Date: (mm/dd/yyyy) ");
    scanf(" %d/%d/%d", &customer.lastpayment.month,
&customer.lastpayment.day, &customer.lastpayment.year);
    return(customer);
}

```

Comparing this program with that shown in the last Example, we see that the two programs are very similar. This program writes the entire customer record to the data file with the single **fwrite** statement

```
fwrite(&customer, sizeof(record), 1, fpt);
```

Note that the data file created by this program is called *data.bin*, as indicated by the first argument within the call to the *fopen* function.

The programmer-defined *writefile* function shown in the last Example is not required in this program, since the **fwrite** library function takes its place. On the other hand, both programs make use of the same programmer-defined function *readscreen*, which causes the information for a given customer record to be entered into the computer interactively.

Execution of this program produces the same interactive dialog as that shown in the last Example.

Once an unformatted data file has been created, the question arises as to how to detect an **end-of-file** condition. The library function **feof** is available for this purpose. (Actually, **feof** will indicate an *end-of-file* condition for any stream-oriented data file, not just an unformatted data file.) This function returns a non-zero value (TRUE) if an end-of-file condition has been detected, and a value of zero (FALSE) if an end of file is not detected. Hence, a program that reads an unformatted data file can utilize a loop that continues to read successive records, as long as the value returned by **feof** is not TRUE.

Reading Customer Record

```

/* create a data file containing customer records */
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

typedef struct
{
    int month;
    int day;
    int year;
} date;

typedef struct
{
    int acct_no;
    char acct_type;
    char name[80];
    float balance;
    date lastpayment;
} record;

void display(record customer);

FILE *fpt;

main()
{
    int n, i;
    record customer;

    if ((fpt = fopen("data.bin", "r")) == NULL)
    {
        printf("\nERROR! File cannot be opened.");
        exit(1);
    }

    do
    {
        fread(&customer, sizeof(record), 1, fpt);
        display(customer);
    } while (!feof(fpt));

    fclose(fpt);
}

void display(record customer)
{
    printf("%d\n%c\n%s\n%.2f\n%d/%d/%d", customer.acct_no,
customer.acct_type, customer.name, customer.balance,
customer.lastpayment.month, customer.lastpayment.day,
customer.lastpayment.year);
}

```

Old Question Solutions

Model Question 2020 Fall

1.a. What is translator? Describe the types of programming language in brief.

A translator is a special type of system software that converts program written in any programming language into MLL. It is also known as language processor. A program code written using any programming language is called source code. A program code obtained after converting source code by translator is called object code or machine code. Mainly, there are three types of language translators: *Assembler, Compiler and Interpreter*. Programming language is defined as a set of rules that provides a platform for instructing computer to perform some specific tasks. Programming languages are classified into various ways. Based on nature of language, they are classified into *general purpose* and *specific purpose* language. On the other hand, programming language are classified into *High-level language* and *Low-level language*.

- iii. Low Level Language (LLL)
 - c. Machine Language
 - d. Assembly Language
- iv. High Level Language (HLL)
 - d. Third Generation Language (3GL)
 - e. Fourth Generation Language (4GL)
 - f. Fifth Generation Language (5GL)

ii) a. Machine Language

Machine language is the language of CPU which consists of set of instructions composed with series of 0s and 1s.

Advantages:

- Machine language does not require translators.
- Execution time of machine language program is extremely fast.

Disadvantages:

- Programming is very complex and tedious job for the programmers.
- They are machine dependent; so, they support only specific hardware.
- It is very difficult to find bugs and hard to debug.

iii) b. Assembly Language

Assembly language is special type of low-level language which consists set of alphanumeric instructions, called *mnemonics*. E.g. ADD A, B

Advantages:

- It is easier to understand than machine language. Also, it consumes less amount of memory than high- level language. Thus, ALL executes much faster than HLL.
- It is efficient for solving hardware interfacing problems. E.g. operating system kernel, programming language, utility software, virus, etc.
- Error finding and debugging process is easier than HLL.

Disadvantages

- ALL is hardware dependent language.
- Programmer should have depth knowledge about the computer hardware during programming.
- Program execution is slower than MLL and the program is lengthy and complex than HLL.

iv) a. Third Generation Language (3GL)

3GL is general-purpose high-level language in which the instructions are expressed in more like English structure and mathematical expressions. E.g. FORTRAN, BASIC, Pascal, C etc. are examples of 3GL.

Advantages

- Machine independent language.
- Easy for program development because the program instructions are English like structure and Mathematical expressions.
- Easy for catching bugs and easy to debug and modify.
- Programmer does not require depth knowledge about computer architecture while programming.

Disadvantages

- 3GL requires translator. So, program execution is slower than LLL.
- They are general-purpose language. They are not suitable for solving complex and specific type of real-world problems.
- They are mostly CUI based language. So, they are less user-friendly.
- Less efficient than 4GL and 5GL.

Fourth Generation Language (4GL)

4GL is a specific-purpose high-level language in which the instructions are expressed by using graphical components and English like structures and Mathematical expressions. E.g.

- *Database and Query Processing Language* (DBase, SyBase, DB2, MySQL, SQL, MS Access, Oracle etc.)
- *Application Generator Language* (Java, C#, Visual C++, Visual.NET etc.)
- *Web Based Scripting Language*: (HTML, XML, JavaScript, VBScript, ASP, PHP, JSP etc.)

Advantages:

- It is efficient and more user friendly.
- It supports rapid application development. Programmers can design, create and debug an application in very short period.
- IDE (Integrated Development Environment) helps developer to include powerful features of different programming languages into one application.
- They are platform independent.

Disadvantages:

- They require more memory and disk storage, hence, take more execution time.
- It is not suitable for solving hardware interfacing problems.

d. Fifth Generation Language (5GL)

5GL is a special type of future generation language in which the statements will be expressed in more like human language. E.g. PROLOG, LISP etc. are considered as 5GL. Some characteristics of 5GL are:

- Natural languages make human and machine closer than before.
- They are designed to allow the computer more natural and intelligent than before.
- They are used in the areas of research, robotics, defense system, and different types of expert systems.

1.b. Why documentation is carried out during problem solving using computer? Write an algorithm to check whether the given number is prime or not.

The same workforce that developed the program may have left the project and gone in search of opportunities. So, a program must be well documented for the new people to understand how the software was developed and how can it be modified. The documentation starts from the very beginning of the problem formulation to the very end of the Evaluation and Maintenance. Program documentation is the systematic writing of activities from problem analysis to debugging/testing phase including operational information and presenting it as a report. There are two types of documentations: Technical documentation for programmer and user manual for user.

Technical documentation is prepared for future reference to the programmers who maintain, redesign and upgrade the system. It may contain detailed requirements of program, format and reports and forms, logic of the program, data flow diagram (DFD), Entity-Relationship diagram (ERD), algorithm and flowchart, input and output data used for testing and other references.

The user manual supports to the user of the program. This provides instructions for installation of the program and use of it effectively.

Algorithm

11. Start.
12. Declare variables n, j, rem.
13. Read a number n.
14. Initialize j=2.
15. Calculate $\text{rem} = n \bmod j$.
16. If $\text{rem} = 0$ then go to step 8.
17. Increment j by 1.
18. If $j < n$, then go to step 5.
19. Check if $n = j$. If true, then print "n is prime". Otherwise, print "n is no prime".
20. Stop.

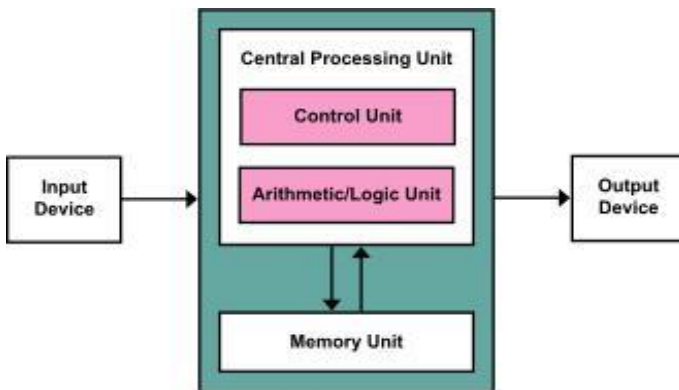
2.a. What is programming language? Draw block diagram of computer and explain each component in brief.

Programming language is defined as a set of rules that provides a platform for instructing computer to perform some specific tasks. Programming languages are classified into various ways. Based on nature of language, they are classified into *general purpose* and *specific purpose* language. On the other hand, programming language are classified into *High-level language* and *Low-level language*.

v. Low Level Language (LLL)

- e. Machine Language
- f. Assembly Language
- vi. High Level Language (HLL)
 - g. Third Generation Language (3GL)
 - h. Fourth Generation Language (4GL)
 - i. Fifth Generation Language (5GL)

The von-Neumann architecture, which is also known as the von Neumann model and Princeton architecture, is a computer architecture based on that described in 1945 by the mathematician and physicist John von Neumann. This describes a design architecture for an electronic digital computer with parts consisting of a processing unit containing an arithmetic logic unit and processor registers; a control unit containing an instruction register and program counter; a memory to store both data and instructions; external mass storage; and input and output mechanisms. The key idea of the Von-Neumann architecture is the stored program concept. A stored-program digital computer is one that keeps its program instructions, as well as its data, in read-write, random-access memory (RAM). Stored-program computers were an advancement over the program-controlled computers of the 1940s.



The main components of a computer are **Input unit (IU)**, **Central Processing unit (CPU)** and **Output unit (OU)**. The information like data, programs etc. are passed to the computer through input devices. The keyboard, mouse, floppy disk, CD, DVD, joystick etc. are certain input devices. The output device is to get information from a computer after processing. VDU (Visual Display Unit), Printer, Floppy disk, CD etc. are

output devices.

The brain of a computer is CPU. It has three components- **Memory unit**, **Control unit** and **Arithmetic and Logical unit (ALU)**. Memory unit also called storage device is to store information. Two types memory are there in a computer. They are RAM (random access memory) and ROM (read only memory). When a program is called, it is loaded and processed in RAM. When the computer is switched-off, whatever stored in RAM will be deleted. So, it is a temporary memory. Whereas, ROM is a permanent memory, where data, program etc. are stored for future use. Inside a computer there is storage device called Hard disk, where data are stored and can be accessed at any time.

The control unit is for controlling the execution and interpreting of instructions stored in the memory. ALU is the unit where the arithmetic and logical operations are performed. The information to a computer is transformed to groups of binary digits, called bit. The length of bit varies from computer to computer, from 8 to 64. A group of 8 bits is called a *Byte* and a byte generally represents one alphanumeric (Alphabets and Numerals) character. The physical components of a computer are called hardware. But for the machine to work it requires certain programs. They are called software.

2.b. What are significances of documentation during problem solving using computer?

Write an algorithm to check if the given number is Armstrong or not.

See solution of 2.a.

Algorithm

13. Start.
14. Declare variables rem, num, Sum.
15. Initialize Sum = 0.
16. Read num.
17. While num > 0, repeat steps 6, 7 and 8. Otherwise, go to step 9.
18. Calculate rem = num mod 10.
19. Sum = Sum + rem * rem * rem.
20. Update num = num / 10 and go to step 5.
21. Check if j = Sum. If true, then print "Armstrong Number". Otherwise print "not Armstrong number".
22. Stop.

3.a. What is data type? Explain relational, logical and bitwise operators in brief.

Data type is a collection of related data and the allowed operations over those data. C supports several different types of data, each of which may be represented differently within the computer's memory. For example, int, float, char, void etc.

h. Relational Operators

They are binary operators which are used to compare two similar operands. They compare their left-hand side operand with their right-hand side operand for comparisons according to magnitudes. The output of a relational operation is either *true* (1) or *false* (0). So, they are used in decision making statements.

Operator	Purpose	Example	Output int a=20, b=6	Output int a=20, b=20
<	Less than	a < b	0	0
>	Greater than	a > b	1	0
<=	Less than or Equal to	a <= b	0	1
>=	Greater than or Equal to	a >= b	1	1

These operators all fall within the same precedence group, which is lower than the arithmetic and unary operators. The associativity of these operators is left to right.

Closely associated with the relational operators are the following two *equality operators*.

Operator	Purpose	Example	Output int a=20, b=6	Output int a=20, b=20
==	Equal to	a == b	0	1
!=	Not Equal to	a != b	1	0

The equality operators fall into a separate precedence group, beneath the relational operators. These operators also have a *left-to-right associativity*.

These six operators are used to form **logical expressions**, which represent conditions that are either true or false. The resulting expressions will be of type integer, since *true* is represented by the integer value 1 and *false* is represented by the value 0.

Suppose that i, j and k are integer variables whose values are 1, 2 and 3, respectively. Several logical expressions involving these variables are shown below.

Expression	Interpretation	Value
i < j	True	1

$(i + j) \geq k$	True	1
$(j + k) > (i + 5)$	False	0
$k \neq 3$	False	0
$j == 2$	True	1

Suppose that i is an integer variable whose value is 7, f is a floating-point variable whose value is 5.5, and c is a character variable that represents the character 'w'. Several logical expressions that make use of these variables are shown below. Each expression involves two different type operands. (Assume that the **ASCII** character set applies.)

Expression	Interpretation	Value
$f > 5$	True	1
$(i + f) \leq 10$	False	0
$c == 119$	True	1
$c \neq 'p'$	True	1
$c \geq 10 * (i + f)$	False	0

i. Logical Operators

In addition to the relational and equality operators, C contains two *logical operators* (also called *logical connectives*). They are

Operator	Meaning
&&	Logical AND
 	Logical OR

The logical operators act upon operands that are themselves logical expressions. The net effect is to combine the individual logical expressions into more complex conditions that are either true or false. The result of a *logical and* operation will be true only if both operands are true, whereas the result of a *logical or* operation will be true if either operand is true or if both operands are true. In other words, the result of a *logical or* operation will be false only if both operands are false.

Suppose that i is an integer variable whose value is 7, f is a floating-point variable whose value is 5.5, and c is a character variable that represents the character 'w'. Several complex logical expressions that make use of these variables are shown below.

Expression	Interpretation	Value
$(i \geq 6) \&\& (c == 'w')$	True	1
$(i \geq 6) (c == 119)$	True	1
$(f < 11) \&\& (i > 100)$	False	0
$(c \neq 'p') ((i + f) \leq 10)$	True	1

The first expression is true because both operands are true. In the second expression, both operands are again true; hence the overall expression is true. The third expression is false because the second operand is false. And finally, the fourth expression is true because the first operand is true.

Each of the logical operators falls into its own precedence group. *Logical and* has a higher precedence than *logical or*. Both precedence groups are lower than the group containing the equality operators. The associativity is left to right. The precedence groups are summarized below.

C also includes the unary operator **!** that negates the value of a logical expression; i.e., it causes an expression that is originally true to become false, and vice versa. This operator is referred to as the *logical negation* (or *logical not*) operator.

Suppose that i is an integer variable whose value is 7, and f is a floating-point variable whose value is 5.5. Several logical expressions which make use of these variables and the logical negation operator are shown below.

Expression	Interpretation	Value
$f > 5$	True	1
$!(f > 5)$	False	0
$i \leq 3$	False	0
$!(i \leq 3)$	True	1
$i > (f + 1)$	True	1
$!(i > (f + 1))$	False	0

The hierarchy of operator precedence covering all the operators discussed so far has become extensive. These operator's precedence is summarized below, from highest to lowest.

j. Bitwise Operators

Bitwise operators are used for manipulating data at bit level. These operators are used for testing bits or shifting them to the left or to the right. Bitwise operators can be applied only to integer-type operands. There are three types of bitwise operators:

- Bitwise logical operators
- Bitwise shift operators
- One's complement operators

IV. Bitwise logical operators

Bitwise logical operators perform logical tests between two integer operands bit by bit starting from Least Significant Bit (LSB) or right most bit. There are three bitwise logical operators: bitwise AND (&), bitwise OR (|) and bitwise Exclusive OR (^).

Bitwise AND (&)

It performs logical *ANDing* between two operands. The result of *ANDing* operation is 1 if both bits are 1, otherwise it is 0. Let us consider two variables, $n1 = 60$ and $n2 = 15$. The binary representation of these values is:

$n1 \rightarrow 0000\ 0000\ 0011\ 1100$

$n2 \rightarrow 0000\ 0000\ 0000\ 1111$

$n3 = n1 \ \& \ n2;$

$n3 \rightarrow 0000\ 0000\ 0000\ 0110$ (i.e., 12)

Bitwise OR (|)

It performs logical *ORing* between two operands. The result of *ORing* operation is 1 if either of bits is 1, otherwise it is 0. Let us consider two variables, $n1 = 60$ and $n2 = 15$. The binary representation of these values is:

$n1 \rightarrow 0000\ 0000\ 0011\ 1100$

$n2 \rightarrow 0000\ 0000\ 0000\ 1111$

$n3 = n1 \ | \ n2;$

$n3 \rightarrow 0000\ 0000\ 0011\ 1111$ (i.e., 63)

Bitwise XOR (^)

The result of Exclusive *ORing* operation is 1 only if one of the bits have the value 1, otherwise it is 0. Let us consider two variables, $n1 = 60$ and $n2 = 15$. The binary representation of these values is:

$n1 \rightarrow 0000\ 0000\ 0011\ 1100$

$n2 \rightarrow 0000\ 0000\ 0000\ 1111$

$n3 = n1 \wedge n2;$

$n3 \rightarrow 0000\ 0000\ 0011\ 0011$ (i.e., 51)

V. Bitwise Shift Operators

Bitwise shift operators are used to move bit patterns either to the left or to the right. There are two bitwise shift operators: *left shift* (<<) and *right shift* (>>).

Left Shift (<<)

The left-shift operation causes operand to be shifted to the left by some bit positions. The general form of left-shift operation is:

operand << n;

The bits in the operand are shifted to the left by n positions. The *leftmost n bits* in the original bit pattern will be *lost* and the *rightmost n bits* empty positions will be filled with 0s. For example, if $n1 = 60$, the statement $n2 = n1 \ll 3$ will be processed as follows.

$n1$ 0000 0000 0011 1100

First Shift 0000 0000 0111 1000

Second Shift 0000 000 1111 0000

Third Shift 0000 0001 1110 0000 (i.e., 480)

Right Shift (>>)

The right-shift operation causes operand to be shifted to the *right* by some bit positions. The general form of right-shift operation is:

operand >> n;

The bits in the operand are shifted to the *right* by n positions. The *rightmost n bits* in the original bit pattern will be *lost* and the leftmost n bits empty positions will be filled with 0s. For example, if $n1 = 60$, the statement $n2 = n1 \gg 3$ will be processed as follows.

$n1$ 0000 0000 0011 1100

First Shift 0000 0000 0001 1110

Second Shift 0000 000 0000 1111

Third Shift 0000 0000 0000 0111 (i.e., 7)

VI. Bitwise One's Complement

Bitwise One's complement operator is a unary operator which *inverts* (i.e., 1 to 0 and vice versa) all the bits represented by its operand. For example, if $n1 = 60$, the statement $n2 = \sim n1;$

$n1 \rightarrow 0000\ 0000\ 0011\ 1100$

$\sim n2 \rightarrow 1111\ 1111\ 1100\ 0011$

3.b. Write a program to find the Lowest Common Multiple of two numbers using function.

```
#include <stdio.h>
```

```
int lcm(int a, int b)
```

```
{
    int temp, i, HCF, LCM;
    if (a > b)
```

```

{
    temp = a;
    a = b;
    b = temp;
}

for (i = a; i >= 1; --i)
{
    if (a % i == 0 && b % i == 0)
    {
        HCF = i;
        break;
    }
}
LCM = (a * b) / HCF;
return LCM;
}
void main()
{
    int a, b, LCM;
    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);
    LCM = lcm(a, b);
    printf("LCM: %d", LCM);
}

```

OUTPUT

Enter two numbers: 25 10

LCM: 50

4.a. What is an operator? Explain any four data types in brief.

An *operator* is a symbol that operates on single or multiple data items. They are used to perform mathematical or logical manipulations. The data items that operators act upon are called *operands*. Some operators require two operands, while others act upon only one operand.

2. According to number of operands

d) Unary Operators (single operand)

e.g., unary minus (-), unary plus (+), ++, --, !, ~, sizeof, type etc.

e) Binary Operators (two operands)

e.g., Binary plus (+), Binary minus (-), *, /, >, <, &&, << etc.

f) Ternary Operators (three operands)

e.g., conditional operator (?:)

2. According to Operations

i. Arithmetic Operators

j. Relational Operators

k. Logical Operators

l. Assignment Operators

m. Increment/Decrement

n. Conditional Operator

- o. Bitwise Operators
- p. Special operators

The basic data types can be augmented (prefixed) using the data type *qualifiers* **short**, **long**, **signed** and **unsigned**. For example, integer quantities can be defined as *short int*, *long int* or *unsigned int* (these data types are usually written simply as *short*, *long* or *unsigned*, and are understood to be integers).

The interpretation of a qualified integer data type will vary from one C compiler to another, though there are some commonsense relationships. Thus, a *short int* may require less memory than an ordinary *int* or it may require the same amount of memory as an ordinary *int*, but it will never exceed an ordinary *int* in word length. Similarly, a *long int* may require the same amount of memory as an ordinary *int* or it may require more memory, but it will never be less than an ordinary *int*.

If *short int* and *int* both have the same memory requirements (e.g., 2 bytes), then *long int* will generally have double the requirements (e.g., 4 bytes). Or if *int* and *long int* both have the same memory requirements (e.g., 4 bytes) then *short int* will generally have half the memory requirements (e.g., 2 bytes). Remember that the specifics will vary from one C compiler to another.

An *unsigned int* has the same memory requirements as an ordinary *int*. However, in the case of an ordinary *int* (or a *short int* or a *long int*), the leftmost bit **is** reserved for the sign. With an *unsigned int*, all the bits are used to represent the numerical value.

Thus, an *unsigned int* can be approximately twice as large as an ordinary *int* (though, of course, *negative values are not permitted*). For example, if an ordinary *int* can vary from -32,768 to +32,767 (which is typical for a 2-byte *int*), then an *unsigned int* will be allowed to vary from 0 to 65,535. The *unsigned* qualifier can also be applied to other qualified integers, e.g., *unsigned short int* or *unsigned long int*.

The *char* type is used to represent individual characters. Hence, the *char* type will generally require only one byte of memory. Each *char* type has an equivalent integer interpretation, however, so that a *char* is really a special kind of *short integer*. With most compilers, a *char* data type will permit a range of values extending from 0 to 255. Some compilers represent the *char* data type as having a range of values extending from -128 to +127. There may also be *unsigned char* data (with typical values ranging from 0 to 255), or *signed char* data (with values ranging from -128 to +127).

Some compilers permit the qualifier *long* to be applied to *float* or to *double*, e.g., *long float*, or *long double*. However, the meaning of these data types will vary from one C compiler to another. Thus, *long float* may be equivalent to *double*.

Moreover, *long double* may be equivalent to *double*, or it may refer to a separate, “extra-large” double-precision data type requiring more than two words of memory.

Two additional data types, **void** and **enum** are also used frequently. Every identifier that represents a number or a character within a C program must be associated with one of the basic data types before the identifier appears in an executable statement. This is accomplished via a *type declaration*.

Data Type Summary

<u>Data Type</u>	<u>Description</u>	<u>Typical Memory Requirements</u>
int	<i>Integer quantity</i>	2 bytes or 1 word <i>(varies from one computer to another)</i>
short	<i>Short integer quantity (may contain fewer digits than int)</i>	2 bytes or 1 word <i>(varies from one computer to another)</i>
long	<i>Long integer quantity (may contain more digits than int)</i>	1 or 2 words <i>(varies from one computer to another)</i>
unsigned	<i>Unsigned (positive) integer quantity (maximum permissible quantity is approximately twice as large as int)</i>	2 bytes or 1 word <i>(varies from one computer to another)</i>
char	<i>Single character</i>	1 byte
signed char	<i>Single character, with numerical values ranging from -128 to +127</i>	1 byte
unsigned char	<i>Single character, with numerical values ranging from 0 to 255</i>	1 byte
float	<i>Floating-point number (i.e., a number containing a decimal point and or an exponent)</i>	1 word
double	<i>Double-precision floating-point number (i.e., more significant figures, and an exponent that may be larger in magnitude)</i>	2 words
long	<i>Double-precision floating-point number</i>	2 or more words
double	<i>(may have higher precision than double)</i>	<i>(varies from one computer to another computer)</i>
void	<i>Special data type for functions that do not return any value</i>	(not applicable)
enum	<i>Enumeration constant (special type of int)</i>	2 bytes or 1 word <i>(varies from one computer to another)</i>

4.b. Write a program to find the Greatest Common Divisor (GCD) of two numbers using function.

```
#include <stdio.h>

int gcd(int a, int b)
{
    int temp, i, HCF;
    if (a > b)
    {
        temp = a;
        a = b;
        b = temp;
    }

    for (i = a; i >= 1; --i)
    {
        if (a % i == 0 && b % i == 0)
        {
            HCF = i;
            break;
        }
    }
    return HCF;
}

void main()
{
    int a, b, hcf;
    printf("Enter two numbers: ");
    scanf(" %d %d", &a, &b);
    hcf = gcd(a, b);
    printf("GCD= %d", hcf);
}
```

OUTPUT

Enter two numbers: 25 10

GCD: 5

5.a. What is an array? Write a program to find product of two matrixes using pointer.

```
//matrix multiplication
#include<stdio.h>

#define M 3
#define N 2
#define P 2
#define Q 3

void main()
{
```

```

int a[M][N], b[P][Q], R[M][Q], i, j, k;

printf("Enter elements of first matrix %dx%d\n", M, N);
for (i = 0; i < M; ++i)
{
    for (j = 0; j < N; ++j)
    {
        scanf(" %d", *(a + i) + j);
    }
}

printf("Enter elements of second matrix %dx%d\n", P, Q);
for (i = 0; i < P; ++i)
{
    for (j = 0; j < Q; ++j)
    {
        scanf(" %d", *(b + i) + j);
    }
}

for (i = 0; i < M; ++i)
{
    for (j = 0; j < Q; ++j)
    {
        *(*(R + i) + j) = 0;
        for (k = 0; k < P; ++k)
        {
            *(*(R+i)+j) += *(*(a+i)+k) * *(*(b+k)+j);
        }
    }
}

printf("\nProduct is\n");
for (i = 0; i < M; ++i)
{
    for (j = 0; j < Q; ++j)
    {
        printf("%d\t", *(*(R + i) + j));
    }
    printf("\n");
}
}

```

OUTPUT

Enter elements of first matrix 3x2

1 1
1 1
1 1

Enter elements of second matrix 2x3

1 1 1

1 1 1

Product is

2	2	2
2	2	2
2	2	2

5.b. Write a program to find out whether the given string is palindrome or not without using string handling function.

*/*To check whether the given string is palindrome or not without using string handling functions*/*

```
#include<stdio.h>

void main()
{
    char str[20];
    int len = 0, i;

    printf("Enter a string: ");
    gets(str);

    //finding length
    i = 0;
    while (str[i] != '\0')
    {
        len++;
        i++;
    }
    //check for reverse
    for (i = 0; i < len / 2; ++i)
    {
        if (str[i] != str[len - 1 - i])
            break;
    }

    if (i == len / 2)
        printf("%s is palindrome.", str);
    else
        printf("%s is not palindrome.", str);
}
```

OUTPUT

RUN 1:

```
Enter a string: LIRIL
LIRIL is palindrome.
```

RUN 2:

```
Enter a string: father
father is not palindrome.
```

6.a. Differentiate between iteration and recursion. Find the sum of n natural numbers using recursive function.

Recursion	Iteration
A function calls to itself until certain condition is satisfied and repeated task.	Loop is used to perform repeated task.
Recursion is a top-down approach to problem solving.	Iteration is a bottom-up approach to problem solving.
A problem should be written or defined in term of its previous result.	It is not necessary to define a problem in term of its previous result.
All problems cannot be solved using recursion.	All problems can be solved using iteration.

//Sum of N natural numbers

```
#include <stdio.h>
int sum(int n)
{
    if(n == 1)//stopping condition
        return 1;
    else
        return(n + sum(n - 1));
}

void main()
{
    int n, s;
    printf("Enter value of n: ");
    scanf("%d", &n);
    s = sum(n);
    printf("Total sum = %d", s);
}
```

OUTPUT

Enter value of n: 10
Total sum = 55

6.b. Which method is default argument passing to function in C? How many ways do you pass arguments to function? Explain with suitable example.

In C, all arguments are *passed by value* by default. There are two ways of passing arguments to function: pass by value and pass by reference.

Call by Value

When a single value is passed to a function via an actual argument, the value of the actual argument is *copied* into the function. Therefore, *the value of the corresponding formal argument can be altered within the Function, but the value of the actual argument within the calling routine will not change*. This procedure for passing the value of an argument to a function is known as *passing by value*.

Here is a simple C program containing a function that alters the value of its argument.

```
#include <stdio.h>

void modify(int a);
```

```

main()
{
    int a = 2;

    printf("\na = %d (from main, before calling the
function)", a);
    modify(a) ;// function call a is a normal variable
    printf("\na = %d (from main, after calling the
function)", a);
}

void modify(int a)
{
    a *= 3;
    printf("\na = %d (from the function, after being
modified)", a);
    return;
}

```

The original value of **a** (i.e., **a = 2**) is displayed when main begins execution. This value is then passed to the function **modify**, where it is multiplied by **3** and the new value displayed. Note that it is the altered value of the formal argument that is displayed within the function. Finally, the value of **a** within main (i.e., the actual argument) is again displayed, after control is transferred back to main from **modify**. When the program is executed, the following output is generated.

```

a = 2 (from main, before calling the function)
a = 6 (from the function, after being modified)
a = 2 (from main, after calling the function)

```

These results show that **a** is not altered within main, even though the corresponding value of **a** is changed within **modify()**.

Passing an argument by value has advantages and disadvantages. On the plus side, it allows a single-valued actual argument to be written as an expression rather than being restricted to a single variable. Moreover, if the actual argument is expressed simply as a single variable, it protects the value of this variable from alterations within the function. On the other hand, it does not allow information to be transferred back to the calling portion of the program via arguments. Thus, *passing by value is restricted to a one-way transfer of information.*

Call by Reference

To pass an array to a function, the array name must appear by itself, without brackets or subscripts, as an actual argument within the function call. The corresponding formal argument is written in the same manner, though it must be declared as an array within the formal argument declarations. When declaring a one-dimensional array as a formal argument, the array name is written with a pair of empty square brackets. The size of the array is *not specified* within the formal argument declaration.

When an array is passed to a function, however, the values of the array elements *are not* passed to the function. Rather, the array name is interpreted as the **address** of the first array element (i.e., the address of the memory location containing the first array element). This address is assigned to the corresponding formal argument when the function is called. The formal argument therefore becomes a **pointer** to the first array element (more about this in the

next chapter). Arguments that are passed in this manner are said to be passed **by reference** rather than passed *by value*.

When a reference is made to an array element within the function, the value of the element's subscript is added to the value of the pointer to indicate the address of the specified array element. Therefore, any array element can be accessed from within the function. Moreover, *if an array element is altered within the function, the alteration will be recognized in the calling portion of the program* (actually, throughout the entire scope of the array).

Here is a simple C program that passes a three-element integer array to a function, where the array elements are altered. The values of the array elements are displayed at three different places in the program, thus illustrating the effects of the alterations.

```
#include <stdio.h>
void modify(int a[]); /* function prototype */

main()
{
    int count, a[3];

    printf("\nEnter any three numbers:\n");
    for (count = 0; count < 3; ++count)
    {
        printf("\na[%d] = ", count);
        scanf(" %d", &a[count]);
    }

    printf("From main(), before modification:\n");
    for (count = 0; count < 3; ++count)
    {
        printf("\na[%d] = %d", count, a[count]);
    }

    modify(a); /* function call a is an array*/

    printf("From main(), after modification:\n");
    for (count = 0; count < 3; ++count)
    {
        printf("\na[%d] = %d", count, a[count]);
    }
}

void modify(int a[]) /* function definition */
{
    int count;

    printf("\nFrom the function, after modification the values:\n");
    for (count = 0; count < 3; ++count)
    {
        a[count] = -9;
        printf("a[%d] = %d\n", count, a[count]);
    }
}
```

```

    }
    //return;
}

```

The array elements are read from the user within the first loop appearing in *main*. These values are displayed as soon as they are assigned. The array is then passed to the function ***modify***, when each array element is assigned the value **-9**. These new values are then displayed from within the function. Finally, the values of array elements are again displayed from ***main()***, after control has been transferred back to *main* from *modify*. When the program is executed, the following output is generated.

```

Enter any three numbers:
a[0] = 1
a[1] = 2
a[2] = 3
From main(), before modification:
a[0] = 1
a[1] = 2
a[2] = 3
From the function, after modification the values:
a[0] = -9
a[1] = -9
a[2] = -9
From main(), after modification:
a[0] = -9
a[1] = -9
a[2] = -9

```

These results show that the elements of ***a*** are altered within *main* as a result of the changes that were made within *modify*.

7.a. What is the meaning of the following pointer declaration?

i. *float *m[20];*

m is a 20-element array of pointers to floating point quantities.

ii. *char (*ch) [30];*

ch is a pointer to 30-element character array.

Write a program to find out the top 5 highest marks out of 48 students using DMA.

```

/* top 5 highest marks out of 48 students using DMA */

#include <stdio.h>
#include <stdlib.h>

#define SIZE 48

void reorder(int n, int *x);

main()
{
    int i, *x;

    /* dynamically allocate memory for SIZE numbers */

```

```

x = (int *) malloc(SIZE * sizeof(int));

/*read in the list of numbers */
for (i = 0; i < SIZE; ++i)
{
    printf("i = %d x = ", i + 1);
    scanf(" %d", x + i);
}

/* reorder all array elements */
reorder(SIZE, x);

/* display the reordered list of numbers */
printf("\nTop 5 Highest Marks are:\n");
for (i = 0; i < 5; ++i)
    printf("i = %d x = %d\n", i+1, *(x + i));
free(x);
}

void reorder(int n, int *x)
{
    int i, item, temp;
    for (item = 0; item < n - 1; ++item)
        for (i = item + 1; i < n; ++i)
            if(*(x + i) > *(x + item))
            {
                temp = *(x + item);
                *(x + item) = *(x + i);
                *(x + i) = temp;
            }
    return;
}

```

7.b. Evaluate: $1+4+9/2+16/6+\dots$ up to n terms.

```

/* 1+4+9/2+16/6+...up to n terms
(hint  $1^2/0!$   $2^2/1!$   $3^2/2!$   $4^2/3!$ ...) */

```

```

#include <stdio.h>
#include <math.h>

```

```

main()
{
    int n, j, i, f;
    float term, sum = 0.;
    printf("How many terms? ");
    scanf(" %d", &n);
    for (i = 0; i < n; ++i)
    {
        f = 1;
        for (j = 1; j <= i; ++j)

```

```

        f *= j;
        term = pow((i + 1), 2) / (float)f; //type casting to float
        printf("%+f ", term);
        sum += term;
    }
    printf(" = %f", sum);
}

```

OUTPUT

How many terms? 5
 +1.000000 +4.000000 +4.500000 +2.666667 +1.041667 = **13.208334**

8.a. What is the meaning of the following pointer declarations?

i. `char *ch[5];`

ch is a 5-element array of pointers to character.

ii. `int (*ptr)[10];`

ptr is a pointer to 10-element integer array.

Write a program to sort marks of 48 students in ascending order using DMA.

```

/* reordering marks of 48 students in ascending order using DMA */

#include <stdio.h>
#include <stdlib.h>

#define SIZE 48
void reorder(int n, int *x);

main()
{
    int i, *x;

    /* dynamically allocate memory for n numbers */
    x = (int *) malloc(SIZE * sizeof(int));

    /*read in the list of numbers */
    for (i = 0; i < SIZE; ++i)
    {
        printf("i = %d x = ", i + 1);
        scanf(" %d", x + i);
    }

    /* reorder all array elements */
    reorder(SIZE, x);

    /* display the reordered list of numbers */
    printf("\nReordered list of numbers:\n");
    for (i = 0; i < SIZE; ++i)
        printf("i = %d x = %d\n", i+1, *(x + i));
    free(x);
}

```

void reorder(int n, int *x)

```

{
    int i, item, temp;
    for (item = 0; item < n - 1; ++item)
        for (i = item + 1; i < n; ++i)
            if(*(x + i) < *(x + item))
            {
                temp = *(x + item);
                *(x + item) = *(x + i);
                *(x + i) = temp;
            }
    return;
}

```

8.b. Evaluate: $1+1/4+2/9+6/16+\dots$ up to n terms.

/* $1+1/4+2/9+6/16+\dots$ up to n terms.

$0!/1^2 + 1!/2^2 + 2!/3^2 + 3!/4^2 + \dots$ */

```
#include <stdio.h>
```

```
#include <math.h>
```

```
main()
```

```

{
    int n, j, i, f;
    float term, sum = 0.;
    printf("How many terms? ");
    scanf(" %d", &n);
    for (i = 0; i < n; ++i)
    {
        f = 1;
        for (j = 1; j <= i; ++j)
            f *= j;
        term = (float)f / pow((i + 1), 2); //type casting to
float
        printf(" %f", term);
        sum += term;
    }
    printf(" = %f", sum);
}

```

OUTPUT

How many terms? 5

+1.000000 +0.250000 +0.222222 +0.375000 +0.960000 = **2.807222**

9.a. Create structure to keep record about student having members roll number, name, total marks and date of birth. Write a program to insert information about 96 students and display them. Also count the number of students who have secured marks greater than 60.

```
#include <stdio.h>
```

```
#define N 96
```

```
typedef struct
```

```

{
    int mm;

```



```

        int dd;
        int yy;
    } date;

typedef struct
{
    int roll;
    char name[30];
    float marks;
    date dob;
} student;
main()
{
    int i, count = 0;
    student s[N];
    printf("Enter Information about\n");
    for (i = 0; i < N; ++i)
    {
        printf("Student no. %d\n", i + 1);
        printf("Roll Number: ");
        scanf(" %d", &s[i].roll);
        printf("Name: ");
        scanf(" %[^\\n]", s[i].name);
        printf("Marks: ");
        scanf(" %f", &s[i].marks);
        if (s[i].marks > 60)
            ++count;
        printf("Date of Birth (mm-dd-yy): ");
        scanf(" %d-%d-%d", &s[i].dob.mm, &s[i].dob.dd,
&s[i].dob.yy);
    }

    printf("\nDetails about all students\n");
    printf("SN Roll# Name           Marks DOB(mm-dd-yy)");
    for (i = 0; i < N; ++i)
    {
        printf("\n%2d %2d %s %.2f %d-%d-%d", i + 1,
s[i].roll, s[i].name, s[i].marks, s[i].dob.mm, s[i].dob.dd,
s[i].dob.yy);
    }

    printf("\nTotal students who secured marks greater than
60: %d", count);
}

```

9.b. Consider a structure Book having attributes ISBN, Title, Price and Author name. Create a file named "book.txt" to store records of 50 books. Also display the record of books from file which are written by Gottfried.

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define N 3

```

```

typedef struct
{
    int ISBN;
    char title[60];
    float price;
    char author_name[30];
} book;

main()
{
    int i;
    FILE *fptr;
    book b;
    if ((fptr = fopen("book.txt", "w+")) == NULL)
    {
        printf("File cannot be created.");
        exit(1);
    }
    for (i = 0; i < N; ++i)
    {
        printf("\nBook no. %d\n", i + 1);
        printf("ISBN: ");
        scanf(" %d", &b.ISBN);
        printf("Author: ");
        scanf(" %[^\\n]", b.author_name);
        printf("Title: ");
        scanf(" %[^\\n]", b.title);
        printf("Price: ");
        scanf(" %f", &b.price);
        fwrite(&b, sizeof(book), 1, fptr);
    }
    rewind(fptr);
    printf("\nISBN\\tAuthor\\tTitle\\tPrice");
    for (i = 0; i < N; ++i)
    {
        fread(&b, sizeof(book), 1, fptr);
        if (strcmp(b.author_name, "Gottfried") == 0)
            printf("\n%d\\t%s\\t%s\\t%.2f", b.ISBN,
b.author_name, b.title, b.price);
    }
}

```

10.a. What is an array? Write a program to find product of squares of diagonal elements of $M \times N$ matrix using pointer.

Arrays are defined in much the same manner as ordinary variables, except that each array name must be accompanied by a size specification (i.e., the number of elements). For a one-dimensional array, the size is specified by a positive integer expression, enclosed in square brackets. The expression is usually written as a positive integer constant.

In general terms, a one-dimensional array definition may be expressed as

storage-class datatype array[expression];

where *storage-class* refers to the storage class of the array, *datatype* is the data type, *array* is the array name, and *expression* is a positive-valued integer expression which indicates the number of array elements. The *storage-class* is optional; default values are **automatic** for arrays that are defined within a function or a block, and **external** for arrays that are defined outside of a function.

//to find product of squares of diagonal elements of M*N matrix using pointer.

```
#include <stdio.h>
```

```
main()
```

```
{
    int m, n, i, j, sum = 0;
    int mat[10][10];

    printf("Enter no. of rows and columns: ");
    scanf(" %d %d", &m, &n);
    printf("Enter elements of %d*%d matrix\n", m, n);
    for (i = 0; i < m; ++i)
    {
        for (j = 0; j < n; ++j)
        {
            printf("mat[%d][%d]: ", i, j);
            scanf(" %d", *(mat + i) + j);
        }
    }

    for (i = 0; i < m; ++i)
    {
        for (j = 0; j < n; ++j)
        {
            if (i == j) //for diagonal element, i = j
            {
                sum += (*(mat+i)+j) * (*(mat+i)+j));
            }
        }
    }

    printf("Sum of square of diagonal elements: %d", sum);
}
```

OUTPUT

Enter no. of rows and columns: 3 3

Enter elements of 3*3 matrix

mat[0][0]: 1

mat[0][1]: 2

mat[0][2]: 3

mat[1][0]: 4

mat[1][1]: 5

mat[1][2]: 6

mat[2][0]: 7

```
mat[2][1]:  $\frac{8}{9}$ 
mat[2][2]:  $\frac{9}{9}$ 
Sum of square of diagonal elements: 107
```

10.b. Find out the reverse of the given string without using string handling function.

/*To check whether the given string is reverse or not
without using string handling functions*/

```
#include<stdio.h>

void main()
{
    char str[20], rev[20];
    int len = 0, i;

    printf("Enter a string: ");
    gets(str);

    //finding length
    i = 0;
    while (str[i] != '\0')
    {
        len++;
        i++;
    }
    for (i = 0; i < len; ++i)
    {
        rev[i] = str[len - 1 - i];
    }
    rev[i] = '\0'; //the last character is null character
    printf("Reverse string is: %s", rev);
}
```

OUTPUT

```
Enter a string: Nepal
Reverse string is: lapeN
```

11.a. Explain the static and local storage classes in C in brief. Find the sum of digits of given number using recursive function.

Static Variables

Static variables are defined within individual functions and therefore have the same scope as automatic variables; i.e., they are local to the functions in which they are defined. Unlike automatic variables, however, static variables retain their values throughout the life of the program. Thus, if a function is exited and then re-entered later, the static variables defined within that function will retain their former values. This feature allows functions to retain information permanently throughout the execution of a program.

Static variables are defined within a function in the same manner as automatic variables, except that the variable declaration must begin with the **static** storage-class designation. Static variables can be utilized within the function in the same manner as other variables. They cannot, however, be accessed outside of their defining function.

It is not unusual to define automatic or static variables having the same names as external variables. In such situations the local variables will *take precedence over* the external variables, though the values of the external variables will be unaffected by any manipulation of the local variables. Thus, the external variables maintain their independence from locally defined automatic and static variables. The same is true of local variables within one function that have the same names as local variables within another function.

Initial values can be included in the static variable declarations. The rules associated with the assignment of these values are essentially the same as the rules associated with the initialization of external variables, even though the static variables are defined locally within a function. In particular:

- The initial values must be expressed as constants, not expressions.
- The initial values are assigned to their respective variables at the beginning of program execution. The variables retain these values throughout the life of the program, unless different values are assigned during the computation.
- **Zeros** will be assigned to all static variables whose declarations do not include explicit initial values. Hence, static variables will always have assigned values.

Automatic Variables

Automatic variables are always declared within a function and are local to the function in which they are declared; that is, their scope is confined to that function. Automatic variables defined in different functions will therefore be independent of one another, even though they may have the same name.

Any variable declared within a function is interpreted as an automatic variable unless a different storage-class specification is shown within the declaration. This includes formal argument declarations. All the variables in the programming examples encountered earlier have been automatic variables.

Since the location of the variable declarations within the program determines the *automatic storage class*, the keyword **auto** is not required at the beginning of each variable declaration. There is no harm in including an auto specification within a declaration, though this is normally not done.

Automatic variables can be assigned initial values by including appropriate expressions within the variable declarations, as in the above example, or by explicit assignment expressions elsewhere in the function. Such values will be reassigned each time the function is reentered. If an automatic variable is not initialized in some manner, however, its initial value will be *unpredictable*, and probably *unintelligible* (i.e., *garbage* value).

An automatic variable does not retain its value once control is transferred out of its defining function.

```
//sum of digits of a number using recursion

#include<stdio.h>

int sum_digits(int n)
{
    if (n == 0)
        return 0;
    else
        return((n % 10) + sum_digits(n / 10));
}
```

```

}

void main()
{
    int n, s;
    printf("Enter a number: ");
    scanf(" %d", &n);
    s = sum_digits(n);
    printf("Sum of digits: %d", s);
}

```

OUTPUT

```

Enter a number: 2650
Sum of digits: 13

```

11.b. How does a function return multiple values? Explain with suitable example.

Since external variables are recognized globally, they can be accessed from any function that falls within their scope. They retain their assigned values within this scope. Therefore, an external variable can be assigned a value within one function, and this value can be used (by accessing the external variable) within another function.

The use of external variables provides a convenient mechanism for **transferring information back and forth between functions**. We can transfer information into a function without using arguments. This is especially convenient when a function requires numerous input data items. Moreover, we now have a way to transfer multiple data items out of a function, since the *return* statement can return only one data item.

```

/* read several lines of text and determine the average number of
characters per line */
#include <stdio.h>

/* global variable */
int sum = 0;    /* total number of character */
int lines = 0; /*total number of lines */

int linecount(void);

main()
{
    int n;          /* no of chars in given line */
    float avg; /*average number of chars per line */
    printf("Enter the text below:\n");
    /* read a line of text and update the cumulative counters */
    while ((n = linecount()) > 0)
    {
        sum += n;
        ++lines;
    }
    avg = (float) sum / lines;
    printf("Average number of characters per line: %5.2f", avg);
}
/* read a line of text and count the number of characters */
int linecount(void)
{
    char line[80];

```

```

int count = 0;

while ((line[count] = getchar()) != '\n')
    ++count;
return count;
}

```

OUTPUT

Enter the text below:

Please Help us to Keep the
Book Clean and Moving

Average number of characters per line: **23.50**

When an array is passed to a function, however, the values of the array elements *are not* passed to the function. Rather, the array name is interpreted as the **address** of the first array element (i.e., the address of the memory location containing the first array element). This address is assigned to the corresponding formal argument when the function is called. The formal argument therefore becomes a **pointer** to the first array element (more about this in the next chapter). Arguments that are passed in this manner are said to be passed **by reference** rather than passed *by value*.

When a reference is made to an array element within the function, the value of the element's subscript is added to the value of the pointer to indicate the address of the specified array element. Therefore, any array element can be accessed from within the function. Moreover, *if an array element is altered within the function, the alteration will be recognized in the calling portion of the program* (actually, throughout the entire scope of the array).

Here is a simple C program that passes a three-element integer array to a function, where the array elements are altered. The values of the array elements are displayed at three different places in the program, thus illustrating the effects of the alterations.

```

#include <stdio.h>
void modify(int a[]); /* function prototype */

main()
{
    int count, a[3];

    printf("\nEnter any three numbers:\n");
    for (count = 0; count < 3; ++count)
    {
        printf("\na[%d] = ", count);
        scanf(" %d", &a[count]);
    }

    printf("From main(), before modification:\n");
    for (count = 0; count < 3; ++count)
    {
        printf("\na[%d] = %d", count, a[count]);
    }

    modify(a); /* function call */

    printf("From main(), after modification:\n");
    for (count = 0; count < 3; ++count)

```

```

    {
        printf("\na[%d] = %d", count, a[count]);
    }
}

void modify(int a[]) /* function definition */
{
    int count;

    printf("\nFrom the function, after modification the values:\n");
    for (count = 0; count < 3; ++count)
    {
        a[count] = -9;
        printf("a[%d] = %d\n", count, a[count]);
    }
    //return;
}

```

The array elements are read from the user within the first loop appearing in *main*. These values are displayed as soon as they are assigned. The array is then passed to the function ***modify***, when each array element is assigned the value **-9**. These new values are then displayed from within the function. Finally, the values of array elements are again displayed from ***main()***, after control has been transferred back to *main* from *modify*. When the program is executed, the following output is generated.

Enter any three numbers:

```

a[0] = 1
a[1] = 2
a[2] = 3

```

From *main()*, before modification:

```

a[0] = 1
a[1] = 2
a[2] = 3

```

From the function, after modification the values:

```

a[0] = -9
a[1] = -9
a[2] = -9

```

From *main()*, after modification:

```

a[0] = -9
a[1] = -9
a[2] = -9

```

These results show that the elements of ***a*** are altered within *main* as a result of the changes that were made within *modify*.

12.a. Create structure to keep record about student having members roll numbers, name, GPA and date of birth. Write a program to insert information about 48 students and display the name of students whose date of month is April.

```

#include <stdio.h>
#define N 48

```



```

typedef struct
{
    int mm;
    int dd;
    int yy;
} date;

typedef struct
{
    int roll;
    char name[30];
    float gpa;
    date dob;
} student;

main()
{
    int i;
    student s[N];
    printf("Enter Information about\n");
    for (i = 0; i < N; ++i)
    {
        printf("Student no. %d\n", i + 1);
        printf("Roll Number: ");
        scanf(" %d", &s[i].roll);
        printf("Name: ");
        scanf(" %[^\n]", s[i].name);
        printf("GPA: ");
        scanf(" %f", &s[i].gpa);
        printf("Date of Birth (mm-dd-yy): ");
        scanf(" %d-%d-%d", &s[i].dob.mm, &s[i].dob.dd,
&s[i].dob.yy);
    }

    printf("\nDetails about all students\n");
    printf("SN\tRoll#\tName\tMarks\tDOB (mm-dd-yy)");
    for (i = 0; i < N; ++i)
    {
        if (s[i].dob.mm == 4) //April = 4
            printf("\n%2d %2d %s %.2f %d-%d-%d", i + 1,
s[i].roll, s[i].name, s[i].gpa, s[i].dob.mm, s[i].dob.dd,
s[i].dob.yy);
    }
}

```

12.b. Consider a structure Book having attributes ISBN, Title, Price and Author name. Create a file named "book.txt" to store records of 500 books. Also display the record of books from file and count the books which are written by Gottfried.

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define N 500

```

```

typedef struct
{
    int ISBN;
    char title[60];
    float price;
    char author_name[30];
} book;

main()
{
    int i, count = 0;
    FILE *fptr;
    book b;
    if ((fptr = fopen("book.txt", "w+")) == NULL)
    {
        printf("File cannot be created.");
        exit(1);
    }
    for (i = 0; i < N; ++i)
    {
        printf("\nBook no. %d\n", i + 1);
        printf("ISBN: ");
        scanf(" %d", &b.ISBN);
        printf("Author: ");
        scanf(" %[^\\n]", b.author_name);
        if (strcmp(b.author_name, "Gottfried") == 0)
            ++count;
        printf("Title: ");
        scanf(" %[^\\n]", b.title);
        printf("Price: ");
        scanf(" %f", &b.price);
        fwrite(&b, sizeof(book), 1, fptr);
    }
    rewind(fptr);
    printf("\nISBN\\tAuthor\\tTitle\\tPrice");
    for (i = 0; i < N; ++i)
    {
        fread(&b, sizeof(book), 1, fptr);
        printf("\n%d\\t%s\\t%s\\t%.2f", b.ISBN, b.author_name,
b.title, b.price);
    }
    printf("\nTotal books written by Gottfried: %d", count);
}

```

13.a. Macros

The **#define** statement can be used for more, however, than simply defining symbolic constants. It can be used to define *macros*; i.e., single identifiers that are equivalent to expressions, complete statements or groups of statements. Macros resemble functions in this

sense. They are defined in an altogether different manner than functions, however, and they are treated differently during the compilation process.

Consider the simple C program shown below.

```
#include <stdio.h>

#define AREA length * width

main()
{
    float length, width;

    printf("Length = ");
    scanf(" %f", &length);
    printf("\nWidth = ");
    scanf(" %f", &width);

    printf("\narea = %.2f", AREA);
}
```

This program contains the macro **AREA**, which represents the expression `length * width`. When the program is compiled, the expression `length * width` will replace the identifier **AREA** within the `printf` statement, so that the `printf` statement will become

```
printf("\narea = %.2f", length * width);
```

When the program is executed, the values for `length` and `width` are entered interactively from the keyboard, and the corresponding value for `area` is displayed. A typical interactive session is shown below. The user's responses are underlined.

```
Length = 4
Width = 3
area = 12.00
```

Macro definitions are customarily placed at the beginning of a file, ahead of the first function definition. The scope of a macro definition extends from its point of definition to the end of the file. Multiline macros can be defined by placing a *backward slash* (`\`) at the end of each line except the last. This feature permits a single macro (i.e., a single identifier) to represent a compound statement. Here is another simple C program that contains a macro.

```
#include <stdio.h>
#define LOOP for (lines = 1; lines <= n; ++lines)          \
    {                                                       \
        for(count = 1; count <= lines; ++count) \
        {                                                       \
            printf("%d ", count);
        }                                                       \
        printf("\n");
    }

main()
{
    int count, lines, n;
    printf("number of lines ");
```

```
scanf(" %d", &n);
```

```
LOOP
```

```
}
```

This program contains a multiline macro, which represents a compound statement. The compound statement consists of several embedded for loops. Notice the backward slash (\) at the end of each line, except the last.

Many commercial C compilers offer certain library functions both as macros and as true functions. The macros are defined in the various header files. You may then choose which form is most appropriate for each particular application. Keep in mind, however, that there are certain disadvantages associated with the use of macros in place of functions, aside from the potentially significant increase in program length. In particular:

- When passing arguments to a macro, the number of arguments will be checked, but their data types will not. Thus, there is less error checking than with a function call.
- A macro identifier is not associated with an address, so that a macro cannot be utilized as a pointer. Thus, a macro identifier cannot be passed to a function as an argument, in the same sense that a function can be passed to another function as an argument. Moreover, a macro cannot call itself recursively.
- There are possible undesirable side effects associated with the use of macros, particularly when calling arguments are involved.

Consider the macro definition

```
#define ROOT(a, b) sqrt(a * a + b * b)
```

Now suppose that this macro is utilized within a program in the following manner.

```
ROOT(a+1, b+2)
```

The intent is to evaluate the formula

```
sqrt((a + 1) * (a + 1) + (b + 2) * (b + 2))
```

However, each appearance of a is replaced by the expression a + 1 (without parentheses), and each appearance of b is replaced by b + 1. Therefore, the result of the macro substitution will be

```
sqrt(2 * a + 1 + b + 2 * b + 2)
```

this expression is equivalent to

```
sqrt(2 * a + 3 * b + 3)
```

which is clearly incorrect. The source of error can be corrected, however, by placing additional parentheses within the macro definition; i.e.,

```
#define ROOT(a, b) sqrt((a) * (a) + (b) * (b))
```

A more subtle error occurs if we write

```
ROOT(a++, b++)
```

The macro substitution results in the expression

```
sqrt(a * (a + 1) + b * (b + 1))
```

rather than

```
sqrt(a * a + b * b)
```

as intended. This is an example of an undesired side effect. The placement of additional parentheses within the macro definition will not correct this problem.

b. File opening modes

File Opening Modes

<i>File-Type</i>	<i>Meaning</i>
“r”	Open an existing file for reading only.
“w”	Open a new file for writing only. If a file with the specified <i>file_name</i> currently exists, it will be destroyed and a <i>new_file</i> created in its place.
“a”	Open an existing file for appending. A new file will be created if the file with the specified <i>file_name</i> does not exist.
“r+”	Open an existing file for both reading and writing.
“w+”	Open a new file for both writing and reading. If a file with the specified <i>file_name</i> currently exists, it will be destroyed, and a new file created in its place.
“a+”	Open an existing file for both reading and appending. A new file will be created if the file with the specified <i>file_name</i> does not exist.

c. Union vs structure

Structure	Union
Each member of structure is assigned its own <i>unique storage</i> . It takes more memory spaces than that of union.	All members of a union share the <i>same storage</i> area of computer memory. It takes less memory than that of structure.
The amount of memory required to store a structure is the sum of the memory sizes required by <i>all</i> its members.	The amount of memory required to store a union is same as memory size occupied by a member which requires <i>largest</i> memory space among the members.
<i>All</i> the members of structure can be accessed at any point of time.	<i>Only one</i> member of union can be accessed at any given time.
Structure is suitable when you have <i>enough memory spaces</i> . It is very efficient to use and handle.	Union is used when you have <i>lack of memory</i> . It is easy but not realistic.
Structure declaration: struct student { char name[20]; //20 int roll; //2 float marks; //4 char address[50]; //50	Union declaration: union student { char name[20]; int roll; float marks; char address[50];

<code>} st;</code> Total size of struct student: 76B	<code>} st;</code> Total size of union student: 50B
--	---

k. *Jumping Statements*

The Break Statement

If a `break` statement is included in a *while*, *do - while* or *for* loop, then control will immediately be transferred out of the loop when the `break` statement is encountered. This provides a convenient way to terminate the loop if an error or other irregular condition is detected.

Here are some illustrations of loops that contain ***break*** statements. In each situation, the loop will continue to execute as long as the current value for the floating-point variable `x` does not exceed **100**. However, the computation will break out of the loop if a negative value for `x` is detected.

First, consider a ***while*** loop.

```
scanf(" %f", &x);
while (x <= 100)
{
    if (x < 0)
    {
        printf("ERROR! NEGATIVE VALUE FOR x");
        break;
    }
    /* process the non-negative value of x */
    ...
    scanf(" %f", &x);
}
```

The continue Statement

The `continue` statement is used to *bypass* the remainder of the current pass through a loop. The loop does not terminate when a `continue` statement is encountered. Rather, the remaining loop statements are skipped and the computation proceeds directly to the next pass through the loop. (Note the distinction between `continue` and `break`.)

The *continue* statement can be included within a *while*, a *do - while* or a *for* statement. It is written simply as

`continue;`

without any embedded statements or expressions.

Here are some illustrations of loops that contain `continue` statements. First, consider a *do - while* loop.

```
do
{
    scanf(" %f", &x);
    if (x < 0)
    {
        printf("ERROR! NEGATIVE VALUE FOR x");
        continue;
    }
}
```

```

    /* process the non-negative value of x */
    ...
}while (x <= 100);

```

The goto Statement

The *goto* statement is used to alter the normal sequence of program execution by transferring control to some other part of the program. In its general form, the goto statement is written as

goto label;

where *label* is an identifier that is used to label the target statement to which control will be transferred. Control may be transferred to any other statement within the program. (To be more precise, control may be transferred anywhere within the current *function*. The target statement must be labeled, and the label must be followed by a colon. Thus, the target statement will appear as

label: statement

Each labeled statement within the program (more precisely, within the current function) must have a unique label; i.e., no two statements can have the same label.

The following skeletal outline illustrates how the goto statement can be used to transfer control out of a loop if an unexpected condition arises.

```

/* main loop */
scanf(" %f", &x);
while (x <= 100)
{
    ...
    if (x < 0) goto errorcheck;
    ...
    scanf(" %f", &x);
}
...
/* error detection routine */
errorcheck:
{
    printf("ERROR! NEGATIVE VALUE FOR x");
    ...
}

```

In this example control is transferred out of the while loop, to the compound statement whose label is **errorcheck**, if a negative value is detected for the input variable *x*.

2019 Fall

1.a. What is programming language? Explain low-level and high-level programming language.

Programming language is defined as a set of rules that provides a platform for instructing computer to perform some specific tasks. Programming languages are classified into various ways. Based on nature of language, they are classified into *general purpose* and *specific*

purpose language. On the other hand, programming language are classified into *High-level language* and *Low-level language*.

- vii. Low Level Language (LLL)
 - g. Machine Language
 - h. Assembly Language
- viii. High Level Language (HLL)
 - j. Third Generation Language (3GL)
 - k. Fourth Generation Language (4GL)
 - l. Fifth Generation Language (5GL)

iii) a. Machine Language

Machine language is the language of CPU which consists of set of instructions composed with series of 0s and 1s.

Advantages:

- Machine language does not require translators.
- Execution time of machine language program is extremely fast.

Disadvantages:

- Programming is very complex and tedious job for the programmers.
- They are machine dependent; so, they support only specific hardware.
- It is very difficult to find bugs and hard to debug.

b. Assembly Language

Assembly language is special type of low-level language which consists set of alphanumeric instructions, called *mnemonics*. E.g. ADD A, B

Advantages:

- It is easier to understand than machine language. Also, it consumes less amount of memory than high-level language. Thus, ALL executes much faster than HLL.
- It is efficient for solving hardware interfacing problems. E.g. operating system kernel, programming language, utility software, virus, etc.
- Error finding and debugging process is easier than HLL.

Disadvantages

- ALL is hardware dependent language.
- Programmer should have depth knowledge about the computer hardware during programming.
- Program execution is slower than MLL and the program is lengthy and complex than HLL.

Third Generation Language (3GL)

3GL is general-purpose high-level language in which the instructions are expressed in more like English structure and mathematical expressions. E.g. FORTRAN, BASIC, Pascal, C etc. are examples of 3GL.

Advantages

- Machine independent language.

- Easy for program development because the program instructions are English like structure and Mathematical expressions.
- Easy for catching bugs and easy to debug and modify.
- Programmer does not require depth knowledge about computer architecture while programming.

Disadvantages

- 3GL requires translator. So, program execution is slower than LLL.
- They are general-purpose language. They are not suitable for solving complex and specific type of real-world problems.
- They are mostly CUI based language. So, they are less user-friendly.
- Less efficient than 4GL and 5GL.

Fourth Generation Language (4GL)

4GL is a specific-purpose high-level language in which the instructions are expressed by using graphical components and English like structures and Mathematical expressions. E.g.

- *Database and Query Processing Language* (DBase, SyBase, DB2, MySQL, SQL, MS Access, Oracle etc.)
- *Application Generator Language* (Java, C#, Visual C++, Visual.NET etc.)
- *Web Based Scripting Language*: (HTML, XML, JavaScript, VBScript, ASP, PHP, JSP etc.)

Advantages:

- It is efficient and more user friendly.
- It supports rapid application development. Programmers can design, create and debug an application in very short period.
- IDE (Integrated Development Environment) helps developer to include powerful features of different programming languages into one application.
- They are platform independent.

Disadvantages:

- They require more memory and disk storage, hence, take more execution time.
- It is not suitable for solving hardware interfacing problems.

Fifth Generation Language (5GL)

5GL is a special type of future generation language in which the statements will be expressed in more like human language. E.g. PROLOG, LISP etc. are considered as 5GL. Some characteristics of 5GL are:

- Natural languages make human and machine closer than before.
- They are designed to allow the computer more natural and intelligent than before.

They are used in the areas of research, robotics, defense system, and different types of expert systems.

1. b. What is the role of algorithm in programming? Write down a program to check if the given number is prime or not.

An algorithm is a well-defined step by step solution or a series of instructions to solve a problem. A computer algorithm is a computational procedure that takes in a set of finite input

and transforms it into output by applying some math & logic. An algorithm in programming will have several steps as follows –

Problem definition – What is to be done?

Data collection – What do we have to solve the problem? Or inputs.

Data processing – Understanding what we have or transforming them into a usable form.

Logical approach – Employing the collected & created data against logic to solve.

Solution – Present the solution in the way you want in a GUI or a terminal or a diagram or a chart.

An algorithm helps us decide if a problem is solvable or not. If yes, then how, how fast and how accurate? If not, then an algorithm again helps us decide, if we can solve a part of it. Talking about why we should use algorithms in programming, we must understand that computer programs adopt different algorithms run on computer hardware which has a processor & memory and these components have limitations. A processor is not infinitely fast and the memory we have is not free. They are bounded resources. They must be used wisely and a good algorithm that is efficient in terms of time complexities and space complexities will help you do so.

```
#include <stdio.h>

int prime(int);

void main()
{
    int n;
    printf("Enter a number to be checked: ");
    scanf(" %d", &n);
    if (prime(n) == 1)
        printf("%d is Prime.", n);
    else
        printf("%d is not Prime.", n);
}

int prime(int n)
{
    int i;
    for (i = 2; i < n; ++i)
    {
        if (n % i == 0)
            break;
    }
    if(i == n)
        return 1;
    else
        return 0;
}
```

OUTPUT

RUN 1

Enter a number to be checked: 15

15 is not Prime.

RUN 2

Enter a number to be checked: 11

11 is Prime.

2.a. Which of the following are invalid variables and why?

Minimum First.name Row Total &name
Doubles 3rd_row Column-total float

Describe the four basic data types along their size and range.

Invalid variable	Reason
First.name	Special character dot (.) is not allowed.
Row Total	White space is not allowed.
&name	Special character ampersand (&) is not allowed.
3 rd _row	Starting with digit (3) is not allowed.
Column-total	Special character (-) is not allowed.
float	Keyword is not allowed.

Data Type	Category	Range	Memory (Bytes)	Format Specifier
integer	short signed	-2^{15} to $2^{15} - 1$	2	%d or %i
	short unsigned	0 to $2^{16} - 1$	2	%u
	signed	-2^{15} to $2^{15} - 1$	2	%d
	unsigned	0 to $2^{16} - 1$	2	%u
	long signed	-2^{31} to $2^{31} - 1$	4	%ld
	long unsigned	0 to $2^{32} - 1$	4	%lu
	hexadecimal			%x
	Octal			%o
float	float	1.2×10^{-38} to $3.4 \times 10^{+38}$	4	%f
double	double	1.7×10^{-308} to $3.4 \times 10^{+308}$	8	%lf
	long double	3.4×10^{-4932} to $1.1 \times 10^{+4932}$	10	%Lf
char	signed char	-128 to +127	1	%c
	unsigned char	0 to 255	1	%c

2. b. What is entry-controlled and exit-controlled loop? Write a program to print the following pattern.

```

1 2 3 4 5
1 2 3 4
1 2 3
1 2
1

```

Entry Controlled Loop

While loop and for loop are *entry-controlled (pre-test)* loops, i.e. test condition is evaluated first and body of loop is executed only if the test is true. The body of loop may not be executed at all if the condition is not satisfied at the very first attempt.

Exit Controlled Loop

Do-while loop is *exit-controlled (post-test)* loop, i.e. the body of loop is executed first without checking any condition and at the end of loop body, the condition is evaluated for repetition of next time. The body of the loop is executed at least once even the tested condition is false.

```
#include <stdio.h>

main()
{
    int i, j, k;
    for (i = 5; i >= 1; --i)
    {
        k = 1;
        for (j = 1; j <= i; ++j)
        {
            printf("%d\t", k);
            ++k;
        }
        printf("\n");
    }
}
```

3.a. Write a program to print the Fibonacci numbers up to N terms.

```
#include<stdio.h>

int fibo(int);
void main()
{
    int i, f, n;

    printf("How many terms do you need? ");
    scanf(" %d", &n);
    for (i = 1; i <= n; i++)
    {
        f = fibo(i);
        printf("%d ", f);
    }
}

int fibo(int n)
{
    if (n == 1)
        return 0;
    else if (n == 2)
        return 1;
    else
        return(fibo(n - 1) + fibo(n - 2));
}
```

OUTPUT

How many terms do you need? 10
0 1 1 2 3 5 8 13 21 34

3. b. What is an Array? Write a program to enter values in 3x3 matrix and compute the sum of odd elements.

Many applications require the processing of multiple data items that have common characteristics (e.g., a set of numerical data, represented by $x_1, x_2, \dots, x_n$). In such situations

it is often convenient to place the data items into an **array**, where they will all share the same name (e.g., **x**). The individual data items can be characters, integers, floating-point numbers, etc. However, they must all be of the same type and the same storage class. Each array element (i.e., each individual data item) is referred to by specifying the array name followed by one or more *subscripts*, with each subscript enclosed in square brackets. Each subscript must be expressed as a *nonnegative integer*. In an n-element array, the array elements are **x[0]**, **x[1]**, **x[2]**, ..., **x[n - 1]**. The value of each subscript can be expressed as an *integer constant*, an *integer variable* or a more *complex integer expression*.

```
#include<stdio.h>

#define M 3
#define N 3

main()
{
    int mat[10][10], i, j, sum = 0;
    printf("Enter elements of matrix %dx%d\n", M, N);
    for (i = 0; i < M; i++)
    {
        for (j = 0; j < N; j++)
        {
            scanf(" %d", &mat[i][j]);
        }
    }

    /* adding odd elements only */
    for (i = 0; i < M; i++)
    {
        for (j = 0; j < N; j++)
        {
            if (mat[i][j] % 2 != 0)
                sum += mat[i][j];
        }
    }

    printf("\nSum of elements: %d", sum);
}
```

OUTPUT

Enter elements of matrix 3x3

```
5 10 15
20 25 30
35 40 45
```

Sum of elements: 125

4. a. Explain any seven functions that are related to strings.

nts the null character (\0) which signifies the end of the string (letter).

K	a	t	h	m	a	n	d	u	\0
0	1	2	3	4	5	6	7	8	9

String Handling Functions

Here are some string handling functions which are used to manipulate strings on C. These functions are defined at header file ***string.h***.

i. **strlen()**

It returns the length of string parameter. Syntax:

```
len = strlen(str);
```

Here *str* is string variable or string constant and *len* is an integer variable which holds the length returned by *strlen()*. E.g.:

```
len = strlen("Nepal");
```

The *len* holds 5 which is length of string constant “Nepal”.

ii. **strlwr()**

It converts each uppercase character into lowercase character presented in given string. Syntax:

```
strlwr(str);
```

Here *str* is string variable or string constant. The new string (after conversion) is stored at the same variable. E.g.:

```
strlwr("Nepal");
```

It returns string “nepal”.

```
char str[] = "Nepal";
```

```
strlwr(str);
```

Then new string at *str* is “nepal”.

iii. **strupr()**

It converts each lowercase character into uppercase character presented in given string. Syntax:

```
strupr(str);
```

Here *str* is string variable or string constant. The new string (after conversion) is stored at the same variable. E.g.:

```
strupr("Nepal");
```

It returns string “NEPAL”.

```
char str[] = "Nepal";
```

```
strupr(str);
```

Then new string at *str* is “NEPAL”.

iv. **strrev()**

It reverses the contents of string (except \0) supplied. The new string is stored at the same variable.

Syntax:

```
strrev(str);
```

where *str* is string variable or string constant. For example,

```
char str[] = "Nepal";
```

```
strrev(str);
```

the *str* now has string “lapeN”.

v. **strcpy()**

Syntax:

```
strcpy(dest_string, src_string);
```

It copies contents of source string (`src_string`) to destination string (`dest_string`). After copying, the `dest_string` has the same value of `src_string`. E.g.:

```
char text[20] = "Nepal";
char str[20];
strcpy(str, text);
```

Now, the `str` and `text` has the same content i.e. *"Nepal"*.

vi. **strcat()**

Syntax:

```
strcat(str1, str2);
```

It concatenates `str2` to end of `str1`. The new string is stored at `str1`. For e.g.,

```
char fname[20] = "My";
char lname[10] = "Nepal";
strcat(fname, lname);
```

The new string *"MyNepal"* is stored at `fname`.

vii. **strcmp()**

Syntax:

```
strcmp(str1, str2);
```

it compares two strings, `str1` and `str2`, character by character then returns

- *positive* value if `str1` is greater than `str2`
- *negative* value if `str1` is less than `str2`
- *zero* if `str1` and `str2` both are identical.

For example:

```
strcmp("ABC", "abc");
```

It returns negative value since ASCII value of 'A' is less than 'a'.

```
strcmp("abc", "ABC");
```

It returns positive value since ASCII value of 'a' is greater than 'A'.

```
strcmp("xyz", "xyz");
```

It returns zero since both strings are same.

4. b. How can we return multiple values from a function? Explain with an appropriate example.

Since external variables are recognized globally, they can be accessed from any function that falls within their scope. They retain their assigned values within this scope. Therefore, an external variable can be assigned a value within one function, and this value can be used (by accessing the external variable) within another function.

The use of external variables provides a convenient mechanism for **transferring information back and forth between functions**. We can transfer information into a function without using arguments. This is especially convenient when a function requires numerous input data items. Moreover, we now have a way to transfer multiple data items out of a function, since the *return* statement can return only one data item.

```
#include <stdio.h>
```

```
int a = 10;
```

```
int b = 20;
```

```

void change();
main()
{
    printf("Before change\na = %d\nb = %d", a, b);
    change();
    printf("\nAfter change\na = %d\nb = %d", a, b);
}

```

```

void change()
{
    a = 2 * a;
    b = 2 * b;
}

```

OUTPUT

Before change

a = 10

b = 20

After change

a = 20

b = 40

When an array is passed to a function, however, the values of the array elements *are not* passed to the function. Rather, the array name is interpreted as the **address** of the first array element (i.e., the address of the memory location containing the first array element). This address is assigned to the corresponding formal argument when the function is called. The formal argument therefore becomes a **pointer** to the first array element (more about this in the next chapter). Arguments that are passed in this manner are said to be passed **by reference** rather than passed *by value*.

When a reference is made to an array element within the function, the value of the element's subscript is added to the value of the pointer to indicate the address of the specified array element. Therefore, any array element can be accessed from within the function. Moreover, *if an array element is altered within the function, the alteration will be recognized in the calling portion of the program* (actually, throughout the entire scope of the array).

Here is a simple C program that passes a three-element integer array to a function, where the array elements are altered. The values of the array elements are displayed at three different places in the program, thus illustrating the effects of the alterations.

```

#include <stdio.h>
void modify(int a[]); /* function prototype */

main()
{
    int count, a[3];

    printf("\nEnter any three numbers:\n");
    for (count = 0; count < 3; ++count)
    {
        printf("\na[%d] = ", count);
        scanf(" %d", &a[count]);
    }
}

```



```

printf("From main(), before modification:\n");
for (count = 0; count < 3; ++count)
{
    printf("\na[%d] = %d", count, a[count]);
}

modify(a); /* function call */

printf("From main(), after modification:\n");
for (count = 0; count < 3; ++count)
{
    printf("\na[%d] = %d", count, a[count]);
}

void modify(int a[]) /* function definition */
{
    int count;

    printf("\nFrom the function, after modification the values:\n");
    for (count = 0; count < 3; ++count)
    {
        a[count] = -9;
        printf("a[%d] = %d\n", count, a[count]);
    }
    //return;
}

```

The array elements are read from the user within the first loop appearing in *main*. These values are displayed as soon as they are assigned. The array is then passed to the function **modify**, when each array element is assigned the value **-9**. These new values are then displayed from within the function. Finally, the values of array elements are again displayed from **main()**, after control has been transferred back to *main* from *modify*. When the program is executed, the following output is generated.

Enter any three numbers:

```

a[0] = 1
a[1] = 2
a[2] = 3

```

From main(), before modification:

```

a[0] = 1
a[1] = 2
a[2] = 3

```

From the function, after modification the values:

```

a[0] = -9
a[1] = -9
a[2] = -9

```

From main(), after modification:

```

a[0] = -9
a[1] = -9

```

```
a[2] = -9
```

These results show that the elements of **a** are altered within *main* as a result of the changes that were made within *modify*.

5.a. What is function prototype? Write a program to pass one dimensional array to a function and display that array in that called function.

when these programs are compiled, the programmer-defined function will have been defined before the first function access. However, many programmers prefer a "**top-down**" approach, in which *main* appears ahead of the programmer-defined function definition. In such situations the function access (within *main*) will precede the function definition. This can be confusing to the compiler, unless the compiler is first alerted to the fact that the function being accessed will be defined later in the program. A *function prototype* is used for this purpose.

Function prototypes are usually written at the beginning of a program, ahead of any programmer-defined functions (including *main*). The general form of a function prototype is

```
data-type name(type 1 arg 1, type 2 arg 2..., type n arg n);
```

where *data-type* represents the data type of the item that is returned by the function, *name* represents the function name, and *type 1*, *type 2*, . . . , *type n* represent the data types of the arguments *arg 1*, *arg 2*, ..., *arg n* respectively. Notice that a function prototype resembles the first line of a function definition (though a function prototype ends with a semicolon).

The names of the arguments within the function prototype need not be declared elsewhere in the program, since these are "**dummy**" argument names that are recognized only within the prototype. In fact, the argument names can be omitted (though it is not a good idea to do so); however, the argument *data types* are essential. For example,

```
long int factorial(int n);
```

5. b. What are function call by value and call by reference? Explain with example.

Call by Value

When a single value is passed to a function via an actual argument, the value of the actual argument is *copied* into the function. Therefore, *the value of the corresponding formal argument can be altered within the Function, but the value of the actual argument within the calling routine will not change*. This procedure for passing the value of an argument to a function is known as *passing by value*.

Here is a simple C program containing a function that alters the value of its argument.

```
#include <stdio.h>
```

```
void modify(int a);
```

```
main()
```

```
{
```

```
    int a = 2;
```

```
    printf("\na = %d (from main, before calling the  
function)", a);
```

```
    modify(a); // function call a is a normal variable
```

```
    printf("\na = %d (from main, after calling the  
function)", a);
```

```
}
```

```

void modify(int a)
{
    a *= 3;
    printf("\na = %d (from the function, after being
modified)", a);
    return;
}

```

The original value of *a* (i.e., *a* = 2) is displayed when main begins execution. This value is then passed to the function *modify*, where it is multiplied by 3 and the new value displayed. Note that it is the altered value of the formal argument that is displayed within the function. Finally, the value of *a* within main (i.e., the actual argument) is again displayed, after control is transferred back to main from *modify*. When the program is executed, the following output is generated.

```

a = 2 (from main, before calling the function)
a = 6 (from the function, after being modified)
a = 2 (from main, after calling the function)

```

These results show that *a* is not altered within main, even though the corresponding value of *a* is changed within *modify()*.

Passing an argument by value has advantages and disadvantages. On the plus side, it allows a single-valued actual argument to be written as an expression rather than being restricted to a single variable. Moreover, if the actual argument is expressed simply as a single variable, it protects the value of this variable from alterations within the function. On the other hand, it does not allow information to be transferred back to the calling portion of the program via arguments. Thus, *passing by value is restricted to a one-way transfer of information*.

Call by Reference

To pass an array to a function, the array name must appear by itself, without brackets or subscripts, as an actual argument within the function call. The corresponding formal argument is written in the same manner, though it must be declared as an array within the formal argument declarations. When declaring a one-dimensional array as a formal argument, the array name is written with a pair of empty square brackets. The size of the array is *not specified* within the formal argument declaration.

When an array is passed to a function, however, the values of the array elements *are not* passed to the function. Rather, the array name is interpreted as the **address** of the first array element (i.e., the address of the memory location containing the first array element). This address is assigned to the corresponding formal argument when the function is called. The formal argument therefore becomes a **pointer** to the first array element (more about this in the next chapter). Arguments that are passed in this manner are said to be passed **by reference** rather than passed *by value*.

When a reference is made to an array element within the function, the value of the element's subscript is added to the value of the pointer to indicate the address of the specified array element. Therefore, any array element can be accessed from within the function. Moreover, *if an array element is altered within the function, the alteration will be recognized in the calling portion of the program* (actually, throughout the entire scope of the array).

Here is a simple C program that passes a three-element integer array to a function, where the array elements are altered. The values of the array elements are displayed at three different places in the program, thus illustrating the effects of the alterations.

```

#include <stdio.h>
void modify(int a[]); /* function prototype */

main()
{
    int count, a[3];

    printf("\nEnter any three numbers:\n");
    for (count = 0; count < 3; ++count)
    {
        printf("\na[%d] = ", count);
        scanf(" %d", &a[count]);
    }

    printf("From main(), before modification:\n");
    for (count = 0; count < 3; ++count)
    {
        printf("\na[%d] = %d", count, a[count]);
    }

    modify(a); /* function call a is an array*/

    printf("From main(), after modification:\n");
    for (count = 0; count < 3; ++count)
    {
        printf("\na[%d] = %d", count, a[count]);
    }
}

void modify(int a[]) /* function definition */
{
    int count;

    printf("\nFrom the function, after modification the
values:\n");
    for (count = 0; count < 3; ++count)
    {
        a[count] = -9;
        printf("a[%d] = %d\n", count, a[count]);
    }
    //return;
}

```

The array elements are read from the user within the first loop appearing in *main*. These values are displayed as soon as they are assigned. The array is then passed to the function **modify**, when each array element is assigned the value **-9**. These new values are then displayed from within the function. Finally, the values of array elements are again displayed from **main()**, after control has been transferred back to *main* from *modify*. When the program is executed, the following output is generated.

```

Enter any three numbers:
a[0] = 1
a[1] = 2
a[2] = 3
From main(), before modification:
a[0] = 1
a[1] = 2
a[2] = 3
From the function, after modification the values:
a[0] = -9
a[1] = -9
a[2] = -9
From main(), after modification:
a[0] = -9
a[1] = -9
a[2] = -9

```

These results show that the elements of *a* are altered within *main* as a result of the changes that were made within *modify*.

6. a. What is a pointer variable? How can memory of a variable be initialized dynamically? Explain with example.

A *pointer* is a variable that represents the *location* (rather than the *value*) of a data item, such as a variable or an array element. Pointers are used frequently in C, as they have several useful applications. For example, pointers can be used to pass information *back and forth* between a function and its reference point. Pointers provide a way to return multiple data items from a function via function arguments. Pointers also permit references to other functions to be specified as arguments to a given function. This has the effect of passing functions as arguments to the given function.

Suppose *v* is a variable that represents some data item. The compiler will automatically assign memory cells for this data item. The data item can then be accessed if we know the location (i.e., the *address*) of the first memory cell. The address of *v*'s memory location can be determined by the expression *&v*, where *&* is a unary operator, called the *address operator*, that evaluates the address of its operand. Now let us assign the address of *v* to another variable, *pv*. Thus,

```
pv = &v
```

This new variable is called a *pointer* to *v*, since it points to the location where *v* is stored in memory. Remember, however, that *pv* represents *v*'s *address*, not its value. Thus, *pv* is referred to as a *pointer variable*.

Dynamic Memory Allocation

Since an array name is a pointer to the first element within the array, it should be possible to define the array as a pointer variable rather than as a conventional array. Syntactically, the two definitions are equivalent. However, a conventional array definition results in a fixed block of memory being reserved at the beginning of program execution, whereas this does not occur if the array is represented in terms of a pointer variable. Therefore, the use of a pointer variable to represent an array requires some type of initial memory assignment before the array elements are processed. This is known as **dynamic memory allocation**. Generally, the **malloc** library function is used for this purpose, as illustrated in the next example.

Suppose **x** is a one-dimensional, 10-element array of integers. It is possible to define **x** as a pointer variable rather than an array. Thus, we can write

```
int *x;
rather than
```

```
int x[10];
```

However, **x** is not automatically assigned a memory block when it is defined as a pointer variable, though a block of memory large enough to store 10 integer quantities will be reserved when **x** is defined as an array.

To assign enough memory for **x**, we can make use of the library function **malloc**, as follows.

```
x = (int *) malloc(10 * sizeof(int));
```

This function reserves a block of memory whose size (in bytes) is equivalent to 10 integer quantities. As written, the function returns a pointer to an integer. This pointer indicates the beginning of the memory block. In general, the type cast preceding **malloc** must be consistent with the data type of the pointer variable. Thus, if **y** were defined as a pointer to a double-precision quantity and we wanted enough memory to store 10 double-precision quantities, we would write

```
y = (double *) malloc(10 * sizeof(double));
```

*If the declaration is to include the assignment of initial values, however, then **x** must be defined as an array rather than a pointer variable.* For example,

```
int x[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
```

or

```
int x[] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
```

When programming in C, it is not unusual to use pointer expressions rather than references to individual array elements. The resulting programs may appear strange at first, though they are straightforward once you become comfortable accessing values that are stored in specified addresses. Generally, a small amount of practice is all that is required.

Reordering a List of Numbers To illustrate the use of pointers with dynamic memory allocation, let us once again consider the problem of reordering a list of integers, as described in earlier Example. Now, however, we will utilize pointer expressions to access numerical values rather than refer explicitly to individual array elements. Here is the complete C program.

```
/* reordering a one-dimensional, integer array from
smallest to largest, using pointer notation */
```

```
#include <stdio.h>
#include <stdlib.h>
```

```
void reorder(int n, int *x);
```

```
main()
```

```
{
    int i, n, *x;

    /* read in a value for n */
    printf("\nHow many numbers will be entered? ");
```

```

scanf(" %d", &n);

/* dynamically allocate memory for n numbers */
x = (int *) malloc(n * sizeof(int));

/*read in the list of numbers */
for (i = 0; i < n; ++i)
{
    printf("i = %d x = ", i + 1);
    scanf(" %d", x + i);
}

/* reorder all array elements */
reorder(n, x);

/* display the reordered list of numbers */
printf("\nReordered list of numbers:\n");
for (i = 0; i < n; ++i)
    printf("i = %d x = %d\n", i+1, *(x + i));
free(x);
}

```

```

void reorder(int n, int *x)
{
    int i, item, temp;
    for (item = 0; item < n - 1; ++item)
        for (i = item + 1; i < n; ++i)
            if(*(x + i) < *(x + item))
            {
                temp = *(x + item);
                *(x + item) = *(x + i);
                *(x + i) = temp;
            }
    return;
}

```

OUTPUT

How many numbers will be entered? 10

i = 1	x = <u>20</u>
i = 2	x = <u>30</u>
i = 3	x = <u>10</u>
i = 4	x = <u>5</u>
i = 5	x = <u>-15</u>
i = 6	x = <u>15</u>
i = 7	x = <u>25</u>
i = 8	x = <u>35</u>
i = 9	x = <u>22</u>
i = 10	x = <u>40</u>

Reordered list of numbers:

```

i = 1    x = -15
i = 2    x = 5
i = 3    x = 10
i = 4    x = 15
i = 5    x = 20
i = 6    x = 22
i = 7    x = 25
i = 8    x = 30
i = 9    x = 35
i = 10   x = 40

```

6.b. Create a structure named 'Employee' with structure members Name, Eid, Address and Gender. The Structure need to read information of 50 employees. Write all contents 'info.dat' and while retrieving, display only the information of those employees whose address is 'Kathmandu'.

```

#include <stdio.h>
#include <string.h>
#include <stdlib.h>

#define N 50

typedef struct
{
    char Name[30];
    int Eid;
    char Address[80];
    char Gender; //M for male and F for female
} Employee;

main()
{
    FILE *fp;
    Employee e;
    int i;

    if ((fp = fopen("info.dat", "w+")) == NULL)
    {
        printf("File cannot be created.");
        exit(1);
    }

    printf("Enter details of employee\n");
    for (i = 0; i < N; ++i)
    {
        printf("\nStudent no. %d\n", i + 1);
        printf("Name: ");
        scanf(" %[^\\n]", e.Name);
        printf("ID: ");
        scanf(" %d", &e.Eid);
    }
}

```



```

        printf("Address: ");
        scanf(" %[^\\n]", e.Address);
        printf("Gender: ");
        scanf(" %c", &e.Gender);
        fwrite(&e, sizeof(Employee), 1, fp);
    }
    rewind(fp);

    printf("\\nStudents from Kathmandu\\n");
    for (i = 0; i < N; ++i)
    {
        fread(&e, sizeof(Employee), 1, fp);
        if((strcmp(e.Address, "Kathmandu")) == 0)
        {
            printf("\\nName: %s", e.Name);
            printf("\\nID: %d", e.Eid);
            printf("\\nAddress: %s", e.Address);
            printf("\\nGender: %c", e.Gender);
        }
    }
    fclose(fp);
}

```

7. Write short notes on:

a. Union in C

Unions, like structures, contain members whose individual data types may differ from one another. However, the members within a union *all share the same storage area* within the computer's memory, whereas each member within a structure is assigned its own unique storage area. Thus, unions are used to conserve memory. They are useful for applications involving multiple members, where values need not be assigned to all of the members at any one time.

Within a union, the bookkeeping required to store members whose data types are different (having different memory requirements) is handled automatically by the compiler.

However, the user must keep track of what type of information is stored at any given time. An attempt to access the wrong type of information will produce meaningless results. In general terms, the composition of a union may be defined as

```

union tag
{
    member 1;
    member 2;
    .....
    member m;
};

```

where **union** is a required keyword and the other terms have the same meaning as in a structure definition. Individual union variables can then be declared as

```

storage-class union tag variable 1, variable 2, ..., variable n;

```

where **storage-class** is an optional storage class specifier, union is a required keyword, **tag** is the name that appeared in the union definition, and **variable1, variable 2, ..., variable n** are union variables of type **tag**.

The two declarations may be combined, just as we did with structures. Thus, we can write

```
storage-class union tag
{
member 1;
member 2;
...
member m;
} variable 1, variable 2, ..., variable n;
```

The **tag** is optional in this type of declaration. A C program contains the following union declaration.

```
union id
{
char color[12];
int size;
} shirt, blouse;
```

Here we have two union variables, *shirt* and *blouse*, of type **id**. Each variable can represent either a 12-character string (**color**) or an integer quantity (**size**) at any one time.

The 12-character string will require more storage area within the computer's memory than the integer quantity. Therefore, a block of memory large enough for the 12-character string will be allocated to each union variable. The compiler will automatically distinguish between the 12-character array and the integer quantity within the given block of memory, as required.

b. Macros

the **#define** statement can be used to define symbolic constants within a C program. At the beginning of the compilation process, all symbolic constants are replaced by their equivalent text. Thus, symbolic constants provide a form of shorthand notation that can simplify the organization of a program.

The **#define** statement can be used for more, however, than simply defining symbolic constants. It can be used to define *macros*; i.e., single identifiers that are equivalent to expressions, complete statements or groups of statements. Macros resemble functions in this sense. They are defined in an altogether different manner than functions, however, and they are treated differently during the compilation process.

Consider the simple C program shown below.

```
#include <stdio.h>

#define AREA length * width

main()
{
    float length, width;

    printf("Length = ");
    scanf(" %f", &length);
    printf("\nWidth = ");
    scanf(" %f", &width);
```

```
printf("\narea = %.2f", AREA);
}
```

This program contains the macro **AREA**, which represents the expression *length * width*. When the program is compiled, the expression *length * width* will replace the identifier **AREA** within the printf statement, so that the printf statement will become

```
printf("\narea = %.2f", length * width);
```

When the program is executed, the values for length and width are entered interactively from the keyboard, and the corresponding value for area is displayed. A typical interactive session is shown below. The user's responses are underlined.

```
Length = 4
Width = 3
area = 12.00
```

Many commercial C compilers offer certain library functions both as macros and as true functions. The macros are defined in the various header files. You may then choose which form is most appropriate for each particular application. Keep in mind, however, that there are certain disadvantages associated with the use of macros in place of functions, aside from the potentially significant increase in program length. In particular:

- When passing arguments to a macro, the number of arguments will be checked, but their data types will not. Thus, there is *less error checking* than with a function call.
- A macro identifier is not associated with an address, so that a macro cannot be utilized as a pointer. Thus, a macro identifier cannot be passed to a function as an argument, in the same sense that a function can be passed to another function as an argument. Moreover, a macro cannot call itself recursively.
- There are possible undesirable side effects associated with the use of macros, particularly when calling arguments are involved.

Consider the macro definition

```
#define ROOT(a, b) sqrt(a * a + b * b)
```

Now suppose that this macro is utilized within a program in the following manner.

```
ROOT(a+1, b+2)
```

The intent is to evaluate the formula

```
sqrt((a + 1) * (a + 1) + (b + 2) * (b + 2))
```

However, each appearance of *a* is replaced by the expression *a + 1* (without parentheses), and each appearance of *b* is replaced by *b + 1*. Therefore, the result of the macro substitution will be

```
sqrt(2 * a + 1 + b + 2 * b + 2)
```

this expression is equivalent to

```
sqrt(2 * a + 3 * b + 3)
```

which is clearly incorrect. The source of error can be corrected, however, by placing additional parentheses within the macro definition; i.e.,

```
#define ROOT(a, b) sqrt((a) * (a) + (b) * (b))
```

c. *getc()* and *putc()*

The `getc()` and `putc()` functions are file handling function in C programming language which is used to read a character from a file and display on standard output or write into a file respectively. The *getc* and *putc* functions are analogous to *getchar* and *putchar* functions and handle one character at a time. The *putc* function writes the character contained in character variable *c* to the file associated with the pointer *fp*.

```
putc(c, fp)
```

Similarly *getc* function is used to read a character from a file that has been open in read mode.

```
c=getc(fp)
//use of getc() and putc()
#include <stdio.h>

main()
{
    FILE *fp;
    char c;

    printf("Data input to file\n");
    fp = fopen("Input.txt", "w+");
    while ((c = getchar()) != '\n') //input character until new
line
        putc(c, fp);    //write a character to input
    rewind(fp);

    printf("Data output from file\n");
    while ((c = getc(fp)) != EOF)
        printf("%c", c);

    fclose(fp);
}
```

OUTPUT

```
Data input to file
Solution of old question 2019 fall.
Data output from file
Solution of old question 2019 fall.
```

2019 Spring

1.a. What do you mean by instruction and software? Explain various types of software.

An instruction is an order given to a computer processor by a computer program. At the lowest level, each instruction is a sequence of 0s and 1s that describes a physical operation the computer is to perform (such as "Add") and, depending on the particular instruction type, the specification of special storage areas called registers that may contain data to be used in carrying out the instruction, or the location in computer memory of data.

Software is the collection of computer programs, procedures and documentation that performs different tasks on a computer system. The types of computer software are system software and application software.

1. System Software

System software is the computer software designed to operate the computer hardware and to provide a platform for running application software. System software is the collection of operating system, device drivers, servers, windowing systems and utilities. Some Common types of system software are given below:

iii. Operating system

The OS is the most important program that runs on a computer. Every general-purpose computer must have an operating system. Some functions of OS are:

- Recognize input from keyboard, send output to display screen.
- Keep track of files and directories on the disk.
- Controlling peripheral devices such as disk drives and printers.
- Provides interface between hardware and software.
- Provides security and backup, memory management, scheduling processes etc.

Some examples of OS are: Windows 2000, Mac-OS, UNIX, DOS etc.

iv. Utility Software

Utility software helps to manage, maintain and control computer resources. These programs are available to help you with the day-to-day chores associated with personal computing and to keep your system running at peak performance. Some examples of utility programs include:

Antivirus: designed to protect your computer from viruses.

Backup Software: designed to assist you in backing up your files and even the entire computer hard drives.

Disk Defragmentation Software: designed to assist you in keeping reorganized your disk drives. After files are saved, deleted and resaved again, the disk becomes fragmented. Disk defragmenters gather those free spaces and put them together to enable you to continue to save your data in the most efficient manner.

Language Processor: used to translate program to machine level from other languages. The types of language translators are *compiler, interpreter and assembler*.

2. Application Software

Application software enable the users to accomplish specific tasks. Business software, databases and educational software are some application software. Other application software is word processing, spreadsheet, presentation packages etc. Mainly, there are two types of application software: *package software* and *tailored software*.

iii. Package Software

They are mainly designed by software companies to generalize the tasks. Some common types of package software are:

Word Processing Software: enables the users to create and edit documents. The most popular word processors are MS Word, Word-Pad, Notepad and some other text editors.

Database Software: allows the users to store and retrieve data from database. The Oracle, MS Access, MySQL, SQL etc. are database software.

Spreadsheet Software: allows users to perform different kinds of numeric calculations. E.g. MS Excel, Lotus, Apple numbers etc.

Multimedia Software: allows the users to create and play with picture, audio, video media. E.g. Media Player, Real Player etc.

Presentation Software: allows users to present their information in the form of slides. E.g. MS PowerPoint etc.

iv. Tailored Software

It is specific purpose software which is developed for specific task. They are mainly written in high level language such as Java, C, C++, COBOL. E.g. banking software, hospital software, hotel reservation software, billing software etc.

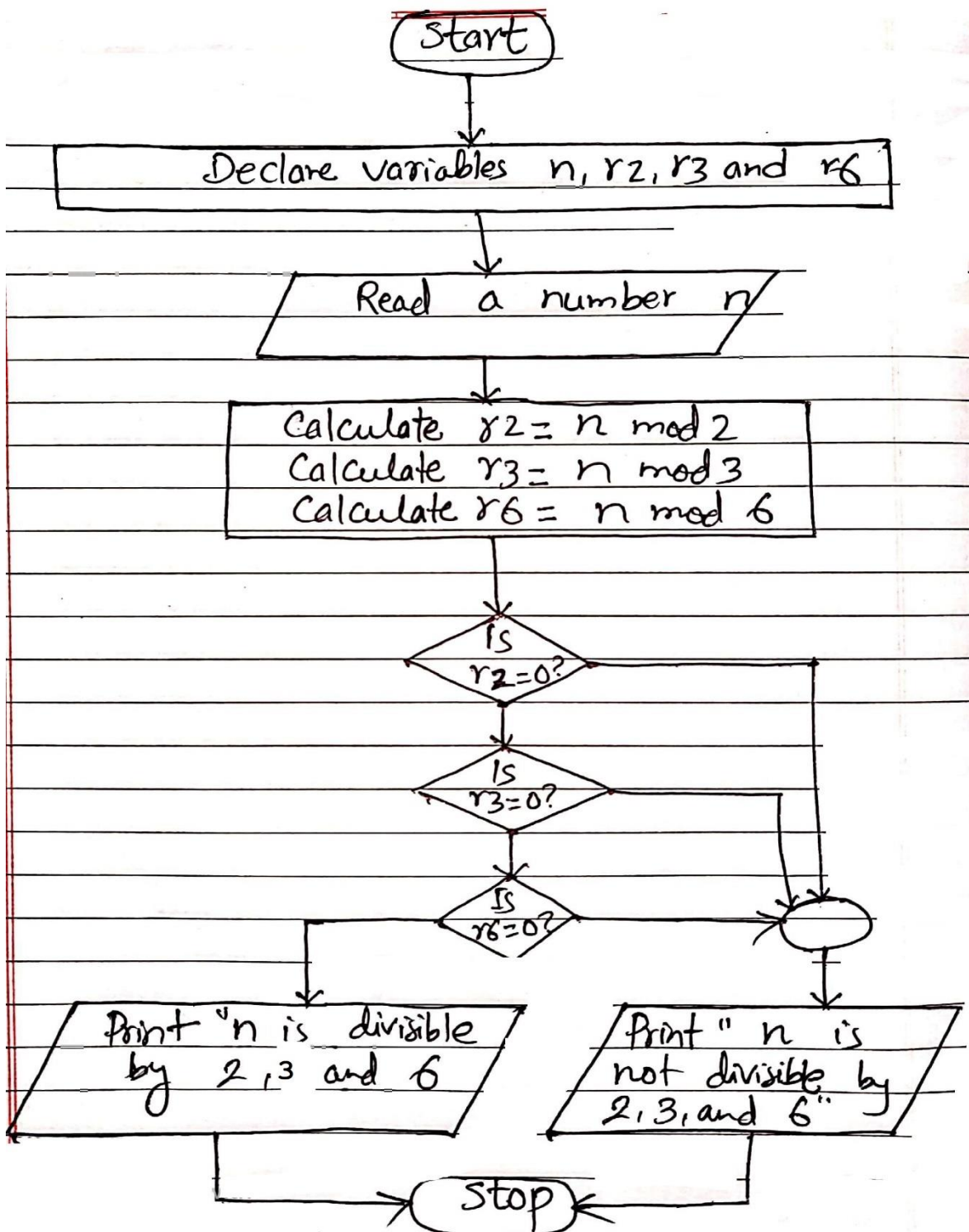
1. b. Differentiate algorithm and flowchart. Write an algorithm and draw flowchart to find the number given by the user is divisible by 2, 3, and 6 or not.

Basis for Comparison	Algorithm	Flowchart
Basic	Includes sequence of steps which depicts the procedure of the solution	An information diagram made up of different shapes shows the data flow.
Comprehensibility	Hard to understand	Easily interpreted
Uses	Text	Symbols
Implements	No rules are implemented	Pre-defined rules are implemented
Debugging	Easier	Difficult
Ease of construction	Hard	Simple
Ease of modification	Easy	Hard and tedious

Algorithm

12. Start
13. Declare variables n, r2, r3 and r6.
14. Read a number n.
15. Calculate $r2 = n \bmod 2$.
16. Calculate $r3 = n \bmod 3$.
17. Calculate $r6 = n \bmod 6$.
18. Check if $r2=0$. If true, go to step 8 otherwise go to step 10.
19. Check if $r3=0$. If true, go to step 9 otherwise, go to step 10.
20. Check if $r6=0$. If true, print "n is divisible by 2, 3 and 6". Otherwise go to step 10.
21. Print "n is not divisible by 2 ,3 and 6."
22. Stop.

Flowchart to find the number given by the user is divisible by 2, 3, and 6 or not.



2.a. Define operand and operator. List the types of operators and explain any five of them.

The operand is the value or variable which is processed by operator. An operator is a symbol that operates on single or multiple data items. They are used to perform mathematical or logical manipulations. The data items that operators act upon are called operands.

For example, consider an expression

$a + b = 10$

here a , b , and 10 are operands and $+$ and $=$ are operators.

Some operators require two operands, while others act upon only one operand.

3. According to number of operands

g) Unary Operators (single operand)

e.g., unary minus ($-$), unary plus ($+$), $++$, $--$, $!$, $\sim$, sizeof , type etc.

h) Binary Operators (two operands)

e.g., Binary plus ($+$), Binary minus ($-$), $*$, $/$, $>$, $<$, $\&\&$, $<<$ etc.

i) Ternary Operators (three operands)

e.g., conditional operator ($?:$)

2. According to Operations

q. Arithmetic Operators

r. Relational Operators

s. Logical Operators

t. Assignment Operators

u. Increment/Decrement

v. Conditional Operator

w. Bitwise Operators

x. Special operators

1. Arithmetic Operators

The C provides all the Arithmetic Operators (*binary*) which perform arithmetic operations.

Operator	Purpose	Example	Output int a=20, b=6
+	Addition	$a + b$	26
-	Subtraction	$a - b$	14
*	Multiplication	$a * b$	120
/	Division	a / b	3
%	Remainder after integer division	$a \% b$	2

The operands acted upon by arithmetic operators must represent numeric values. The division operator requires that the second operand must be non-zero. The remainder operator ($\%$) requires that both operands must be integers and the second operand should be non-zero.

m. Relational Operators

They are binary operators which are used to compare two similar operands. They compare their left-hand side operand with their right-hand side operand for comparisons according to magnitudes. The output of a relational operation is either *true* (1) or *false* (0). So, they are used in decision making statements.

Operator	Purpose	Example	Output int a=20, b=6	Output int a=20, b=20
<	Less than	$a < b$	0	0
>	Greater than	$a > b$	1	0
<=	Less than or Equal to	$a <= b$	0	1
>=	Greater than or Equal to	$a >= b$	1	1

These operators all fall within the same precedence group, which is lower than the arithmetic and unary operators. The associativity of these operators is left to right.

Closely associated with the relational operators are the following two *equality operators*.

Operator	Purpose	Example	Output int a=20, b=6	Output int a=20, b=20
==	Equal to	a == b	0	1
!=	Not Equal to	a != b	1	0

The equality operators fall into a separate precedence group, beneath the relational operators. These operators also have a *left-to-right associativity*.

These six operators are used to form **logical expressions**, which represent conditions that are either true or false. The resulting expressions will be of type integer, since *true* is represented by the integer value 1 and *false* is represented by the value 0.

a. Logical Operators

In addition to the relational and equality operators, C contains two *logical operators* (also called *logical connectives*). They are

Operator	Meaning
&&	Logical AND
	Logical OR

The logical operators act upon operands that are themselves logical expressions. The net effect is to combine the individual logical expressions into more complex conditions that are either true or false. The result of a *logical and* operation will be true only if both operands are true, whereas the result of a *logical or* operation will be true if either operand is true or if both operands are true. In other words, the result of a *logical or* operation will be false only if both operands are false.

Suppose that *i* is an integer variable whose value is 7, *f* is a floating-point variable whose value is 5.5, and *c* is a character variable that represents the character 'w'. Several complex logical expressions that make use of these variables are shown below.

Expression	Interpretation	Value
(i >= 6) && (c == 'w')	True	1
(i >= 6) (c == 119)	True	1
(f < 11) && (i > 100)	False	0
(c != 'p') ((i + f) <= 10)	True	1

The first expression is true because both operands are true. In the second expression, both operands are again true; hence the overall expression is true. The third expression is false because the second operand is false. And finally, the fourth expression is true because the first operand is true.

Each of the logical operators falls into its own precedence group. *Logical and* has a higher precedence than *logical or*. Both precedence groups are lower than the group containing the equality operators. The associativity is left to right. The precedence groups are summarized below.

C also includes the unary operator **!** that negates the value of a logical expression; i.e., it causes an expression that is originally true to become false, and vice versa. This operator is referred to as the *logical negation* (or *logical not*) operator.

Suppose that *i* is an integer variable whose value is 7, and *f* is a floating-point variable whose value is 5.5. Several logical expressions which make use of these variables and the logical negation operator are shown below.

Expression	Interpretation	Value
f > 5	True	1
! (f > 5)	False	0

<code>i <= 3</code>	False	0
<code>!(i <= 3)</code>	True	1
<code>i > (f + 1)</code>	True	1
<code>!(i > (f + 1))</code>	False	0

n. Assignment Operator

There are several different assignment operators in C. All of them are used to form *assignment expressions*, which assign the value of an expression to an identifier. The most used assignment operator is `=`. Assignment expressions that make use of this operator are written in the form

```
identifier = expression
```

where *identifier* generally represents a variable, and *expression* represents a constant, a variable or a more complex expression.

Here are some typical assignment expressions that make use of the `=` operator.

```
a = 3;
x = y;
delta = 0.001;
sum = a + b;
area = length * width;
```

The first assignment expression causes the integer value **3** to be assigned to the variable *a*, and the second assignment causes the value of *y* to be assigned to *x*. In the third assignment, the floating-point value **0.001** is assigned to *delta*. The last two assignments each result in the value of an arithmetic expression being assigned to a variable (i.e., the value of **a + b** is assigned to *sum*, and the value of **length * width** is assigned to *area*).

Under some circumstances, this automatic type conversion can result in an alteration of the data being assigned. For example:

- A floating-point value may be rounded if assigned to an integer identifier.
- A double-precision value may be rounded if assigned to a floating-point (single-precision) identifier.
- An integer quantity may be altered if assigned to a shorter integer identifier or to a character identifier (some high-order bits may be lost).

Multiple assignments of the form

```
identifier 1 = identifier 2 = ... = expression
```

are permissible in C. In such situations, the assignments are carried out from right to left. Thus, the multiple assignment

```
identifier 1 = identifier 2 = expression
```

is equivalent to

```
identifier 1 = (identifier 2 = expression)
```

and so on, with right-to-left nesting for additional multiple assignments.

Suppose that *i* and *j* are integer variables. The multiple assignment expression

```
i = j = 5;
```

will cause the integer value 5 to be assigned to both *i* and *j*. (To be more precise, 5 is first assigned to *j*, and the value of *j* is then assigned to *i*.) Similarly, the multiple assignment expression

```
i = j = 5.9;
```

will cause the integer value 5 to be assigned to both *i* and *j*. Remember that truncation occurs when the floating-point value 5.9 is assigned to the integer variable *j*. C contains the following five additional assignment operators: `+=`, `-=`, `*=`, `/=` and `%=`. To see how they are used, consider the first operator, `+=`. The assignment expression

```
expression 1 += expression 2
```

is equivalent to

```
expression 1 = expression 1 + expression 2
```

Similarly, the assignment expression

```
expression 1 -= expression 2
```

is equivalent to

```
expression 1 = expression 1 - expression 2
```

and so on for all five operators.

Usually, *expression 1* is an identifier, such as a variable or an array element.

Suppose that *i* and *j* are integer variables whose values are 5 and 7, and *f* and *g* are floating-point variables whose values are 5.5 and -3.25. Several assignment expressions that make use of these variables are shown below. Each expression utilizes the original values of *i*, *j*, *f* and *g*.

Expression	Equivalent Expression	Final Value
<code>i += 5</code>	<code>i = i + 5</code>	10
<code>f -= g</code>	<code>f = f - g</code>	8.75
<code>j *= (i - 3)</code>	<code>j = j * (i - 3)</code>	14
<code>f /= 3</code>	<code>f = f / 3</code>	1.833333
<code>i %= (j - 2)</code>	<code>i = i % (j - 2)</code>	0

Conditional Operator

Simple conditional operations can be carried out with the *conditional operator* (`? :`). An expression that makes use of the conditional operator is called a *conditional expression*. Such an expression can be written in place of the more traditional *if-else* statement. The conditional expression is written in the form

```
expression 1 ? expression 2 : expression 3
```

When evaluating a conditional expression, *expression 1* is evaluated first. If *expression 1* is true (i.e., if its value is nonzero), then *expression 2* is evaluated and this becomes the value of the conditional expression. However, if *expression 1* is false (i.e., if its value is zero), then *expression 3* is evaluated and this becomes the value of the conditional expression. Note that only one of the embedded expressions (either *expression 2* or *expression 3*) is evaluated when determining the value of a conditional expression.

In the conditional expression shown below, assume that *i* is an integer variable.

```
(i < 0) ? 0 : 100;
```

The expression `(i < 0)` is evaluated first. If it is true (i.e., if the value of *i* is less than 0), the entire conditional expression takes on the value 0. Otherwise (if the value of *i* is not less than 0), the entire conditional expression takes on the value 100.

In the following conditional expression, assume that *f* and *g* are floating-point variables.

```
(f < g) ? f : g;
```

This conditional expression takes on the value of *f* if *f* is less than *g*; otherwise, the conditional expression takes on the value of *g*. In other words, the conditional expression returns the value of the *smaller* of the two variables.

2. b. Write a recursive program to raise number *X* to power *n*, i.e., X^n .

```
#include <stdio.h>
```

```

float expo(float, int);
void main()
{
    int n;
    float x, r;
    printf("Enter value of base:");
    scanf(" %f", &x);
    printf("Enter exponent:");
    scanf(" %d", &n);
    r = expo(x, n);
    printf("%.2f^%d: %.2f", x, n, r);
}
float expo(float x, int n)
{
    if (n == 0)
        return 1.0;
    else if (n == 1)
        return x;
    else
        return(x * expo(x, n - 1));
}

```

OUTPUT

```

Enter value of base:5
Enter exponent:3
5.00^3: 125.00

```

3.a. Write a program to generate the following pattern by initializing the string at first.

PROGRAMMING

PROGRAMMIN

PROGRAMMI

PROGRAMM

PROGRAM

PROGRA

PROGR

PROG

PRO

PR

P

```

#include <stdio.h>
#include <string.h>

```

```

#define EOL '\0'

```

```

void main()
{
    char str[] = "PROGRAMMING";
    int len, i, j;
    len = strlen(str);
    for (i = len - 1; i >= 0; --i)
    {

```

```

        for (j = 0; j <= i; ++j)
        {
            printf("%c", str[j]);
        }
        printf("\n");
    }
}

```

3. b. Explain the run time and compile time initialization of an array with example.

Compile Time Initialization

The initial values must appear in the order in which they will be assigned to the individual array elements, enclosed in braces and separated by commas. The general form is

```
storage-class data-type array[expression] = {value 1, value 2, ..., value n};
```

where **value 1** refers to the value of the first array element, **value 2** refers to the value of the second element, and so on. The appearance of the **expression**, which indicates the number of array elements, is *optional* when initial values are present.

Shown below are several array definitions that include the assignment of initial values.

```
int digits[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
static float x[6] = {0, 0.25, 0, -0.50, 0, 0};
char color[3] = {'R', 'E', 'D',};
```

Note that **x** is a static array. The other two arrays (**digits** and **color**) are assumed to be external arrays by virtue of their placement within the program. The results of these initial assignments, in terms of the individual array elements, are as follows. (Remember that the subscripts in an n-element array range from **0** to **n - 1**.)

digits[0] = 1	x[0] = 0	color[0] = 'R'
digits[1] = 2	x[1] = 0.25	color[1] = 'E'
digits[2] = 3	x[2] = 0	color[2] = 'D'
digits[3] = 4	x[3] = -0.50	
digits[4] = 5	x[4] = 0	
digits[5] = 6	x[5] = 0	
digits[6] = 7		
digits[7] = 8		
digits[8] = 9		
digits[9] = 10		

All individual array elements that are not assigned explicit initial values will automatically be set to **zero**. This includes the remaining elements of an array in which some elements have been assigned nonzero values.

Consider the following array definitions.

```
int digits[10] = {3, 3, 3};
static float x[6] = {-0.3, 0, 0.25};
```

The results, on an element-by-element basis, are as follows.

digits[0] = 3	x[0] = -0.3
----------------------	--------------------

```

digits[1] = 3    x[1] = 0
digits[2] = 3    x[2] = 0.25
digits[3] = 0    x[3] = 0
digits[4] = 0    x[4] = 0
digits[5] = 0    x[5] = 0
digits[6] = 0
digits[7] = 0
digits[8] = 0
digits[9] = 0

```

In each case, all the array elements are automatically set to **zero** except those that have been explicitly initialized within the array definitions.

Run Time Initialization

The array can be initialized at run time by accepting data from users. In that case, you may use a loop (for 1-D array) for initialization of array element. The following code skeletal shows the run time initialization.

```

int age[10], i;
for (i = 0; i < 10; ++i)
    scanf("%d", &age[i]);

```

Here scanf() will run at ten times and initialize array elements one at a time. While executing, if user supplies 10, 20, 15, 22, 18, 16, 14, 25, 23, 19 then

```

age[0] = 10  age[1] = 20  age[2] = 15  age[3] = 22  age[4] = 18  age[5] = 16
age[6] = 14  age[7] = 25  age[8] = 23  age[9] = 19

```

4. a. Write a program to find the transpose of 4*3 matrix.

```

//transpose of matrix
#include<stdio.h>

#define m 4
#define n 3

void main()
{
    int mat[m][n], trans[n][m], i, j;
    printf("Enter elements of %dx%d matrix\n", m, n);
    for (i = 0; i < m; ++i)
    {
        for (j = 0; j < n; ++j)
        {
            scanf("%d", &mat[i][j]);
        }
    }
    /* printf("\nBefore transpose\n");
    for (i = 0; i < m; i++)
    {
        for (j = 0; j < n; j++)
        {
            printf("%d\t", mat[i][j]);

```

```

    }
    printf("\n");
}*/

//transposing
for (i = 0; i < m; ++i)
{
    for (j = 0; j < n; ++j)
    {
        trans[j][i] = mat[i][j];
    }
}
printf("\nAfter transpose\n");
for (i = 0; i < n; ++i)
{
    for (j = 0; j < m; ++j)
    {
        printf("%d\t", trans[i][j]);
    }
    printf("\n");
}
}

```

OUTPUT

Enter elements of 4x3 matrix

```

1 2 3
4 5 6
7 8 9
-5 -10 -15

```

After transpose

```

1      4      7      -5
2      5      8      -10
3      6      9      -15

```

4. b. What do you mean by recursive function? What are the advantage and disadvantage of using recursive function? Write a program to find whether the number is prime or not using function.

Recursion is a process by which a function calls itself repeatedly, until some specified condition has been satisfied. The process is used for repetitive computations in which each action is stated in terms of a previous result. Many iterative (i.e., repetitive) problems can be written in this form.

In order to solve a problem recursively, two conditions must be satisfied. First, the problem must be written in a *recursive form*, and second, the problem statement must *include a stopping condition*.

Advantages of Recursive function are listed below.

- The repetitive structure results in shorter codes.
- Complex case analysis and nested loops can be avoided.
- Recursion can lead to more readable and efficient algorithm descriptions.

- Recursion is very useful to solve those problems which are recursive in nature (e.g., Tower of Hanoi)

Disadvantages

- Fairly slower than its iterative solution.
- For each step we make a recursive call to a function. For which it occupies significant amount of stack memory with each step.
- May cause stack-overflow if the recursion goes too deep to solve the problem.
- Difficult to debug and trace the values with each step of recursion.
- All problems cannot be solved by recursion.

```
#include <stdio.h>
```

```
int prime(int);
```

```
void main()
```

```
{
    int n;
    printf("Enter a number to be checked: ");
    scanf(" %d", &n);
    if (prime(n) == 1)
        printf("%d is Prime.", n);
    else
        printf("%d is not Prime.", n);
}
```

```
int prime(int n)
{
    int i;
    for (i = 2; i < n; ++i)
    {
        if (n % i == 0)
            break;
    }
    if(i == n)
        return 1;
    else
        return 0;
}
```

OUTPUT

RUN 1

Enter a number to be checked: 15

15 is not Prime.

RUN 2

Enter a number to be checked: 11

11 is Prime.

5.a. Differentiate between dynamic memory allocation and static memory allocation. Explain with suitable program example.

Basis for Comparison	Static Memory allocation	Dynamic Memory Allocation
Basic	Static memory allocation is a method of allocating memory, and once the memory is allocated, it is fixed.	Dynamic memory allocation is a method of allocating memory, and once the memory is allocated, it can be changed.
Modification	In static memory allocation, it is not possible to resize after initial allocation.	In dynamic memory allocation, the memory can be minimized or maximize accordingly.
Implementation	Static memory allocation is easy to implement.	Dynamic memory allocation is complex to implement.
Speed	execution is faster than dynamic memory allocation.	execution is slower than dynamic memory allocation.
Memory utilization	execution is faster than dynamic memory allocation.	Dynamic memory allocation allows reusing the memory. The programmer can allocate more memory when required . He can release the memory when necessary.
Allocation time	Allocation is done before program execution	Allocation is done during program execution
Data structure	It uses the data structure called stack for implementing static allocation	It uses the data structure called heap for implementing dynamic allocation

```
//static memory allocation
```

```
#include <stdio.h>
```

```
#define N 10
```

```
main()
```

```
{
```

```
    int a[100], i, sum = 0;
```

```
    printf("Enter any 10 numbers\n");
```

```
    for (i = 0; i < N; ++i)
```

```
        scanf(" %d", &a[i]);
```

```
    printf("Total sum: ");
```

```
    for (i = 0; i < N; ++i)
```

```
        sum += a[i];
```

```
    printf("%d", sum);
```

```
}
```

In the above program, the memory for 100-elements array is allocated at compile time.

```
//DMA
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
main()
```

```
{
```

```
    int *a, i, n, sum = 0;
```

```

printf("How many numbers? ");
scanf(" %d", &n);
a = (int *)malloc(n * sizeof(int));
printf("Enter %d numbers\n", n);
for (i = 0; i < n; ++i)
    scanf(" %d", a + i);

printf("Total sum: ");
for (i = 0; i < n; ++i)
    sum += *(a + i);
printf("%d", sum);
free(a);
}

```

Here memory required for n elements is allocated at run time using malloc() function. The output of the above programs is given below.

```

How many numbers? 10
Enter 10 numbers
1 2 3 4 5 6 7 8 9 10
Total sum: 55

```

5. b. Write a program to pass array element to function using pointer.

```

/* reordering a one-dimensional, integer array from
smallest to largest, using pointer notation */
#include <stdio.h>
#include <stdlib.h>

void reorder(int n, int *x);

main()
{
    int i, n, *x;

    /* read in a value for n */
    printf("\nHow many numbers will be entered? ");
    scanf(" %d", &n);

    /* dynamically allocate memory for n numbers */
    x = (int *) malloc(n * sizeof(int));

    /*read in the list of numbers */
    for (i = 0; i < n; ++i)
    {
        printf("i = %d x = ", i + 1);
        scanf(" %d", x + i);
    }

    /* reorder all array elements */
    reorder(n, x);
}

```

```

/* display the reordered list of numbers */
printf("\nReordered list of numbers:\n");
for (i = 0; i < n; ++i)
    printf("i = %d x = %d\n", i+1, *(x + i));
free(x);
}

```

```

void reorder(int n, int *x)
{
    int i, item, temp;
    for (item = 0; item < n - 1; ++item)
        for (i = item + 1; i < n; ++i)
            if(*(x + i) < *(x + item))
            {
                temp = *(x + item);
                *(x + item) = *(x + i);
                *(x + i) = temp;
            }
    return;
}

```

How many numbers will be entered? 10

```

i = 1    x = 20
i = 2    x = 30
i = 3    x = 10
i = 4    x = 5
i = 5    x = -15
i = 6    x = 15
i = 7    x = 25
i = 8    x = 35
i = 9    x = 22
i = 10   x = 40

```

Reordered list of numbers:

```

i = 1    x = -15
i = 2    x = 5
i = 3    x = 10
i = 4    x = 15
i = 5    x = 20
i = 6    x = 22
i = 7    x = 25
i = 8    x = 30
i = 9    x = 35
i = 10   x = 40

```

6. a. Create a structure for the following data. And also write a program to input 100 employees and display whose department is 'Sales'.

Emp_id	Emp_Name	Address	Department	Date of Birth		
				mm	dd	yy

```
#include<stdio.h>
```

```

#include<string.h>

#define N 100
struct Date
{
    int mm;
    int dd;
    int yr;
};

struct Employee
{
    int Emp_id;
    char Emp_name[30];
    char Address[50];
    char Department[20];
    struct Date dob;
};

void main()
{
    struct Employee e[N];
    int i;

    printf("Enter Details about %d employees\n", N);
    for (i = 0; i < N; i++)
    {
        printf("\nEmployee ID: ");
        scanf(" %d", &e[i].Emp_id);
        printf("\nEmployee Name: ");
        scanf(" %[^\\n]", e[i].Emp_name);
        printf("\nEmployee Address: ");
        scanf(" %[^\\n]", e[i].Address);
        printf("\nEmployee Department: ");
        scanf(" %[^\\n]", e[i].Department);

        printf("\nDate of Birth(mm-dd-yy): ");
        scanf(" %d %d %d", &e[i].dob.mm, &e[i].dob.dd,
&e[i].dob.yr);
    }

    printf("\nDetails about employee\n");
    printf("\nEmp_id\tEmp_name\tAddress\tDepartment mm-dd-yr\n");
    for (i = 0; i < N; i++)
    {
        if(strcmp("Sales", e[i].Department) == 0)
            printf("\n%d\t%s\t\t%s\t%s\t%d-%d-%d",
e[i].Emp_id, e[i].Emp_name, e[i].Address, e[i].Department,
e[i].dob.mm, e[i].dob.dd, e[i].dob.yr);
    }
}

```

```
}
```

6.b. What is the significance of file pointer in file handling? Consider the following structure:

Roll no.	Name	Address	Faculty	Date of Birth		
				mm	dd	yy

Write a program to create 'Student.txt' file to store the above records for 100 employees.

A file pointer is a pointer to a structure, which contains information about the file, including its name, current position of the file, whether the file is being read or written, and whether errors or end of the file have occurred. The user does not need to know the details, because the definitions obtained from *stdio.h* include a structure declaration called FILE. The only declaration needed for a file pointer is symbolized by

FILE *fp;

This says that **fp** is the file pointer that points to a FILE structure.

```
#include <stdio.h>
#include <stdlib.h>

typedef struct
{
    int mm;
    int dd;
    int yy;
} date;

typedef struct
{
    int roll;
    char name[30];
    char address[80];
    char faculty[30];
    date dob;
} student;

main()
{
    student s;
    FILE *fp;
    int i;

    if ((fp = fopen("Student.txt", "w")) == NULL)
    {
        printf("File cannot be created.");
        exit(1);
    }
    printf("Enter details about 100 Students\n");
    for (i = 0; i < 100; ++i)
    {
        printf("Student no. %d\n", i + 1);
```

```

printf("Roll Number: ");
scanf(" %d", &s.roll);
printf("Name: ");
scanf(" %[^\\n]", s.name);
printf("Address: ");
scanf(" %[^\\n]", s.address);
printf("Faculty: ");
scanf(" %[^\\n]", s.faculty);
printf("Date of Birth(mm-dd-yy): ");
scanf(" %d %d %d", &s.dob.mm, &s.dob.dd, &s.dob.yy);
fwrite(&s, sizeof(student), 1, fp);
}

fclose(fp);
}

```

7. Write short notes on:

a. Preprocessor Directives

The C preprocessor is a collection of special statements, called directives, that are executed at the beginning of the compilation process. The **#include** and **#define** statements considered are preprocessor directives. Additional preprocessor directives are **#if**, **#elif**, **#else**, **#endif**, **#if def**, **#ifndef**, **#line** and **#undef**. The preprocessor also includes three special operators: **defined**, **#**, and **##**.

Preprocessor directives usually appear at the beginning of a program, though this is not a firm requirement. Thus, a preprocessor directive may appear anywhere within a program. However, the directive will apply only to the portion of the program following its appearance.

For the beginning programmer, some of the preprocessor directives are relatively unimportant. Hence, we will not describe each preprocessor feature in detail. The more important features are discussed below.

The **#if**, **#elif**, **#else** and **#endif** directives are used most frequently. They permit conditional compilation of the source program, depending on the value of one or more true/false conditions. They are sometimes used in conjunction with the **defined** operator, which is used to determine whether a symbolic constant or a macro identifier has been defined within a program.

The following preprocessor directives illustrate the conditional compilation of a C program. The conditional compilation depends on the status of the symbolic constant **FOREGROUND**.

```

#if defined (FOREGROUND)
    #define BACKGROUND 0
#else
    #define FOREGROUND 0
    #define BACKGROUND 7
#endif

```

Thus, if **FOREGROUND** has already been defined, the symbolic constant **BACKGROUND** will represent the value 0. Otherwise, **FOREGROUND** and **BACKGROUND** will represent the values 0 and 7 respectively.

Here is another way to accomplish the same thing.

```

#ifndef FOREGROUND

```

```

    #define BACKGROUND 0
#else
    #define FOREGROUND 0
    #define BACKGROUND 7
#endif

```

The directive **#ifdef** is equivalent to **#if defined()**. Similarly, the directive **#ifndef** is equivalent to **#if !defined()**, i.e., “if not defined.” The original approach, in which the **defined** operator appears explicitly, is the preferred form.

In each of these examples, the last directive is **#endif**. The preprocessor requires that any set of directives beginning with **#if**, **#ifdef** or **#ifndef** must end with **#endif**.

The directive **#elif** is analogous to an **else - if** clause using ordinary C control statements. An **#if** directive can be followed by any number of **#elif** directives, though there can be only one **#else** directive. The appearance of the **#else** directive is optional, as determined by the required program logic.

Here is another illustration of conditional compilation. In this situation the conditional compilation will depend on the value that is represented by the symbolic constant BACKGROUND.

```

#if BACKGROUND == 7
    #define FOREGROUND 0
#elif BACKGROUND == 6
    #define FOREGROUND 1
#else
    #define FOREGROUND 6
#endif

```

In this example we see that FOREGROUND will represent **0** if BACKGROUND represents **7**, and FOREGROUND will represent **1** if BACKGROUND represents **6**. Otherwise, FOREGROUND will represent **6**.

The **#undef** directive “undefines” a symbolic constant or a macro identifier; i.e., it negates the effect of a **#define** directive that may have appeared earlier in the program.

The following example illustrates the use of the **#undef** directive within a C program.

```

#define FOREGROUND 7
#define BACKGROUND 0

main()
{
    .....
    #undef    FOREGROUND
    .....
    #undef    BACKGROUND
    .....
}

```

The symbolic constants FOREGROUND and BACKGROUND are defined by the first two directives. These definitions are then negated by the **#undef** directives, when they appear later

in the program. Prior to the **#undef** directives, any references to FOREGROUND or BACKGROUND will be associated with the values 7 and 0, respectively. After the **#undef** directives, any references to the corresponding identifiers will be ignored.

b. Union

Unions, like structures, contain members whose individual data types may differ from one another. However, the members within a union *all share the same storage area* within the computer's memory, whereas each member within a structure is assigned its own unique storage area. Thus, unions are used to conserve memory. They are useful for applications involving multiple members, where values need not be assigned to all of the members at any one time.

Within a union, the bookkeeping required to store members whose data types are different (having different memory requirements) is handled automatically by the compiler. However, the user must keep track of what type of information is stored at any given time. An attempt to access the wrong type of information will produce meaningless results. In general terms, the composition of a union may be defined as

```
union tag
{
member 1;
member 2;
.....
member m;

};
```

where **union** is a required keyword and the other terms have the same meaning as in a structure definition. Individual union variables can then be declared as

```
storage-class union tag variable 1, variable 2, ..., variable n;
```

where **storage-class** is an optional storage class specifier, **union** is a required keyword, **tag** is the name that appeared in the union definition, and **variable1, variable 2, ..., variable n** are union variables of type **tag**.

The two declarations may be combined, just as we did with structures. Thus, we can write

```
storage-class union tag
{
member 1;
member 2;
...
member m;
} variable 1, variable 2, ..., variable n;
```

The **tag** is optional in this type of declaration. A C program contains the following union declaration.

```
union id
{
char color[12];
int size;
} shirt, blouse;
```


Here we have two union variables, *shirt* and *blouse*, of type **id**. Each variable can represent either a 12-character string (**color**) or an integer quantity (**size**) at any one time. The 12-character string will require more storage area within the computer's memory than the integer quantity. Therefore, a block of memory large enough for the 12-character string will be allocated to each union variable. The compiler will automatically distinguish between the 12-character array and the integer quantity within the given block of memory, as required.

c. *break* and *continue* statement

The Break Statement

The **break** statement is used to *terminate loops* or to *exit from a switch*. It can be used within a *for*, *while*, *do - while*, or *switch* statement. The break statement is written simply as

break;

without any embedded expressions or statements.

In the following situation, the loop will continue to execute as long as the current value for the floating-point variable *x* does not exceed **100**. However, the computation will break out of the loop if a negative value for *x* is detected.

First, consider a **while** loop.

```
scanf(" %f", &x); //initialize
while (x <= 100)
{
    if (x < 0)
    {
        printf("ERROR! NEGATIVE VALUE FOR x");
        break;
    }
    /* process the non-negative value of x */
    ...
    scanf(" %f", &x); //update
}
```

The continue Statement

The continue statement is used to *bypass* the remainder of the current pass through a loop.

The loop does not terminate when a continue statement is encountered. Rather, the remaining loop statements are skipped and the computation proceeds directly to the next pass through the loop. (Note the distinction between continue and break.)

The *continue* statement can be included within a *while*, a *do - while* or a *for* statement. It is written simply as

continue;

without any embedded statements or expressions.

Consider a *do - while* loop.

```
do
{
    scanf(" %f", &x);
    if (x < 0)
    {
```

```

    printf("ERROR! NEGATIVE VALUE FOR x");
    continue;
}
/* process the non-negative value of x */
...
}while (x <= 100);

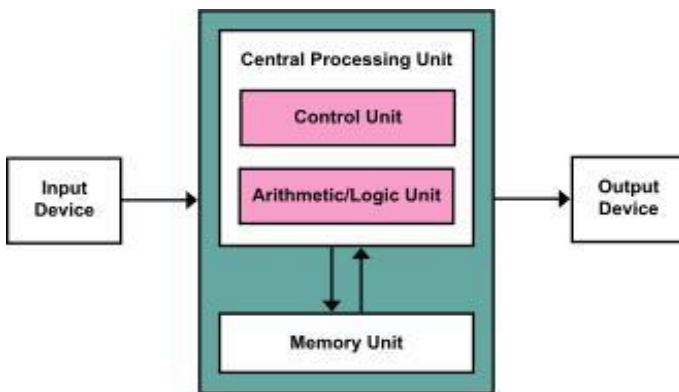
```

The processing of the current value of x will be bypassed if the value of x is negative. Execution of the loop will then continue with the next pass.

2018 Spring

1.a. Draw block diagram of digital computer. Explain each component in brief.

The von-Neumann architecture, which is also known as the von Neumann model and Princeton architecture, is a computer architecture based on that described in 1945 by the mathematician and physicist John von Neumann. This describes a design architecture for an electronic digital computer with parts consisting of a processing unit containing an arithmetic logic unit and processor registers; a control unit containing an instruction register and program counter; a memory to store both data and instructions; external mass storage; and input and output mechanisms. The key idea of the Von-Neumann architecture is the stored program concept. A stored-program digital computer is one that keeps its program instructions, as well as its data, in read-write, random-access memory (RAM). Stored-program computers were an advancement over the program-controlled computers of the 1940s.



The main components of a computer are **Input unit (IU)**, **Central Processing unit (CPU)** and **Output unit (OU)**. The information like data, programs etc. are passed to the computer through input devices. The keyboard, mouse, floppy disk, CD, DVD, joystick etc. are certain input devices. The output device is to get information from a computer after processing. VDU (Visual Display Unit), Printer, Floppy disk, CD etc. are

output devices.

The brain of a computer is CPU. It has three components- **Memory unit**, **Control unit** and **Arithmetic and Logical unit (ALU)**. Memory unit also called storage device is to store information. Two types memory are there in a computer. They are RAM (random access memory) and ROM (read only memory). When a program is called, it is loaded and processed in RAM. When the computer is switched-off, whatever stored in RAM will be deleted. So, it is a temporary memory. Whereas, ROM is a permanent memory, where data, program etc. are stored for future use. Inside a computer there is storage device called Hard disk, where data are stored and can be accessed at any time.

The control unit is for controlling the execution and interpreting of instructions stored in the memory. ALU is the unit where the arithmetic and logical operations are performed. The

information to a computer is transformed to groups of binary digits, called bit. The length of bit varies from computer to computer, from 8 to 64. A group of 8 bits is called a *Byte* and a byte generally represents one alphanumeric (Alphabets and Numerals) character. The physical components of a computer are called hardware. But for the machine to work it requires certain programs. They are called software.

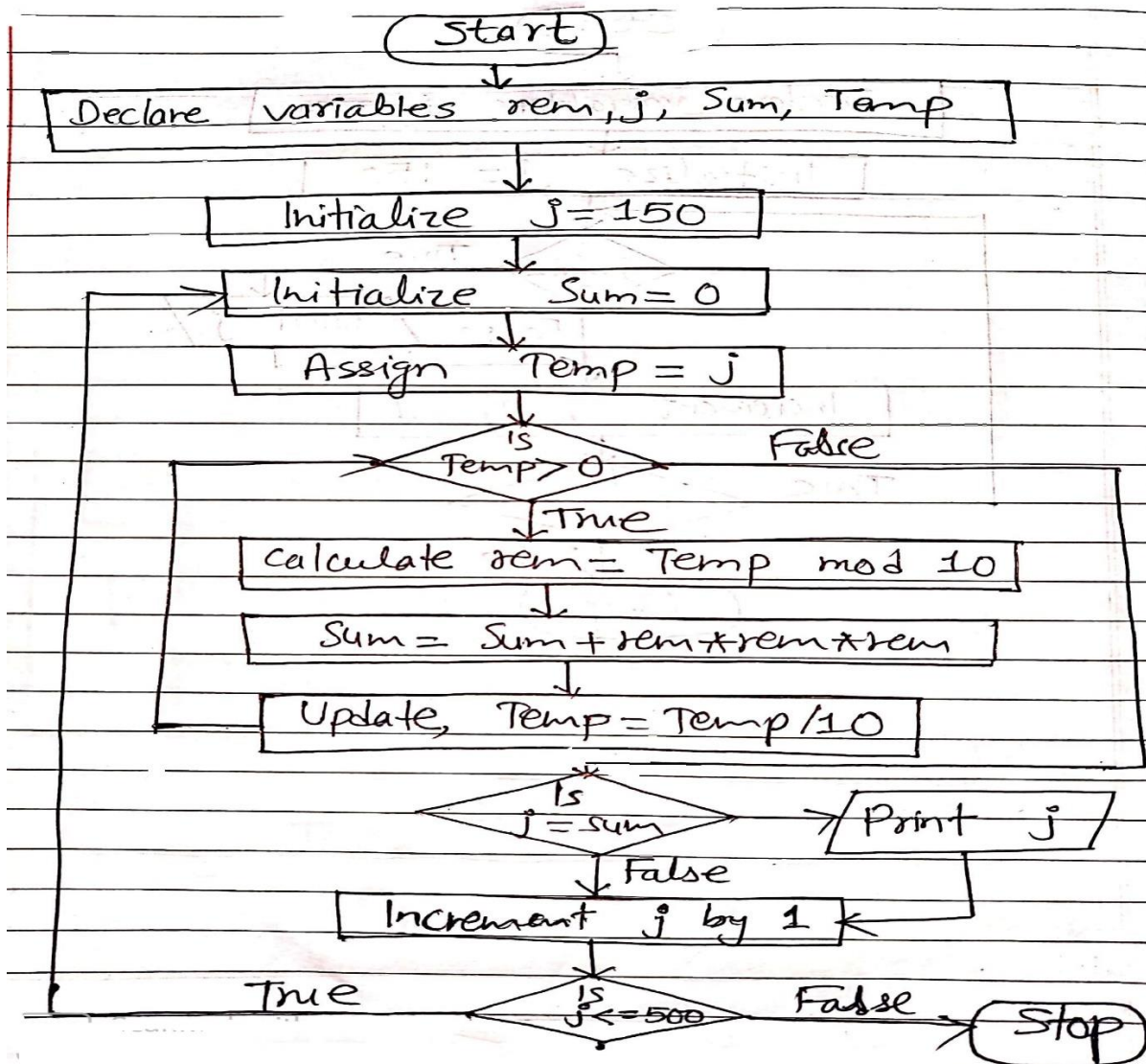
1.b. Define the role of flowchart in efficient program maintenance with its character. Also develop a flowchart to print the Armstrong numbers between 150 and 500.

Flowchart Pictures Detailing with specific blocks detailing out the logical flow of the solution. For a better understanding of an algorithm, it is represented pictorially. The pictorial representation of an algorithm is called a Flowchart.

- *Communication*: Flowcharts are a good way of communication. They quickly and clearly provide logic, ideas and descriptions of algorithms to other programmers, students, teacher, computer operators and users.
- *Effective Analysis*: Flowchart provides a clear overview of the entire problem and its algorithm. They show all the major components and their relationships.
- *Proper Documentation*: The flowchart provides a permanent recording of program logic. It documents the steps followed in an algorithm.
- *Efficient Coding*: The flowchart specifies all the steps to be coded and helps to prevent omission or error. Thus, a flowchart acts as a guide or blueprint during the system analysis and program development phase.

Flowchart

to find Armstrong numbers between 150 and 500



2. a. How can you declare following variables using suitable data types?

Mobile phone numbers, address, body temperature, salary. Also explain each memory occupancy size and range.

Variable	Variable declaration	Memory occupancy	Range
Mobile phone numbers	char phone[10];	1 byte per character (total 10 bytes)	0 to 255 for unsigned characters (-128 to +127 for signed characters)
Address	char address[80];	1 byte per character (total 80 bytes)	
Body temperature	float temperature;	4 bytes	1.2×10^{-38} to $3.4 \times 10^{+38}$
Salary	float salary;	4 bytes	

2. b. Why you use 'continue' and 'break' statement in your program? Explain with suitable program.

The continue Statement

The *continue* statement is used to *bypass* the remainder of the current pass through a loop. The loop does not terminate when a *continue* statement is encountered. Rather, the remaining loop statements are skipped and the computation proceeds directly to the next pass through the loop. (Note the distinction between *continue* and *break*.)

The *continue* statement can be included within a *while*, a *do - while* or a *for* statement. It is written simply as

continue;

without any embedded statements or expressions.

Here are some illustrations of loops that contain *continue* statements. First, consider a *do - while* loop.

```
do
{
    scanf(" %f", &x);
    if (x < 0)
    {
        printf("ERROR! NEGATIVE VALUE FOR x");
        continue;
    }
    /* process the non-negative value of x */
    ...
}while (x <= 100);
```

The Break Statement

The **break** statement is used to *terminate loops* or to *exit from a switch*. It can be used within a *for*, *while*, *do - while*, or *switch* statement. The *break* statement is written simply as

break;

without any embedded expressions or statements.

Here is example of *while* loop that contain **break** statements. The loop will continue to execute as long as the current value for the floating-point variable *x* does not exceed **100**. However, the computation will break out of the loop if a negative value for *x* is detected. First, consider a *while* loop.

```
scanf(" %f", &x); //initialize
while (x <= 100)
{
    if (x < 0)
    {
        printf("ERROR! NEGATIVE VALUE FOR x");
        break;
    }
    /* process the non-negative value of x */
    ...
    scanf(" %f", &x); //update
}
```

3. a. Differentiate pre-test and post-test loop. Write a program to generate fibonacci numbers as per user choice.

Pre-test loop	Post-test loop
While loop and for loop are <i>entry-controlled (pre-test)</i> loops, i.e. test condition is evaluated first and body of loop is executed only if the test is true.	Do-while loop is <i>exit-controlled (post-test)</i> loop, i.e. the body of loop is executed first without checking any condition and at the end of loop body, the condition is evaluated for repetition of next time.
The body of loop may not be executed at all if the condition is not satisfied at the very first attempt.	The body of the loop is executed at least once even the tested condition is false.
Syntax: while (expression) statement The <i>statement</i> will be executed repeatedly, as long as the <i>expression</i> is true (i.e., as long <i>expression</i> has a nonzero value). This <i>statement</i> can be simple or compound.	Syntax: do statement while (expression); The <i>statement</i> will be executed repeatedly, as long as the value of <i>expression</i> is true (i.e., is nonzero). The <i>statement</i> can be either simple or compound.
<pre>int i = 5; while (i < 5) { printf("%d ", i); ++i; }</pre> This loop condition is initially false. So, it does not print any value.	<pre>int i = 5; do { printf("%d ", i); ++i; } while (i < 5);</pre> Although body and conditions in both loops are same, this program will print 5.

```
//fibonacci series upto N terms
#include<stdio.h>
```

```
long int fibo(int n)
{
    if(n == 1)
        return 0;
    else if(n == 2)
        return 1;
    else
        return(fibo(n - 1) + fibo(n - 2));
}

void main()
{
    int i, n;
    long int f;
    printf("How many terms do you want? ");
    scanf(" %d", &n);
    for (i = 1; i <= n; ++i)
    {
        f = fibo(i);
        printf("%d ", f);
    }
}
```

OUTPUT

How many terms do you want? 10

0 1 1 2 3 5 8 13 21 34

3. b. Write a program to read a one-dimensional array, sort the numbers in ascending order and display sorted numbers.

```

/* reorder a one-dimensional, integer
array from smallest to largest */
#include <stdio.h>

#define SIZE 100

void reorder(int n, int x[]); /* function prototype */

main()
{
    int i, n, x[SIZE];

    /* read in a value for n */
    printf("How many numbers will be entered? ");
    scanf(" %d", &n);
    /*read the list of numbers */
    for (i = 0; i < n; ++i)
    {
        printf("i = %d    x = ", i+1);
        scanf(" %d", &x[i]);
    }
    /* reorder all array elements */
    reorder(n, x);
    /*display the reordered list of number */
    printf("\nReordered list of numbers:\n");
    for(i = 0; i < n; ++i)
        printf("i = %d    x = %d\n", i+1, x[i]);
}

void reorder(int n, int x[]) /*rearrange the list of numbers */
{
    int i, item, temp;
    for (item = 0; item < n-1; ++item)
    {
        /* find the smallest of all remaining elements */
        for (i = item + 1; i < n; ++i)
        {
            if (x[i] < x[item])
            {
                /* interchange two elements */
                temp = x[item];
                x[item] = x[i];
                x[i] = temp;
            }
        }
    }
}

```

```

    }
    //return;
}

```

OUTPUT

How many numbers will be entered? 6

```

i = 1    x = 595
i = 2    x = 78
i = 3    x = -1505
i = 4    x = 891
i = 5    x = -29
i = 6    x = -7

```

Reordered list of numbers:

```

i = 1    x = -1505
i = 2    x = -29
i = 3    x = -7
i = 4    x = 78
i = 5    x = 595
i = 6    x = 891

```

4. a. Write a program to add two 3x3 matrices. Display the sum stored in third matrix.

```

//matrix using pointer
#include<stdio.h>
#define M 3
#define N 3
void main()
{
    float m1[M][N],m2[M][N],sum[M][N];
    int i,j;

    printf("Enter elements of First matrix\n");
    for (i = 0; i < M; i++)
    {
        for (j = 0; j < N; j++)
        {
            printf("m1[%d][%d]: ", i, j);
            scanf(" %f", &m1[i][j]);
        }
    }

    printf("Enter elements of second matrix\n");
    for (i = 0; i < M; i++)
    {
        for (j = 0; j < N; j++)
        {
            printf("m2[%d][%d]: ", i, j);
            scanf(" %f", &m2[i][j]);
        }
    }
}

```



```
//calculating sum
for (i = 0; i < M; i++)
{
    for (j = 0; j < N; j++)
    {
        sum[i][j] = m1[i][j] + m2[i][j];
    }
}
//displaying sum
for (i = 0; i < M; i++)
{
    for(j = 0; j < N; j++)
    {
        printf("%.2f\t", sum[i][j]);
    }
    printf("\n");
}
}
```

OUTPUT

Enter elements of First matrix

m1[0][0]: 9

m1[0][1]: 8

m1[0][2]: 7

m1[1][0]: 6

m1[1][1]: 5

m1[1][2]: 4

m1[2][0]: 3

m1[2][1]: 2

m1[2][2]: 1

Enter elements of second matrix

m2[0][0]: 1

m2[0][1]: 2

m2[0][2]: 3

m2[1][0]: 4

m2[1][1]: 5

m2[1][2]: 6

m2[2][0]: 7

m2[2][1]: 8

m2[2][2]: 9

10.00 10.00 10.00

10.00 10.00 10.00

10.00 10.00 10.00

4. b. List the major advantages of recursive function. Write a recursive program to generate 10 terms Fibonacci sequence starting from 2.

Advantages of Recursive function are listed below.

- The repetitive structure results in shorter codes.
- Complex case analysis and nested loops can be avoided.
- Recursion can lead to more readable and efficient algorithm descriptions.

- Recursion is very useful to solve those problems which are recursive in nature (e.g., Tower of Hanoi)

```
//fibonacci series using recursion
#include <stdio.h>
```

```
int fibo(int); //prototype
```

```
void main()
{
    int i, f;
    /* second term is 4 */
    for (i = 4; i <= 13; ++i)
    {
        f = fibo(i);
        printf("%d ", f);
    }
}
```

OUTPUT:

2 3 5 8 13 21 34 55 89 144

5. a. What are the advantages of using Dynamic Memory Allocation over Static Memory Allocation? Explain with suitable example program.

When main memory is allocated statically it cannot be altered during the execution of program. When main memory is allocated dynamically it can be altered during the execution of program as per the user wish. The length of dynamically allocated memory either can be decreased or increased. This is the major advantage of dynamic memory allocation over static memory allocation.

Suppose user wishes to enter elements of an array or a list one by one just after allocating memory and he may stop at any point then definitely it is not possible by means of static memory allocation. So, the dynamic memory allocation certainly has an advantage in this case over static memory allocation. Such dynamically created list is called as linked list.

In case of dynamically created lists insertions and deletions can be done very easily just by the manipulation of addresses whereas in case of statically allocated memory insertions and deletions lead to more number of movements and wastage of memory.

In case of statically allocated memory there is every chance of “overflow” during insertions in the lists, whereas in case of dynamically allocated memory it does not come into picture unless otherwise unavailability of main memory.

For some liner data structures like STACK and QUEUE, dynamic memory allocation proves very much appropriate because the length of such data structure is not fixed. They may increase or decrease dynamically that is during the execution of program. Similarly, non-linear data structures need recursive definitions of “struct” data type, so the need of dynamic memory allocation.

In summary:

Advantages of Dynamic memory allocation

- Data structures can grow and shrink according to the requirement.
- We can allocate (create) additional storage whenever we need them.
- We can de-allocate (free/delete) dynamic space whenever we are done with them.

- Dynamic Allocation is done at run time.

For example,

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int n, i, *p;
    printf("Enter number of elements: ");
    scanf(" %d", &n);
    p = (int *)malloc(n * sizeof(int)); //memory allocated using
malloc
    if (p == NULL)
    {
        printf("Memory cannot be allocated\n");
    }
    else
    {
        printf("Enter elements of array:\n");
        for (i = 0; i < n; ++i)
            scanf(" %d", (p + i));

        printf("Elements of array are\n");
        for (i = 0; i < n; i++)
            printf("%d\n", *(p + i));
    }
    return 0;
}
```

In this example, the memory required for n integers is allocated dynamically using malloc function.

5. b. How can a function return multiple values? Explain with example.

Since external variables are recognized globally, they can be accessed from any function that falls within their scope. They retain their assigned values within this scope. Therefore, an external variable can be assigned a value within one function, and this value can be used (by accessing the external variable) within another function.

The use of external variables provides a convenient mechanism for **transferring information back and forth between functions**. We can transfer information into a function without using arguments. This is especially convenient when a function requires numerous input data items. Moreover, we now have a way to transfer multiple data items out of a function, since the **return** statement can return only one data item.

```
/* read several lines of text and determine the average number of
characters per line */
#include <stdio.h>

/* global variable */
int sum = 0; /* total number of character */
int lines = 0; /*total number of lines */
```

```

int linecount(void);

main()
{
    int n;          /* no of chars in given line */
    float avg; /*average number of chars per line */
    printf("Enter the text below:\n");
    /* read a line of text and update the cumulative counters */
    while ((n = linecount()) > 0)
    {
        sum += n;
        ++lines;
    }
    avg = (float) sum / lines;
    printf("Average number of characters per line: %5.2f", avg);
}
/* read a line of text and count the number of characters */
int linecount(void)
{
    char line[80];
    int count = 0;

    while ((line[count] = getchar()) != '\n')
        ++count;
    return count;
}

```

OUTPUT

Enter the text below:

Please Help us to Keep the
Book Clean and Moving

Average number of characters per line: **23.50**

When an array is passed to a function, however, the values of the array elements *are not* passed to the function. Rather, the array name is interpreted as the **address** of the first array element (i.e., the address of the memory location containing the first array element). This address is assigned to the corresponding formal argument when the function is called. The formal argument therefore becomes a **pointer** to the first array element (more about this in the next chapter). Arguments that are passed in this manner are said to be passed **by reference** rather than passed *by value*.

When a reference is made to an array element within the function, the value of the element's subscript is added to the value of the pointer to indicate the address of the specified array element. Therefore, any array element can be accessed from within the function. Moreover, *if an array element is altered within the function, the alteration will be recognized in the calling portion of the program* (actually, throughout the entire scope of the array).

Here is a simple C program that passes a three-element integer array to a function, where the array elements are altered. The values of the array elements are displayed at three different places in the program, thus illustrating the effects of the alterations.

```

#include <stdio.h>
void modify(int a[]); /* function prototype */

main()

```

```

{
    int count, a[3];

    printf("\nEnter any three numbers:\n");
    for (count = 0; count < 3; ++count)
    {
        printf("\na[%d] = ", count);
        scanf(" %d", &a[count]);
    }

    printf("From main(), before modification:\n\n");
    for (count = 0; count < 3; ++count)
    {
        printf("\na[%d] = %d", count, a[count]);
    }

    modify(a); /* function call */

    printf("From main(), after modification:\n");
    for (count = 0; count < 3; ++count)
    {
        printf("\na[%d] = %d", count, a[count]);
    }
}

void modify(int a[]) /* function definition */
{
    int count;

    printf("\nFrom the function, after modification the values:\n");
    for (count = 0; count < 3; ++count)
    {
        a[count] = -9;
        printf("a[%d] = %d\n", count, a[count]);
    }
    //return;
}

```

The array elements are read from the user within the first loop appearing in *main*. These values are displayed as soon as they are assigned. The array is then passed to the function **modify**, when each array element is assigned the value **-9**. These new values are then displayed from within the function. Finally, the values of array elements are again displayed from **main()**, after control has been transferred back to *main* from *modify*. When the program is executed, the following output is generated.

```

Enter any three numbers:
a[0] = 1
a[1] = 2
a[2] = 3
From main(), before modification:

a[0] = 1
a[1] = 2
a[2] = 3

```

From the function, after modification the values:

```
a[0] = -9
```

```
a[1] = -9
```

```
a[2] = -9
```

From main(), after modification:

```
a[0] = -9
```

```
a[1] = -9
```

```
a[2] = -9
```

These results show that the elements of *a* are altered within *main* as a result of the changes that were made within *modify*.

6. a. Write a program to sort *N* numbers in array dynamically.

```
/* reordering a one-dimensional, integer array from
smallest to largest, using pointer notation */
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
void reorder(int n, int *x);
```

```
main()
```

```
{
```

```
    int i, n, *x;
```

```
    /* read in a value for n */
```

```
    printf("\nHow many numbers will be entered? ");
```

```
    scanf(" %d", &n);
```

```
    /* dynamically allocate memory for n numbers */
```

```
    x = (int *) malloc(n * sizeof(int));
```

```
    /*read in the list of numbers */
```

```
    for (i = 0; i < n; ++i)
```

```
    {
```

```
        printf("i = %d x = ", i + 1);
```

```
        scanf(" %d", x + i);
```

```
    }
```

```
    /* reorder all array elements */
```

```
    reorder(n, x);
```

```
    /* display the reordered list of numbers */
```

```
    printf("\nReordered list of numbers:\n");
```

```
    for (i = 0; i < n; ++i)
```

```
        printf("i = %d x = %d\n", i+1, *(x + i));
```

```
    free(x);
```

```
}
```

```
void reorder(int n, int *x)
```

```
{
```

```

int i, item, temp;
for (item = 0; item < n - 1; ++item)
    for (i = item + 1; i < n; ++i)
        if(*(x + i) < *(x + item))
        {
            temp = *(x + item);
            *(x + item) = *(x + i);
            *(x + i) = temp;
        }
return;
}

```

OUTPUT

How many numbers will be entered? 10

```

i = 1    x = 20
i = 2    x = 30
i = 3    x = 10
i = 4    x = 5
i = 5    x = -15
i = 6    x = 15
i = 7    x = 25
i = 8    x = 35
i = 9    x = 22
i = 10   x = 40

```

Reordered list of numbers:

```

i = 1    x = -15
i = 2    x = 5
i = 3    x = 10
i = 4    x = 15
i = 5    x = 20
i = 6    x = 22
i = 7    x = 25
i = 8    x = 30
i = 9    x = 35
i = 10   x = 40

```

6. b. What is significance of file pointer on file handling? Consider the following structure:

Roll no.	Name	Address	Faculty	Date of Birth		
				mm	dd	yy

Write a program to create 'Student.txt' file to store the above records for 100 students.

Also, display records of these students who are not from 'Kathmandu'.

A file pointer is a pointer to a structure, which contains information about the file, including its name, current position of the file, whether the file is being read or written, and whether errors or end of the file have occurred. The user does not need to know the details, because the definitions obtained from *stdio.h* include a structure declaration called *FILE*. The only declaration needed for a file pointer is symbolized by

FILE *fp;

This says that **fp** is the file pointer that points to a *FILE* structure.

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define N 100

typedef struct
{
    int mm;
    int dd;
    int yy;
} date;

typedef struct
{
    int roll;
    char name[30];
    char add[80];
    char fac[30];
    date dob;
} student;

main()
{
    student s;
    int i;
    FILE *fp;
    if ((fp = fopen("Student.txt", "w+")) == NULL)
    {
        printf("File cannot be created.");
        exit(1);
    }
    printf("Enter Student Information\n");
    for (i = 0; i < N; ++i)
    {
        printf("Student no. %d\n", i + 1);
        printf("Roll Number: ");
        scanf(" %d", &s.roll);
        printf("Name: ");
        scanf(" %[^\\n]", s.name);
        printf("Address: ");
        scanf(" %[^\\n]", s.add);
        printf("Faculty: ");
        scanf(" %[^\\n]", s.fac);
        printf("Date of birth (mm-dd-yy): ");
        scanf(" %d %d %d", &s.dob.mm, &s.dob.dd, &s.dob.yy);
        fwrite(&s, sizeof(student), 1, fp);
    }
}

```



```

rewind(fp);

printf("Records of students who are not from
Kathmandu\n");
for (i = 0; i < N; ++i)
{
    fread(&s, sizeof(student), 1, fp);
    if ((strcmp(s.add, "Kathmandu")) != 0)
    {
        printf("\nRoll Number: %d", s.roll);
        printf("\nName: %s", s.name);
        printf("\nAddress: %s", s.add);
        printf("\nFaculty: %s", s.fac);
        printf("\nDate of Birth(mm-dd-yy):%d%d%d",
s.dob.mm, s.dob.dd, s.dob.yy);

    }
}
}

```

7. Write Short notes on:

a. Documentation

The same workforce that developed the program may have left the project and gone in search of opportunities. So, a program must be well documented for the new people to understand how the software was developed and how can it be modified. The documentation starts from the very beginning of the problem formulation to the very end of the Evaluation and Maintenance. Program documentation is the systematic writing of activities from problem analysis to debugging/testing phase including operational information and presenting it as a report. There are two types of documentations: Technical documentation for programmer and user manual for user.

Technical documentation is prepared for future reference to the programmers who maintain, redesign and upgrade the system. It may contain detailed requirements of program, format and reports and forms, logic of the program, data flow diagram (DFD), Entity-Relationship diagram (ERD), algorithm and flowchart, input and output data used for testing and other references.

The user manual supports to the user of the program. This provides instructions for installation of the program and use of it effectively.

b. Void Pointer

A void pointer is a pointer that has no associated data type with it. A void pointer can hold address of any type and can be typecast to any type.

```

int a = 10;
char b = 'x';

void *p = &a; // void pointer holds address of int 'a'
p = &b; // void pointer holds address of char 'b'

```

The *malloc()* and *calloc()* return **void *** type and this allows these functions to be used to allocate memory of any data type (just because of void *)

```

int *x;
x = (int *)malloc(sizeof(int) * n);

```

The void pointers cannot be *dereferenced*. For example, the following program doesn't compile.

```
#include<stdio.h>
int main()
{
    int a = 10;
    void *ptr = &a;
    printf("%d", *ptr);
    return 0;
}
```

Output:

Compiler Error: 'void*' is not a pointer-to-object type

The following program compiles and runs fine.

```
#include<stdio.h>
int main()
{
    int a = 10;
    void *ptr = &a;
    printf("%d", *(int *)ptr);
    return 0;
}
```

Output:

10

c. Generation of Computers

Introduction

A computer is an electronic device that manipulates information or data. It has the ability to store, retrieve, and process data. Nowadays, a computer can be used to type documents, send email, play games, and browse the Web. It can also be used to edit or create spreadsheets, presentations, and even videos. But the evolution of this complex system started around 1940 with the first Generation of Computer and evolving ever since.

There are five generations of computers.

First Generation

Introduction:

- 1946-1959 is the period of first-generation computer.
- J.P.Eckert and J.W.Mauchy invented the first successful electronic computer called ENIAC, ENIAC stands for “Electronic Numeric Integrated And Calculator”.

Few Examples are:

ENIAC, EDVAC, UNIVAC, IBM-701, IBM-650, and so on.

Advantages:

- It made use of vacuum tubes which are the only electronic component available during those days.
- These computers could calculate in milliseconds.

Disadvantages:

- These were very big in size; weight was about 30 tones.
- These computers were based on vacuum tubes.
- These computers were very costly.
- It could store only a small amount of information due to the presence of magnetic drums.
- As the invention of first-generation computers involves vacuum tubes, so another disadvantage of these computers was, vacuum tubes require a large cooling system.
- Very less work efficiency.
- Limited programming capabilities and punch cards were used to take inputs.
- Large amount of energy consumption.
- Not reliable and constant maintenance is required.

Second Generation**Introduction:**

- 1959-1965 is the period of second-generation computer.
- Second generation computers were based on Transistor instead of vacuum tubes.

Few Examples are:

Honeywell 400, IBM 7094, CDC 1604, CDC 3600, UNIVAC 1108, many more.

Advantages:

- Due to the presence of transistors instead of vacuum tubes, the size of electron component decreased. This resulted in reducing the size of a computer as compared to first generation computers.
- Less energy and not produce as much heat as the first generation.
- Assembly language and punch cards were used for input.
- Low cost than first generation computers.
- Better speed calculate data in microseconds.
- Better portability as compared to first generation

Disadvantages:

- A cooling system was required.
- Constant maintenance was required.
- Only used for specific purposes.

Third Generation**Introduction:**

- 1965-1971 is the period of third generation computer.
- These computers were based on Integrated circuits.
- IC was invented by Robert Noyce and Jack Kilby In 1958-1959.
- IC was a single component containing number of transistors.

Few Examples are:

PDP-8, PDP-11, ICL 2900, IBM 360, IBM 370, and many more

Advantages:

- These computers were cheaper as compared to second-generation computers.
- They were fast and reliable.
- Use of IC in the computer provides the small size of the computer.
- IC not only reduce the size of the computer, but it also improves the performance of the computer as compared to previous computers.
- This generation of computers has big storage capacity.
- Instead of punch cards, mouse and keyboard are used for input.
- They used an operating system for better resource management and used the concept of time-sharing and multiple programming.
- These computers reduce the computational time from microseconds to nanoseconds.

Disadvantages:

- IC chips are difficult to maintain.
- The highly sophisticated technology required for the manufacturing of IC chips.
- Air conditioning is required.

Fourth Generation**Introduction:**

- 1971-1980 is the period of fourth generation computer.
- This technology is based on Microprocessor.
- A microprocessor is used in a computer for any logical and arithmetic function to be performed in any program.
- Graphics User Interface (GUI) technology was exploited to offer more comfort to users.

Few Examples are:

IBM 4341, DEC 10, STAR 1000, PUP 11 and many more.

Advantages:

- Fastest in computation and size get reduced as compared to the previous generation of computer.
- Heat generated is negligible.
- Small in size as compared to previous generation computers.
- Less maintenance is required.
- All types of high-level language can be used in this type of computers.

Disadvantages:

- The Microprocessor design and fabrication are very complex.
- Air conditioning is required in many cases due to the presence of ICs.
- Advance technology is required to make the ICs.

Fifth Generation**Introduction:**

- The period of the fifth generation in 1980-onwards.
- This generation is based on artificial intelligence.
- The aim of the fifth generation is to make a device which could respond to natural language input and are capable of learning and self-organization.
- This generation is based on ULSI(Ultra Large-Scale Integration) technology resulting in the production of microprocessor chips having ten million electronic components.

Few Examples are:

Desktop, Laptop, Notebook, Ultrabook, Chromebook and many more.

Advantages:

- It is more reliable and works faster.
- It is available in different sizes and unique features.
- It provides computers with more user-friendly interfaces with multimedia features.

Disadvantages:

- They need very low-level languages.
- They may make the human brains dull and doomed.

2018 Fall

1.a. What is computer software? Explain different types of software used in today's life.

Software is the collection of computer programs, procedures and documentation that performs different tasks on a computer system. The types of computer software are system software and application software.

1. System Software

System software is the computer software designed to operate the computer hardware and to provide a platform for running application software. System software is the collection of operating system, device drivers, servers, windowing systems and utilities. Some Common types of system software are given below:

v. Operating system

The OS is the most important program that runs on a computer. Every general-purpose computer must have an operating system. Some functions of OS are:

- Recognize input from keyboard, send output to display screen.
- Keep track of files and directories on the disk.
- Controlling peripheral devices such as disk drives and printers.
- Provides interface between hardware and software.
- Provides security and backup, memory management, scheduling processes etc.

Some examples of OS are: Windows 2000, Mac-OS, UNIX, DOS etc.

vi. Utility Software

Utility software helps to manage, maintain and control computer resources. These programs are available to help you with the day-to-day chores associated with personal computing and to keep your system running at peak performance. Some examples of utility programs include:

Antivirus: designed to protect your computer from viruses.

Backup Software: designed to assist you in backing up your files and even the entire computer hard drives.

Disk Defragmentation Software: designed to assist you in keeping reorganized your disk drives. After files are saved, deleted and resaved again, the disk becomes fragmented. Disk defragmenters gather those free spaces and put them together to enable you to continue to save your data in the most efficient manner.

Language Processor: used to translate program to machine level from other languages. The types of language translators are *compiler, interpreter and assembler*.

2. Application Software

Application software enable the users to accomplish specific tasks. Business software, databases and educational software are some application software. Other application software is word processing, spreadsheet, presentation packages etc. Mainly, there are two types of application software: *package software* and *tailored software*.

v. **Package Software**

They are mainly designed by software companies to generalize the tasks. Some common types of package software are:

Word Processing Software: enables the users to create and edit documents. The most popular word processors are MS Word, Word-Pad, Notepad and some other text editors.

Database Software: allows the users to store and retrieve data from database. The Oracle, MS Access, MySQL, SQL etc. are database software.

Spreadsheet Software: allows users to perform different kinds of numeric calculations. E.g. MS Excel, Lotus, Apple numbers etc.

Multimedia Software: allows the users to create and play with picture, audio, video media. E.g. Media Player, Real Player etc.

Presentation Software: allows users to present their information in the form of slides. E.g. MS PowerPoint etc.

vi. **Tailored Software**

It is specific purpose software which is developed for specific task. They are mainly written in high level language such as Java, C, C++, COBOL. E.g. banking software, hospital software, hotel reservation software, billing software etc.

1.b. Write an algorithm and draw flowchart to generate Fibonacci sequence of eight terms.

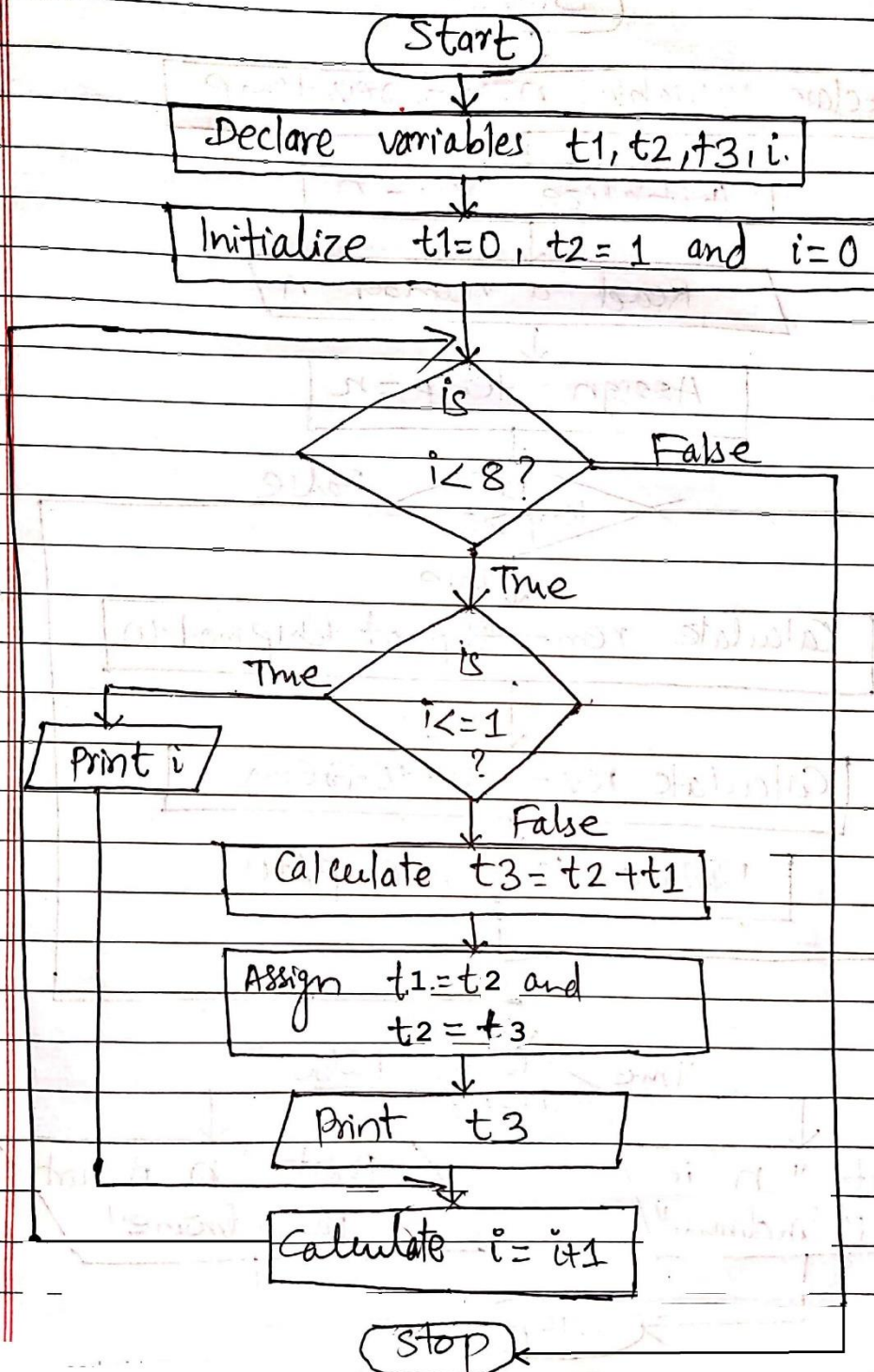
Algorithm

1. Start
2. Declare variables t1, t2, t3, i.
3. Initialize t1 = 0, t2 = 1 and i = 0.
4. If i < 8 then repeat steps 5 to 9. Otherwise goto step 10.
5. If i <= 1 then print i and goto step 9. Else goto next step.
6. Calculate t3 = t1 + t2.
7. Assign t1 = t2 and t2 = t3.
8. Print t3.
9. Calculate i = i + 1.

10. Stop.

Flowchart

Flowchart to print 8 terms of Fibonacci



2.a. What are different data types available in C? Explain their type qualifiers, conversion characters, range of value and storage size in memory occupied by each type.

The data types available in C are summarized in the following table.

Data Type	Category	Range	Memory (Bytes)	Format Specifier
integer	short signed	-2^{15} to $2^{15} - 1$	2	%d or %i
	short unsigned	0 to $2^{16} - 1$	2	%u
	signed	-2^{15} to $2^{15} - 1$	2	%d
	unsigned	0 to $2^{16} - 1$	2	%u
	long signed	-2^{31} to $2^{31} - 1$	4	%ld
	long unsigned	0 to $2^{32} - 1$	4	%lu
	hexadecimal			%x
	Octal			%o
float	float	1.2×10^{-38} to $3.4 \times 10^{+38}$	4	%f
double	double	1.7×10^{-308} to $3.4 \times 10^{+308}$	8	%lf
	long double	3.4×10^{-4932} to $1.1 \times 10^{+4932}$	10	%Lf
char	signed char	-128 to +127	1	%c
	unsigned char	0 to 255	1	%c
void	void	n/a	n/a	n/a

2.b. Differentiate between break and continue statements with a suitable example.

The Break Statement

The **break** statement is used to *terminate loops* or to *exit from a switch*. It can be used within a *for*, *while*, *do - while*, or *switch* statement. The break statement is written simply as

break;

without any embedded expressions or statements.

If a break statement is included in a *while*, *do - while* or *for* loop, then control will immediately be transferred out of the loop when the break statement is encountered. This provides a convenient way to terminate the loop if an error or other irregular condition is detected.

Here are some illustrations of loops that contain **break** statements. In each situation, the loop will continue to execute as long as the current value for the floating-point variable **x** does not exceed **100**. However, the computation will break out of the loop if a negative value for **x** is detected.

First, consider a **while** loop.

```
scanf(" %f", &x); //initialize
while (x <= 100)
{
    if (x < 0)
    {
        printf("ERROR! NEGATIVE VALUE FOR x");
        break;
    }
    /* process the non-negative value of x */
    ...
    scanf(" %f", &x); //update
}
```

Now consider a **do - while** loop that does the same thing.


```

do
{
    scanf(" %f", &x);
    if (x < 0)
    {
        printf("ERROR! NEGATIVE VALUE FOR x");
        break;
    }
    /* process the non-negative value of x */
    ...
}while (x <= 100);

```

The **continue** Statement

The *continue* statement is used to *bypass* the remainder of the current pass through a loop. The loop does not terminate when a *continue* statement is encountered. Rather, the remaining loop statements are skipped and the computation proceeds directly to the next pass through the loop. (Note the distinction between *continue* and *break*.)

The *continue* statement can be included within a *while*, a *do - while* or a *for* statement. It is written simply as

continue;

without any embedded statements or expressions.

Here are some illustrations of loops that contain *continue* statements. First, consider a *do - while* loop.

```

do
{
    scanf(" %f", &x);
    if (x < 0)
    {
        printf("ERROR! NEGATIVE VALUE FOR x");
        continue;
    }
    /* process the non-negative value of x */
    ...
}while (x <= 100);

```

Here is a similar *for* loop

```

for (count = 1; x <= 100; ++count)
{
    scanf(" %f", &x);
    if (x < 0)
    {
        printf("ERROR! NEGATIVE VALUE FOR x");
        continue;
    }
    /* process the non-negative value of x */
    ...
}

```

In each case, the processing of the current value of **x** will be bypassed if the value of **x** is negative. Execution of the loop will then continue with the next pass.

3.a. What do you mean by function? Differentiate between function call by value and function call by reference with suitable program.

A **function** is a self-contained program segment that carries out some specific, well-defined task. Every C program consists of one or more functions. One of these functions must be called **main**. Execution of the program will always begin by carrying out the instructions in **main**. Additional functions will be subordinate to **main**, and perhaps to one another.

If a program contains multiple functions, their definitions may appear in any order, though they must be independent of one another. That is, one function definition cannot be embedded within another.

Function Call by Value

When a single value is passed to a function via an actual argument, the value of the actual argument is *copied* into the function. Therefore, *the value of the corresponding formal argument can be altered within the Function, but the value of the actual argument within the calling routine will not change*. This procedure for passing the value of an argument to a function is known as *passing by value*.

Here is a simple C program containing a function that alters the value of its argument.

```
#include <stdio.h>

void modify(int a);

main()
{
    int a = 2;

    printf("\na = %d (from main, before calling the
function)", a);
    modify(a); // function call a is a normal variable
    printf("\na = %d (from main, after calling the
function)", a);
}

void modify(int a)
{
    a *= 3;
    printf("\na = %d (from the function, after being
modified)", a);
    return;
}
```

The original value of **a** (i.e., **a = 2**) is displayed when **main** begins execution. This value is then passed to the function **modify**, where it is multiplied by **3** and the new value displayed. Note that it is the altered value of the formal argument that is displayed within the function. Finally, the value of **a** within **main** (i.e., the actual argument) is again displayed, after control is transferred back to **main** from **modify**. When the program is executed, the following output is generated.

```
a = 2 (from main, before calling the function)
a = 6 (from the function, after being modified)
a = 2 (from main, after calling the function)
```

These results show that **a** is not altered within main, even though the corresponding value of **a** is changed within **modify()**.

Passing an argument by value has advantages and disadvantages. On the plus side, it allows a single-valued actual argument to be written as an expression rather than being restricted to a single variable. Moreover, if the actual argument is expressed simply as a single variable, it protects the value of this variable from alterations within the function. On the other hand, it does not allow information to be transferred back to the calling portion of the program via arguments. Thus, *passing by value is restricted to a one-way transfer of information.*

Function Call by Reference

To pass an array to a function, the array name must appear by itself, without brackets or subscripts, as an actual argument within the function call. The corresponding formal argument is written in the same manner, though it must be declared as an array within the formal argument declarations. When declaring a one-dimensional array as a formal argument, the array name is written with a pair of empty square brackets. The size of the array is *not specified* within the formal argument declaration.

When an array is passed to a function, however, the values of the array elements *are not* passed to the function. Rather, the array name is interpreted as the **address** of the first array element (i.e., the address of the memory location containing the first array element). This address is assigned to the corresponding formal argument when the function is called. The formal argument therefore becomes a **pointer** to the first array element (more about this in the next chapter). Arguments that are passed in this manner are said to be passed **by reference** rather than passed *by value*.

When a reference is made to an array element within the function, the value of the element's subscript is added to the value of the pointer to indicate the address of the specified array element. Therefore, any array element can be accessed from within the function. Moreover, *if an array element is altered within the function, the alteration will be recognized in the calling portion of the program* (actually, throughout the entire scope of the array).

Here is a simple C program that passes a three-element integer array to a function, where the array elements are altered. The values of the array elements are displayed at three different places in the program, thus illustrating the effects of the alterations.

```
#include <stdio.h>
void modify(int a[]); /* function prototype */

main()
{
    int count, a[3];

    printf("\nEnter any three numbers:\n");
    for (count = 0; count < 3; ++count)
    {
        printf("\na[%d] = ", count);
        scanf(" %d", &a[count]);
    }

    printf("From main(), before modification:\n");
```

```

for (count = 0; count < 3; ++count)
{
    printf("\na[%d] = %d", count, a[count]);
}

modify(a); /* function call a is an array*/

printf("From main(), after modification:\n");
for (count = 0; count < 3; ++count)
{
    printf("\na[%d] = %d", count, a[count]);
}
}

void modify(int a[])    /* function definition */
{
    int count;

    printf("\nFrom the function, after modification the
values:\n");
    for (count = 0; count < 3; ++count)
    {
        a[count] = -9;
        printf("a[%d] = %d\n", count, a[count]);
    }
    //return;
}

```

The array elements are read from the user within the first loop appearing in *main*. These values are displayed as soon as they are assigned. The array is then passed to the function **modify**, when each array element is assigned the value **-9**. These new values are then displayed from within the function. Finally, the values of array elements are again displayed from **main()**, after control has been transferred back to *main* from *modify*. When the program is executed, the following output is generated.

```

Enter any three numbers:
a[0] = 1
a[1] = 2
a[2] = 3
From main(), before modification:
a[0] = 1
a[1] = 2
a[2] = 3
From the function, after modification the values:
a[0] = -9
a[1] = -9
a[2] = -9
From main(), after modification:
a[0] = -9
a[1] = -9

```

a[2] = -9

These results show that the elements of **a** are altered within *main* as a result of the changes that were made within *modify*.

3.b. Write a menu-driven program to work following cases; take appropriate input whenever required.

i. reverse number

ii. find sum of individual digits

iii. check for prime

iv. exit

```
/*to display the following menu:
    i. reverse number
    ii. find sum of individual digits
    iii. check for prime
    iv. exit
*/
#include <stdio.h>
#include <stdlib.h>

main()
{
    int choice, n, i, sum, rev, temp, rem, isprime;
    do
    {
        printf("\n-----Menu-----\n");
        printf("1.Reverse Number\n2.Find sum of individual
digits\n3.Check for Prime\n4.Exit from program.");
        printf("\nEnter your choice: ");
        scanf(" %d", &choice);
        switch(choice)
        {
            case 1:
                printf("Enter a number: ");
                scanf(" %d", &n);
                while ( n != 0)
                {
                    rem = n % 10;
                    rev = rev * 10 + rem;
                    n /= 10;
                }
                printf("Reverse number: %d", rev);
                break;
            case 2:
                printf("\nEnter a number: ");
                scanf(" %d", &n);
                sum = 0;
                while (n != 0)
                {
                    rem = n % 10;
```

```

        sum += rem;
        n /= 10;
    }
    printf("Sum of digits: %d", sum);
    break;
case 3:
    printf("\nEnter a number: ");
    scanf("%d", &n);
    isprime = 1;
    for (i = 2; i < n; ++i)
    {
        if (n % i == 0)
        {
            isprime = 0;
            break;
        }
    }
    if (isprime == 1)
        printf("%d is Prime.", n);
    else
        printf("%d is not Prime.", n);
    break;
case 4:
    printf("Exiting...");
    break;
default:
    printf("Invalid choice.");
}
} while (choice != 4);
}

```

OUTPUT

-----Menu-----

```

1.Reverse Number
2.Find sum of individual digits
3.Check for Prime
4.Exit from program
Enter your choice: 1
Enter a number: 123
Reverse number: 321

```

-----Menu-----

```

1.Reverse Number
2.Find sum of individual digits
3.Check for Prime
4.Exit from program
Enter your choice: 2

```

```

Enter a number: 91532
Sum of digits: 20

```

-----Menu-----

1.Reverse Number
2.Find sum of individual digits
3.Check for Prime
4.Exit from program
Enter your choice: 3

Enter a number: 9
9 is not Prime.

-----Menu-----

1.Reverse Number
2.Find sum of individual digits
3.Check for Prime
4.Exit from program
Enter your choice: 3

Enter a number: 17
17 is Prime.

-----Menu-----

1.Reverse Number
2.Find sum of individual digits
3.Check for Prime
4.Exit from program
Enter your choice: 4
Exiting...

4.a. What is dynamic memory allocation? What are different functions used in DMA?

Since an array name is a pointer to the first element within the array, it should be possible to define the array as a pointer variable rather than as a conventional array. Syntactically, the two definitions are equivalent. However, a conventional array definition results in a fixed block of memory being reserved at the beginning of program execution, whereas this does not occur if the array is represented in terms of a pointer variable. Therefore, the use of a pointer variable to represent an array requires some type of initial memory assignment before the array elements are processed. This is known as **dynamic memory allocation**.

Mainly, there are four functions used in DMA. They are as follows:

a. malloc()

Syntax:

```
malloc (number * sizeof(data_type));
```

- The malloc () function is used to allocate space in memory during the execution of the program.
- It does not initialize the memory allocated during execution. It carries *garbage* value.
- It returns *null pointer* if it couldn't able to allocate requested amount of memory.
- It allocates only *single block* of requested memory
- Type cast must be done since this function returns *void pointer*.

```
int *ptr;  
ptr = (int *) malloc(20 * sizeof(int));
```

b. calloc()

Syntax:

```
calloc(number, sizeof(data_type));
```

- It allocates *multiple blocks* of requested memory.
- The `calloc()` initializes the allocated memory to *zero*.
- Type *cast* must be done since this function returns *void pointer*.

```
int *ptr;
ptr = (int *) calloc(20, 20 * sizeof(int));
```

c. `free()`

Syntax:

```
free (pointer_name);
```

The `free ()` function *frees* the allocated memory by `malloc()`, `calloc()`, `realloc()` functions and returns the memory to the system.

```
free (ptr);
```

d. `realloc()`

```
realloc (pointer_name, number * sizeof(data_type));
```

- The `realloc()` function modifies the allocated memory size by `malloc()` and `calloc()` functions to *new size*.
- If enough space doesn't exist in memory of current block to extend, new block is allocated for the full size of reallocation, then copies the existing data to new block and then frees the old block.

4.b. Describe string. Explain any three string handling functions with examples.

A string can be represented as a one-dimensional character-type array. Each character within the string will be stored within one element of the array. Some problems require that the characters within a string be processed individually. Since the array is one-dimensional, there will be a single *subscript* (sometimes called an *index*) whose value refers to individual array elements. If the array contains *n* elements, the subscript will be an integer quantity whose values range from 0 to *n*-1. Note that an *n*-character string will require an **(*n*+1)**-element array, because of the *null* character (`\0`) that is automatically placed at the end of the string.

Suppose that the string "Kathmandu" is to be stored in a one-dimensional character array called *letter*. Since "Kathmandu" contains 9 characters, *letter* will be a 10-element array. Thus, *letter*[0] will represent *K*, *letter*[1] represents *a*, and so on. Note that the last (10th) array element, *letter*[9], represents the *null character* (`\0`) which signifies the end of the string (*letter*).

K	a	T	H	m	a	n	d	u	\0
0	1	2	3	4	5	6	7	8	9

String Handling Functions

Here are some string handling functions which are used to manipulate strings on C. These functions are defined at header file ***string.h***.

i. `strlen()`

It returns the length of string parameter. Syntax:

```
len = strlen(str);
```

Here *str* is string variable or string constant and *len* is an integer variable which holds the length returned by *strlen()*. E.g.:

```
len = strlen("Nepal");
```


The *len* holds 5 which is length of string constant “Nepal”.

ii. **strlwr()**

It converts each uppercase character into lowercase character presented in given string. Syntax:

```
strlwr(str);
```

Here *str* is string variable or string constant. The new string (after conversion) is stored at the same variable. E.g.:

```
strlwr("Nepal");
```

It returns string “nepal”.

```
char str[] = "Nepal";
```

```
strlwr(str);
```

Then new string at *str* is “nepal”.

iii. **strupr()**

It converts each lowercase character into uppercase character presented in given string. Syntax:

```
strupr(str);
```

Here *str* is string variable or string constant. The new string (after conversion) is stored at the same variable. E.g.:

```
strupr("Nepal");
```

It returns string “NEPAL”.

```
char str[] = "Nepal";
```

```
strupr(str);
```

Then new string at *str* is “NEPAL”.

5.a. Why array is important in programming? How can you initialize different types of array? Explain 2-Dimensional array in C.

Array is very useful property of C programming. The following reasons emphasize the importance of array.

- **Saves memory**

Memory can be allocated dynamically in an array. This advantage of array helps to save the memory of the system. It also helps when the pre-defined array has insufficient memory. At runtime memory can be allocated manually during run time. Also when memory allocation is not dynamic it stored the data in contiguous memory locations. The amount of storage required depends on the data type or size.

- **Cache Friendly**

In an array, values are near each other in memory. They can be accessed easily from CPU to cache. This concludes that iteration over an array is much faster than any other iteration. It parses the array an array of reference may be an advantage here.

- **More compact in memory usage**

An array requires memory space only for the values, the start address and its length. On the contrary, a linked list needs a pointer for every value which is inserted. It acquires memory for every address and also when extra data is inserted it also needs memory for the same. Hash table also needs memory depending on how it is implemented. This implementation decides how memory is allocated and usually, it requires extra allocation.

- **Advantage over variables**

An array is considered to be a homogenous collection of data. Here the word collection means that it helps in storing multiple values which are under the same variable. For any purpose, if the user wishes to store multiple values of similar type, an array is the best option which can be used. As a result for any purpose if a user wishes to store multiple values of a similar type then arrays can be used and utilized efficiently.

- **Multi- dimensional array**

These can be defined as an array of arrays. The data, which is present in tabular format like 1D, 2D, etc. can be defined. The total number of elements can be stored in the multi-dimensional array and can be calculated by multiplying the size of all dimensions.

Initialization of one-dimensional array

The initial values must appear in the order in which they will be assigned to the individual array elements, enclosed in braces and separated by commas. The general form is

```
storage-class data-type array[expression] = {value 1, value 2, ..., value n};
```

where **value 1** refers to the value of the first array element, **value 2** refers to the value of the second element, and so on. The appearance of the **expression**, which indicates the number of array elements, is *optional* when initial values are present.

Shown below are several array definitions that include the assignment of initial values.

```
int digits[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
static float x[6] = {0, 0.25, 0, -0.50, 0, 0};
char color[3] = {'R', 'E', 'D', };
```

Note that **x** is a static array. The other two arrays (**digits** and **color**) are assumed to be external arrays by virtue of their placement within the program. The results of these initial assignments, in terms of the individual array elements, are as follows. (Remember that the subscripts in an n-element array range from **0** to **n - 1**.)

digits[0] = 1	x[0] = 0	color[0] = 'R'
digits[1] = 2	x[1] = 0.25	color[1] = 'E'
digits[2] = 3	x[2] = 0	color[2] = 'D'
digits[3] = 4	x[3] = -0.50	
digits[4] = 5	x[4] = 0	
digits[5] = 6	x[5] = 0	
digits[6] = 7		
digits[7] = 8		
digits[8] = 9		
digits[9] = 10		

All individual array elements that are not assigned explicit initial values will automatically be set to **zero**. This includes the remaining elements of an array in which some elements have been assigned nonzero values.

Initialization of two-dimensional array

Consider the following two-dimensional array definition.

```
int values[3][4] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12};
```

Note that **values** can be thought of as a *table* having **3** rows and **4** columns (**4** elements per row). Since the initial values are assigned by rows (i.e., last subscript increasing most rapidly), the results of this initial assignment are as follows.

```
values[0][0] = 1  values[0][1] = 2  values[0][2] = 3  values[0][3] = 4
values[1][0] = 5  values[1][1] = 6  values[1][2] = 7  values[1][3] = 8
values[2][0] = 9  values[2][1] = 10 values[2][2] = 11 values[2][3] = 12
```

Here is a variation of the two-dimensional array definition presented in the last example.

```
int values[3][4] = {
    {1, 2, 3, 4},
    {5, 6, 7, 8},
    {9, 10, 11, 12}
};
```

This definition results in the same initial assignments as in the last example. Thus, the four values in the first inner pair of braces are assigned to the array elements in the first row, the values in the second inner pair of braces are assigned to the array elements in the second row, etc. Note that an outer pair of braces is required, containing the inner pairs.

Now consider the following two-dimensional array definition.

```
int values[3][4] = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};
```

This definition assigns values only to the first three elements in each row. Therefore, the array elements will have the following initial values.

```
values[0][0] = 1  values[0][1] = 2  values[0][2] = 3  values[0][3] = 0
values[1][0] = 4  values[1][1] = 5  values[1][2] = 6  values[1][3] = 0
values[2][0] = 7  values[2][1] = 8  values[2][2] = 9  values[2][3] = 0
```

Notice that the last element in each row is assigned a value of *zero*. If the preceding array definition is written as

```
int values[3][4] = {1, 2, 3, 4, 5, 6, 7, 8, 9};
```

then three of the array elements will again be assigned zeros, though the order of the assignments will be different. In particular, the array elements will have the following initial values.

```
values[0][0] = 1  values[0][1] = 2  values[0][2] = 3  values[0][3] = 4
values[1][0] = 5  values[1][1] = 6  values[1][2] = 7  values[1][3] = 8
values[2][0] = 9  values[2][1] = 0  values[2][2] = 0  values[2][3] = 0
```

Two-Dimensional Array

A $m \times n$, two-dimensional array can be thought of as a *table* of values having m rows and n columns, as illustrated in the following figure. Extending this idea, a three-dimensional array can be visualized as a *set of tables* (e.g., a **book** in which each page is a table), and so on.


```

main()
{
    int n, s;

    printf("Enter value of n: ");
    scanf(" %d", &n);
    s = sum(n);
    printf("Sum of natural number up to %d: %d", n, s);
}

int sum(int n)
{
    if (n == 1)
        return 1;
    else
        return (n + sum(n - 1));
}

```

OUTPUT

```

Enter value of n: 20
Sum of natural number up to 20: 210

```

Or

What are pre-processor directives? Differentiate between macro and function with describing necessary example.

The C preprocessor is a collection of special statements, called directives, that are executed at the beginning of the compilation process. The **#include** and **#define** statements considered are preprocessor directives. Additional preprocessor directives are **#if**, **#elif**, **#else**, **#endif**, **#if def**, **#ifndef**, **#line** and **#undef**. The preprocessor also includes three special operators: **defined**, **#**, and **##**.

Preprocessor directives usually appear at the beginning of a program, though this is not a firm requirement. Thus, a preprocessor directive may appear anywhere within a program. However, the directive will apply only to the portion of the program following its appearance.

For the beginning programmer, some of the preprocessor directives are relatively unimportant. Hence, we will not describe each preprocessor feature in detail. The more important features are discussed below.

The **#if**, **#elif**, **#else** and **#endif** directives are used most frequently. They permit conditional compilation of the source program, depending on the value of one or more true/false conditions. They are sometimes used in conjunction with the *defined* operator, which is used to determine whether a symbolic constant or a macro identifier has been defined within a program.

The following preprocessor directives illustrate the conditional compilation of a C program. The conditional compilation depends on the status of the symbolic constant **FOREGROUND**.

```

#if defined (FOREGROUND)
    #define BACKGROUND 0
#else
    #define FOREGROUND 0
    #define BACKGROUND 7
#endif

```

Thus, if `FOREGROUND` has already been defined, the symbolic constant `BACKGROUND` will represent the value 0. Otherwise, `FOREGROUND` and `BACKGROUND` will represent the values 0 and 7 respectively.

Here is another way to accomplish the same thing.

```
#ifndef FOREGROUND
    #define BACKGROUND 0
#else
    #define FOREGROUND 0
    #define BACKGROUND 7
#endif
```

The directive **`#ifndef`** is equivalent to **`#if defined()`**. Similarly, the directive **`#ifndef`** is equivalent to **`#if !defined()`**, i.e., “if not defined.” The original approach, in which the **`defined`** operator appears explicitly, is the preferred form.

In each of these examples, the last directive is **`#endif`**. The preprocessor requires that any set of directives beginning with **`#if`**, **`#ifdef`** or **`#ifndef`** must end with **`#endif`**.

The directive **`#elif`** is analogous to an **`else - if`** clause using ordinary C control statements. An **`#if`** directive can be followed by any number of **`#elif`** directives, though there can be only one **`#else`** directive. The appearance of the **`#else`** directive is optional, as determined by the required program logic.

Here is another illustration of conditional compilation. In this situation the conditional compilation will depend on the value that is represented by the symbolic constant `BACKGROUND`.

```
#if BACKGROUND == 7
    #define FOREGROUND 0
#elif BACKGROUND == 6
    #define FOREGROUND 1
#else
    #define FOREGROUND 6
#endif
```

The following example illustrates the use of the **`#undef`** directive within a C program.

```
#define FOREGROUND 7
#define BACKGROUND 0

main()
{
    .....
    #undef    FOREGROUND
    .....
    #undef    BACKGROUND
    .....
}
```

Macros are sometimes used in place of functions within a program. The use of a macro in place of a function eliminates the time delays associated with function calls. If a program

contains many repeated function calls, the time savings resulting from the use of macros can become significant.

On the other hand, macro substitution will take place wherever a reference to a macro appears within a program. Thus, a program that contains several references to the same macro may become unreasonably long. We are therefore faced with a tradeoff between execution speed and size of the compiled object program. The use of a macro is most advantageous in applications where there are relatively few function calls, but the function is called repeatedly (e.g., a single function call within a loop).

Many commercial C compilers offer certain library functions both as macros and as true functions. The macros are defined in the various header files. You may then choose which form is most appropriate for each particular application. Keep in mind, however, that there are certain disadvantages associated with the use of macros in place of functions, aside from the potentially significant increase in program length. In particular:

- When passing arguments to a macro, the number of arguments will be checked, but their data types will not. Thus, there is *less error checking* than with a function call.
- A macro identifier is not associated with an address, so that a macro cannot be utilized as a pointer. Thus, a macro identifier cannot be passed to a function as an argument, in the same sense that a function can be passed to another function as an argument. Moreover, a macro cannot call itself recursively.
- There are possible undesirable side effects associated with the use of macros, particularly when calling arguments are involved.

Consider the macro definition

```
#define ROOT(a, b) sqrt(a * a + b * b)
```

Now suppose that this macro is utilized within a program in the following manner.

```
ROOT(a+1, b+2)
```

The intent is to evaluate the formula

```
sqrt((a + 1) * (a + 1) + (b + 2) * (b + 2))
```

However, each appearance of *a* is replaced by the expression *a + 1* (without parentheses), and each appearance of *b* is replaced by *b + 1*. Therefore, the result of the macro substitution will be

```
sqrt(2 * a + 1 + b + 2 * b + 2)
```

this expression is equivalent to

```
sqrt(2 * a + 3 * b + 3)
```

which is clearly incorrect. The source of error can be corrected, however, by placing additional parentheses within the macro definition; i.e.,

```
#define ROOT(a, b) sqrt((a) * (a) + (b) * (b))
```

The function can handle this problem easily.

```
float root(int a, int b)
{
    return(sqrt(a * a + b * b));
}
```

The function call

```
ROOT(a + 1, b + 2)
```

leads to the following calculation:

```
sqrt((a + 1) * (a + 1) + (b + 2) * (b + 2))
```

6.a. Differentiate between local and global variable. Write a program to illustrate the use of static variable.

Automatic variables are always declared within a function and are local to the function in which they are declared; that is, their scope is confined to that function. Automatic variables defined in different functions will therefore be independent of one another, even though they may have the same name.

Any variable declared within a function is interpreted as an automatic variable unless a different storage-class specification is shown within the declaration. This includes formal argument declarations. All the variables in the programming examples encountered earlier have been automatic variables.

Since the location of the variable declarations within the program determines the *automatic storage class*, the keyword **auto** is not required at the beginning of each variable declaration. There is no harm in including an auto specification within a declaration, though this is normally not done.

Automatic variables can be assigned initial values by including appropriate expressions within the variable declarations, as in the above example, or by explicit assignment expressions elsewhere in the function. Such values will be reassigned each time the function is reentered. If an automatic variable is not initialized in some manner, however, its initial value will be *unpredictable*, and probably *unintelligible* (i.e., *garbage* value).

An automatic variable does not retain its value once control is transferred out of its defining function.

Therefore, any value assigned to an automatic variable within a function will be lost once the function is exited. If the program logic requires that an automatic variable be assigned a particular value each time the function is executed, that value will have to be reset whenever the function is reentered (i.e., whenever the function is accessed).

External variables, in contrast to automatic variables, are not confined to single functions. Their scope extends from the point of definition through the remainder of the program. Hence, they usually span two or more functions, and often an entire program. They are often referred to as **global variables**.

Since external variables are recognized globally, they can be accessed from any function that falls within their scope. They retain their assigned values within this scope. Therefore, an external variable can be assigned a value within one function, and this value can be used (by accessing the external variable) within another function.

The use of external variables provides a convenient mechanism for **transferring information back and forth between functions**. We can transfer information into a function without using arguments. This is especially convenient when a function requires numerous input data items. Moreover, we now have a way to transfer multiple data items out of a function, since the *return* statement can return only one data item.

The storage-class specifier **extern** is not required in an external variable definition, since the external variables will be identified by the location of their definition within the program. In fact, many C compilers forbid the use of **extern** within an external variable definition. External variables can be assigned initial values as a part of the variable definitions, but the initial values must be expressed as **constants** rather than expressions. These initial values will

be assigned only once, at the beginning of the program. The external variables will then retain these initial values unless they are later altered during the execution of the program.

If an initial value is not included in the definition of an external variable, the variable will automatically be assigned a value of **zero**. Thus, external variables are never left dangling with undefined, garbled values.

The following program illustrates the use of static variables.

```
/* calculate successive Fibonacci series */
#include <stdio.h>

long int fibonacci(int count);

main()
{
    int count, n;

    printf("How many Fibonacci Numbers? ");
    scanf(" %d", &n);

    for (count = 0; count < n; ++count)
        printf("\ni = %2d\tF = %ld", count+1, fibonacci(count));
}

long int fibonacci(int count)
{
    static int f1 = 0, f2 = 1;
    long int f;

    f = count < 2 ? count : (f1 + f2);
    f1 = f2;
    f2 = f;
    return f;
}
```

Notice that long integers are used to represent the Fibonacci numbers. Also, note that **f1** and **f2** are static variables that are each assigned an initial value of 0 and 1 respectively. These initial values are assigned only once, at the beginning of the program execution. The subsequent values are retained between successive function calls, as they are assigned. You should understand that **f1** and **f2** are strictly local variables, even though they retain their values from one function call to another.

The output corresponding to a value of **n = 10** is shown below. As usual, the user's response is underlined.

How many Fibonacci Numbers? 10

```
i = 1   F = 0
i = 2   F = 1
i = 3   F = 1
i = 4   F = 2
i = 5   F = 3
i = 6   F = 5
i = 7   F = 8
i = 8   F = 13
```

```
i = 9   F = 21
i = 10  F = 34
```

6.b. Write a program to create structure for the following data for the Cricket game. Country name, player name, playing type (bating, balling or both), number of matches played by player and salary. Save the information in a file cricket.txt and display the information of those players who had played 10 matches.

```
#include <stdio.h>
#include <stdlib.h>

typedef struct
{
    char country[30];
    char name[30];
    char type[10];
    int match;
    float salary;
} cricketer;

main()
{
    cricketer c;
    int n, i;
    FILE *fp;

    if ((fp = fopen("cricket.txt", "w+")) == NULL)
    {
        printf("File cannot be created.");
        exit(1);
    }

    printf("How many records? ");
    scanf(" %d", &n);
    printf("Enter Details Information\n");
    for (i = 0; i < n; ++i)
    {
        printf("\nCricketer no. %d\n", i + 1);
        printf("Country: ");
        scanf(" %[^\\n]", c.country);
        printf("Player Name: ");
        scanf(" %[^\\n]", c.name);
        printf("Playing type: ");
        scanf(" %[^\\n]", c.type);
        printf("Total Matches: ");
        scanf(" %d", &c.match);
        printf("Salary: ");
        scanf(" %f", &c.salary);
        fwrite(&c, sizeof(cricketer), 1, fp);
    }
}
```

```

    }
    rewind(fp);

    printf("\nInformation of player who palyed 10
matches\n");
    for (i = 0; i < n; ++i)
    {
        fread(&c, sizeof(cricketer), 1, fp);
        if (c.match == 10)
        {
            printf("\nCountry: %s", c.country);
            printf("\nPlayer Name: %s", c.name);
            printf("\nPlaying Type: %s", c.type);
            printf("\nTotal Mathces: %d", c.match);
            printf("\nSalary: %f", c.salary);
        }
    }
}

```

7. a. Program testing and debugging.

Once the program code has been written in a selected programming language of choice the next task to complete is the testing and debugging. Testing and debugging help to find the problems associated with the program behavior under normal and abnormal circumstances. Extensive testing like white box testing and black box testing, integration testing needs to do before the program is deployed into the real-world scenario.

Programming theorists often refer debugging to verification and testing to validation.

Verification ensures that the program does what the programmer intends to do. Validation ensures that the program produces the correct results for a set of test data. The different tools like simulators, logic analyzers, breakpoints, trance routines, software interrupts can be used for effective debugging process.

For testing process, test data are supplied to the program and output is observed. If the output is as expected, the program can be considered error free. The error means failure of compiling and executing the program successfully and receiving incorrect results. The types of errors are *syntax error*, *semantic/logical error*, and *runtime error*.

b. operator precedence and associativity

The operators within C are grouped hierarchically according to their **precedence** (i.e., order of evaluation). Operations with a higher precedence are carried out before operations having a lower precedence. The natural order of evaluation can be altered, however, using parentheses. Another important consideration is the order in which consecutive operations within the same precedence group are carried out. This is known as **associativity**. Within each of the precedence groups described above, the associativity is left to right.

Operator category	Operators	Associativity
Function, array, structure member, pointer to structure member	(), [], ., ->	L → R
Unary Operators	-, ++, --, !, sizeof, (type), *, &, ~	R → L
Arithmetic multiply, division and remainder	*, /, %	L → R
Arithmetic Add and Subtract	+, -	L → R
Bitwise shift operators	<<, >>	L → R
Relational operators	<, >, <=, >=	L → R
Equality	==, !=	L → R
Bitwise AND	&	L → R
Bitwise XOR	^	L → R

Bitwise OR		L → R
Logical AND	&&	L → R
Logical OR		L → R
Conditional Operator	? :	R → L
Assignment operators	=, +=, -=, *=, /=, % =, &=, ^=, =, <<=, >>=	R → L
Comma Operator	,	L → R

c. Software development life cycle (SDLC)

An effective System Development Life Cycle (SDLC) should result in a high-quality system that meets customer expectations, reaches completion within time and cost evaluations, and works effectively and efficiently in the current and planned Information Technology infrastructure.

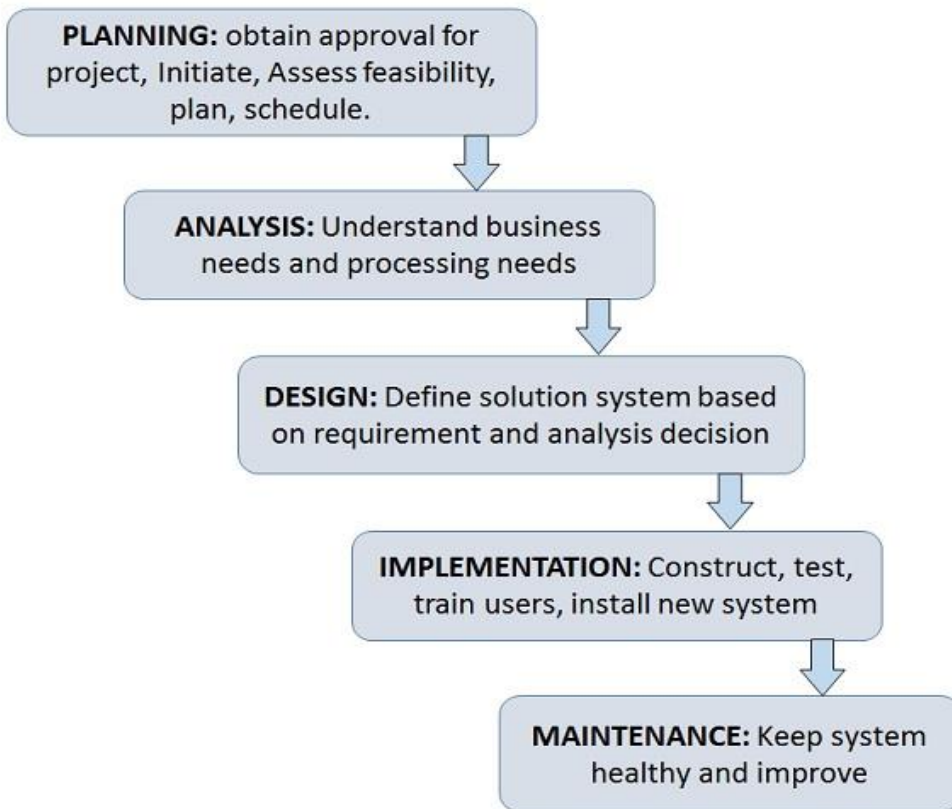
System Development Life Cycle (SDLC) is a conceptual model which includes policies and procedures for developing or altering systems throughout their life cycles.

SDLC is used by analysts to develop an information system. SDLC includes the following activities

- requirements
- design
- implementation
- testing
- deployment
- operations
- maintenance

Phases of SDLC

Systems Development Life Cycle is a systematic approach which explicitly breaks down the work into phases that are required to implement either new or modified Information System.



Feasibility Study or Planning

- Define the problem and scope of existing system.
- Overview the new system and determine its objectives.
- Confirm project feasibility and produce the project Schedule.
- During this phase, threats, constraints, integration and security of system are also considered.
- A feasibility report for the entire project is created at the end of this phase.

Analysis and Specification

- Gather, analyze, and validate the information.
- Define the requirements and prototypes for new system.
- Evaluate the alternatives and prioritize the requirements.
- Examine the information needs of end-user and enhances the system goal.
- A Software Requirement Specification (SRS) document, which specifies the software, hardware, functional, and network requirements of the system is prepared at the end of this phase.

System Design

- Includes the design of application, network, databases, user interfaces, and system interfaces.

- Transform the SRS document into logical structure, which contains detailed and complete set of specifications that can be implemented in a programming language.
- Create a contingency, training, maintenance, and operation plan.
- Review the proposed design. Ensure that the final design must meet the requirements stated in SRS document.
- Finally, prepare a design document which will be used during next phases.

Implementation

- Implement the design into source code through coding.
- Combine all the modules together into training environment that detects errors and defects.
- A test report which contains errors is prepared through test plan that includes test related tasks such as test case generation, testing criteria, and resource allocation for testing.
- Integrate the information system into its environment and install the new system.

Maintenance/Support

- Include all the activities such as phone support or physical on-site support for users that is required once the system is installing.
- Implement the changes that software might undergo over a period of time or implement any new requirements after the software is deployed at the customer location.
- It also includes handling the residual errors and resolve any issues that may exist in the system even after the testing phase.
- Maintenance and support may be needed for a longer time for large systems and for a short time for smaller systems.

2017Fall

1. a. What do you mean by programming language? Discuss on machine language, assembly language and high-level language.

Programming language is defined as a set of rules that provides a platform for instructing computer to perform some specific tasks. Programming languages are classified into various ways. Based on nature of language, they are classified into *general purpose* and *specific purpose* language. On the other hand, programming language are classified into *High-level language* and *Low-level language*.

- ix. Low Level Language (LLL)
 - i. Machine Language
 - j. Assembly Language
- x. High Level Language (HLL)
 - m. Third Generation Language (3GL)
 - n. Fourth Generation Language (4GL)
 - o. Fifth Generation Language (5GL)

iv) a. Machine Language

Machine language is the language of CPU which consists of set of instructions composed with series of 0s and 1s.

Advantages:

- Machine language does not require translators.
- Execution time of machine language program is extremely fast.

Disadvantages:

- Programming is very complex and tedious job for the programmers.
- They are machine dependent; so, they support only specific hardware.
- It is very difficult to find bugs and hard to debug.

v) b. Assembly Language

Assembly language is special type of low-level language which consists set of alphanumeric instructions, called *mnemonics*. E.g. ADD A, B

Advantages:

- It is easier to understand than machine language. Also, it consumes less amount of memory than high-level language. Thus, ALL executes much faster than HLL.
- It is efficient for solving hardware interfacing problems. E.g. operating system kernel, programming language, utility software, virus, etc.
- Error finding and debugging process is easier than HLL.

Disadvantages

- ALL is hardware dependent language.
- Programmer should have depth knowledge about the computer hardware during programming.
- Program execution is slower than MLL and the program is lengthy and complex than HLL.

vi) a. Third Generation Language (3GL)

3GL is general-purpose high-level language in which the instructions are expressed in more like English structure and mathematical expressions. E.g. FORTRAN, BASIC, Pascal, C etc. are examples of 3GL.

Advantages

- Machine independent language.
- Easy for program development because the program instructions are English like structure and Mathematical expressions.
- Easy for catching bugs and easy to debug and modify.
- Programmer does not require depth knowledge about computer architecture while programming.

Disadvantages

- 3GL requires translator. So, program execution is slower than LLL.
- They are general-purpose language. They are not suitable for solving complex and specific type of real-world problems.
- They are mostly CUI based language. So, they are less user-friendly.

- Less efficient than 4GL and 5GL.

e. Fourth Generation Language (4GL)

4GL is a specific-purpose high-level language in which the instructions are expressed by using graphical components and English like structures and Mathematical expressions. E.g.

- *Database and Query Processing Language* (DBase, SyBase, DB2, MySQL, SQL, MS Access, Oracle etc.)
- *Application Generator Language* (Java, C#, Visual C++, Visual.NET etc.)
- *Web Based Scripting Language*: (HTML, XML, JavaScript, VBScript, ASP, PHP, JSP etc.)

Advantages:

- It is efficient and more user friendly.
- It supports rapid application development. Programmers can design, create and debug an application in very short period.
- IDE (Integrated Development Environment) helps developer to include powerful features of different programming languages into one application.
- They are platform independent.

Disadvantages:

- They require more memory and disk storage, hence, take more execution time.
- It is not suitable for solving hardware interfacing problems.

f. Fifth Generation Language (5GL)

5GL is a special type of future generation language in which the statements will be expressed in more like human language. E.g. PROLOG, LISP etc. are considered as 5GL. Some characteristics of 5GL are:

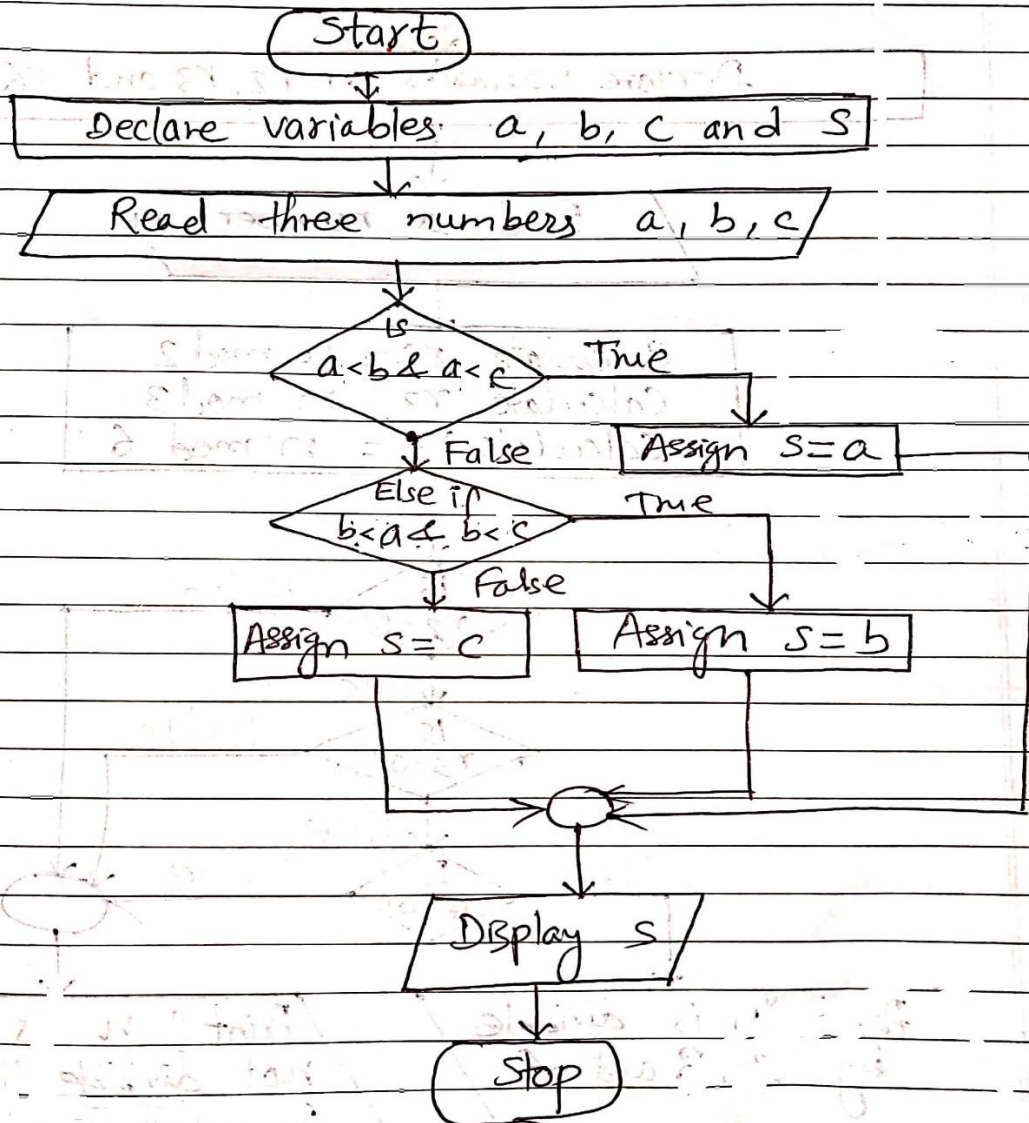
- Natural languages make human and machine closer than before.
- They are designed to allow the computer more natural and intelligent than before.
- They are used in the areas of research, robotics, defense system, and different types of expert systems.

1. b. Define algorithm and flowchart. Draw a flowchart to read three numbers from the user and find the smallest one.

An algorithm is a solution to a computer programming problem. In other words, a step by step procedure for developing a problem is called an algorithm. Algorithms can be written in two different ways.

Flowchart Pictures Detailing with specific blocks detailing out the logical flow of the solution. For a better understanding of an algorithm, it is represented pictorially. The pictorial representation of an algorithm is called a Flowchart.

Flowchart to find smallest among three numbers



2. a. What is an operator? Explain conditional operator with suitable example.

An *operator* is a symbol that operates on single or multiple data items. They are used to perform mathematical or logical manipulations. The data items that operators act upon are called *operands*. Some operators require two operands, while others act upon only one operand.

4. According to number of operands

j) Unary Operators (single operand)

e.g., unary minus (-), unary plus (+), ++, --, !, ~, sizeof, type etc.

k) Binary Operators (two operands)

e.g., Binary plus (+), Binary minus (-), *, /, >, <, &&, << etc.

l) Ternary Operators (three operands)

e.g., conditional operator (?:)

2. According to Operations

- y. Arithmetic Operators
- z. Relational Operators
- aa. Logical Operators
- bb. Assignment Operators
- cc. Increment/Decrement
- dd. Conditional Operator
- ee. Bitwise Operators
- ff. Special operators

Conditional Operator

Simple conditional operations can be carried out with the *conditional operator* (`? :`). An expression that makes use of the conditional operator is called a *conditional expression*. Such an expression can be written in place of the more traditional *if-else* statement. The conditional expression is written in the form

`expression 1 ? expression 2 : expression 3`

When evaluating a conditional expression, *expression 1* is evaluated first. If *expression 1* is true (i.e., if its value is nonzero), then *expression 2* is evaluated and this becomes the value of the conditional expression. However, if *expression 1* is false (i.e., if its value is zero), then *expression 3* is evaluated and this becomes the value of the conditional expression. Note that only one of the embedded expressions (either *expression 2* or *expression 3*) is evaluated when determining the value of a conditional expression.

In the conditional expression shown below, assume that *i* is an integer variable.

`(i < 0) ? 0 : 100;`

The expression `(i < 0)` is evaluated first. If it is true (i.e., if the value of *i* is less than 0), the entire conditional expression takes on the value 0. Otherwise (if the value of *i* is not less than 0), the entire conditional expression takes on the value 100.

In the following conditional expression, assume that *f* and *g* are floating-point variables.

`(f < g) ? f : g;`

This conditional expression takes on the value of *f* if *f* is less than *g*; otherwise, the conditional expression takes on the value of *g*. In other words, the conditional expression returns the value of the *smaller* of the two variables.

If the operands (i.e., *expression 2* and *expression 3*) differ in type, then the resulting data type of the conditional expression will be determined by the rules given in previous section (Arithmetic Operator).

Conditional expressions frequently appear on the right-hand side of a simple assignment statement. The resulting value of the conditional expression is assigned to the identifier on the left. Here is an assignment statement that contains a conditional expression on the right-hand side.

`flag = (i < 0) ? 0 : 100;`

If the value of *i* is negative, then 0 will be assigned to *flag*. If *i* is not negative, however, then 100 will be assigned to *flag*. Here is another assignment statement that contains a conditional expression on the right-hand side.

`min = (f < g) ? f : g;`

This statement causes the value of the smaller of *f* and *g* to be assigned to *min*.

The conditional operator has its own precedence, just above the assignment operators. The associativity is right to left.

5. b. What are control statements? Differentiate between while and do-while loop with suitable example.

A realistic C program may require that a logical test be carried out at some point within the program. One of several possible actions will then be carried out, depending on the outcome of the logical test. This is known as **branching**. There is also a special kind of branching, called **selection**, in which one group of statements is selected from several available groups. In addition, the program may require that a group of instructions be executed repeatedly, until some logical condition has been satisfied. This is known as **looping**. Sometimes the required number of repetitions is known in *advance*; and sometimes the computation continues *indefinitely* until the logical condition becomes true. All these operations can be carried out using the various control statements.

while	do-while
While loop is <i>entry-controlled (pre-test)</i> loop, i.e. test condition is evaluated first and body of loop is executed only if the test is true.	Do-while loop is <i>exit-controlled (post-test)</i> loop, i.e. the body of loop is executed first without checking any condition and at the end of loop body, the condition is evaluated for repetition of next time.
The body of loop may not be executed at all if the condition is not satisfied at the very first attempt.	The body of the loop is executed at least once even the tested condition is false.
Syntax: while (expression) statement The statement will be executed repeatedly, as long as the expression is true (i.e., as long expression has a nonzero value). This statement can be simple or compound.	Syntax: do statement while (expression) ; The statement will be executed repeatedly, as long as the value of expression is true (i.e., is nonzero). The statement can be either simple or compound.
<pre>int i = 5; while (i < 5) { printf("%d ", i); ++i; }</pre> This loop condition is initially false. So, it does not print any value.	<pre>int i = 5; do { printf("%d ", i); ++i; } while (i < 5);</pre> Although body and conditions in both loops are same, this program will print 5.

3. a. Write a program to print the following pattern.

1

1 2

1 2 3

```
#include <stdio.h>
```

```
main()
```

```
{
```

```
    int i, j, k;
```

```
    for(i = 1; i <= 3; ++i)
```

```

{
    k = 1;
    for(j = 1; j <= i; ++j)
    {
        printf("%d\t", k);
        ++k;
    }
    printf("\n");
}
}

```

6. b. How can you initialize one-dimensional array? Write a program to search an element in one-dimensional array having five integer elements.

To initialize 1-D array, the initial values must appear in the order in which they will be assigned to the individual array elements, enclosed in braces and separated by commas. The general form is

storage-class data-type array[expression] = {value 1, value 2, ..., value n};

where **value 1** refers to the value of the first array element, **value 2** refers to the value of the second element, and so on. The appearance of the **expression**, which indicates the number of array elements, is *optional* when initial values are present.

Shown below are several array definitions that include the assignment of initial values.

```

int digits[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
static float x[6] = {0, 0.25, 0, -0.50, 0, 0};
char color[3] = {'R', 'E', 'D', };

```

Note that **x** is a static array. The other two arrays (**digits** and **color**) are assumed to be external arrays by virtue of their placement within the program. The results of these initial assignments, in terms of the individual array elements, are as follows. (Remember that the subscripts in an n-element array range from **0** to **n - 1**.)

digits[0] = 1	x[0] = 0	color[0] = 'R'
digits[1] = 2	x[1] = 0.25	color[1] = 'E'
digits[2] = 3	x[2] = 0	color[2] = 'D'
digits[3] = 4	x[3] = -0.50	
digits[4] = 5	x[4] = 0	
digits[5] = 6	x[5] = 0	
digits[6] = 7		
digits[7] = 8		
digits[8] = 9		
digits[9] = 10		

```

/* to search an element in one-dimensional array having five
integer elements. */
#include <stdio.h>

```

```

main()
{
    int a[] = {5, 10, 15, 20, 25};

```

```

int i, item, flag = 1;

printf("Enter an item to be searched: ");
scanf(" %d", &item);
for (i = 0; i < 5; ++i)
{
    if (a[i] == item)
    {
        flag = 0;
        break;
    }
}
if (flag == 0)
    printf("The item is currently at the list.");
else
    printf("The item is not currently at the list.");
}

```

OUTPUT

RUN 1:

Enter an item to be searched: 15

The item is currently at the list.

RUN 2:

Enter an item to be searched: 50

The item is not currently at the list.

7. a. Does a function return multiple values? When and how functions return single or multiple values? Illustrate with suitable example.

Yes, a function can return multiple values to the calling function. The function returns single value when the formal arguments are passed by value. In this approach of argument passing, only the copy of formal parameter is passed and whatever changes occur in called function do not reflect in calling function.

Since external variables are recognized globally, they can be accessed from any function that falls within their scope. They retain their assigned values within this scope. Therefore, an external variable can be assigned a value within one function, and this value can be used (by accessing the external variable) within another function.

The use of external variables provides a convenient mechanism for **transferring information back and forth between functions**. We can transfer information into a function without using arguments. This is especially convenient when a function requires numerous input data items. Moreover, we now have a way to transfer multiple data items out of a function, since the *return* statement can return only one data item.

```
#include <stdio.h>
```

```
int a = 10;
```

```
int b = 20;
```

```
void change();
```

```
main()
```

```
{
    printf("Before change\n a = %d\n b = %d", a, b);
```

```

    change();
    printf("\nAfter change\na = %d\nb = %d", a, b);
}

```

```

void change()
{
    a = 2 * a;
    b = 2 * b;
}

```

OUTPUT

Before change

a = 10

b = 20

After change

a = 20

b = 40

When an array is passed to a function, however, the values of the array elements *are not* passed to the function. Rather, the array name is interpreted as the **address** of the first array element (i.e., the address of the memory location containing the first array element). This address is assigned to the corresponding formal argument when the function is called. The formal argument therefore becomes a **pointer** to the first array element (more about this in the next chapter). Arguments that are passed in this manner are said to be passed **by reference** rather than passed *by value*.

When a reference is made to an array element within the function, the value of the element's subscript is added to the value of the pointer to indicate the address of the specified array element. Therefore, any array element can be accessed from within the function. Moreover, *if an array element is altered within the function, the alteration will be recognized in the calling portion of the program* (actually, throughout the entire scope of the array).

Here is a simple C program that passes a three-element integer array to a function, where the array elements are altered. The values of the array elements are displayed at three different places in the program, thus illustrating the effects of the alterations.

```

#include <stdio.h>
void modify(int a[]); /* function prototype */

main()
{
    int count, a[3];

    printf("\nEnter any three numbers:\n");
    for (count = 0; count < 3; ++count)
    {
        printf("\na[%d] = ", count);
        scanf(" %d", &a[count]);
    }

    printf("From main(), before modification:\n");
    for (count = 0; count < 3; ++count)
    {
        printf("\na[%d] = %d", count, a[count]);
    }
}

```

```

    }

    modify(a); /* function call */

    printf("From main(), after modification:\n");
    for (count = 0; count < 3; ++count)
    {
        printf("\na[%d] = %d", count, a[count]);
    }
}

void modify(int a[]) /* function definition */
{
    int count;

    printf("\nFrom the function, after modification the values:\n");
    for (count = 0; count < 3; ++count)
    {
        a[count] = -9;
        printf("a[%d] = %d\n", count, a[count]);
    }
    //return;
}

```

The array elements are read from the user within the first loop appearing in *main*. These values are displayed as soon as they are assigned. The array is then passed to the function **modify**, when each array element is assigned the value **-9**. These new values are then displayed from within the function. Finally, the values of array elements are again displayed from **main()**, after control has been transferred back to *main* from *modify*. When the program is executed, the following output is generated.

Enter any three numbers:

```

a[0] = 1
a[1] = 2
a[2] = 3

```

From main(), before modification:

```

a[0] = 1
a[1] = 2
a[2] = 3

```

From the function, after modification the values:

```

a[0] = -9
a[1] = -9
a[2] = -9

```

From main(), after modification:

```

a[0] = -9
a[1] = -9
a[2] = -9

```

These results show that the elements of **a** are altered within *main* as a result of the changes that were made within *modify*.

4 b. How arguments are passed by value and passed by reference? Explain with suitable example.

Call by Value

When a single value is passed to a function via an actual argument, the value of the actual argument is *copied* into the function. Therefore, *the value of the corresponding formal argument can be altered within the Function, but the value of the actual argument within the calling routine will not change.* This procedure for passing the value of an argument to a function is known as *passing by value*.

Here is a simple C program containing a function that alters the value of its argument.

```
#include <stdio.h>

void modify(int a);

main()
{
    int a = 2;

    printf("\na = %d (from main, before calling the
function)", a);
    modify(a) ;// function call a is a normal variable
    printf("\na = %d (from main, after calling the
function)", a);
}

void modify(int a)
{
    a *= 3;
    printf("\na = %d (from the function, after being
modified)", a);
    return;
}
```

The original value of **a** (i.e., **a = 2**) is displayed when main begins execution. This value is then passed to the function **modify**, where it is multiplied by **3** and the new value displayed. Note that it is the altered value of the formal argument that is displayed within the function. Finally, the value of **a** within main (i.e., the actual argument) is again displayed, after control is transferred back to main from **modify**. When the program is executed, the following output is generated.

```
a = 2 (from main, before calling the function)
a = 6 (from the function, after being modified)
a = 2 (from main, after calling the function)
```

These results show that **a** is not altered within main, even though the corresponding value of **a** is changed within **modify()**.

Passing an argument by value has advantages and disadvantages. On the plus side, it allows a single-valued actual argument to be written as an expression rather than being restricted to a single variable. Moreover, if the actual argument is expressed simply as a single variable, it

protects the value of this variable from alterations within the function. On the other hand, it does not allow information to be transferred back to the calling portion of the program via arguments. Thus, *passing by value is restricted to a one-way transfer of information.*

Call by Reference

To pass an array to a function, the array name must appear by itself, without brackets or subscripts, as an actual argument within the function call. The corresponding formal argument is written in the same manner, though it must be declared as an array within the formal argument declarations. When declaring a one-dimensional array as a formal argument, the array name is written with a pair of empty square brackets. The size of the array is *not specified* within the formal argument declaration.

When an array is passed to a function, however, the values of the array elements *are not* passed to the function. Rather, the array name is interpreted as the **address** of the first array element (i.e., the address of the memory location containing the first array element). This address is assigned to the corresponding formal argument when the function is called. The formal argument therefore becomes a **pointer** to the first array element (more about this in the next chapter). Arguments that are passed in this manner are said to be passed **by reference** rather than passed *by value*.

When a reference is made to an array element within the function, the value of the element's subscript is added to the value of the pointer to indicate the address of the specified array element. Therefore, any array element can be accessed from within the function. Moreover, *if an array element is altered within the function, the alteration will be recognized in the calling portion of the program* (actually, throughout the entire scope of the array).

Here is a simple C program that passes a three-element integer array to a function, where the array elements are altered. The values of the array elements are displayed at three different places in the program, thus illustrating the effects of the alterations.

```
#include <stdio.h>
void modify(int a[]); /* function prototype */

main()
{
    int count, a[3];

    printf("\nEnter any three numbers:\n");
    for (count = 0; count < 3; ++count)
    {
        printf("\na[%d] = ", count);
        scanf(" %d", &a[count]);
    }

    printf("From main(), before modification:\n");
    for (count = 0; count < 3; ++count)
    {
        printf("\na[%d] = %d", count, a[count]);
    }

    modify(a); /* function call a is an array*/

    printf("From main(), after modification:\n");
```

```

    for (count = 0; count < 3; ++count)
    {
        printf("\na[%d] = %d", count, a[count]);
    }
}

void modify(int a[])    /* function definition */
{
    int count;

    printf("\nFrom the function, after modification the
values:\n");
    for (count = 0; count < 3; ++count)
    {
        a[count] = -9;
        printf("a[%d] = %d\n", count, a[count]);
    }
    //return;
}

```

The array elements are read from the user within the first loop appearing in *main*. These values are displayed as soon as they are assigned. The array is then passed to the function ***modify***, when each array element is assigned the value **-9**. These new values are then displayed from within the function. Finally, the values of array elements are again displayed from ***main()***, after control has been transferred back to *main* from *modify*. When the program is executed, the following output is generated.

```

Enter any three numbers:
a[0] = 1
a[1] = 2
a[2] = 3
From main(), before modification:
a[0] = 1
a[1] = 2
a[2] = 3
From the function, after modification the values:
a[0] = -9
a[1] = -9
a[2] = -9
From main(), after modification:
a[0] = -9
a[1] = -9
a[2] = -9

```

These results show that the elements of ***a*** are altered within *main* as a result of the changes that were made within *modify*.

8. a. Explain the relationship between arrays and pointers. How can a pointer variable be used to access and modify single-dimensional and multidimensional array?

Pointers and One-Dimensional Arrays

Recall that an array name is really a pointer to the first element in the array. Therefore, if **x** is a one-dimensional array, then the address of the first array element can be expressed as either **&x[0]** or simply as **x**. Moreover, the address of the second array element can be written as either **&x[1]** or as **(x + 1)**, and so on. In general, the address of array element **(i + 1)** can be expressed as either **&x[i]** or as **(x + i)**. Thus, we have two different ways to write the address of any array element: We can write the actual array element, preceded by an ampersand; or we can write an expression in which the subscript is added to the array name.

In the latter case, we are dealing with a very special and unusual type of expression. In the expression **(x + i)**, for example, **x** represents an address, whereas **i** represents an integer quantity. Moreover, **x** is the name of an array whose elements may be characters, integers, floating-point quantities, etc. (though all the array elements must be of the same data type). Thus, we are not simply adding numerical values. Rather, we are specifying an address that is a certain number of memory cells beyond the address of the first array element. Or, in simpler terms, we are specifying a location that is **i** array elements beyond the first. Hence, the expression **(x + i)** is a symbolic representation for an address specification rather than an arithmetic expression.

Recall that the number of memory cells associated with an array element will depend upon the data type of the array as well as the particular computer's architecture. With some computers, for example, an integer quantity occupies two bytes (two memory cells), a floating-point quantity requires four bytes, and a double-precision quantity requires eight bytes of memory. With other computers, an integer quantity may require four bytes, and floating-point and double-precision quantities may each require eight bytes. And so on.

When writing the address of an array element in the form **(x + i)**, however, you need not be concerned with the number of memory cells associated with each type of array element; the C compiler adjusts for this automatically. You must specify only the address of the first array element (i.e., the name of the array) and the number of array elements beyond the first (i.e., a value for the subscript). The value of **i** is sometimes referred to as an **offset** when used in this manner.

Since **&x[i]** and **(x + i)** both represent the address of the **ith** element of **x**, it would seem reasonable that **x[i]** and ***(x + i)** both represent the contents of that address, i.e., the *value* of the **ith** element of **x**. This is indeed the case. The two terms are interchangeable. Hence, either term can be used in any application. The choice depends upon your individual preferences. Here is a simple program that illustrates the relationship between array elements and their addresses.

```
#include <stdio.h>

main()
{
    static int x[10] = {10, 11, 12, 13, 14, 15, 16, 17, 18, 19};
    int i;

    for (i = 0; i <= 9; ++i)
    {
        /*display an array element*/
        printf("\ni = %d      x[i] = %d      *(x+i) = %d", i, x[i], *(x + i));

        /*display the corresponding array address*/
        printf("      &x[i] = %X      x+i = %X", &x[i], (x + i));
    }
}
```

This program defines a one-dimensional, 10-element integer array **x**, whose elements are assigned the values 10, 11, ..., 19. The action portion of the program consists of a loop that displays the value and the corresponding address of each array element. Note that the value of each array element is specified in two different ways, as **x[i]** and as ***(x+i)**, in order to illustrate their equivalence. Similarly, the address of each array element is specified in two different ways, as **&x[i]** and as **(x + i)**, for the same reason. Therefore, the value and the address of each array element should appear twice. Execution of this program results in the following output.

i = 0	x[i] = 10	*(x+i) = 10	&x[i] = 403020	x+i = 403020
i = 1	x[i] = 11	*(x+i) = 11	&x[i] = 403024	x+i = 403024
i = 2	x[i] = 12	*(x+i) = 12	&x[i] = 403028	x+i = 403028
i = 3	x[i] = 13	*(x+i) = 13	&x[i] = 40302C	x+i = 40302C
i = 4	x[i] = 14	*(x+i) = 14	&x[i] = 403030	x+i = 403030
i = 5	x[i] = 15	*(x+i) = 15	&x[i] = 403034	x+i = 403034
i = 6	x[i] = 16	*(x+i) = 16	&x[i] = 403038	x+i = 403038
i = 7	x[i] = 17	*(x+i) = 17	&x[i] = 40303C	x+i = 40303C
i = 8	x[i] = 18	*(x+i) = 18	&x[i] = 403040	x+i = 403040
i = 9	x[i] = 19	*(x+i) = 19	&x[i] = 403044	x+i = 403044

The output clearly illustrates the distinction between **x[i]**, which represents the value of the i^{th} array element, and **&x[i]**, which represents its address. Moreover, we see that the value of the i^{th} array element can be represented by either **x[i]** or **(x+i)**, and the address of the i^{th} element can be represented by either **&x[i]** or **x+i**. Thus, we see another comparison, between ***(x+i)**, which also represents the value of the i^{th} element, and **x+i**, which also represents its address. Notice, for example, that the first array element (corresponding to **i = 0**) has been assigned a value of 10 and a (hexadecimal) address of 403020. The second array element has a value of 11 and an address of 403024, etc. Thus, memory location 403020 will contain the integer value 10, location 403024 will contain 11, and so on. You should understand that these addresses are assigned automatically by the compiler.

When assigning a value to an array element such as **x[i]**, the left side of the assignment statement may be written as either **x[i]** or as ***(x+i)**. Thus, a value may be assigned directly to an array element, or it may be assigned to the memory area whose address is that of the array element.

Pointers and Multidimensional Arrays

A two-dimensional array, for example, is a collection of one-dimensional arrays. Therefore, we can define a two-dimensional array as a pointer to a group of contiguous one-dimensional arrays. Thus, a two-dimensional array declaration can be written as

```
data-type (*ptvar) [expression 2];
```

rather than

```
data-type array[expression 1][expression 2];
```

This concept can be generalized to higher-dimensional arrays; that is,

```
data-type (*ptvar) [expression 2][expression 3]...[expression n];
```

replaces

```
data-type array[expression 1][expression 2]...[expression n];
```

In these declarations **data-type** refers to the data type of the array, **ptvar** is the name of the pointer variable, **array** is the corresponding array name, and **expression 1**, **expression 2**, ...,

expression n are positive-valued integer expressions that indicate the maximum number of array elements associated with each subscript.

Notice the parentheses that surround the array name and the preceding asterisk in the pointer version of each declaration. These parentheses must be present. Without them we would be defining an array of pointers rather than a pointer to a group of arrays, since these symbols (i.e., the **square brackets** and the **asterisk**) would normally be evaluated *right to left*.

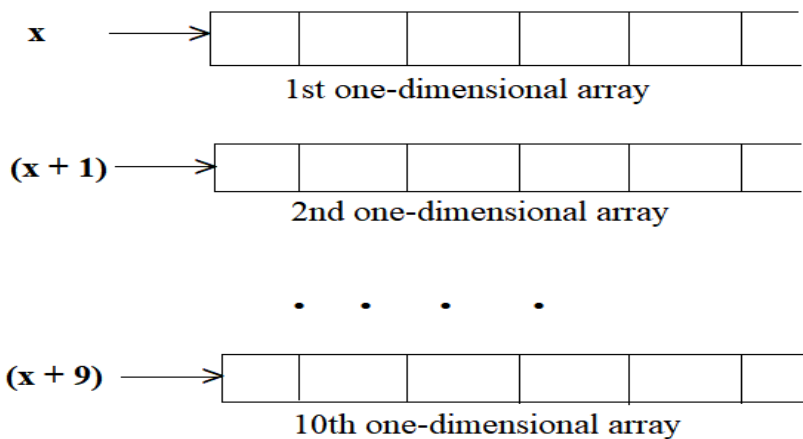
Suppose **x** is a two-dimensional integer array having **10** rows and **20** columns. We can declare **x** as

```
int (*x) [20];
```

rather than

```
int x[10][20];
```

In the first declaration, **x** is defined to be a *pointer* to a group of contiguous, one-dimensional, 20-element integer arrays. Thus, **x** points to the first 20-element array, which is the *first row* (i.e., **row 0**) of the original two-dimensional array. Similarly, **(x + 1)** points to the second 20-element array, which is the second row (row 1) of the original two-dimensional array, and so on, as illustrated in figure below.



Now consider a three-dimensional floating-point array **t**. This array can be defined as

```
float (*t) [20][30];
```

rather than

```
float t[10][20][30];
```

In the first declaration, **t** is defined as a pointer to a group of contiguous two-dimensional, 20 x 30 floating-point arrays.

Hence, **t** points to the first 20 x 30 array, **(t + 1)** points to the second 20 x 30 array, and so on.

An individual array element within a multidimensional array can be accessed by the repeated use of the indirection operator. Usually, however, this procedure is more awkward than the conventional method for accessing an array element. The following example illustrates the use of the indirection operator.

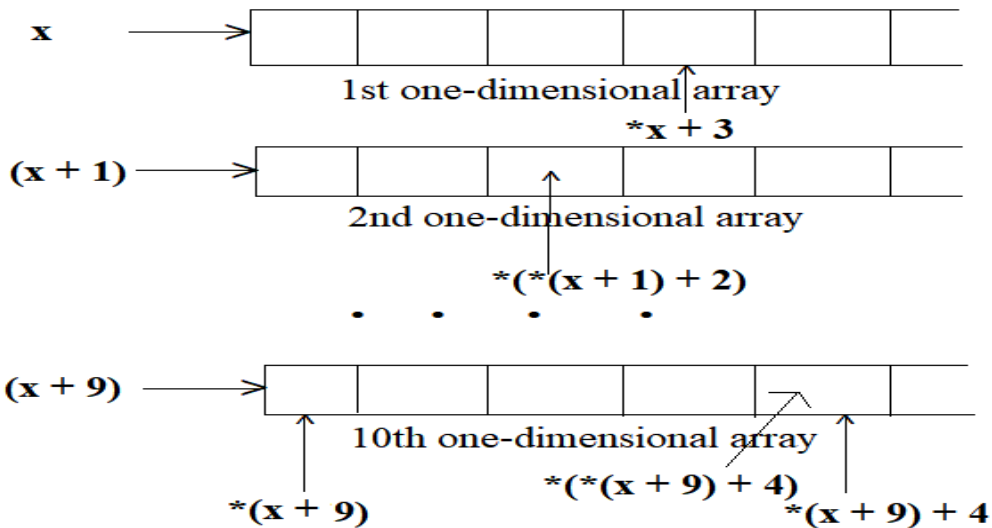
Suppose **x** is a two-dimensional integer array having **10** rows and **20** columns, as declared in the previous example. The item in row **2**, column **5** can be accessed by writing either

```
x[2][5]
```

or

$*(*(\mathbf{x} + 2) + 5)$

The second form requires some explanation. First, note that $(\mathbf{x} + 2)$ is a pointer to row 2. Therefore, the object of this pointer, $*(\mathbf{x} + 2)$, refers to the entire row. Since row 2 is a one-dimensional array, $*(\mathbf{x} + 2)$ is a pointer to the first element in row 2. We now add 5 to this pointer. Hence, $(*(\mathbf{x} + 2) + 5)$ is a pointer to element 5 (i.e., the sixth element) in row 2. The object of this pointer, $*(*(\mathbf{x} + 2) + 5)$, therefore refers to the item in column 5 of row 2, which is $\mathbf{x}[2][5]$. These relationships are illustrated in Figure below.

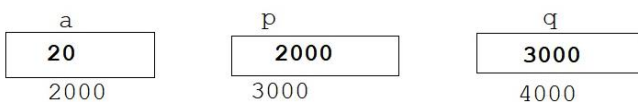


5. b. How do you define double indirection pointers, pointer to array and array of pointers? Give example codes.

Double Indirection Pointer

C allows the use of a pointer that points to other pointers. In this case, we need to add asterisk (*) for each level of reference. For example,

```
int a = 20;
int *p;
int **q; //pointer to pointer
p = &a;
q = &p;
```



An individual array element within a multidimensional array can be accessed by the repeated use of the indirection operator. Usually, however, this procedure is more awkward than the conventional method for accessing an array element. The following example illustrates the use of the indirection operator.

Suppose $\mathbf{x}$ is a two-dimensional integer array having **10** rows and **20** columns, as declared in the previous example. The item in row 2, column 5 can be accessed by writing either

$\mathbf{x}[2][5]$

or

$*(*(\mathbf{x} + 2) + 5)$

Pointer to Array

A two-dimensional array declaration can be written as

```
data-type (*ptvar) [expression 2];
```

rather than

```
data-type array[expression 1][expression 2];
```

This concept can be generalized to higher-dimensional arrays; that is,

```
data-type (*ptvar) [expression 2][expression 3]...[expression n];
```

replaces

```
data-type array[expression 1][expression 2]...[expression n];
```

In these declarations **data-type** refers to the data type of the array, **ptvar** is the name of the pointer variable, **array** is the corresponding array name, and **expression 1**, **expression 2**, ..., **expression n** are positive-valued integer expressions that indicate the maximum number of array elements associated with each subscript.

Suppose **x** is a two-dimensional integer array having **10** rows and **20** columns. We can declare **x** as

```
int (*x) [20];
```

rather than

```
int x[10][20];
```

In the first declaration, **x** is defined to be a *pointer* to a group of contiguous, one-dimensional, *20-element integer arrays*. Thus, **x** points to the first *20-element array*, which is the *first row* (i.e., **row 0**) of the original two-dimensional array. Similarly, **(x + 1)** points to the second *20-element array*, which is the second row (row 1) of the original two-dimensional array, and so on.

Array of Pointer

In general terms, a two-dimensional array can be defined as a one-dimensional array of pointers by writing

```
data-type *array[expression 1];
```

rather than the conventional array definition,

```
data-type array[expression 1][expression 2];
```

Similarly, an n-dimensional array can be defined as an (n - 1)-dimensional array of pointers by writing

```
data-type *array[expression 1][expression 2]...[expression n-1];
```

rather than

```
data-type array[expression 1][expression 2]...[expression n];
```

In these declarations **data-type** refers to the data type of the original n-dimensional array, **array** is the array name, and **expression 1**, **expression 2**, ..., **expression n** are positive-valued integer expressions that indicate the maximum number of elements associated with each subscript.

Suppose **x** is a two-dimensional integer array having 10 rows and 20 columns. We can define **x** as a one-dimensional array of pointers by writing

```
int *x[10];
```

Hence, $x[0]$ points to the beginning of the first row, $x[1]$ points to the beginning of the second row, and so on. Note that the number of elements within each row is not explicitly specified. An individual array element, such as $x[2][5]$, can be accessed by writing

```
*(x[2] + 5)
```

In this expression, $x[2]$ is a pointer to the first element in row 2, so that $(x[2] + 5)$ points to element 5 (actually, the sixth element) within row 2. The object of this pointer, $*(x[2] + 5)$, therefore refers to $x[2][5]$.

9. a. How do you declare and initialize array of structure variables? How is structure different from union? Give example.

It is also possible to define an array of structures; i.e., an array in which each element is a structure. The procedure is illustrated in the following example.

```
struct date
{
    int month;
    int day;
    int year;
};
struct account
{
    int acct_no;
    char acct_type;
    char name[80];
    float balance;
    struct date lastpayment;
};
struct account customer[100];
```

In this declaration **customer** is a 100-element array of structures. Hence, each element of customer is a separate structure of type account (i.e., each element of customer represents an individual customer record).

Note that each structure of type account includes an array (**name[80]**) and another structure (**date**) as members. Thus, we have an array and a structure embedded within another structure, which is itself an element of an array.

An array of structures can be assigned initial values just as any other array. Remember that each array element is a structure that must be assigned a corresponding set of initial values, as illustrated below. A C program contains the following declarations.

```
struct date
{
    char name[80];
    int month;
    int day;
    int year;
};
static struct data birthday[] = {"Amy", 12, 30, 73,
                                "Gail", 5, 13, 66,
                                "Marc", 7, 15, 72,
                                "Marla", 11, 29, 70,
                                "Megan", 2, 4, 77,
```



```
"Sharon", 12, 29, 63,
"Susan", 4, 12, 69};
```

In this example **birthday** is an array of structures whose size is unspecified. The initial values will define the size of the array, and the amount of memory required to store the array.

Notice that each row in the variable declaration contains *four constants*. These constants represent the initial values, i.e., the *name, month, day* and *year*, for one array element. Since there are 7 rows (7 sets of constants), the array will contain 7 elements, numbered 0 to 6.

Some programmers may prefer to embed each set of constants within a separate pair of braces, in order to delineate the individual array elements more clearly. This is entirely permissible. Thus, the array declaration can be written

```
static struct data birthday[] = {
    {"Amy", 12, 30, 73},
    {"Gail", 5, 13, 66},
    {"Marc", 7, 15, 72},
    {"Marla", 11, 29, 70},
    {"Megan", 2, 4, 77},
    {"Sharon", 12, 29, 63},
    {"Susan", 4, 12, 69}
};
```

Structure	Union
Each member of structure is assigned its own <i>unique storage</i> . It takes more memory spaces than that of union.	All members of a union share the <i>same storage</i> area of computer memory. It takes less memory than that of structure.
The amount of memory required to store a structure is the sum of the memory sizes required by <i>all</i> its members.	The amount of memory required to store a union is same as memory size occupied by a member which requires <i>largest</i> memory space among the members.
<i>All</i> the members of structure can be accessed at any point of time.	<i>Only one</i> member of union can be accessed at any given time.
Structure is suitable when you have <i>enough memory spaces</i> . It is very efficient to use and handle.	Union is used when you have <i>lack of memory</i> . It is easy but not realistic.
Structure example: <pre>struct student { char name[20]; //20 int roll; //2 float marks; //4 char address[50]; //50 } st;</pre> Total size of struct student: 76B	Union example: <pre>union student { char name[20]; int roll; float marks; char address[50]; } st;</pre> Total size of union student: 50B

6. b. What are different modes of opening a file? Write a program to create a file 'hello.txt', write data into to the file and also read the data from file.

```
#include <stdio.h>
#include <stdlib.h>
```

```

main()
{
    FILE *fp;
    char name[30];
    int roll;
    float marks;

    if ((fp = fopen("hello.txt", "w+")) == NULL)
    {
        printf("Cannot create the file.");
        exit(1);
    }

    printf("Enter Information about Student:\n");
    printf("Name: ");
    scanf(" %[^\\n]", name);
    printf("Roll number: ");
    scanf(" %d", &roll);
    printf("Marks: ");
    scanf(" %f", &marks);

    fprintf(fp, "%s\\n%d\\n%f", name, roll, marks);

    rewind(fp);

    printf("\\nReading from the file...\\n");
    fscanf(fp, "%[^\\n]%d%f", name, &roll, &marks);
    printf("Name: %s", name);
    printf("\\nRoll number: %d", roll);
    printf("\\nMarks: %f", marks);
    fclose(fp);
}

```

OUTPUT

Enter Information about Student:

Name: Dennis Ritchie

Roll number: 5

Marks: 89.5

Reading from the file...

Name: Dennis Ritchie

Roll number: 5

Marks: 89.500000

7.a. Dynamic Memory Management

Since an array name is a pointer to the first element within the array, it should be possible to define the array as a pointer variable rather than as a conventional array. Syntactically, the two definitions are equivalent. However, a conventional array definition results in a fixed block of memory being reserved at the beginning of program execution, whereas this does not occur if the array is represented in terms of a pointer variable. Therefore, the use of a pointer

variable to represent an array requires some type of initial memory assignment before the array elements are processed. This is known as **dynamic memory allocation**. Generally, the **malloc** library function is used for this purpose, as illustrated in the next example. Suppose **x** is a one-dimensional, 10-element array of integers. It is possible to define **x** as a pointer variable rather than an array. Thus, we can write

```
int *x;
rather than
```

```
int x[10];
```

However, **x** is not automatically assigned a memory block when it is defined as a pointer variable, though a block of memory large enough to store 10 integer quantities will be reserved when **x** is defined as an array.

To assign enough memory for **x**, we can make use of the library function **malloc**, as follows.

```
x = (int *) malloc(10 * sizeof(int));
```

This function reserves a block of memory whose size (in bytes) is equivalent to 10 integer quantities. As written, the function returns a pointer to an integer. This pointer indicates the beginning of the memory block. In general, the type cast preceding **malloc** must be consistent with the data type of the pointer variable. Thus, if **y** were defined as a pointer to a double-precision quantity and we wanted enough memory to store 10 double-precision quantities, we would write

```
y = (double *) malloc(10 * sizeof(double));
```

*If the declaration is to include the assignment of initial values, however, then **x** must be defined as an array rather than a pointer variable.* For example,

```
int x[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
or
```

```
int x[] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
```

When programming in C, it is not unusual to use pointer expressions rather than references to individual array elements. The resulting programs may appear strange at first, though they are straightforward once you become comfortable accessing values that are stored in specified addresses. Generally, a small amount of practice is all that is required.

Also, there are another functions related to DMA; **calloc()**, **realloc()** and **free()**.

10. b. String Handling Functions

Here are some string handling functions which are used to manipulate strings on C. These functions are defined at header file **string.h**.

i. strlen()

It returns the length of string parameter. Syntax:

```
len = strlen(str);
```

Here **str** is string variable or string constant and **len** is an integer variable which holds the length returned by **strlen()**. E.g.:

```
len = strlen("Nepal");
```

The **len** holds 5 which is length of string constant "Nepal".

ii. strlwr()

It converts each uppercase character into lowercase character presented in given string.

Syntax: **strlwr(str)** ;

Here *str* is string variable or string constant. The new string (after conversion) is stored at the same variable. E.g.:

```
strlwr("Nepal");
```

It returns string "*nepal*".

```
char str[] = "Nepal";
```

```
strlwr(str);
```

Then new string at *str* is "*nepal*".

iii. **strupr()**

It converts each lowercase character into uppercase character presented in given string.

Syntax: **strupr(str)** ;

Here *str* is string variable or string constant. The new string (after conversion) is stored at the same variable. E.g.:

```
strupr("Nepal");
```

It returns string "*NEPAL*".

```
char str[] = "Nepal";
```

```
strupr(str);
```

Then new string at *str* is "*NEPAL*".

iv. **strrev()**

It reverses the contents of string (except \0) supplied. The new string is stored at the same variable.

Syntax:

strrev(str) ;

where *str* is string variable or string constant. For example,

```
char str[] = "Nepal";
```

```
strrev(str);
```

the *str* now has string "*lapeN*".

v. **strcpy()**

Syntax:

strcpy(dest_string, src_string) ;

It copies contents of source string (*src_string*) to destination string (*dest_string*). After copying, the *dest_string* has the same value of *src_string*. E.g.:

```
char text[20] = "Nepal";
```

```
char str[20];
```

```
strcpy(str, text);
```

Now, the *str* and *text* has the same content i.e. "*Nepal*".

7. c. **Self-referential structure**

It is sometimes desirable to include within a structure one member that is a pointer to the parent structure type. In general terms, this can be expressed as

```
struct tag
{
    member 1;
    member 2;
    ...
    struct tag *name;
```

```
};
```

where **name** refers to the name of a pointer variable. Thus, the structure of type **tag** will contain a member that points to another structure of type **tag**. Such structures are known as **self-referential** structures.

A C program contains the following structure declaration.

```
struct list_element
{
    char item[40];
    struct list_element *next;
};
```

This is a structure of type **list-element**. The structure contains two members: a 40-element character array, called *item*, and a pointer to a structure of the same type (i.e., a pointer to a structure of type **list-element**), called *next*. Therefore, this is a self-referential structure. Self-referential structures are very useful in applications that involve *linked data structures*, such as **lists** and **trees**. First, however, we present a brief summary of linked data structures. The basic idea of a linked data structure is that each component within the structure includes a pointer indicating where the next component can be found. Therefore, the relative order of the components can easily be changed simply by altering the pointers. In addition, individual components can easily be added or deleted, again by altering the pointers. As a result, a linked data structure is not confined to some maximum number of components. Rather, the data structure can expand or contract in size as required.

Figure below illustrates a linked list containing three components. Each component consists of two data items: a string, and a pointer that references the next component within the list. Thus, the first component contains the string **red**, the second contains **green** and the third contains **blue**. The beginning of the list is indicated by a separate pointer, which is labeled start. Also, the end of the list is indicated by a special pointer, called NULL.



Finally, in the Figure shown below we see an example of a **tree**. Trees consist of *nodes* and *branches*, arranged in some hierarchical manner which indicates a corresponding hierarchical structure within the data. (A **binary tree** is a tree in which every node has no more than two branches.) In the Figure (a) the *root node* has the value **screen**, and the associated branches lead to the nodes whose values are **foreground** and **background**, respectively. Similarly, the branches associated with **foreground** lead to the nodes whose values are **white**, **green** and **amber**, and the branches associated with **background** lead to the nodes whose values are **black**, **blue** and **white**. Figure (b) illustrates the manner in which pointers are used to construct the tree.

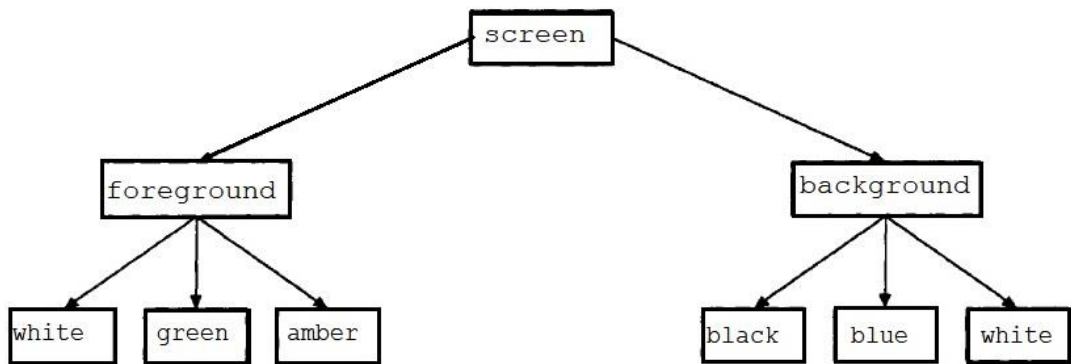


figure (a)

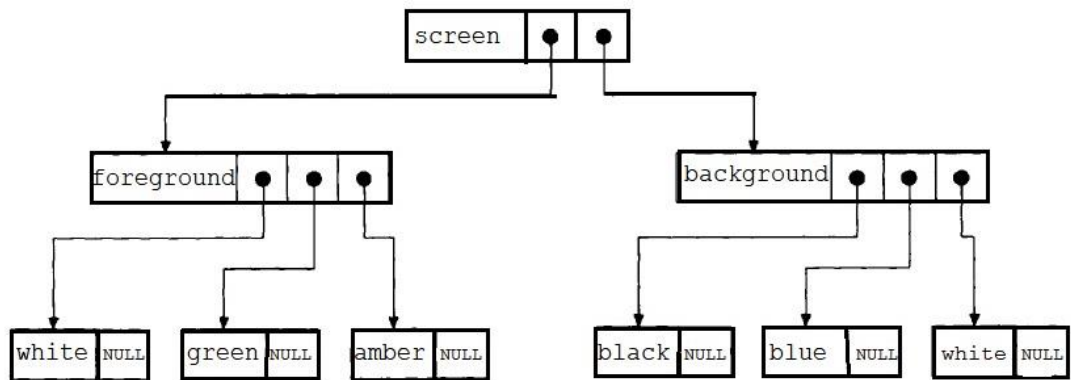


figure (b)

2016 Fall

1.a. What are functional differences between primary and secondary memory? Explain the function of control unit with the help of block diagram of digital computer.

Primary memory is the **main memory** of computer system. The instructions that have to be currently executed is copied to the primary memory because CPU can directly access the data from primary memory. Accessing data from primary memory is *faster* as it is an *internal memory* and processor accesses data from primary memory using *data bus*.

The primary memory is usually **volatile** in nature which means data in primary memory does not exist if not saved, in case the *power failure* occurs. The primary memory is *semiconductor memory* and is more *expensive* than secondary memory. The primary memory capacity is *limited* in computer and is always *smaller* than secondary memory.

Primary memory can be divided into two types of memory that are *RAM (Random Access Memory)* and *ROM (Read Only Memory)*.

RAM is both **read and write** memory. The data which has to be currently processed is kept in RAM which can be quickly accessed by the CPU. RAM is *volatile* and loses data if the power is switched off. RAM can be *static* or *dynamic*.

ROM is a **read only** memory; its content cannot be *altered*. It has the instructions that are used when the system is *booted up*. ROM is a *non-volatile* memory i.e. it retains its content even if the power is switched off. The types of ROM are PROM, EPROM and EEPROM.

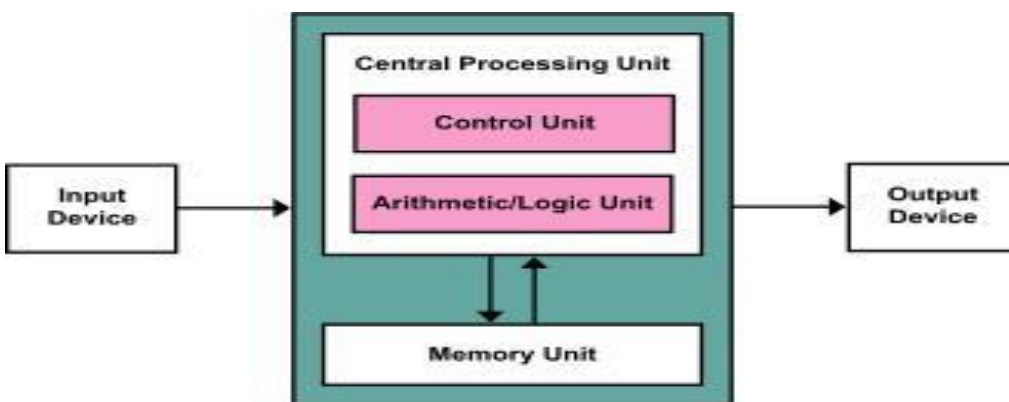
Secondary memory is an **auxiliary memory** of the computer. The data that has to be *permanently* stored is kept in secondary memory. The CPU cannot directly access the data in secondary memory. The data has to be initially copied to primary memory then only it can be processed by CPU. Hence, accessing data from secondary memory is *slower*. The secondary memory can be accessed using the input-output channel.

The secondary memory is **nonvolatile** in nature, which means that the content of the secondary memory exists even if the power is switched off. The secondary memory is *magnetic memory* or *optical memory* and it available at *cheaper* rates as compared to the primary memory.

Secondary memory is available in *bulk* and always *larger* than primary memory. A computer can even work without secondary memory as it an external memory. The examples of secondary memory are the hard disk, floppy disk, CD, DVD, etc.

Key Differences Between Primary and Secondary Memory

- ✓ Primary memory can be directly accessed by the CPU whereas, the CPU cannot directly access the secondary memory.
- ✓ The primary memory of the computer is also known as the main memory of the computer. However, secondary memory is known as auxiliary memory.
- ✓ The data that is to be currently processed is in primary memory whereas, the data that has to be permanently stored is kept in secondary memory.
- ✓ Primary memory is a volatile memory whereas, the secondary memory is a non-volatile memory.
- ✓ Primary memories are semiconductor memories whereas; the secondary memories are the magnetic and optical memories.
- ✓ Data accessing speed of the primary memory is faster than secondary memory.
- ✓ Primary memory is accessed by the data bus. On the other hand, secondary memory is accessed using input-output channels.
- ✓ Primary memory's capacity is always smaller than secondary memory's capacity.
- ✓ Primary memory is costlier than secondary memory.
- ✓ Primary memory is an internal memory whereas, secondary memory is an external memory.



The control unit is for controlling the execution and interpreting of instructions stored in the memory. ALU is the unit where the arithmetic and logical operations are performed. The information to a computer is transformed to groups of binary digits, called bit. The length of bit varies from computer to computer, from 8 to 64. A group of 8 bits is called a *Byte* and a byte generally represents one alphanumeric (Alphabets and Numerals) character. The physical

components of a computer are called hardware. But for the machine to work it requires certain programs. They are called software.

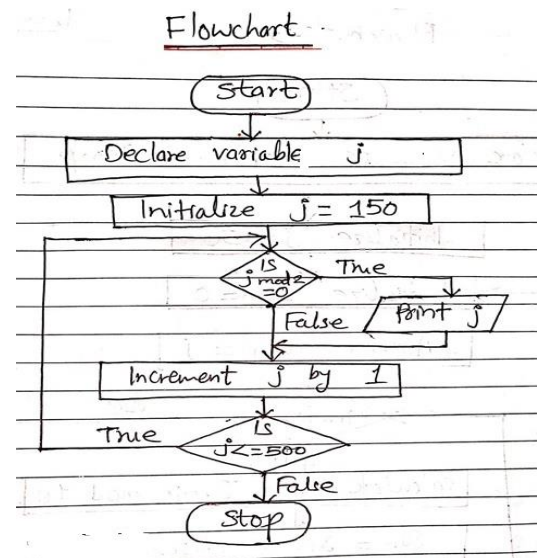
Functions of Control Unit

- ✓ The control unit directs the entire computer system to carry out stored program instructions.
- ✓ The control unit must communicate with both the arithmetic logic unit (ALU) and main memory.
- ✓ The control unit instructs the arithmetic logic unit that which logical or arithmetic operation is to be performed.
- ✓ The control unit co-ordinates the activities of the other two units as well as all peripherals and auxiliary storage devices linked to the computer.

1.b. Define the role of flowchart in the efficient program maintenance with its character. Also develop a flowchart to print even numbers between 150 to 500.

Flowchart Pictures Detailing with specific blocks detailing out the logical flow of the solution. For a better understanding of an algorithm, it is represented pictorially. The pictorial representation of an algorithm is called a Flowchart.

- **Communication:** Flowcharts are a good way of communication. They quickly and clearly provide logic, ideas and descriptions of algorithms to other programmers, students, teacher, computer operators and users.
- **Effective Analysis:** Flowchart provides a clear overview of the entire problem and its algorithm. They show all the major components and their relationships.
- **Proper Documentation:** The flowchart provides a permanent recording of program logic. It documents the steps followed in an algorithm.
- **Efficient Coding:** The flowchart specifies all the steps to be coded and helps to prevent omission or error. Thus, a flowchart acts as a guide or blueprint during the system analysis and program development phase.



2.a. What is an operator? Explain arithmetic, relational, logical and assignment operators in C language.

An *operator* is a symbol that operates on single or multiple data items. They are used to perform mathematical or logical manipulations. The data items that operators act upon are called *operands*. Some operators require two operands, while others act upon only one operand.

11. According to number of operands

m) Unary Operators (single operand)

e.g., unary minus (-), unary plus (+), ++, --, !, ~, sizeof, type etc.

n) Binary Operators (two operands)

e.g., Binary plus (+), Binary minus (-), *, /, >, <, &&, << etc.

o) Ternary Operators (three operands)

e.g., conditional operator (?:)

2. According to Operations

- gg. Arithmetic Operators
- hh. Relational Operators
- ii. Logical Operators
- jj. Assignment Operators
- kk. Increment/Decrement
- ll. Conditional Operator
- mm. Bitwise Operators
- nn. Special operators

o. Arithmetic Operators

The C provides all the Arithmetic Operators (*binary*) which perform arithmetic operations.

Operator	Purpose	Example	Output int a=20, b=6
+	Addition	a + b	26
-	Subtraction	a - b	14
*	Multiplication	a * b	120
/	Division	a / b	3
%	Remainder after integer division	a % b	2

The operands acted upon by arithmetic operators must represent numeric values. The division operator requires that the second operand must be non-zero. The remainder operator (%) requires that both operands must be integers and the second operand should be non-zero.

p. Relational Operators

They are binary operators which are used to compare two similar operands. They compare their left-hand side operand with their right-hand side operand for comparisons according to magnitudes. The output of a relational operation is either *true* (1) or *false* (0). So, they are used in decision making statements.

Operator	Purpose	Example	Output int a=20, b=6	Output int a=20, b=20
<	Less than	a < b	0	0
>	Greater than	a > b	1	0
<=	Less than or Equal to	a <= b	0	1
>=	Greater than or Equal to	a >= b	1	1

These operators all fall within the same precedence group, which is lower than the arithmetic and unary operators. The associativity of these operators is left to right.

Closely associated with the relational operators are the following two *equality operators*.

Operator	Purpose	Example	Output int a=20, b=6	Output int a=20, b=20
==	Equal to	a == b	0	1
!=	Not Equal to	a != b	1	0

The equality operators fall into a separate precedence group, beneath the relational operators. These operators also have a *left-to-right associativity*.

These six operators are used to form **logical expressions**, which represent conditions that are either true or false. The resulting expressions will be of type integer, since *true* is represented by the integer value 1 and *false* is represented by the value 0.

c. Logical Operators

In addition to the relational and equality operators, C contains two *logical operators* (also called *logical connectives*). They are

Operator	Meaning
&&	Logical AND
	Logical OR

The logical operators act upon operands that are themselves logical expressions. The net effect is to combine the individual logical expressions into more complex conditions that are either true or false. The result of a *logical and* operation will be true only if both operands are true, whereas the result of a *logical or* operation will be true if either operand is true or if both operands are true. In other words, the result of a *logical or* operation will be false only if both operands are false.

Suppose that *i* is an integer variable whose value is 7, *f* is a floating-point variable whose value is 5.5, and *c* is a character variable that represents the character 'w'. Several complex logical expressions that make use of these variables are shown below.

Expression	Interpretation	Value
(i >= 6) && (c == 'w')	True	1
(i >= 6) (c == 119)	True	1
(f < 11) && (i > 100)	False	0
(c != 'p') ((i + f) <= 10)	True	1

The first expression is true because both operands are true. In the second expression, both operands are again true; hence the overall expression is true. The third expression is false because the second operand is false. And finally, the fourth expression is true because the first operand is true.

Each of the logical operators falls into its own precedence group. *Logical and* has a higher precedence than *logical or*. Both precedence groups are lower than the group containing the equality operators. The associativity is left to right. The precedence groups are summarized below.

C also includes the unary operator **!** that negates the value of a logical expression; i.e., it causes an expression that is originally true to become false, and vice versa. This operator is referred to as the *logical negation* (or *logical not*) operator.

Suppose that *i* is an integer variable whose value is 7, and *f* is a floating-point variable whose value is 5.5. Several logical expressions which make use of these variables and the logical negation operator are shown below.

Expression	Interpretation	Value
f > 5	True	1
! (f > 5)	False	0
i <= 3	False	0
! (i <= 3)	True	1
i > (f + 1)	True	1
! (i > (f + 1))	False	0

q. Assignment Operator

There are several different assignment operators in C. All of them are used to form *assignment expressions*, which assign the value of an expression to an identifier. The most used assignment operator is **=**. Assignment expressions that make use of this operator are written in the form

identifier = expression

where *identifier* generally represents a variable, and *expression* represents a constant, a variable or a more complex expression.

Here are some typical assignment expressions that make use of the = operator.

```
a = 3;
x = y;
delta = 0.001;
sum = a + b;
area = length * width;
```

The first assignment expression causes the integer value **3** to be assigned to the variable *a*, and the second assignment causes the value of *y* to be assigned to *x*. In the third assignment, the floating-point value **0.001** is assigned to *delta*. The last two assignments each result in the value of an arithmetic expression being assigned to a variable (i.e., the value of **a + b** is assigned to *sum*, and the value of **length * width** is assigned to *area*).

Under some circumstances, this automatic type conversion can result in an alteration of the data being assigned. For example:

- A floating-point value may be rounded if assigned to an integer identifier.
- A double-precision value may be rounded if assigned to a floating-point (single-precision) identifier.
- An integer quantity may be altered if assigned to a shorter integer identifier or to a character identifier (some high-order bits may be lost).

Multiple assignments of the form

```
identifier 1 = identifier 2 = ... = expression
```

are permissible in C. In such situations, the assignments are carried out from right to left. Thus, the multiple assignment

```
identifier 1 = identifier 2 = expression
```

is equivalent to

```
identifier 1 = (identifier 2 = expression)
```

and so on, with right-to-left nesting for additional multiple assignments.

Suppose that *i* and *j* are integer variables. The multiple assignment expression

```
i = j = 5;
```

will cause the integer value 5 to be assigned to both *i* and *j*. (To be more precise, 5 is first assigned to *j*, and the value of *j* is then assigned to *i*.) Similarly, the multiple assignment expression

```
i = j = 5.9;
```

will cause the integer value 5 to be assigned to both *i* and *j*. Remember that truncation occurs when the floating-point value 5.9 is assigned to the integer variable *j*.

C contains the following five additional assignment operators: **+=**, **-=**, ***=**, **/=** and **%=**. To see how they are used, consider the first operator, **+=**. The assignment expression

```
expression 1 += expression 2
```

is equivalent to

```
expression 1 = expression 1 + expression 2
```

Similarly, the assignment expression

```
expression 1 -= expression 2
```

is equivalent to

`expression 1 = expression 1 - expression 2`

and so on for all five operators.

Usually, *expression 1* is an identifier, such as a variable or an array element.

Suppose that **i** and **j** are integer variables whose values are 5 and 7, and **f** and **g** are floating-point variables whose values are 5.5 and -3.25. Several assignment expressions that make use of these variables are shown below. Each expression utilizes the original values of *i*, *j*, *f* and *g*.

Expression	Equivalent Expression	Final Value
<code>i += 5</code>	<code>i = i + 5</code>	10
<code>f -= g</code>	<code>f = f - g</code>	8.75
<code>j *= (i - 3)</code>	<code>j = j * (i - 3)</code>	14
<code>f /= 3</code>	<code>f = f / 3</code>	1.833333
<code>i %= (j - 2)</code>	<code>i = i % (j - 2)</code>	0

2.b. Explain entry-controlled and exit-controlled loops with example. Compare continue and break statements.

Entry-Controlled Loop

The *while* loop and *for* loop are entry-controlled loop.

The while Statement

The **while** statement is used to carry out looping operations, in which a group of statements is executed repeatedly, until some condition has been satisfied. The general form of the **while** statement is

while (expression) statement

The **statement** will be executed repeatedly, as long as the **expression** is true (i.e., as long as **expression** has a nonzero value). This **statement** can be simple or compound, though it is usually a compound statement. It must include some feature that eventually alters the value of the **expression**, thus providing a stopping condition for the loop.

Consecutive Integer Quantities Suppose we want to display the consecutive digits 0, 1, 2, . . . , 9, with one digit on each line. This can be accomplished with the following program.

```
#include <stdio.h>
main()
{
    int digit = 0;

    while (digit<=9)
    {
        printf("%d\n", digit);
        ++digit;
    }
}
```

Initially, **digit** is assigned a value of 0. The **while** loop then displays the current value of **digit**, increases its value by 1 and then repeats the cycle, until the value of **digit** exceeds 9. The net effect is that the body of the loop will be repeated 10 times, resulting in 10 consecutive lines of output. Each line will contain a successive integer value, beginning with 0 and ending with 9. Thus, when the program is executed, the following output will be generated.

0
1

2
3
4
5
6
7
8
9

The for Statement

The **for** statement is the third and perhaps the most used looping statement in C. This statement includes an expression that specifies an initial value for an index, another expression that determines whether the loop is continued, and a third expression that allows the index to be modified at the end of each pass. The general form of the **for** statement is

for (expression 1; expression 2; expression 3) statement

where **expression 1** is used to initialize some parameter (called an **index**) that controls the looping action, **expression 2** represents a condition that must be true for the loop to continue execution, and **expression 3** is used to alter the value of the parameter initially assigned by **expression 1**. Typically, **expression 1** is an assignment expression, **expression 2** is a logical expression and **expression 3** is a unary expression or an assignment expression.

When the **for** statement is executed, **expression 2** is evaluated and tested at the **beginning** of each pass through the loop, and **expression 3** is evaluated at the end of each pass. Thus, the **for** statement is equivalent to

```
expression 1;
while (expression 2)
{
    statement
    expression 3;
}
```

The looping action will continue as long as the value of **expression 2** is not zero, that is, as long as the logical condition represented by **expression 2** is true.

The **for** statement, like the **while** and the **do-while** statements, can be used to carry out looping actions where the number of passes through the loop is not known in advance. Because of the features that are built into the **for** statement, however, it is particularly well suited for loops in which the number of passes is known in advance. As a rough rule of thumb, **while** loops are generally used when the number of passes is **not** known in advance, and **for** loops are generally used when the number of passes is known in advance.

Exit-Controlled Loop

The **do-while** loop is exit-controlled loop.

The do-while Statement

When a loop is constructed using the **while** statement, the test for continuation of the loop is carried out at the **beginning** of each pass. Sometimes, however, it is desirable to have a loop with the test for continuation at the **end** of each pass. This can be accomplished by means of the **do - while** statement. The general form of the **do - while** statement is

```
do statement while (expression);
```

The **statement** will be executed repeatedly, as long as the value of **expression** is true (i.e., is nonzero). Notice that **statement** will always be executed *at least once*, since the test for repetition does not occur until the end of the first pass through the loop. The **statement** can be either simple or compound, though most applications will require it to be a compound statement. It must include some feature that eventually alters the value of **expression** so the looping action can terminate.

For many applications it is more natural to test for continuation of a loop at the beginning rather than at the end of the loop. For this reason, the **do-while** statement is used less frequently than the **while** statement. For illustrative purposes, however, the programming examples shown in last section are repeated below using the **do-while** statement for the conditional loops.

Consecutive Integer Quantities In the above example, we saw two complete C programs that use the **while** statement to display the consecutive digits 0, 1, 2, . . . , 9. Here is another program to do the same thing, using the **do-while** statement in place of the **while** statement.

```
#include <stdio.h>
main()
{
    int digit = 0;

    do
    {
        printf("%d\n", digit++);
    }while(digit<=9);
}
```

As in the earlier example, *digit* is initially assigned a value of 0. The *do-while* loop displays the current value of *digit*, increases its value by 1, and then tests to see if the current value of *digit* exceeds 9. If so, the loop terminates; otherwise, the loop continues, using the new value of *digit*. Note that the test is carried out at the end of each pass through the loop. The net effect is that the loop will be repeated 10 times, resulting in 10 successive lines of output. Each line will appear exactly as shown in the above Example.

The break Statement

The **break** statement is used to *terminate loops* or to *exit from a switch*. It can be used within a *for*, *while*, *do - while*, or *switch* statement. The break statement is written simply as

```
break;
```

without any embedded expressions or statements.

We have already seen several examples of the use of the break statement within a switch statement, in the last section. The *break* statement causes a transfer of control out of the entire switch statement, to the first statement following the switch statement.

Consider once again the *switch* statement originally presented in the last Example.

```
switch (choice = toupper(getchar()))
{
    case 'R':
        printf("red");
```

```

        break;
    case 'W':
        printf("white");
        break;
    case 'B':
        printf("blue");
        break;
    default:
        printf("Error");
        break;
}

```

Notice that each group of statements ends with a **break** statement, in order to transfer control out of the **switch** statement. The **break** statement is required within each of the first three groups, in order to prevent the succeeding groups of statements from executing. The last group does not require a **break** statement, since control will automatically be transferred out of the **switch** statement after the last group has been executed. This last **break** statement is included, however, as a matter of good programming practice, so that it will be present if another group of statements is added later.

If a break statement is included in a *while*, *do - while* or *for* loop, then control will immediately be transferred out of the loop when the break statement is encountered. This provides a convenient way to terminate the loop if an error or other irregular condition is detected.

Here are some illustrations of loops that contain **break** statements. In each situation, the loop will continue to execute as long as the current value for the floating-point variable *x* does not exceed **100**. However, the computation will break out of the loop if a negative value for *x* is detected.

First, consider a **while** loop.

```

scanf(" %f", &x);
while (x <= 100)
{
    if (x < 0)
    {
        printf("ERROR! NEGATIVE VALUE FOR x");
        break;
    }
    /* process the non-negative value of x */
    ...
    scanf(" %f", &x);
}

```

Now consider a **do - while** loop that does the same thing.

```

do
{
    scanf(" %f", &x);
    if (x < 0)
    {
        printf("ERROR! NEGATIVE VALUE FOR x");
        break;
    }
}

```

```
/* process the non-negative value of x */
...
}while (x <= 100);
```

Finally, here is a **for** loop that is similar.

```
for (count = 1; x <= 100; ++count)
{
    scanf(" %f", &x);
    if (x < 0)
    {
        printf("ERROR! NEGATIVE VALUE FOR x");
        break;
    }
    /* process the non-negative value of x */
    ...
}
```

In the event of several *nested while*, *do - while*, *for* or *switch* statements, a **break** statement will cause a transfer of control out of the immediate enclosing statement, but not out of the outer surrounding statements.

Consider the following outline of a **while** loop embedded within a **for** loop.

```
for (count = 0; count <= n; ++count)
{
    ...
    while (c = getchar() != '\n')
    {
        if (c = '*')
            break;
        ...
    }
}
```

If the character variable *c* is assigned an asterisk (*), then the **while** loop will be terminated. However, the **for** loop will continue to execute. Thus, if the value of *count* is less than *n* when the breakout occurs, the computer will increment *count* and make another pass through the **for** loop.

The continue Statement

The **continue** statement is used to *bypass* the remainder of the current pass through a loop. The loop does not terminate when a **continue** statement is encountered. Rather, the remaining loop statements are skipped and the computation proceeds directly to the next pass through the loop. (Note the distinction between **continue** and **break**.)

The *continue* statement can be included within a *while*, a *do - while* or a *for* statement. It is written simply as

continue;

without any embedded statements or expressions.

Here are some illustrations of loops that contain **continue** statements. First, consider a *do - while* loop.

do


```

{
    scanf(" %f", &x);
    if (x < 0)
    {
        printf("ERROR! NEGATIVE VALUE FOR x");
        continue;
    }
    /* process the non-negative value of x */
    ...
}while (x <= 100);

```

Here is a similar *for* loop

```

for (count = 1; x <= 100; ++count)
{
    scanf(" %f", &x);
    if (x < 0)
    {
        printf("ERROR! NEGATIVE VALUE FOR x");
        continue;
    }
    /* process the non-negative value of x */
    ...
}

```

In each case, the processing of the current value of *x* will be bypassed if the value of *x* is negative. Execution of the loop will then continue with the next pass.

3.a. Why array is called static data type? Write a program to find sum of diagonal elements of $m \times n$ matrix.

Unlike pointer declaration, once array is declared, then fixed stack memory is allocated which in later cannot be unchanged (increased or decreased). So, array is called static data type.

For example, if we declare an array as follows:

```
char Name[50];
```

then 50 bytes (Consider character variable needs 1 byte) of stack is allocated, if available, which in later the allocated size be neither increased nor decreased as needed.

```
//sum of diagonal elements of mxn matrix
```

```
#include<stdio.h>
```

```
void main()
```

```

{
    int mat[10][10], i, j, sum = 0, m, n;

    printf("Enter no. of rows and columns: ");
    scanf(" %d %d", &m, &n);
    printf("Enter elements of %dx%d matrix: ", m, n);
    for (i = 0; i < m; ++i)
    {
        for (j = 0; j < n; ++j)
        {
            scanf(" %d", &mat[i][j]);
        }
    }
}

```

```

}

for (i = 0; i < m; ++i)
{
    for (j = 0; j < n; ++j)
    {
        if (i == j)
        {
            sum += mat[i][j];
        }
    }
}

printf("Sum of square of diagonal matrix = %d", sum);
}

```

OUTPUT

Enter no. of rows and columns: 3 3

Enter elements of 3x3 matrix:

```

1 2 3
4 5 6
7 8 9

```

Sum of square of diagonal matrix = **15**

3.b. Define string. Explain the string handling functions with suitable example.

The one-dimensional, char-type array (often called a one-dimensional *character* array). This type of array is generally used to represent a *string*. Each array element will represent one character within the string. Thus, the entire array can be thought of as an ordered list of characters.

Since the array is one-dimensional, there will be a single *subscript* (sometimes called an *index*) whose value refers to individual array elements. If the array contains *n* elements, the subscript will be an integer quantity whose values range from 0 to *n*-1. Note that an *n*-character string will require an (*n*+1)-element array, because of the *null* character (*\0*) that is automatically placed at the end of the string.

Suppose that the string “Kathmandu” is to be stored in a one-dimensional character array called *letter*. Since “Kathmandu” contains 9 characters, *letter* will be a 10-element array. Thus, *letter*[0] will represent *K*, *letter*[1] represents *a*, and so on. Note that the last (10th) array element, *letter*[9], represents the *null character* (*\0*) which signifies the end of the string (*letter*).

K	a	t	h	m	a	n	d	u	\0
0	1	2	3	4	5	6	7	8	9

String Handling Functions

Here are some string handling functions which are used to manipulate strings on C. These functions are defined at header file ***string.h***.

i. **strlen()**

It returns the length of string parameter. Syntax:

```
len = strlen(str);
```

Here *str* is string variable or string constant and *len* is an integer variable which holds the length returned by *strlen()*. E.g.:

```
len = strlen("Nepal");
```

The *len* holds 5 which is length of string constant “Nepal”.

ii. **strlwr()**

It converts each uppercase character into lowercase character presented in given string. Syntax:

```
strlwr(str) ;
```

Here *str* is string variable or string constant. The new string (after conversion) is stored at the same variable. E.g.:

```
strlwr("Nepal");
```

It returns string *"nepal"*.

```
char str[] = "Nepal";
```

```
strlwr(str);
```

Then new string at *str* is *"nepal"*.

iii. **strupr()**

It converts each lowercase character into uppercase character presented in given string. Syntax:

```
strupr(str) ;
```

Here *str* is string variable or string constant. The new string (after conversion) is stored at the same variable. E.g.:

```
strupr("Nepal");
```

It returns string *"NEPAL"*.

```
char str[] = "Nepal";
```

```
strupr(str);
```

Then new string at *str* is *"NEPAL"*.

iv. **strrev()**

It reverses the contents of string (except \0) supplied. The new string is stored at the same variable.

Syntax:

```
strrev(str) ;
```

where *str* is string variable or string constant. For example,

```
char str[] = "Nepal";
```

```
strrev(str);
```

the *str* now has string *"lapeN"*.

v. **strcpy()**

Syntax:

```
strcpy(dest_string, src_string) ;
```

It copies contents of source string (*src_string*) to destination string (*dest_string*). After copying, the *dest_string* has the same value of *src_string*. E.g.:

```
char text[20] = "Nepal";
```

```
char str[20];
```

```
strcpy(str, text);
```

Now, the *str* and *text* has the same content i.e. *"Nepal"*.

vi. **strcat()**

Syntax:

```
strcat(str1, str2) ;
```

It concatenates *str2* to end of *str1*. The new string is stored at *str1*. For e.g.,

```
char fname[20] = "My";
char lname[10] = "Nepal";
strcat(fname, lname);
```

The new string "MyNepal" is stored at *fname*.

vii. **strcmp()**

Syntax:

```
strcmp(str1, str2);
```

it compares two strings, str1 and str2, character by character then returns

- *positive* value if str1 is greater than str2
- *negative* value if str1 is less than str2
- *zero* if str1 and str2 both are identical.

For example:

```
strcmp("ABC", "abc");
```

It returns negative value since ASCII value of 'A' is less than 'a'.

```
strcmp("abc", "ABC");
```

It returns positive value since ASCII value of 'a' is greater than 'A'.

```
strcmp("xyz", "xyz");
```

It returns zero since both strings are same.

4.a. List the advantage of recursive function. Write a recursive program to generate the 10 terms of fibonacci sequence starting from 2.

Advantages of Recursive function are listed below.

- The repetitive structure results in shorter codes.
- Complex case analysis and nested loops can be avoided.
- Recursion can lead to more readable and efficient algorithm descriptions.
- Recursion is very useful to solve those problems which are recursive in nature (e.g., Tower of Hanoi)

```
//fibonacci series using recursion
#include <stdio.h>
```

```
int fibo(int); //prototype
```

```
void main()
{
    int i,f;
    /* second term is 4 */
    for (i = 4; i <= 13; ++i)
    {
        f = fibo(i);
        printf("%d ", f);
    }
}
```

OUTPUT:

2 3 5 8 13 21 34 55 89 144

4.b. What do you mean by storage class? Define its type with suitable example.

There are two different ways to characterize variables: by **data type**, and by **storage class**. Data type refers to the type of information represented by a variable, e.g., integer number, floating-point number, character, etc. Storage class refers to the **permanence** of a variable, and its **scope** within the program, i.e., the portion of the program over which the variable is recognized.

There are four different storage-class specifications in C: **automatic**, **external**, **static** and **register**. They are identified by the keywords **auto**, **extern**, **static**, and **register**, respectively. The storage class associated with a variable can sometimes be established simply by the location of the variable declaration within the program. In other situations, however, the keyword that specifies a particular storage class must be placed at the beginning of the variable declaration.

Automatic Variables

Automatic variables are always declared within a function and are local to the function in which they are declared; that is, their scope is confined to that function. Automatic variables defined in different functions will therefore be independent of one another, even though they may have the same name.

Any variable declared within a function is interpreted as an automatic variable unless a different storage-class specification is shown within the declaration. This includes formal argument declarations. All the variables in the programming examples encountered earlier have been automatic variables.

Since the location of the variable declarations within the program determines the *automatic storage class*, the keyword **auto** is not required at the beginning of each variable declaration. There is no harm in including an auto specification within a declaration, though this is normally not done.

Automatic variables can be assigned initial values by including appropriate expressions within the variable declarations, as in the above example, or by explicit assignment expressions elsewhere in the function. Such values will be reassigned each time the function is reentered. If an automatic variable is not initialized in some manner, however, its initial value will be *unpredictable*, and probably *unintelligible* (i.e., *garbage value*).

An automatic variable does not retain its value once control is transferred out of its defining function.

External (Global) Variables

External variables, in contrast to automatic variables, are not confined to single functions.

Their scope extends from the point of definition through the remainder of the program. Hence, they usually span two or more functions, and often an entire program. They are often referred to as **global variables**.

Since external variables are recognized globally, they can be accessed from any function that falls within their scope. They retain their assigned values within this scope. Therefore, an external variable can be assigned a value within one function, and this value can be used (by accessing the external variable) within another function.

The use of external variables provides a convenient mechanism for **transferring information back and forth between functions**. We can transfer information into a function without using arguments. This is especially convenient when a function requires numerous input data items. Moreover, we now have a way to transfer multiple data items out of a function, since the *return* statement can return only one data item.

The storage-class specifier **extern** is not required in an external variable definition, since the external variables will be identified by the location of their definition within the program. In fact, many C compilers forbid the use of **extern** within an external variable definition. External variables can be assigned initial values as a part of the variable definitions, but the initial values must be expressed as **constants** rather than expressions. These initial values will be assigned only once, at the beginning of the program. The external variables will then retain these initial values unless they are later altered during the execution of the program. If an initial value is not included in the definition of an external variable, the variable will automatically be assigned a value of **zero**.

Static Variables

Static variables are defined within individual functions and therefore have the same scope as automatic variables; i.e., they are local to the functions in which they are defined. Unlike automatic variables, however, static variables retain their values throughout the life of the program. Thus, if a function is exited and then re-entered later, the static variables defined within that function will retain their former values. This feature allows functions to retain information permanently throughout the execution of a program.

Static variables are defined within a function in the same manner as automatic variables, except that the variable declaration must begin with the **static** storage-class designation. Static variables can be utilized within the function in the same manner as other variables. They cannot, however, be accessed outside of their defining function.

It is not unusual to define automatic or static variables having the same names as external variables. In such situations the local variables will *take precedence over* the external variables, though the values of the external variables will be unaffected by any manipulation of the local variables. Thus, the external variables maintain their independence from locally defined automatic and static variables. The same is true of local variables within one function that have the same names as local variables within another function.

Initial values can be included in the static variable declarations. The rules associated with the assignment of these values are essentially the same as the rules associated with the initialization of external variables, even though the static variables are defined locally within a function. In particular:

- The initial values must be expressed as constants, not expressions.
- The initial values are assigned to their respective variables at the beginning of program execution. The variables retain these values throughout the life of the program, unless different values are assigned during the computation.
- **Zeros** will be assigned to all static variables whose declarations do not include explicit initial values. Hence, static variables will always have assigned values.

Register Variables

Registers are special storage areas within the computer's central processing unit. The actual arithmetic and logical operations that comprise a program are carried out within these registers. Normally, these operations are carried out by transferring information from the computer's memory to these registers, carrying out the indicated operations, and then transferring the results back to the computer's memory. This general procedure is repeated many times during a program's execution.

For some programs, the execution time can be reduced considerably if certain values can be stored within these registers rather than in the computer's memory. Such programs may also be somewhat smaller in size (i.e., they may require fewer instructions), since fewer data

transfers will be required. Usually, however, the size reduction will not be dramatic and will be less significant than the savings in execution time.

In C, the values of **register variables** are stored within the registers of the central processing unit. A variable can be assigned this storage class simply by preceding the type declaration with the keyword **register**. There can, however, be only a few register variables (typically, two or three) within any one function. The exact number depends upon the particular computer, and the specific C compiler. Usually, only integer variables are assigned the register storage class.

The register and automatic storage classes are closely related. Their visibility (i.e., their scope) is the same. Thus, **register** variables, like automatic variables, are local to the function in which they are declared. Furthermore, the rules governing the use of register variables are the same as those for automatic variables, except that the address operator (**&**) cannot be applied to register variables.

The similarities between register and automatic variables is not coincidental, because the register storage class can be assigned only to variables that would otherwise have the storage class automatic. Moreover, declaring certain variables to be register variables does not guarantee that they will be treated as register variables. The declaration will be valid only if the requested register space is available. If a register declaration is not honored, the variables will be treated as having the storage class automatic. A program that makes use of **register** variables should run faster than the corresponding program without **register** variables. It may also be somewhat smaller in size.

The storage class associated with a variable can sometimes be established simply by the location of the variable declaration within the program. In other situations, however, the keyword that specifies a particular storage class must be placed at the beginning of the variable declaration.

Shown below are several typical variable declarations that include the specification of a storage class.

```
auto int a, b, c;
extern float root1, root2;
static int count = 0;
extern char star;
```

The first declaration states that **a**, **b** and **c** are automatic integer variables, and the second declaration establishes **root1** and **root2** as external floating-point variables. The third declaration states that **count** is a static integer variable whose initial value is 0, and the last declaration establishes **star** as an external character-type variable.

5.a. Write a program to sort the array using Dynamic memory allocation.

```
/* reordering a one-dimensional, integer array from
smallest to largest, using pointer notation */
#include <stdio.h>
#include <stdlib.h>

void reorder(int n, int *x);

main()
{
    int i, n, *x;

    /* read in a value for n */
```

```

printf("\nHow many numbers will be entered? ");
scanf(" %d", &n);

/* dynamically allocate memory for n numbers */
x = (int *) malloc(n * sizeof(int));

/*read in the list of numbers */
for (i = 0; i < n; ++i)
{
    printf("i = %d x = ", i + 1);
    scanf(" %d", x + i);
}

/* reorder all array elements */
reorder(n, x);

/* display the reordered list of numbers */
printf("\nReordered list of numbers:\n");
for (i = 0; i < n; ++i)
    printf("i = %d x = %d\n", i+1, *(x + i));
free(x);
}

void reorder(int n, int *x)
{
    int i, item, temp;
    for (item = 0; item < n - 1; ++item)
        for (i = item + 1; i < n; ++i)
            if(*(x + i) < *(x + item))
            {
                temp = *(x + item);
                *(x + item) = *(x + i);
                *(x + i) = temp;
            }
    return;
}

```

OUTPUT

How many numbers will be entered? 10

```

i = 1    x = 20
i = 2    x = 30
i = 3    x = 10
i = 4    x = 5
i = 5    x = -15
i = 6    x = 15
i = 7    x = 25
i = 8    x = 35
i = 9    x = 22
i = 10   x = 40

```


Reordered list of numbers:

```
i = 1    x = -15
i = 2    x = 5
i = 3    x = 10
i = 4    x = 15
i = 5    x = 20
i = 6    x = 22
i = 7    x = 25
i = 8    x = 30
i = 9    x = 35
i = 10   x = 40
```

5.b. Differentiate between pass by value and pass by reference with suitable example.

Call by Value

When a single value is passed to a function via an actual argument, the value of the actual argument is *copied* into the function. Therefore, *the value of the corresponding formal argument can be altered within the Function, but the value of the actual argument within the calling routine will not change*. This procedure for passing the value of an argument to a function is known as *passing by value*.

Here is a simple C program containing a function that alters the value of its argument.

```
#include <stdio.h>

void modify(int a);

main()
{
    int a = 2;

    printf("\na = %d (from main, before calling the
function)", a);
    modify(a); // function call a is a normal variable
    printf("\na = %d (from main, after calling the
function)", a);
}

void modify(int a)
{
    a *= 3;
    printf("\na = %d (from the function, after being
modified)", a);
    return;
}
```

The original value of a (i.e., **a = 2**) is displayed when main begins execution. This value is then passed to the function modify, where it is multiplied by **3** and the new value displayed. Note that it is the altered value of the formal argument that is displayed within the function. Finally, the value of a within main (i.e., the actual argument) is again displayed, after control

is transferred back to main from modify. When the program is executed, the following output is generated.

```
a = 2 (from main, before calling the function)
a = 6 (from the function, after being modified)
a = 2 (from main, after calling the function)
```

These results show that **a** is not altered within main, even though the corresponding value of **a** is changed within **modify()**.

Passing an argument by value has advantages and disadvantages. On the plus side, it allows a single-valued actual argument to be written as an expression rather than being restricted to a single variable. Moreover, if the actual argument is expressed simply as a single variable, it protects the value of this variable from alterations within the function. On the other hand, it does not allow information to be transferred back to the calling portion of the program via arguments. Thus, *passing by value is restricted to a one-way transfer of information.*

Call by Reference

To pass an array to a function, the array name must appear by itself, without brackets or subscripts, as an actual argument within the function call. The corresponding formal argument is written in the same manner, though it must be declared as an array within the formal argument declarations. When declaring a one-dimensional array as a formal argument, the array name is written with a pair of empty square brackets. The size of the array is *not specified* within the formal argument declaration.

When an array is passed to a function, however, the values of the array elements *are not* passed to the function. Rather, the array name is interpreted as the **address** of the first array element (i.e., the address of the memory location containing the first array element). This address is assigned to the corresponding formal argument when the function is called. The formal argument therefore becomes a **pointer** to the first array element (more about this in the next chapter). Arguments that are passed in this manner are said to be passed **by reference** rather than passed *by value*.

When a reference is made to an array element within the function, the value of the element's subscript is added to the value of the pointer to indicate the address of the specified array element. Therefore, any array element can be accessed from within the function. Moreover, *if an array element is altered within the function, the alteration will be recognized in the calling portion of the program* (actually, throughout the entire scope of the array).

Here is a simple C program that passes a three-element integer array to a function, where the array elements are altered. The values of the array elements are displayed at three different places in the program, thus illustrating the effects of the alterations.

```
#include <stdio.h>
void modify(int a[]); /* function prototype */

main()
{
    int count, a[3];

    printf("\nEnter any three numbers:\n");
    for (count = 0; count < 3; ++count)
    {
        printf("\na[%d] = ", count);
        scanf(" %d", &a[count]);
    }
}
```

```

    }

    printf("From main(), before modification:\n");
    for (count = 0; count < 3; ++count)
    {
        printf("\na[%d] = %d", count, a[count]);
    }

    modify(a); /* function call a is an array*/

    printf("From main(), after modification:\n");
    for (count = 0; count < 3; ++count)
    {
        printf("\na[%d] = %d", count, a[count]);
    }
}

void modify(int a[])    /* function definition */
{
    int count;

    printf("\nFrom the function, after modification the
values:\n");
    for (count = 0; count < 3; ++count)
    {
        a[count] = -9;
        printf("a[%d] = %d\n", count, a[count]);
    }
    //return;
}

```

The array elements are read from the user within the first loop appearing in *main*. These values are displayed as soon as they are assigned. The array is then passed to the function **modify**, when each array element is assigned the value **-9**. These new values are then displayed from within the function. Finally, the values of array elements are again displayed from **main()**, after control has been transferred back to *main* from *modify*. When the program is executed, the following output is generated.

```

Enter any three numbers:
a[0] = 1
a[1] = 2
a[2] = 3
From main(), before modification:
a[0] = 1
a[1] = 2
a[2] = 3
From the function, after modification the values:
a[0] = -9
a[1] = -9
a[2] = -9

```

From `main()`, after modification:

```
a[0] = -9
a[1] = -9
a[2] = -9
```

These results show that the elements of *a* are altered within *main* as a result of the changes that were made within *modify*.

6.a. Write a program to create structure for the following data of student (RN, name, phone, address and semester). Read the 10 students by user and write only those students whose semester is 1 in file 'student.txt'.

```
#include <stdio.h>
#include <stdlib.h>

typedef struct
{
    int rn;
    char name[30];
    char phone[10];
    char address[50];
    int semester;
} Student;

void main()
{
    Student s;
    int i;
    FILE *fp;

    if ((fp = fopen("student.txt", "w")) == NULL)
    {
        printf("The designated file cannot be created.");
        exit(1);
    }

    printf("\nEnter details about students\n");
    for (i = 0; i < 10; ++i)
    {
        printf("\n\tStudent %d\n", i+1);
        printf("Roll Number: ");
        scanf(" %d", &s.rn);
        printf("Name: ");
        scanf(" %[^\n]", s.name);
        printf("Phone: ");
        scanf(" %[^\n]", s.phone);
        printf("Address: ");
        scanf(" %[^\n]", s.address);
        printf("Semester: ");
        scanf(" %d", &s.semester);

        if (s.semester == 1)
```

```

        fwrite(&s, sizeof(Student), 1, fp);
    }
    fclose(fp);
}

```

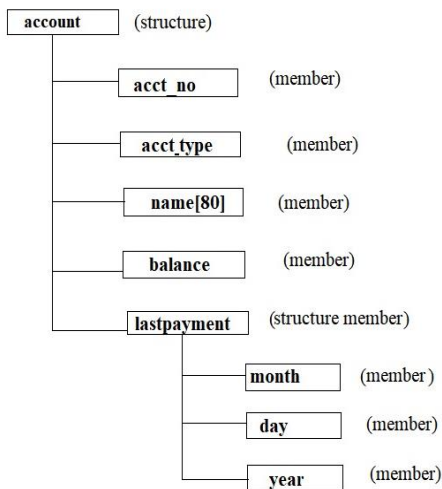
6. b. Differentiate structure and union. How the members of nested structure are accessed? Show it with example.

Differences between Structure and Union

Structure	Union
Each member of structure is assigned its own <i>unique storage</i> . It takes more memory spaces than that of union.	All members of a union share the <i>same storage</i> area of computer memory. It takes less memory than that of structure.
The amount of memory required to store a structure is the sum of the memory sizes required by <i>all</i> its members.	The amount of memory required to store a union is same as memory size occupied by a member which requires <i>largest</i> memory space among the members.
<i>All</i> the members of structure can be accessed at any point of time.	<i>Only one</i> member of union can be accessed at any given time.
Structure is suitable when you have <i>enough memory spaces</i> . It is very efficient to use and handle.	Union is used when you have <i>lack of memory</i> . It is easy but not realistic.
Structure declaration: struct student { char name[20]; //20 int roll; //2 float marks; //4 char address[50]; //50 } st; Total size of struct student: 76B	Union declaration: union student { char name[20]; int roll; float marks; char address[50]; } st; Total size of union student: 50B

A structure variable may be defined as a member of another structure. In such situations, the declaration of the embedded structure must appear before the declaration of the outer structure.

A C program contains the following structure declarations.



```

struct date
{
    int month;
    int day;
    int year;
};

struct account
{
    int acct_no;
    char acct_type;
    char name[80];
    float balance;
    struct date lastpayment;
} oldcustomer, newcustomer;
  
```

The second structure (**account**) now contains another structure (**date**) as one of its members. Note that the declaration of **date** precedes the declaration of **account**. The composition of **account** is shown schematically in the Figure. The members of a structure variable can be assigned initial values in much the same manner as the elements of an array. The initial values must appear in the order in which they will be assigned to their corresponding structure members, enclosed in braces and separated by commas. The general form is

storage-class struct tag variable = {value 1, value 2, ..., value m}

where **value 1**, refers to the value of the first member, **value 2** refers to the second member, and so on. A structure variable, like an array, can be initialized only if its storage class is either *external* or *static*.

More complex expressions involving the repeated use of the period operator may also be written. For example, if a structure member is itself a structure, then a member of the embedded structure can be accessed by writing

variable.member.sub_member

where **member** refers to the name of the member within the outer structure, and **sub_member** refers to the name of the member within the embedded structure.

Consider the following structure declarations, which were presented in the previous example.

```

struct date
{
    int month;
    int day;
    int year;
};

struct account
{
    int acct_no;
    char acct_type;
    char name[80];
    float balance;
    struct date lastpayment;
};

struct account customer;
  
```

If we wanted to access the account number customer, we would write *customer.acct_no*. Similarly, this customer's balance can be accessed by writing *customer.balance*. The address can be obtained as *&customer.balance*.

The customer's name can be accessed by writing *customer.name*. Moreover, we can access an individual character within the name by specifying a subscript. For example, the 8th character within the customer's name can be accessed by writing *customer.name[7]*. In a similar manner we can access the month, day and year customer's last payment by specifying the individual members of *customer.lastpayment*, i.e., ***customer.lastpayment.month***, ***customer.lastpayment.day***, ***customer.lastpayment.year***. Moreover, the expression *++customer.lastpayment.day* causes the value of the day to be incremented.

7.a. Void Pointer

A void pointer is a pointer that has no associated data type with it. A void pointer can hold address of any type and can be typecast to any type.

```
int a = 10;
char b = 'x';
```

```
void *p = &a; // void pointer holds address of int 'a'
p = &b; // void pointer holds address of char 'b'
```

The *malloc()* and *calloc()* return **void *** type and this allows these functions to be used to allocate memory of any data type (just because of void *)

```
int *x;
x = (int *)malloc(sizeof(int) * n);
```

The void pointers cannot be *dereferenced*. For example, the following program doesn't compile.

```
#include<stdio.h>
int main()
{
    int a = 10;
    void *ptr = &a;
    printf("%d", *ptr);
    return 0;
}
```

Output:

Compiler Error: 'void*' is not a pointer-to-object type

The following program compiles and runs fine.

```
#include<stdio.h>
int main()
{
    int a = 10;
    void *ptr = &a;
    printf("%d", *(int *)ptr);
    return 0;
}
```

Output:

7.b. Escape Sequence

Certain nonprinting characters, as well as the backslash (\) and the apostrophe ('), can be expressed in terms of *escape sequences*. An escape sequence always begins with a backward slash and is followed by one or more special characters. For example, a line feed (**LF**), which is referred to as a *newline* in C, can be represented as \n. Such escape sequences always represent single characters, even though they are written in terms of two or more characters. The commonly used escape sequences are listed below.

<u>Character</u>	<u>Escape Sequence</u>	<u>ASCII Value</u>
bell (alert)	\a	007
backspace	\b	008
horizontal tab	\t	009
vertical tab	\v	011
newline (line feed)	\n	010
form feed	\f	012
carriage return	\r	013
quotation mark (")	\"	034
apostrophe (')	\'	039
question mark (?)	\?	063
backslash ()	\\	092
null	\0	000

Shown below are several character constants, expressed in terms of escape sequences.

'\n' '\t' '\b' '\\" x'\\'x '\'

Note that the last three escape sequences represent an apostrophe, a backslash and a quotation mark, respectively.

Of interest is the escape sequence \0. This represents the null character (ASCII 000), which is used to indicate the end of a string. Note that, the null character constant '\0' is not equivalent to the character constant '0'.

7.c. Go to Statement

The *goto* statement is used to alter the normal sequence of program execution by transferring control to some other part of the program. In its general form, the goto statement is written as

goto label;

where *label* is an identifier that is used to label the target statement to which control will be transferred. Control may be transferred to any other statement within the program. (To be more precise, control may be transferred anywhere within the current *function*. The target statement must be labeled, and the label must be followed by a colon. Thus, the target statement will appear as

label: statement

Each labeled statement within the program (more precisely, within the current function) must have a unique label; i.e., no two statements can have the same label.

The following skeletal outline illustrates how the goto statement can be used to transfer control out of a loop if an unexpected condition arises.

```
/* main loop */
```



```
scanf(" %f", &x);
while (x <= 100)
{
    ...
    if (x < 0) goto errorcheck;
    ...
    scanf(" %f", &x);
}
...
/* error detection routine */
errorcheck:
{
    printf("ERROR! NEGATIVE VALUE FOR x");
    ...
}
```

In this example control is transferred out of the while loop, to the compound statement whose label is **errorcheck**, if a negative value is detected for the input variable **x**.

The same thing could have been accomplished using the **break** statement, as illustrated in the last Example. The use of the **break** statement is actually the preferred approach. The use of the **goto** statement is presented here only to illustrate the syntax.

All of the popular general-purpose programming languages contain a **goto** statement, though modern programming practice discourages its use. The **goto** statement was used extensively, however, in early versions of some older languages, such as Fortran and BASIC. The most common applications were:

4. Branching around statements or groups of statements under certain conditions.
5. Jumping to the end of a loop under certain conditions, thus bypassing the remainder of the loop during the current pass.
6. Jumping completely out of a loop under certain conditions, thus terminating the execution of a loop.

The structured features in C enable all these operations to be carried out without resorting to the **goto** statement. For example, branching around statements can be accomplished with the **if - else** statement; jumping to the end of a loop can be carried out with the **continue** statement; and jumping out of a loop is easily accomplished using the **break** statement. The use of these structured features is preferable to the use of the **goto** statement, because the use of **goto** tends to encourage (or at least, not discourage) logic that skips all over the program whereas the structured features in C require that the entire program be written in an orderly, sequential manner. For this reason, **use of the goto statement should generally be avoided**.

2016 Spring

1.a. What is programming language? Why is high level programming language is preferred to low level language?

Solution:

Programming language is defined as a set of rules that provides a platform for instructing computer to perform some specific tasks. Programming languages are classified into various ways. Based on nature of language, they are classified into *general purpose* and *specific*

purpose language. On the other hand, programming language are classified into High-level language and Low-level language.

- xi. Low Level Language (LLL)
 - k. Machine Language
 - l. Assembly Language
- xii. High Level Language (HLL)
 - p. Third Generation Language (3GL)
 - q. Fourth Generation Language (4GL)
 - r. Fifth Generation Language (5GL)

High level programming language is preferred over low-level language because of

- i. **User Interface:** HLL provides better user interface than LLL. The user interface is user friendly.
- ii. **Modular Programming:** HLL supports programming which allows program divided into multiple modules (functions) and may distribute the whole task over several programmers.
- iii. **Machine Independent:** The programmer should not know about the internal architecture of the particular computer system while programming. Only the programming knowledge (without hardware knowledge) is sufficient.
- iv. **Easy to Use:** We can easily learn HLL. Also, coding, debugging, and maintenance is very easy.
- v. **Programming Productivity:** All the above-mentioned advantages lead to the better programming productivity than LLL.

1.b. What is the significance of algorithm and flowcharting in programming? Write an algorithm and draw neat flowchart to input a number to find whether the number is palindrome or not.

Solution:

Algorithm: It is very efficient tool during system development.

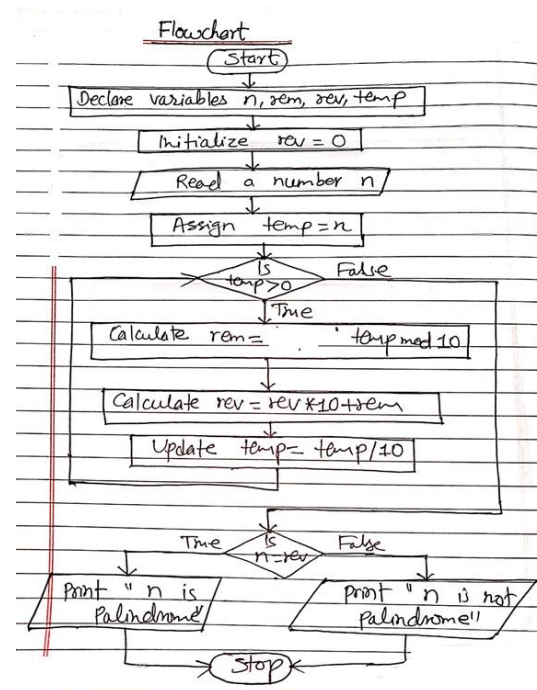
- It provides the core solution to a given problem. This solution can be implemented on a computer system using any programming language of users' choice.
- It facilitates program development by acting as a design document or blueprint of a given problem solution.
- It eases identification and removal of logical errors in a program.

Flowchart: It is a graphical representation of the particular algorithm.

- **Communication:** Flowcharts are a good way of communication. They quickly and clearly provide logic, ideas and descriptions of algorithms to other programmers, students, teacher, computer operators and users.
- **Effective Analysis:** Flowchart provides a clear overview of the entire problem and its algorithm. They show all the major components and their relationships.
- **Proper Documentation:** The flowchart provides a permanent recording of program logic. It documents the steps followed in an algorithm.
- **Efficient Coding:** The flowchart specifies all the steps to be coded and helps to prevent omission or error. Thus, a flowchart acts as a guide or blueprint during the system analysis and program development phase.

Algorithm to check whether the given number is palindrome or not.

1. Start
2. Declare variables n , rem , rev , $temp$.
3. Initialize $rev = 0$.
4. Read a number n .
5. Assign $temp = n$.
6. While **temp** > 0, repeat steps 7, 8, 9. Otherwise, go to step 10.
7. Calculate $rem = temp \text{ mod } 10$.
8. Calculate $rev = rev * 10 + rem$.
9. Update $temp = temp / 10$ and go to step 6.
10. Check if $n = rev$. If true, print "**n is palindrome**". Otherwise, print "**n is not palindrome.**"
11. Stop.



2 a. What is operator? Explain unary, binary and ternary operator with examples.

Solution

An *operator* is a symbol that operates on single or multiple data items. They are used to perform mathematical or logical manipulations. The data items that operators act upon are called *operands*. Some operators require two operands, while others act upon only one operand.

12. According to number of operands
 - p) Unary Operators (single operand)

e.g., unary minus (-), unary plus (+), ++, --, !, ~, sizeof, type etc.
 - q) Binary Operators (two operands)

e.g., Binary plus (+), Binary minus (-), *, /, >, <, &&, << etc.
 - r) Ternary Operators (three operands)

e.g., conditional operator (?:)
2. According to Operations
 - oo. Arithmetic Operators
 - pp. Relational Operators
 - qq. Logical Operators
 - rr. Assignment Operators
 - ss. Increment/Decrement
 - tt. Conditional Operator
 - uu. Bitwise Operators
 - vv. Special operators

Unary Operator:

C includes a class of operators that act upon a single operand to produce a new value. Such operators are known as **unary operators**. Unary operators usually precede their single operands, though some unary operators are written after their operands.

Perhaps the most common unary operation is **unary minus**, where a numerical constant, variable or expression is preceded by a minus sign. Note that the *unary minus* operation is distinctly different from the arithmetic operator which denotes *subtraction* (-). The subtraction operator requires two separate operands. Here are several examples which illustrate the use of the unary minus operation.

```
-723          -0X7FFF          -0.2          -5E-8
-root1       -(x + y)         -3 * (x + y)
```

There are two others commonly used unary operators: The **increment operator**, ++, and the **decrement operator**, --. The increment operator causes its operand to be increased by 1, whereas the decrement operator causes its operand to be decreased by 1. The operand used with each of these operators must be a single variable.

Suppose that *i* is an integer variable that has been assigned a value of 5. The expression ++*i*, which is equivalent to writing $i = i + 1$, causes the value of *i* to be increased to 6. Similarly, the expression --*i*, which is equivalent to $i = i - 1$, causes the (original) value of *i* to be decreased to 4.

The increment and decrement operators can each be utilized two different ways, depending on whether the operator is written *before* or *after* the operand. If the operator precedes the operand (e.g., ++*i*), then the operand will be altered in value *before* it is utilized for its intended purpose within the program. If, however, the operator *follows* the operand (e.g., *i*++), then the value of the operand will be altered *after* it is utilized.

A C program includes an integer variable *i* whose initial value is 1. Suppose the program includes the following three *printf* statements.

```
printf("i = %d\n", i);
printf("i = %d\n", ++i);
printf("i = %d\n", i);
```

These *printf* statements will generate the following three lines of output.

```
i = 1
i = 2
i = 2
```

The first statement causes the original value of *i* to be displayed. The second statement increments *i* and then displays its value. The final value of *i* is displayed by the last statement. Now suppose that the program includes the following three *printf* statements, rather than the three statements given above.

```
printf("i = %d\n", i);
printf("i = %d\n", i++);
printf("i = %d\n", i);
```

These statements will generate the following three lines of output.

```
i = 1
i = 1
i = 2
```

The first statement causes the original value of *i* to be displayed, as before. The second statement causes the current value of *i* (1) to be displayed and then incremented (to 2). The final value of *i* (2) is displayed by the last statement.

Another unary operator that is worth mentioning currently is the **sizeof** operator. This operator returns the size of its operand, in bytes. The *sizeof* operator always precedes its operand. The

operand may be an expression, or it may be a cast. Suppose that *i* is an integer variable, *x* is a floating-point variable, *d* is a double-precision variable, and *c* is a character-type variable. The statements

```
printf("integer: %d\n", sizeof i);
printf("float: %d\n", sizeof x);
printf("double: %d\n", sizeof d);
printf("character: %d\n", sizeof c);
```

might generate the following output.

```
integer: 4
float: 4
double: 8
character: 1
```

Thus, we see that this version of C allocates 4 bytes to each integer quantity, 4 bytes to each floating-point quantity, 8 bytes to each double-precision quantity, and 1 byte to each character. These values may vary from one version of C to another.

Another way to generate the same information is to use a cast rather than a variable within each printf statement. Thus, the printf statements could have been written as

```
printf("integer: %d\n", sizeof(int));
printf("float: %d\n", sizeof (float));
printf("double: %d\n", sizeof (double));
printf("character: %d\n", sizeof (char));
```

These printf statements will generate the same output as that shown above. Note that the cast is enclosed in parentheses.

Finally, consider the array declaration

```
char text [ ] = "Kathmandu";
```

The statement

```
printf ("number of characters = %d", sizeof text);
```

will generate the following output.

```
number of characters = 10
```

A **cast** is also considered to be a unary operator. In general terms, a reference to the cast operator is written as (type).

```
(data type) expression
```

This type of construction is known as a cast.

Suppose that *i* is an integer variable whose value is 7, and *f* is a floating-point variable whose value is 5. The expression

```
(i + f)%4
```

is invalid, because the first operand (*i + f*) is floating-point rather than integer.

However, the expression

```
((int)(i + f))%4
```

forces the first operand to be an integer and is therefore valid, resulting in the integer remainder 3.

Note that the explicit type specification applies only to the first operand, not the entire expression.

Unary operators have a higher precedence than arithmetic operators. Hence, if a unary minus operator acts upon an arithmetic expression that contains one or more arithmetic operators, the unary minus operation will be carried out first (unless, of course, the arithmetic expression is enclosed in parentheses). Also, the associativity of the unary operators is right to left, though consecutive unary operators rarely appear in elementary programs.

Suppose that x and y are integer variables whose values are 10 and 20, respectively. The value of the expression $-x + y$ will be $-10 + 20 = 10$. Note that the unary minus operation is carried out before the addition.

Now suppose that parentheses are introduced, so that the expression becomes $-(10 + 20)$. The value of this expression is $-(10 + 20) = -30$. Note that the addition now precedes the unary minus operation.

Binary Operators

Binary Operators operates on two operands. There are several operators which belongs to binary operators.

Arithmetic Operators

The C provides all the Arithmetic Operators (*binary*) which perform arithmetic operations.

Operator	Purpose	Example	Output int a=20, b=6
+	Addition	$a + b$	26
-	Subtraction	$a - b$	14
*	Multiplication	$a * b$	120
/	Division	a / b	3
%	Remainder after integer division	$a \% b$	2

The operands acted upon by arithmetic operators must represent numeric values. The division operator requires that the second operand must be non-zero. The remainder operator (%) requires that both operands must be integers and the second operand should be non-zero.

Relational Operators

They are binary operators which are used to compare two similar operands. They compare their left-hand side operand with their right-hand side operand for comparisons according to magnitudes. The output of a relational operation is either *true* (1) or *false* (0). So, they are used in decision making statements.

Operator	Purpose	Example	Output int a=20, b=6	Output int a=20, b=20
<	Less than	$a < b$	0	0
>	Greater than	$a > b$	1	0
<=	Less than or Equal to	$a <= b$	0	1
>=	Greater than or Equal to	$a >= b$	1	1

These operators all fall within the same precedence group, which is lower than the arithmetic and unary operators. The associativity of these operators is left to right.

Closely associated with the relational operators are the following two *equality operators*.

Operator	Purpose	Example	Output	Output
----------	---------	---------	--------	--------

			int a=20, b=6	int a=20, b=20
==	Equal to	a == b	0	1
!=	Not Equal to	a != b	1	0

The equality operators fall into a separate precedence group, beneath the relational operators. These operators also have a *left-to-right associativity*.

These six operators are used to form **logical expressions**, which represent conditions that are either true or false. The resulting expressions will be of type integer, since *true* is represented by the integer value 1 and *false* is represented by the value 0.

Logical Operators

In addition to the relational and equality operators, C contains two *logical operators* (also called *logical connectives*). They are

<u>Operator</u>	<u>Meaning</u>
&&	Logical AND
 	Logical OR

The logical operators act upon operands that are themselves logical expressions. The net effect is to combine the individual logical expressions into more complex conditions that are either true or false. The result of a *logical and* operation will be true only if both operands are true, whereas the result of a *logical or* operation will be true if either operand is true or if both operands are true. In other words, the result of a *logical or* operation will be false only if both operands are false.

Suppose that *i* is an integer variable whose value is 7, *f* is a floating-point variable whose value is 5.5, and *c* is a character variable that represents the character 'w'. Several complex logical expressions that make use of these variables are shown below.

Expression	Interpretation	Value
(i >= 6) && (c == 'w')	True	1
(i >= 6) (c == 119)	True	1
(f < 11) && (i > 100)	False	0
(c != 'p') ((i + f) <= 10)	True	1

Assignment Operator

There are several different assignment operators in C. All of them are used to form *assignment expressions*, which assign the value of an expression to an identifier. The most used assignment operator is =. Assignment expressions that make use of this operator are written in the form

```
identifier = expression
```

where *identifier* generally represents a variable, and *expression* represents a constant, a variable or a more complex expression.

Here are some typical assignment expressions that make use of the = operator.

```
a = 3;
x = y;
delta = 0.001;
sum = a + b;
area = length * width;
```

Bitwise Operators

Bitwise operators are used for manipulating data at bit level. These operators are used for testing bits or shifting them to the left or to the right. Bitwise operators can be applied only to integer-type operands. There are three types of bitwise operators:

- Bitwise logical operators
- Bitwise shift operators
- One's complement operators

Bitwise logical operators: Bitwise logical operators perform logical tests between two integer operands bit by bit starting from Least Significant Bit (LSB) or right most bit. There are three bitwise logical operators: bitwise AND (&), bitwise OR (|) and bitwise Exclusive OR (^).

Bitwise Shift Operators: Bitwise shift operators are used to move bit patterns either to the left or to the right. There are two bitwise shift operators: *left shift* (<<) and *right shift* (>>).

Bitwise One's Complement: Bitwise One's complement operator is a unary operator which *inverts* (i.e., 1 to 0 and vice versa) all the bits represented by its operand.

Ternary Operator (? :)

Simple conditional operations can be carried out with the *conditional operator* (? :). An expression that makes use of the conditional operator is called a *conditional expression*. Such an expression can be written in place of the more traditional *if-else* statement. The conditional expression is written in the form

expression 1 ? *expression 2* : *expression 3*

When evaluating a conditional expression, *expression 1* is evaluated first. If *expression 1* is true (i.e., if its value is nonzero), then *expression 2* is evaluated and this becomes the value of the conditional expression. However, if *expression 1* is false (i.e., if its value is zero), then *expression 3* is evaluated and this becomes the value of the conditional expression. Note that only one of the embedded expressions (either *expression 2* or *expression 3*) is evaluated when determining the value of a conditional expression.

In the conditional expression shown below, assume that *i* is an integer variable.

$(i < 0) ? 0 : 100;$

The expression $(i < 0)$ is evaluated first. If it is true (i.e., if the value of *i* is less than 0), the entire conditional expression takes on the value 0. Otherwise (if the value of *i* is not less than 0), the entire conditional expression takes on the value 100.

In the following conditional expression, assume that *f* and *g* are floating-point variables.

$(f < g) ? f : g;$

This conditional expression takes on the value of *f* if *f* is less than *g*; otherwise, the conditional expression takes on the value of *g*. In other words, the conditional expression returns the value of the *smaller* of the two variables.

If the operands (i.e., *expression 2* and *expression 3*) differ in type, then the resulting data type of the conditional expression will be determined by the rules given in previous section (Arithmetic Operator).

Conditional expressions frequently appear on the right-hand side of a simple assignment statement. The resulting value of the conditional expression is assigned to the identifier on the left. Here is an assignment statement that contains a conditional expression on the right-hand side.

`flag = (i < 0) ? 0 : 100;`

If the value of *i* is negative, then 0 will be assigned to *flag*. If *i* is not negative, however, then 100 will be assigned to *flag*. Here is another assignment statement that contains a conditional expression on the right-hand side.

```
min = (f < g) ? f : g;
```

This statement causes the value of the smaller of *f* and *g* to be assigned to *min*.

The conditional operator has its own precedence, just above the assignment operators. The associativity is right to left.

2.b. Differentiate between while loop and do-while loop. Write a program to print the sum of digits of a number.

Solution:

while	do-while
While loop is <i>entry-controlled (pre-test)</i> loop, i.e. test condition is evaluated first and body of loop is executed only if the test is true.	Do-while loop is <i>exit-controlled (post-test)</i> loop, i.e. the body of loop is executed first without checking any condition and at the end of loop body, the condition is evaluated for repetition of next time.
The body of loop may not be executed at all if the condition is not satisfied at the very first attempt.	The body of the loop is executed at least once even the tested condition is false.
Syntax: while (expression) statement The statement will be executed repeatedly, as long as the expression is true (i.e., as long expression has a nonzero value). This statement can be simple or compound.	Syntax: do statement while (expression) ; The statement will be executed repeatedly, as long as the value of expression is true (i.e., is nonzero). The statement can be either simple or compound.
<pre>int i = 5; while (i < 5) { printf("%d ", i); ++i; }</pre> This loop condition is initially false. So, it does not print any value.	<pre>int i = 5; do { printf("%d ", i); ++i; } while (i < 5);</pre> Although body and conditions in both loops are same, this program will print 5.

```
//to print the sum of digits of a number
```

```
#include <stdio.h>
```

```
void main()
```

```
{
```

```
    int n, r, sum = 0;
```

```
    printf("Enter a number: ");
```

```
    scanf(" %d", &n); //initialization
```

```
    while (n > 0)
```

```
    {
```

```
        r = n % 10;
```

```
        sum += r;
```

```

        n /= 10; //update expression
    }
    printf("Sum of digits: %d", sum);
}

```

OUTPUT

```

Enter a number: 2077
Sum of digits: 16

```

3.a. Why are functions used? Explain call by value and call by reference with examples.

Solution:

The use of functions allows a large program to be broken down into a few smaller, self-contained components, each of which has some unique, identifiable purpose. Thus, a C program can be **modularized** through the intelligent use of such functions.

There are several *advantages* to this modular approach to program development. For example, many programs require that a group of instructions be accessed repeatedly, from several different places within the program. The repeated instructions can be placed within a single function, which can then be accessed whenever it is needed. Moreover, a different set of data can be transferred to the function each time it is accessed. *Thus, the use of a function avoids the need for redundant (repeated) programming of the same instructions.*

Equally important is the **logical clarity** resulting from the decomposition of a program into several concise functions, where each function represents some well-defined part of the overall problem. Such programs are easier to write and easier to debug, and their logical structure is more apparent than programs which lack this type of structure. This is especially true of lengthy, complicated programs. Most C programs are therefore modularized in this manner, even though they may not involve repeated execution of the same tasks. In fact, the decomposition of a program into individual program modules is generally considered to be an important part of good programming practice.

Call by Value

When a single value is passed to a function via an actual argument, the value of the actual argument is *copied* into the function. Therefore, *the value of the corresponding formal argument can be altered within the Function, but the value of the actual argument within the calling routine will not change.* This procedure for passing the value of an argument to a function is known as *passing by value*.

Here is a simple C program containing a function that alters the value of its argument.

```

#include <stdio.h>

void modify(int a);

main()
{
    int a = 2;

    printf("\na = %d (from main, before calling the
function)", a);
    modify(a); // function call a is a normal variable
    printf("\na = %d (from main, after calling the
function)", a);
}

```

```

void modify(int a)
{
    a *= 3;
    printf("\na = %d (from the function, after being
modified)", a);
    return;
}

```

The original value of *a* (i.e., *a* = 2) is displayed when main begins execution. This value is then passed to the function *modify*, where it is multiplied by 3 and the new value displayed. Note that it is the altered value of the formal argument that is displayed within the function. Finally, the value of *a* within main (i.e., the actual argument) is again displayed, after control is transferred back to main from *modify*. When the program is executed, the following output is generated.

```

a = 2 (from main, before calling the function)
a = 6 (from the function, after being modified)
a = 2 (from main, after calling the function)

```

These results show that *a* is not altered within main, even though the corresponding value of *a* is changed within *modify()*.

Passing an argument by value has advantages and disadvantages. On the plus side, it allows a single-valued actual argument to be written as an expression rather than being restricted to a single variable. Moreover, if the actual argument is expressed simply as a single variable, it protects the value of this variable from alterations within the function. On the other hand, it does not allow information to be transferred back to the calling portion of the program via arguments. Thus, *passing by value is restricted to a one-way transfer of information*.

Call by Reference

To pass an array to a function, the array name must appear by itself, without brackets or subscripts, as an actual argument within the function call. The corresponding formal argument is written in the same manner, though it must be declared as an array within the formal argument declarations. When declaring a one-dimensional array as a formal argument, the array name is written with a pair of empty square brackets. The size of the array is *not specified* within the formal argument declaration.

When an array is passed to a function, however, the values of the array elements *are not* passed to the function. Rather, the array name is interpreted as the **address** of the first array element (i.e., the address of the memory location containing the first array element). This address is assigned to the corresponding formal argument when the function is called. The formal argument therefore becomes a **pointer** to the first array element (more about this in the next chapter). Arguments that are passed in this manner are said to be passed **by reference** rather than passed *by value*.

When a reference is made to an array element within the function, the value of the element's subscript is added to the value of the pointer to indicate the address of the specified array element. Therefore, any array element can be accessed from within the function. Moreover, *if an array element is altered within the function, the alteration will be recognized in the calling portion of the program* (actually, throughout the entire scope of the array).

Here is a simple C program that passes a three-element integer array to a function, where the array elements are altered. The values of the array elements are displayed at three different places in the program, thus illustrating the effects of the alterations.

```

#include <stdio.h>
void modify(int a[]); /* function prototype */

main()
{
    int count, a[3];

    printf("\nEnter any three numbers:\n");
    for (count = 0; count < 3; ++count)
    {
        printf("\na[%d] = ", count);
        scanf(" %d", &a[count]);
    }

    printf("From main(), before modification:\n");
    for (count = 0; count < 3; ++count)
    {
        printf("\na[%d] = %d", count, a[count]);
    }

    modify(a); /* function call a is an array*/

    printf("From main(), after modification:\n");
    for (count = 0; count < 3; ++count)
    {
        printf("\na[%d] = %d", count, a[count]);
    }
}

void modify(int a[])    /* function definition */
{
    int count;

    printf("\nFrom the function, after modification the
values:\n");
    for (count = 0; count < 3; ++count)
    {
        a[count] = -9;
        printf("a[%d] = %d\n", count, a[count]);
    }
    //return;
}

```

The array elements are read from the user within the first loop appearing in *main*. These values are displayed as soon as they are assigned. The array is then passed to the function **modify**, when each array element is assigned the value **-9**. These new values are then displayed from within the function. Finally, the values of array elements are again displayed from **main()**, after control has been transferred back to *main* from *modify*. When the program is executed, the following output is generated.

Enter any three numbers:

a[0] = 1

a[1] = 2

a[2] = 3

From main(), before modification:

a[0] = 1

a[1] = 2

a[2] = 3

From the function, after modification the values:

a[0] = -9

a[1] = -9

a[2] = -9

From main(), after modification:

a[0] = -9

a[1] = -9

a[2] = -9

These results show that the elements of **a** are altered within *main* as a result of the changes that were made within *modify*.

3.b. Define function, function definition, function calling and function declaration with code example.

Solution:

A **function** is a self-contained program segment that carries out some specific, well-defined task. Every C program consists of one or more functions. One of these functions must be called **main**. Execution of the program will always begin by carrying out the instructions in **main**. Additional functions will be subordinate to **main**, and perhaps to one another.

If a program contains multiple functions, their definitions may appear in any order, though they must be independent of one another. That is, one function definition cannot be embedded within another.

A function will carry out its intended action whenever it is *accessed* (i.e., whenever the function is "called") from some other portion of the program. The same function can be accessed from several different places within a program. Once the function has carried out its intended action, control will be returned to the point from which the function was accessed.

Generally, a function will process information that is passed to it from the calling portion of the program and return a single value. Information is passed to the function via special identifiers called **arguments** (also called **parameters**) and returned via the **return** statement. Some functions, however, accept information but do not return anything (as, for example, the library function **printf**), whereas other functions (e.g., the library function **scanf**) return multiple values.

Function Definition

A function definition has two principal components: the *first line* (including the *argument declarations*), and the *body* of the function.

The first line of a function definition contains the type specification of the **value returned** by the function, followed by the **function name**, and (optionally) a **set of arguments**, separated by **commas** and enclosed in **parentheses**. Each argument is preceded by its associated type declaration. An empty pair of parentheses must follow the function name if the function definition does not include any arguments.

In general terms, the first line can be written as

data-type name(type 1 arg 1, type 2 arg 2, ..., type n arg n)

where **data - type** represents the data type of the item that is returned by the function, **name** represents the function name, and **type 1, type 2, ... , type n** represent the data types of the arguments **arg 1, arg 2, ..., arg n** respectively. The data types are assumed to be of type **int** if they are not shown explicitly. However, the omission of the data types is considered poor programming practice, even if the data items are integers.

Function Prototypes (Declaration)

In the programs that we have examined the programmer-defined function has always *preceded* main. Thus, when these programs are compiled, the programmer-defined function will have been defined before the first function access. However, many programmers prefer a "**top-down**" approach, in which main appears ahead of the programmer-defined function definition. In such situations the function access (within main) will precede the function definition. This can be confusing to the compiler, unless the compiler is first alerted to the fact that the function being accessed will be defined later in the program. A *function prototype* is used for this purpose.

Function prototypes are usually written at the beginning of a program, ahead of any programmer-defined functions (including main). The general form of a function prototype is

```
data-type name(type 1 arg 1, type 2 arg 2, ..., type n arg n) ;
```

where *data-type* represents the data type of the item that is returned by the function, *name* represents the function name, and *type 1, type 2, ... , type n* represent the data types of the arguments *arg 1, arg 2, ..., arg n* respectively. Notice that a function prototype resembles the first line of a function definition (though a function prototype ends with a semicolon).

The names of the arguments within the function prototype need not be declared elsewhere in the program, since these are "*dummy*" argument names that are recognized only within the prototype. In fact, the argument names can be omitted (though it is not a good idea to do so); however, the argument *data types* are essential.

```
/* calculate the factorial of an integer quantity */
#include <stdio.h>
```

```
long int factorial(int n);      /* function
prototype/declaration */
```

```
main()
{
    int n;
    /* read in the integer quantity */
    printf("\n n= ");
    scanf(" %d", &n);
```

```
/* calculate and display factorial */
printf("n! = %ld", factorial(n));
}
```

```
long int factorial(int n)//Function definition
/* calculate the factorial of n */
{
    int i;
    long int prod = 1;
```

```

if (n > 1)
{
    for (i= 1; i <= n; ++i)
        prod *= i;
}
return prod;
}

```

OUTPUT

```

n= 5
n! = 120

```

4.a. Define local variables and global variables. Explain different storage classes with examples.

Solution:

There are four different storage-class specifications in C: **automatic**, **external**, **static** and **register**. They are identified by the keywords **auto**, **extern**, **static**, and **register**, respectively.

Automatic Variables

Automatic variables are always declared within a function and are local to the function in which they are declared; that is, their scope is confined to that function. Automatic variables defined in different functions will therefore be independent of one another, even though they may have the same name.

Any variable declared within a function is interpreted as an automatic variable unless a different storage-class specification is shown within the declaration. This includes formal argument declarations. All the variables in the programming examples encountered earlier have been automatic variables.

Since the location of the variable declarations within the program determines the *automatic storage class*, the keyword **auto** is not required at the beginning of each variable declaration. There is no harm in including an auto specification within a declaration, though this is normally not done.

Automatic variables can be assigned initial values by including appropriate expressions within the variable declarations, as in the above example, or by explicit assignment expressions elsewhere in the function. Such values will be reassigned each time the function is reentered. If an automatic variable is not initialized in some manner, however, its initial value will be *unpredictable*, and probably *unintelligible* (i.e., *garbage* value).

An automatic variable does not retain its value once control is transferred out of its defining function.

External (Global) Variables

External variables, in contrast to automatic variables, are not confined to single functions. Their scope extends from the point of definition through the remainder of the program. Hence, they usually span two or more functions, and often an entire program. They are often referred to as **global variables**.

Since external variables are recognized globally, they can be accessed from any function that falls within their scope. They retain their assigned values within this scope. Therefore, an external variable can be assigned a value within one function, and this value can be used (by accessing the external variable) within another function.

The use of external variables provides a convenient mechanism for **transferring information back and forth between functions**. We can transfer information into a function without using

arguments. This is especially convenient when a function requires numerous input data items. Moreover, we now have a way to transfer multiple data items out of a function, since the *return* statement can return only one data item.

The storage-class specifier **extern** is not required in an external variable definition, since the external variables will be identified by the location of their definition within the program. In fact, many C compilers forbid the use of **extern** within an external variable definition.

External variables can be assigned initial values as a part of the variable definitions, but the initial values must be expressed as **constants** rather than expressions. These initial values will be assigned only once, at the beginning of the program. The external variables will then retain these initial values unless they are later altered during the execution of the program. If an initial value is not included in the definition of an external variable, the variable will automatically be assigned a value of **zero**.

Static Variables

Static variables are defined within individual functions and therefore have the same scope as automatic variables; i.e., they are local to the functions in which they are defined. Unlike automatic variables, however, static variables retain their values throughout the life of the program. Thus, if a function is exited and then re-entered later, the static variables defined within that function will retain their former values. This feature allows functions to retain information permanently throughout the execution of a program.

Static variables are defined within a function in the same manner as automatic variables, except that the variable declaration must begin with the **static** storage-class designation. Static variables can be utilized within the function in the same manner as other variables. They cannot, however, be accessed outside of their defining function.

It is not unusual to define automatic or static variables having the same names as external variables. In such situations the local variables will *take precedence over* the external variables, though the values of the external variables will be unaffected by any manipulation of the local variables. Thus, the external variables maintain their independence from locally defined automatic and static variables. The same is true of local variables within one function that have the same names as local variables within another function.

Initial values can be included in the static variable declarations. The rules associated with the assignment of these values are essentially the same as the rules associated with the initialization of external variables, even though the static variables are defined locally within a function. In particular:

- The initial values must be expressed as constants, not expressions.
- The initial values are assigned to their respective variables at the beginning of program execution. The variables retain these values throughout the life of the program, unless different values are assigned during the computation.
- **Zeros** will be assigned to all static variables whose declarations do not include explicit initial values. Hence, static variables will always have assigned values.

Register Variables

Registers are special storage areas within the computer's central processing unit. The actual arithmetic and logical operations that comprise a program are carried out within these registers. Normally, these operations are carried out by transferring information from the computer's memory to these registers, carrying out the indicated operations, and then transferring the results back to the computer's memory. This general procedure is repeated many times during a program's execution.

For some programs, the execution time can be reduced considerably if certain values can be stored within these registers rather than in the computer's memory. Such programs may also be somewhat smaller in size (i.e., they may require fewer instructions), since fewer data transfers will be required. Usually, however, the size reduction will not be dramatic and will be less significant than the savings in execution time.

In C, the values of **register variables** are stored within the registers of the central processing unit. A variable can be assigned this storage class simply by preceding the type declaration with the keyword **register**. There can, however, be only a few register variables (typically, two or three) within any one function. The exact number depends upon the particular computer, and the specific C compiler. Usually, only integer variables are assigned the register storage class.

The register and automatic storage classes are closely related. Their visibility (i.e., their scope) is the same. Thus, **register** variables, like automatic variables, are local to the function in which they are declared. Furthermore, the rules governing the use of register variables are the same as those for automatic variables, except that the address operator (**&**) cannot be applied to register variables.

The similarities between register and automatic variables is not coincidental, because the register storage class can be assigned only to variables that would otherwise have the storage class automatic. Moreover, declaring certain variables to be register variables does not guarantee that they will be treated as register variables. The declaration will be valid only if the requested register space is available. If a register declaration is not honored, the variables will be treated as having the storage class automatic. A program that makes use of **register** variables should run faster than the corresponding program without **register** variables. It may also be somewhat smaller in size.

The storage class associated with a variable can sometimes be established simply by the location of the variable declaration within the program. In other situations, however, the keyword that specifies a particular storage class must be placed at the beginning of the variable declaration.

Shown below are several typical variable declarations that include the specification of a storage class.

```
auto int a, b, c;
extern float root1, root2;
static int count = 0;
extern char star;
```

The first declaration states that **a**, **b** and **c** are automatic integer variables, and the second declaration establishes **root1** and **root2** as external floating-point variables. The third declaration states that **count** is a static integer variable whose initial value is 0, and the last declaration establishes **star** as an external character-type variable.

4.b. How dynamic memory allocation can be achieved? Explain with a suitable example. What are advantages of DMA?

Solution:

Since an array name is a pointer to the first element within the array, it should be possible to define the array as a pointer variable rather than as a conventional array. Syntactically, the two definitions are equivalent. However, a conventional array definition results in a fixed block of memory being reserved at the beginning of program execution, whereas this does not occur if the array is represented in terms of a pointer variable. Therefore, the use of a pointer variable to represent an array requires some type of initial memory assignment before the

array elements are processed. This is known as **dynamic memory allocation**. Generally, the **malloc** library function is used for this purpose, as illustrated in the next example.

Suppose **x** is a one-dimensional, 10-element array of integers. It is possible to define **x** as a pointer variable rather than an array. Thus, we can write

```
int *x;
```

rather than

```
int x[10];
```

However, **x** is not automatically assigned a memory block when it is defined as a pointer variable, though a block of memory large enough to store 10 integer quantities will be reserved when **x** is defined as an array.

To assign enough memory for **x**, we can make use of the library function **malloc**, as follows.

```
x = (int *) malloc(10 * sizeof(int));
```

This function reserves a block of memory whose size (in bytes) is equivalent to 10 integer quantities. As written, the function returns a pointer to an integer. This pointer indicates the beginning of the memory block. In general, the type cast preceding **malloc** must be consistent with the data type of the pointer variable. Thus, if **y** were defined as a pointer to a double-precision quantity and we wanted enough memory to store 10 double-precision quantities, we would write

```
y = (double *) malloc(10 * sizeof(double));
```

*If the declaration is to include the assignment of initial values, however, then **x** must be defined as an array rather than a pointer variable.* For example,

```
int x[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
```

or

```
int x[] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
```

When programming in C, it is not unusual to use pointer expressions rather than references to individual array elements. The resulting programs may appear strange at first, though they are straightforward once you become comfortable accessing values that are stored in specified addresses. Generally, a small amount of practice is all that is required.

Other functions which are involved with DMA are **calloc()**, **realloc()** and **free()**.

Reordering a List of Numbers To illustrate the use of pointers with dynamic memory allocation, let us once again consider the problem of reordering a list of integers. Now, however, we will utilize pointer expressions to access numerical values rather than refer explicitly to individual array elements. Here is the complete C program.

```
/* reordering a one-dimensional, integer array from  
smallest to largest, using pointer notation */
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
void reorder(int n, int *x);
```

```
main()
```

```
{
```

```
    int i, n, *x;
```

```
    /* read in a value for n */
```

```

printf("How many numbers will be entered? ");
scanf(" %d", &n);

/* dynamically allocate memory for n numbers */
x = (int *) malloc(n * sizeof(int));

/*read in the list of numbers */
for (i = 0; i < n; ++i)
{
    printf("i = %d x = ", i + 1);
    scanf(" %d", x + i);
}

/* reorder all array elements */
reorder(n, x);

/* display the reordered list of numbers */
printf("\nReordered list of numbers:\n");
for (i = 0; i < n; ++i)
    printf("i = %d x = %d\n", i+1, *(x + i));
free(x);
}

void reorder(int n, int *x)
{
    int i, item, temp;
    for (item = 0; item < n - 1; ++item)
        for (i = item + 1; i < n; ++i)
            if(*(x + i) < *(x + item))
            {
                temp = *(x + item);
                *(x + item) = *(x + i);
                *(x + i) = temp;
            }
    return;
}

```

In this program, the integer array is defined as a pointer to an integer. Memory is initially assigned to the pointer variable via the malloc library function. Elsewhere in the program, pointer notation is used to process the individual array elements. For example, the function prototype now specifies that the second argument is a pointer to an integer quantity rather than an integer array. This pointer will identify the beginning of the integer array.

OUTPUT

How many numbers will be entered? 6

```

i = 1    x = 20
i = 2    x = 30
i = 3    x = 15
i = 4    x = 5
i = 5    x = 25
i = 6    x = -35

```

Reordered list of numbers:

```
i = 1    x = -35
i = 2    x = 5
i = 3    x = 15
i = 4    x = 20
i = 5    x = 25
i = 6    x = 30
```

5.a. What do you mean by array? How do you initialize one-dimensional array at compile time and run time? Explain with example.

Solution:

Arrays are defined in much the same manner as ordinary variables, except that each array name must be accompanied by a size specification (i.e., the number of elements). For a one-dimensional array, the size is specified by a positive integer expression, enclosed in square brackets. The expression is usually written as a positive integer constant.

In general terms, a one-dimensional array definition may be expressed as

storage-class data-type array[expression] ;

where *storage-class* refers to the storage class of the array, *data-type* is the data type, *array* is the array name, and *expression* is a positive-valued integer expression which indicates the number of array elements. The *storage-class* is optional; default values are **automatic** for arrays that are defined within a function or a block, and **external** for arrays that are defined outside of a function.

Several typical one-dimensional array definitions are shown below.

```
int x[100];
char text[80];
static char message[25];
static float n[12];
```

Compile Time Initialization

The initial values must appear in the order in which they will be assigned to the individual array elements, enclosed in braces and separated by commas. The general form is

```
storage-class data-type array[expression] = {value 1, value 2, ..., value n};
```

where **value 1** refers to the value of the first array element, **value 2** refers to the value of the second element, and so on. The appearance of the **expression**, which indicates the number of array elements, is *optional* when initial values are present.

Shown below are several array definitions that include the assignment of initial values.

```
int digits[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
static float x[6] = {0, 0.25, 0, -0.50, 0, 0};
char color[3] = {'R', 'E', 'D', };
```

Note that **x** is a static array. The other two arrays (**digits** and **color**) are assumed to be external arrays by virtue of their placement within the program. The results of these initial assignments, in terms of the individual array elements, are as follows. (Remember that the subscripts in an n-element array range from **0** to **n - 1**.)

```

digits[0] = 1    x[0]    = 0    color[0]    = 'R'
digits[1] = 2    x[1]    = 0.25  color[1]    = 'E'
digits[2] = 3    x[2]    = 0    color[2]    = 'D'
digits[3] = 4    x[3]    = -0.50
digits[4] = 5    x[4]    = 0
digits[5] = 6    x[5]    = 0
digits[6] = 7
digits[7] = 8
digits[8] = 9
digits[9] = 10

```

All individual array elements that are not assigned explicit initial values will automatically be set to **zero**. This includes the remaining elements of an array in which some elements have been assigned nonzero values.

Consider the following array definitions.

```

int digits[10] = {3, 3, 3};
static float x[6] = {-0.3, 0, 0.25};

```

The results, on an element-by-element basis, are as follows.

```

digits[0] = 3    x[0] = -0.3
digits[1] = 3    x[1] = 0
digits[2] = 3    x[2] = 0.25
digits[3] = 0    x[3] = 0
digits[4] = 0    x[4] = 0
digits[5] = 0    x[5] = 0
digits[6] = 0
digits[7] = 0
digits[8] = 0
digits[9] = 0

```

In each case, all the array elements are automatically set to **zero** except those that have been explicitly initialized within the array definitions.

Run Time Initialization

The array can be initialized at run time by accepting data from users. In that case, you may use a loop (for 1-D array) for initialization of array element. The following code skeletal shows the run time initialization.

```

int age[10], i;
for (i = 0; i < 10; ++i)
    scanf("%d", &age[i]);

```

Here scanf() will run at ten times and initialize array elements one at a time. While executing, if user supplies 10, 20, 15, 22, 18, 16, 14, 25, 23, 19 then

```

age[0] = 10  age[1] = 20  age[2] = 15  age[3] = 22  age[4] = 18  age[5] = 16
age[6] = 14  age[7] = 25  age[8] = 23  age[9] = 19

```

5.b. Use recursive function calls to evaluate

$$f(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots$$

Solution

```

//f(x) = x -  $\frac{x^3}{3!}$  +  $\frac{x^5}{5!}$  -  $\frac{x^7}{7!}$  + ...
#include <stdio.h>
#include <math.h>

float term(float x, int n)
{
    if(n == 0)
        return 1;
    else
        return((x * expo(x, n - 1)) / (n * fact(n - 1)));
}

main()
{
    int n, i, j;
    float x, T, sum = 0;

    printf("How many terms? ");
    scanf(" %d",&n);
    printf("Enter value of x: ");
    scanf(" %f",&x);

    for(j = 0, i = 1; j < n; ++j, i += 2)
    {
        T = pow(-1, j) * term(x, i);
        printf("%+f ", T); // '+' flag is used to display
sign
        sum += T;
    }
    printf("\nf(x) = %+f", sum);
}

```

OUTPUT

```

How many terms? 5
Enter value of x: 1
+1.000000 -0.166667 +0.008333 -0.000198 +0.000003
f(x) = +0.841471

```

6.a. What is menu driven structure? Explain with suitable example.

Solution:

Menu driven structure is a programming logic which uses structure along with switch case. The switch case is used to select one alternative among several operations. It is a real programming practice and is very useful for problem solving.

A menu driven program in C to create a structure employee having fields empid, empname, salary. Accept the details of 'n' Employees from user and perform the following operations using function.

- Search by Emp-Id
- Display All Records
- Display Names of Employee having Salary > 10000.

```

#include <stdio.h>
#include <stdlib.h>

Struct Employee
{
    char name[30];
    int eid;
    int salary;
}emp[5];

void emp_search(int n)
{
    int id, i;
    printf("\nEnter Employee-Id to be Searched : ");
    scanf(" %d", &id);
    printf("-----\n");
    for(i = 0; i < n; ++i)
    {
        if(emp[i].eid == id)
        {
            printf("Employee Id      : %d", emp[i].eid);
            printf("\nName          : %s", emp[i].name);
            printf("\nSalary         : %d\n", emp[i].salary);
        }
    }
}

void display(int n)
{
    int i;
    printf("\nList of All Employees:\n");
    printf("-----\n");
    printf("Emp-Id\tEmp-Name   Salary\n");
    printf("-----\n");
    for (i = 0; i < n; ++i)
    {
        printf("%d\t%s\t  %d\n", emp[i].eid, emp[i].name,
emp[i].salary);
    }
}

void greater(int n)
{
    int i;
    printf("\nDetails of Employee Whose Salary > 10000\n");
    printf("-----
");
    for (i = 0; i < n; ++i)
    {
        if (emp[i].salary > 10000)

```

```

        {
            printf("\n Employee Name : %s", emp[i].name);
            printf("\n Employee-Id    : %d", emp[i].eid);
            printf("\n Salary          : %d\n",
emp[i].salary);
        }
    }
}
main()
{
    int n, i, ch;
    printf("How Many Employee Record You Want to Add: ");
    scanf(" %d", &n);
    for (i = 0; i < n; ++i)
    {
        printf("\n\tEnter Details of Employee:%d", i + 1);
        printf("\n-----");
        printf("\nName of Employee: ");
        scanf(" %[^\\n]", emp[i].name);
        printf("Employee-Id: ");
        scanf(" %d", &emp[i].eid);
        printf("Salary: ");
        scanf(" %d", &emp[i].salary);
    }
    while(ch != 4)
    {
        printf("\n\tMenu\n");
        printf("-----");
        printf("\n 1:Search Employee by E-ID");
        printf("\n 2:List of All Employee");
        printf("\n 3:Display Employee Name whose Salary >
10000 ");
        printf("\n 4:Exit");
        printf("\n-----
\n");
        printf("Enter Your Choice: ");
        scanf(" %d", &ch);
        switch(ch)
        {
            case 1: emp_search(n);
                    break;
            case 2: display(n);
                    break;
            case 3: greater(n);
                    break;
            case 4: exit(0);
            default:
                printf("\nInvalid choice!");
        }
    }
}

```



```

    }
    return 0;
}

```

OUTPUT

How Many Employee Record You Want to Add: 3

Enter Details of Employee:1

```

-----
Name of Employee: Rajkumar Mahato
Employee-Id: 123
Salary: 50000

```

Enter Details of Employee:2

```

-----
Name of Employee: Purna Magar
Employee-Id: 234
Salary: 10000

```

Enter Details of Employee:3

```

-----
Name of Employee: Apekshya Pokhrel
Employee-Id: 1000
Salary: 8500

```

Menu

```

-----
1:Search Employee by E-ID
2:List of All Employee
3:Display Employee Name whose Salary > 10000
4:Exit
-----

```

Enter Your Choice: 1

Enter Employee-Id to be Searched : 234

```

-----
Employee Id    : 234
Name           : Purna Magar
Salary         : 10000

```

Menu

```

-----
1:Search Employee by E-ID
2:List of All Employee
3:Display Employee Name whose Salary > 10000
4:Exit
-----

```

Enter Your Choice: 3

Details of Employee Whose Salary > 10000

```

-----
Employee Name : Rajkumar Mahato
Employee-Id   : 123
Salary       : 50000

```

Menu

```

-----
1:Search Employee by E-ID
2:List of All Employee
3:Display Employee Name whose Salary > 10000
4:Exit
-----

```

Enter Your Choice: 2

List of All Employees:

```

-----
Emp-Id  Emp-Name          Salary
-----
123      Rajkumar Mahato      50000
234      Purna Magar          10000
1000     Apekshya Pokhrel     8500

```

Menu

```

-----
1:Search Employee by E-ID
2:List of All Employee
3:Display Employee Name whose Salary > 10000
4:Exit
-----

```

Enter Your Choice: 4

6.b. Why is file handling necessary in C programming? Write a program to input name, address, faculty, program, and GPA (in maximum of 4.0) of 500 students and store them in 'RESULT.DAT' data file and display the records of those students whose faculty is 'Engineering' and GPA > 3.5.

Solution:

There can be many reasons to use files. The first and the foremost reason is need of *permanency* of storage of data. If we want to preserve the output of any program for future use, it is not possible without files. All messages or values printed with help of any output statements like printf, putchar, etc. are never available for future use and the only solution in that case is to write the output in files. Similarly, if there is the large amount of data generated as output by a program, storing that output in file will help in easy handling/analysis of the output, as user can see the whole output at any time even after complete execution of the program.

The input files are also equally important and useful. If a program needs a lot of data to be inputted, user cannot keep on typing that again and again for repeated execution of the program. In that case, all input data can be once written in a file and then that file can be

easily used as the input file. Moreover, the transfer of input-data and/or output-data from one computer to another can be easily done by using files.

```
//2016 spring
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

#define N 500

struct Student
{
    char name[30];
    char add[50];
    char fac[20];
    char pro[20];
    float gpa;
};

void main()
{
    struct Student s;
    int i;
    FILE *fp;

    if ((fp = fopen("RESULT.dat", "w+")) == NULL)
    {
        printf("\nRESULT.dat cannot be created.");
        exit(1);
    }
    printf("Enter details of %d Students:", N);
    for (i = 0; i < N; ++i)
    {
        printf("\nName: ");
        scanf("%[^\\n]", s.name);

        printf("\nAddress: ");
        scanf("%[^\\n]", s.add);

        printf("\nFaculty: ");
        scanf("%[^\\n]", s.fac);

        printf("\nProgram: ");
        scanf("%[^\\n]", s.pro);

        printf("\nGPA: ");
        scanf("%f", &s.gpa);

        fwrite(&s, sizeof(struct Student), 1, fp);
    }
}
```

```

rewind(fp);

printf("\nDetails of Students:\n");
printf("\nName\tAddress\tFaculty\tProgram\tGPA\n");
for (i = 0; i < N; ++i)
{
    fread(&s, sizeof(struct Student), 1, fp);
    if(strcmp("Engineering",s.fac) == 0 && s.gpa > 3.5)
    {
        printf("\n%s\t%s\t%s\t%s\t%f", s.name, s.add,
s.fac, s.pro, s.gpa);
    }
}
}

```

7. Write short notes on:

i. Pointer Arithmetic

The permissible operations are summarized below.

- vii. A pointer variable can be assigned the address of an ordinary variable (e.g., **pv = &v**).
- viii. A pointer variable can be assigned the value of another pointer variable (e.g., **pv = px**) provided both pointers point to objects of the same data type.
- ix. A pointer variable can be assigned a null (zero) value (e.g., **pv = NULL**, where **NULL** is a symbolic constant that represents the value **0**).
- x. An integer quantity can be added to or subtracted from a pointer variable (e.g., **pv + 3**, **++pv**, **--pv**, etc.)
- xi. One pointer variable can be subtracted from another provided both pointers point to elements of the same array.
- xii. Two pointer variables can be compared provided both pointers point to objects of the same data type.

Other arithmetic operations on pointers are not allowed. Thus, a pointer variable cannot be multiplied by a constant; two pointer variables cannot be added; and so on. Also, you are again reminded that an ordinary variable cannot be assigned an arbitrary address (i.e., an expression such as **&x** cannot appear on the left side of an assignment statement).

ii. Differences between Structure and Union

Structure	Union
Each member of structure is assigned its own <i>unique storage</i> . It takes more memory spaces than that of union.	All members of a union share the <i>same storage</i> area of computer memory. It takes less memory than that of structure.
The amount of memory required to store a structure is the sum of the memory sizes required by <i>all</i> its members.	The amount of memory required to store a union is same as memory size occupied by a member which requires <i>largest</i> memory space among the members.
<i>All</i> the members of structure can be accessed at any point of time.	<i>Only one</i> member of union can be accessed at any given time.

Structure is suitable when you have <i>enough memory spaces</i> . It is very efficient to use and handle.	Union is used when you have <i>lack of memory</i> . It is easy but not realistic.
Structure declaration: <pre>struct student { char name[20]; //20 int roll; //2 float marks; //4 char address[50]; //50 } st;</pre> Total size of struct student: 76B	Union declaration: <pre>union student { char name[20]; int roll; float marks; char address[50]; } st;</pre> Total size of union student: 50B

iii. Testing and Debugging

Once the program code has been written in a selected programming language of choice the next task to complete is the testing and debugging. Testing and debugging help to find the problems associated with the program behavior under normal and abnormal circumstances. Extensive testing like white box testing and black box testing, integration testing needs to do before the program is deployed into the real-world scenario.

Programming theorists often refer debugging to verification and testing to validation. Verification ensures that the program does what the programmer intends to do. Validation ensures that the program produces the correct results for a set of test data. The different tools like simulators, logic analyzers, breakpoints, trace routines, software interrupts can be used for effective debugging process.

For testing process, test data are supplied to the program and output is observed. If the output is as expected, the program can be considered error free. The error means failure of compiling and executing the program successfully and receiving incorrect results. The types of errors are: *syntax error, semantic/logical error, and runtime error.*